abLEMON — A User Manual

For abLEMON 1.9.0 (November 2026)

Donato Meoli

Contents

abLEMON — A User Manual **5**
Abstract . . . 5
Conventions . . . 5
Contents . . . 5

1. Introduction **7**
1.1 What abLEMON changes . . . 7
1.2 How the manual is organized . . . 8
1.3 Conventions . . . 9

2. Installation and first program **10**
2.1 Packages . . . 10
2.2 Building from the sources . . . 10
2.3 Using the installed library . . . 11
2.4 Optional backends . . . 11
2.5 Exercises . . . 11

3. Graphs, items and maps **12**
3.1 A first program . . . 12
3.2 Items, identifiers and INVALID . . . 13
3.3 Iterators and ranges . . . 14
3.4 Maps . . . 15
3.5 Concepts and their checking . . . 17
3.6 The algorithm classes . . . 17
3.7 Tolerances and numeric traits . . . 19
3.8 Exercises . . . 21

4. Graph structures **22**
4.1 The graph concepts . . . 22
4.2 List graphs . . . 23
4.3 Smart graphs . . . 25
4.4 Static graphs . . . 26
4.5 Full graphs . . . 29
4.6 Grid and hypercube graphs . . . 29
4.7 Arc sets and edge sets . . . 30
4.8 Counting and finding arcs . . . 30
4.9 Copying graphs . . . 32
4.10 Exercises . . . 34

5. Graph adaptors **36**
5.1 Views of a graph . . . 36
5.2 Reversing and filtering . . . 37
5.3 Changing the orientation . . . 37
5.4 The residual digraph . . . 39
5.5 Splitting the nodes . . . 41
5.6 Exercises . . . 43

6. Maps **44**
6.1 The map concepts . . . 44
6.2 Graph maps . . . 44
6.3 Basic maps . . . 45
6.4 Map adaptors and arithmetic . . . 45
6.5 Maps of the items of a graph . . . 47
6.6 Iterable maps . . . 48
6.7 Map utilities . . . 51
6.8 Exercises . . . 51

7. Heaps **52**
7.1 The heap concept . . . 52
7.2 Implicit d-ary heaps . . . 53
7.3 Linked heaps . . . 54
7.4 Heaps of integer priorities . . . 55
7.5 Choosing a heap . . . 56
7.6 Exercises . . . 60

8. Paths and auxiliary structures **61**
8.1 Paths . . . 61
8.2 Union-find structures . . . 63
8.3 The elevator . . . 66
8.4 Two-dimensional geometry . . . 67
8.5 Radix sort . . . 68
8.6 Random numbers . . . 69
8.7 Counting and timing . . . 69
8.8 Colors . . . 70
8.9 Errors, assertions and the command line . . . 70
8.10 Exercises . . . 73

9. Graph search **74**
9.1 The common interface . . . 74
9.2 Breadth-first search . . . 74
9.3 Depth-first search . . . 76
9.4 Function-type interfaces . . . 76
9.5 Visitors . . . 77
9.6 Maximum cardinality search . . . 79
9.7 Exercises . . . 81

10. Shortest paths **82**
10.1 The problem and its optimality conditions . . . 82
10.2 Dijkstra's algorithm . . . 83
10.3 Shortest paths between two nodes . . . 86
10.4 The Bellman-Ford algorithm . . . 88
10.5 Arc-disjoint paths of least total length . . . 90
10.6 Exercises . . . 92

11. Spanning trees and arborescences **94**
11.1 Minimum spanning trees . . . 94
11.2 Minimum cost arborescences . . . 96
11.3 Exercises . . . 99

12. Maximum flows and circulations **101**
12.1 The problem and its dual . . . 101
12.2 The push-relabel algorithm: Preflow . . . 102
12.3 Augmenting paths: EdmondsKarp . . . 105
12.4 Feasible circulations: Circulation . . . 106
12.5 Tolerances and the common interface . . . 108
12.6 Exercises . . . 108

13. Minimum cost flows **110**
13.1 The problem, its forms and its dual . . . 110
13.2 The common interface . . . 111
13.3 Tolerances and floating-point data . . . 114
13.4 The network simplex method . . . 116
13.5 The starting flow and the infinite bounds . . . 118
13.6 Cost scaling . . . 119
13.7 Capacity scaling . . . 119
13.8 Cycle canceling . . . 120
13.9 Choosing an algorithm . . . 121
13.10 Exercises . . . 121

14. Certificates and the k best flows **122**
14.1 Certificates of infeasibility . . . 122
14.2 Certificates of unboundedness . . . 127
14.3 The k best integer flows: KBestFlows . . . 128
14.4 The k best extreme flows: KBestExtremeFlows . . . 130
14.5 Alternative optimal potentials . . . 134
14.6 Exercises . . . 135

15. Minimum cuts **136**
15.1 The problems . . . 136
15.2 The Hao-Orlin algorithm . . . 137
15.3 The Nagamochi-Ibaraki algorithm . . . 140
15.4 Gomory-Hu trees . . . 141
15.5 Choosing an algorithm . . . 144
15.6 Exercises . . . 144

16. Minimum mean cycles **145**
16.1 The problem . . . 145
16.2 Karp's algorithm . . . 145
16.3 The Hartmann-Orlin algorithm . . . 146
16.4 Howard's policy iteration . . . 147
16.5 The common interface . . . 148
16.6 Exercises . . . 151

17. Matchings **152**
17.1 Problems, duality and the choice of an algorithm . . . 152
17.2 Maximum cardinality matching in bipartite graphs . . . 153
17.3 Minimum cost assignment . . . 154
17.4 Maximum cardinality matching in general graphs . . . 157
17.5 Maximum weight matchings in general graphs . . . 159
17.6 Fractional matchings . . . 162
17.7 Exercises . . . 163

18. Connectivity, planarity, isomorphism **165**
18.1 Connected components, acyclicity and simple graphs . . . 165
18.2 Biconnectivity . . . 166
18.3 Strong components and topological order . . . 167
18.4 Euler tours . . . 170
18.5 Planarity, embedding and drawing . . . 172
18.6 Coloring planar graphs . . . 173
18.7 Subgraph isomorphism: VF2 and VF2++ . . . 176
18.8 Exercises . . . 179

19. Travelling salesman and maximum clique **180**
19.1 The symmetric travelling salesman problem . . . 180
19.2 Tour construction heuristics . . . 180
19.3 Improvement by 2-opt . . . 183
19.4 Choosing a heuristic, and an example . . . 184

19.5 The maximum clique problem . . . 186
19.6 The iterated local search of Grosso, Locatelli and Pullan . . . 186
19.7 The exact algorithm of Tomita and Seki . . . 187
19.8 Exercises . . . 190

20. The LP and MIP interface **191**
20.1 The structure of the interface . . . 191
20.2 Columns, rows and expressions . . . 192
20.3 Building and modifying a problem . . . 193
20.4 Solving an LP and reading the solution . . . 193
20.5 Mixed integer programs . . . 194
20.6 The backends . . . 194
20.7 The skeletons . . . 195
20.8 An example . . . 196
20.9 Exercises . . . 198

21. Input and output **199**
21.1 The LEMON Graph Format . . . 199
21.2 Reading . . . 200
21.3 Writing . . . 201
21.4 Extra sections and the contents of a file . . . 201
21.5 The DIMACS formats . . . 202
21.6 The graph6 format of nauty . . . 202
21.7 Drawing a graph in EPS . . . 203
21.8 Examples . . . 203
21.9 Exercises . . . 207

22. The tools **208**
22.1 Building and calling the tools . . . 208
22.2 dimacs-solver . . . 208
22.3 dimacs-to-lgf . . . 209
22.4 lgf-gen . . . 209
22.5 lemon-0.x-to-1.x.sh and the demos . . . 210
22.6 An example . . . 211
22.7 Exercises . . . 212

A. Writing a graph, a map or a heap **214**
A.1 The concepts as contracts . . . 214
A.2 The base class and the extenders . . . 214
A.3 The alteration notifier . . . 219
A.4 Writing a map . . . 220
A.5 Writing a heap . . . 220
A.6 Checking a new class . . . 220

Y. Glossary **222**

Z. Bibliography **228**

abLEMON — A User Manual

For abLEMON 1.9.0 (November 2026).

Abstract

abLEMON is a C++ template library of graphs, maps, heaps and network optimization algorithms, maintained as a fork of LEMON 1.3.1, whose headers and namespace it keeps. This manual describes the library as it is implemented, family by family: the graph structures and the adaptors that present a graph as another, the maps through which data are attached to nodes, arcs and edges, the heaps and the other auxiliary structures, and the algorithms for graph search, shortest paths, spanning trees, maximum and minimum cost flows (with the certificates of infeasibility and unboundedness and the k best flows that abLEMON adds), minimum cuts, minimum mean cycles, matchings, connectivity, planarity and isomorphism, the heuristics for the travelling salesman and the maximum clique, and the interface to LP and MIP solvers. For each algorithm the manual states the problem it solves, the conditions it maintains, its complexity as implemented and the choices it leaves to the user, gives its pseudocode, and shows it at work in complete programs, which are compiled and run as tests of the library. Yet, it does not replace the reference documentation, which lists every member with its exact signature, and to which the chapters refer when a detail is not worth repeating.

Conventions

The headings carry the numbers of the manual, and sections, figures, tables, algorithms and exercises are numbered by chapter. Programs are included from the files of `manual/examples/`, each of which is a test of the library (`ctest -R manual_`). A note headed "abLEMON" marks what abLEMON adds to LEMON 1.3.1, or where its behavior differs from it; the rest of the library behaves as in LEMON. Citations refer to the bibliography (Appendix Z), and the terms defined in the chapters are collected in the glossary (Appendix Y).

Contents

Part I: Foundations

- 1. Introduction
- 2. Installation and first program
- 3. Graphs, items and maps

Part II: Data structures

- 4. Graph structures
- 5. Graph adaptors
- 6. Maps
- 7. Heaps
- 8. Paths and auxiliary structures

Part III: Algorithms

- 9. Graph search
- 10. Shortest paths
- 11. Spanning trees and arborescences
- 12. Maximum flows and circulations
- 13. Minimum cost flows

- 14. Certificates and the k best flows
- 15. Minimum cuts
- 16. Minimum mean cycles
- 17. Matchings
- 18. Connectivity, planarity, isomorphism
- 19. Travelling salesman and maximum clique
- 20. The LP and MIP interface

Part IV: Input, output and tools

- 21. Input and output
- 22. The tools

Part V: Appendices

- A. Writing a graph, a map or a heap
- Y. Glossary
- Z. Bibliography

1. Introduction

We are concerned with abLEMON, a C++ template library of graphs and of the algorithms of network optimization built on them. It provides the structures that represent directed, undirected and bipartite graphs, the maps that attach data to their nodes, arcs and edges, the heaps and the other auxiliary structures that the algorithms need, and the algorithms themselves (searches, shortest paths, spanning trees, maximum and minimum cost flows, minimum cuts and mean cycles, matchings, connectivity, planarity and isomorphism, heuristics for the travelling salesman and the maximum clique, and an interface to LP and MIP solvers). abLEMON is a maintained fork of LEMON, the Library for Efficient Modeling and Optimization in Networks of the Egerváry Research Group of the Eötvös Loránd University of Budapest (Dezső, Jüttner and Kovács 2011), which has had no release since 1.3.1 (2014). It starts from the last revision of the Mercurial repository of LEMON, whose history it keeps, and it keeps its headers (`lemon/`) and its namespace (`lemon`); hence, a program written for LEMON usually compiles with abLEMON unchanged.

This manual describes the library as it is implemented in abLEMON 1.9.0. Each public class of `lemon/` is described in the chapter of its family, with the theory that one needs to choose among the algorithms and to read their results, and the statements on costs, defaults and behavior have been checked against the code. The manual is a companion to the reference documentation that Doxygen produces from the headers (https://dmeoli.github.io/abLEMON/), which gives each member with its exact signature. We give instead the problem that each class solves, the conditions it maintains, how the classes are meant to be combined, and complete programs; these are compiled and run as tests of the library (cf. chapter 2). We assume that the reader writes C++ and knows the basic notions of graph theory and of linear programming. The network optimization that a chapter needs is recalled in it, with references to the textbook of Ahuja, Magnanti and Orlin (1993) and to the papers of each algorithm.

1.1 What abLEMON changes

The differences from LEMON 1.3.1 fall into four groups, which NEWS lists release by release and which the chapters mark with a note “abLEMON” where they apply. The first group is portability: the library compiles without warnings from C++11 to C++23 with GCC, Clang, AppleClang and MSVC. For instance, the allocators go through `std::allocator_traits`, since C++20 removed the members of `std::allocator` that `ArrayMap` and `ListPath` called; also, each header is self-contained, and the headers survive the macros `min` and `max` of `<windows.h>`. The second group is correctness. Several defects of the algorithms and of the data structures are fixed, most of them with a test that failed before the fix (e.g., `RadixHeap` mixed the items of adjacent boxes, `BinomialHeap` truncated fractional priorities, and `Circulation` read past its storage on an empty digraph), and the items of the graphs are `INVALID` when they are default-constructed.

The third group concerns the minimum cost flow algorithms (chapter 13 and chapter 14). These solve problems with floating-point data, under tolerances relative to the data, and they certify an infeasible or unbounded answer (with a set of nodes whose supply exceeds what can leave it, or with a cycle of negative cost and infinite capacity). Furthermore, they give the k most violated such sets, and two further classes give the k best integer flows and the k best extreme flows; alternative optimal potentials are also available. The fourth group is a set of new algorithms and structures, i.e., `HopcroftKarp` and `MinCostAssignment` for bipartite matching (chapter 17), `BidirDijkstra` and `AStar` for the shortest path between two nodes (chapter 10), `TomitaSekiMc` for the exact maximum clique (chapter 19), and `StaticGraph`, the undirected counterpart of `StaticDigraph` (chapter 4). Also, with CMake 3.28 and Clang 16 or MSVC 19.34, the library may be built as the C++20 module `lemon` (whose interface is `lemon/lemon.cppm`, cf. §2.3).

1.2 How the manual is organized

Part I gives what one needs before anything else: the installation (chapter 2) and the model of the library (chapter 3). In this model graphs have nodes and arcs that are handles, data are attached to them by maps, concepts state what an algorithm requires of the types it is given, and algorithms are classes that are configured, run and then queried. Part II describes the data structures, i.e., the graphs, the adaptors that present a graph as another one without copying it, the maps, the heaps, the paths and the auxiliary structures. Part III describes the algorithms, family by family: each chapter states the problem, the optimality conditions and the complexity of each algorithm, gives its pseudocode, compares the variants that the library offers, and ends with programs and exercises. Part IV covers the formats of input and output (LGF, DIMACS, nauty, PostScript) and the tools of `tools/`. Appendix A explains how a new graph, map or heap is written so that the algorithms accept it, while Appendices Y and Z hold the glossary and the bibliography.

A reader who only needs one algorithm may read chapter 3 and then the chapter of that algorithm, since the chapters of Part III are meant to be read independently of each other. They refer to each other where one algorithm uses another one; e.g., the certificates of chapter 14 rest on the maximum flows of chapter 12. Table 1.1 gives the chapter where each header of `lemon/` is described. In it, `core.h` holds the items of the graphs, the basic utilities and the common definitions, and is included by all the other headers; `config.h`, instead, is written by CMake when the library is configured (cf. §2.4).

Table 1.1. The headers of `lemon/` by chapter.

Chapter	Headers
3	`core.h` (items), `concept_check.h`, `tolerance.h`, `math.h`
4	`list_graph.h`, `smart_graph.h`, `static_graph.h`, `compact_graph.h`, `full_graph.h`, `grid_graph.h`, `hypercube_graph.h`, `edge_set.h`, `core.h` (utilities), `concepts/digraph.h`, `concepts/graph.h`, `concepts/bpgraph.h`, `concepts/graph_components.h`
5	`adaptors.h`
6	`maps.h`, `concepts/maps.h`
7	`bin_heap.h`, `quad_heap.h`, `dheap.h`, `fib_heap.h`, `pairing_heap.h`, `binomial_heap.h`, `radix_heap.h`, `bucket_heap.h`, `concepts/heap.h`
8	`path.h`, `concepts/path.h`, `unionfind.h`, `elevator.h`, `dim2.h`, `radix_sort.h`, `random.h`, `counter.h`, `time_measure.h`, `color.h`, `error.h`, `assert.h`, `arg_parser.h`
9	`bfs.h`, `dfs.h`, `max_cardinality_search.h`
10	`dijkstra.h`, `bidir_dijkstra.h`, `astar.h`, `bellman_ford.h`, `suurballe.h`
11	`kruskal.h`, `min_cost_arborescence.h`
12	`preflow.h`, `edmonds_karp.h`, `circulation.h`, `concepts/max_flow.h`
13	`network_simplex.h`, `cost_scaling.h`, `capacity_scaling.h`, `cycle_canceling.h`, `concepts/min_cost_flow.h`
14	`k_best_flows.h`, `k_best_extreme_flows.h`
15	`hao_orlin.h`, `nagamochi_ibaraki.h`, `gomory_hu.h`
16	`karp_mmc.h`, `hartmann_orlin_mmc.h`, `howard_mmc.h`, `concepts/min_mean_cycle.h`
17	`matching.h`, `bipartite_matching.h`, `fractional_matching.h`
18	`connectivity.h`, `euler.h`, `planarity.h`, `vf2.h`, `vf2pp.h`
19	`nearest_neighbor_tsp.h`, `greedy_tsp.h`, `insertion_tsp.h`, `christofides_tsp.h`, `opt2_tsp.h`, `grosso_locatelli_pullan_mc.h`, `tomita_seki_mc.h`
20	`lp_base.h`, `lp.h`, `lp_skeleton.h`, `glpk.h`, `clp.h`, `cbc.h`, `cplex.h`, `soplex.h`
21	`lgf_reader.h`, `lgf_writer.h`, `dimacs.h`, `nauty_reader.h`, `graph_to_eps.h`

1.3 Conventions

Class names, members and code are set in `monospace`. A class is named without the namespace `lemon`, and a member is written as `Class::member()` when the class is not clear from the context. A graph has n nodes and m arcs (or edges), while C and U denote the largest absolute value of a cost and of a capacity when the data are integer. The complexities are those of the implementation, and the chapters say so when they differ from the bounds of the literature. Sections, figures, tables, algorithms and exercises are numbered by chapter; a program is named by the file of `manual/examples/` that holds it, and therefore by the test `manual_<name>` that runs it.

2. Installation and first program

We now describe how abLEMON is obtained, either as a package or from the sources, how a program finds it once it is installed, and which of its parts are optional; the first program, which only needs a compiler and the headers, is the subject of chapter 3. Most of abLEMON is made of templates, and therefore lives in its headers. The compiled library (`libemon`, a name kept from LEMON) holds the few parts that are not templates, i.e., the random number generator, the parser of the command line, the colors of `graph_to_eps.h`, the locking on Windows and, when they are enabled, the interfaces to the LP and MIP solvers of chapter 20.

2.1 Packages

The recipes of the packages are in `packaging/` of the repository, and each release brings them to the archive of its tag (the workflow `release.yml` does so, after taking the notes of the release from NEWS). The Homebrew tap of the project works on macOS and Linux:

```
brew install dmeoli/ablemon/ablemon
```

Also, the recipes for conda-forge (package `ablemon`), vcpkg (port `ablemon`) and Conan have been submitted to the respective indices; at the time of writing (abLEMON 1.9.0) they are still under review. Until they are accepted, the recipes in `packaging/` can be used locally, e.g., as an overlay of vcpkg (`vcpkg install ablemon --overlay-ports=packaging/vcpkg`). Each package installs the headers, the library and the CMake configuration of §2.3. None of them enables the LP and MIP backends of §2.4, which one builds from the sources when they are needed.

2.2 Building from the sources

Building needs CMake 3.19 or later and a compiler that supports C++11. In fact, the library compiles without warnings with GCC, Clang, AppleClang and MSVC, from C++11 to C++23 (the continuous integration builds each change on Linux, macOS and Windows with these compilers, and also under the address and undefined behavior sanitizers). The usual sequence is

```
cmake -S . -B build -DCMAKE_BUILD_TYPE=Release
cmake --build build
ctest --test-dir build
cmake --install build --prefix /usr/local
```

where the third line runs the tests of `test/` and the programs of this manual (those of `manual/examples/`, which ctest names `manual_*`), and the last one installs the library under the given prefix, following `GNUInstallDirs`.

Among the options, `CMAKE_CXX_STANDARD` is honored when it is set, and is 11 otherwise; `BUILD_SHARED_LIBS` builds a shared library (on Windows, a DLL). `LEMON_BUILD_TESTS`, `LEMON_BUILD_DEMOS`, `LEMON_BUILD_TOOLS` and `LEMON_BUILD_DOC` are `ON` by default, and each of them leaves out a directory when it is `OFF`; `LEMON_BUILD_MODULE` is `OFF` by default (cf. §2.3), and the four options of the LP and MIP backends are described in §2.4. The version is usually taken from the tag of git, and from the variable `LEMON_VERSION` when the sources are not a clone (as in the archive of a release). The build types `Debug` and `Maintainer` define `LEMON_ENABLE_DEBUG`, which turns on the checks of `assert.h` (cf. chapter 8); furthermore, `Maintainer` runs the tests at each build. A project may also take the library in by `add_subdirectory()` or by `FetchContent`, in which case only the library is configured, whatever the options say.

For a project built with GNU make, the makefiles of the repository build the library and the tests without a configuration step:

```
make -f makefile-lib            # lib/libemon.a
make -C test check              # the tests, built and run
```

Also, `makefile-c`, which the makefile of a project includes, defines the objects of the library (`LEMONOBJ`), the external libraries it needs (`LEMONLIB`), its headers (`LEMONH`) and the `-I` directive (`LEMONINC`); it takes as input the compiler, its options and the directory of the sources.

2.3 Using the installed library

A CMake project finds the installed library by its configuration, which `find_package()` accepts both as `LEMON` and as `lemon`:

```
find_package(LEMON CONFIG REQUIRED)
target_link_libraries(main PRIVATE LEMON::lemon)
```

Note that the target `LEMON::lemon` carries the include directories, the compile definitions and, when the library was built with them, the LP and MIP backends. Also, the installation may be moved to another prefix, since the configuration only uses paths relative to it, and other build systems use pkg-config (`pkg-config --cflags --libs lemon`). Thus, the headers are included as `<lemon/...>`, and the library is in the namespace `lemon`, as in LEMON 1.3.1.

With `-DLEMON_BUILD_MODULE=ON`, the library is also built and installed as the C++20 module `lemon`, whose interface is `lemon/lemon.cppm`; a translation unit linked to `LEMON::lemon_module` may then write `import lemon;` in place of the includes. This needs CMake 3.28 or later, the Ninja generator (or one of Visual Studio), and Clang 16 or MSVC 19.34 or later. When one of these conditions does not hold, the option is turned off with a warning that names it. GCC (up to version 15, at least) builds the module, but a unit that imports it does not find the operators of the standard library that the templates use; hence, GCC is not accepted for it.

2.4 Optional backends

The LP and MIP interface of chapter 20 needs at least one external solver: GLPK (`LEMON_ENABLE_GLPK`), CPLEX (`LEMON_ENABLE_ILOG`), CLP and CBC of COIN-OR (`LEMON_ENABLE_COIN`), or SoPlex (`LEMON_ENABLE_SOPLEX`). Each option is `YES` by default, i.e., the solver is used if CMake finds it, through its own CMake configuration where it has one. Those found are named by their file names in the installed configuration, so that a project that links `LEMON::lemon` also links them. The variables `LEMON_DEFAULT_LP` and `LEMON_DEFAULT_MIP` choose which of the solvers found the type aliases `Lp` and `Mip` of `lemon/lp.h` refer to. This choice is recorded in `lemon/config.h`, which CMake writes in the build directory, and whose macros `LEMON_HAVE_LP` and `LEMON_HAVE_MIP` tell a program whether there is a solver at all; when there is none, the two aliases are not defined, while the rest of the library is unaffected.

2.5 Exercises

Exercise 2.1. Build the library with `-DCMAKE_CXX_STANDARD=20`, run `ctest --test-dir build -R manual_`, and check that the programs of this manual pass. Then install the library to a temporary prefix, and build the program of chapter 3 against the installation, once with `find_package(LEMON)` and once with pkg-config.

Exercise 2.2. Configure the library on a machine where GLPK is installed, and check in the output of CMake and in `lemon/config.h` of the build directory which solver `Lp` and `Mip` refer to; the test `lp_test` exercises them.

3. Graphs, items and maps

We are concerned in this chapter with the few ideas on which all the headers of the library are built, and which a reader needs before the data structures of Part II and the algorithms of Part III. A graph is a set of items (nodes, arcs and, if it is undirected, edges), which are handled by small value types, visited by iterators and compared with the constant `INVALID`. Data are attached to the items only through maps, i.e., objects that associate a value with each item, and the requirements on a graph, a map or a heap are written as concept classes, against which a type can be checked at compile time, and the algorithms are classes that follow (with few exceptions) one pattern of construction, execution and query. These ideas come from LEMON, whose design is described in Dezső, Jüttner and Kovács 2011, and abLEMON keeps them unchanged. Section 3.1 gives a first complete program; sections 3.2 to 3.4 describe the items, the iterators and the maps, section 3.5 the concepts, section 3.6 the algorithm classes, and section 3.7 the tolerances with which the library compares numbers that may carry a rounding error, while section 3.8 collects the exercises. In all the chapters we write $D = (V, A)$ for a digraph and $G = (V, E)$ for an undirected graph, with $n = |V|$ and $m = |A|$ (or $m = |E|$), and $\delta^+(v)$ and $\delta^-(v)$ for the sets of the arcs that leave and enter a node v.

3.1 A first program

We begin with a program that builds the digraph of Figure 3.1, with four nodes and five arcs, stores a cost for each arc, and computes the shortest path from the node `s` to the node `t` by the algorithm of Dijkstra (cf. chapter 10). As all the programs of the manual, it is built and run by the tests of the library; how to build a user program against an installed abLEMON is explained in chapter 2.

```
// The first program of the manual (section 3.1): a digraph of four nodes
// and five arcs, the cost of each arc in an ArcMap, and the shortest path
// from s to t found by Dijkstra.

#include <iostream>
#include <lemon/list_graph.h>
#include <lemon/dijkstra.h>

using namespace lemon;

int main()
{
 ListDigraph g;
 ListDigraph::Node s = g.addNode() , a = g.addNode() , b = g.addNode() ,
                   t = g.addNode();
 ListDigraph::ArcMap< int > cost( g );
 cost[ g.addArc( s , a ) ] = 2;
 cost[ g.addArc( s , b ) ] = 5;
 cost[ g.addArc( a , b ) ] = 1;
 cost[ g.addArc( a , t ) ] = 4;
 cost[ g.addArc( b , t ) ] = 2;

 Dijkstra< ListDigraph , ListDigraph::ArcMap< int > > dijkstra( g , cost );
 dijkstra.run( s );

 std::cout << "distance of t: " << dijkstra.dist( t ) << std::endl;
 std::cout << "path, backwards:";
 for( ListDigraph::Node v = t ; v != s ; v = dijkstra.predNode( v ) )
```

```
 std::cout << ' ' << g.id( v );
std::cout << ' ' << g.id( s ) << std::endl;
return( dijkstra.dist( t ) == 5 ? 0 : 1 );
}
```

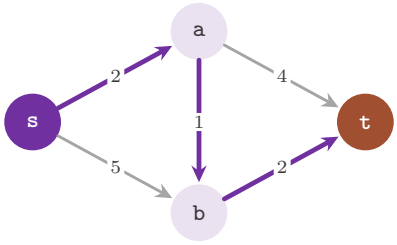


Figure 3.1. The digraph of the first program, with the cost of each arc; the arcs of the shortest path from s to t, of cost 5, are in the primary colour.

Each line of the program uses one of the ideas of this chapter. Its digraph is a `ListDigraph` (cf. chapter 4), which is empty when constructed and grows by `addNode()` and `addArc(s, t)`. Each of these functions returns a handle to the new item, i.e., a small object of type `ListDigraph::Node` or `ListDigraph::Arc` that identifies the item within its digraph (section 3.2). Since an arc has no field that a user could set, the costs are the values of a `ListDigraph::ArcMap< int >`, a map that holds one `int` for each arc of `g` and that is read and written by `cost[a]` (section 3.4). The algorithm is the class template `Dijkstra`, whose parameters are the type of the digraph and the type of the map of the lengths; it is constructed on the digraph and on the map, it runs from a source by `run(s)`, and its results are read afterwards by queries such as `dist(t)` and `predNode(v)` (section 3.6). In the last loop the path is followed backwards from `t` to the source, because `predNode(v)` gives the node that precedes `v` on the shortest path found, and the program prints the identifiers `3 2 1 0` that `g.id(v)` returns for its nodes. Finally, the program checks that the distance of `t` is 5 and returns 1 otherwise; hence, a program of the manual that stops matching the library makes the tests fail.

3.2 Items, identifiers and INVALID

A digraph has two kinds of items, of the types `Node` and `Arc` nested in its class. An undirected graph also has the type `Edge`: each edge e has two end nodes, `u(e)` and `v(e)`, and it is represented by two arcs of opposite directions, which the graph creates and erases together with the edge. A bipartite graph (`BpGraph`) is an undirected graph whose nodes are either red or blue, and whose edges join a red node to a blue one; its types `RedNode` and `BlueNode` derive from `Node`. All these types are handles. An object of type `Node` does not contain a node but identifies one, typically by an integer index, and it is "cheap" to copy, to compare and to store. We remark that a handle is meaningful only together with its graph (the same index may identify another node in another graph), and the graph is therefore passed explicitly to each function that reads the structure, as in `g.source(a)` or `g.id(v)`.

The constant INVALID. In `lemon/core.h` one finds the empty type `Invalid` and its only object, `INVALID`. Indeed, each handle and each iterator converts from `INVALID`, and comparing an item with `INVALID` reveals whether it identifies an item at all. In this way an iteration ends (section 3.3), a function such as `findArc()` of chapter 4 returns the absence of a result, and an algorithm marks a node without a predecessor (e.g., the source of a search has `predArc(s) == INVALID`). Also, the handles have the operators `==`, `!=` and `<`, the last of which is an artificial order (unrelated to the order of iteration) that only lets the handles be the keys of ordered containers.

> **abLEMON.** A default-constructed handle (`ListDigraph::Node v;`) is `INVALID` in all the graph structures and adaptors of abLEMON, while in LEMON 1.3.1 its index was left indeterminate in most of them (and the maps copied that indeterminate value when they built their default values). Yet, the concepts still say that the default constructor leaves the value undefined; hence, a generic algorithm should assign `INVALID` explicitly instead of relying on it.

Identifiers. Each item has an integer identifier, `g.id(item)`, which is unique among the items of its kind in the graph and does not change while the item exists, while `g.nodeFromId(i)`, `g.arcFromId(i)` and `g.edgeFromId(i)` give the item back, and `g.maxNodeId()`, `g.maxArcId()` and `g.maxEdgeId()` give an upper bound on the identifiers in use. Identifiers are the indices of the vectors that store the items and the values of the maps, and they are not necessarily contiguous: a `ListDigraph`, e.g., keeps the identifiers of the erased nodes

on a free list and gives them to the next nodes added. An erased node may thus become valid again, and the structures that allow erasing (the `List` ones, and the `Smart` ones through their snapshots) have the function `val id(item)`, which returns whether a handle identifies an item of the graph at present. In fact, a bipartite graph also numbers the red and the blue nodes separately. Thus, `g.id(RedNode)` is the index of a node among the red ones (bounded by `g.maxRedId()`), and it usually differs from `g.id(Node)` of the same node. Finally, the generic forms `g.maxId(Node())` and `g.fromId(i, Node())`, with a dummy item that selects the kind, serve the templates that work on any kind of item.

Edges and arcs. Each edge has an inherent orientation, from `u(e)` to `v(e)`, given by the order in which its end nodes were passed to `addEdge(u, v)`. Of the two arcs of an edge, the forward one goes from `u(e)` to `v(e)` and the backward one from `v(e)` to `u(e)`. These arcs are `g.direct(e, true)` and `g.direct(e, false)`, while `g.direct(e, w)` is the arc of e whose source is the node `w`. Also, `g.direction(a)` is `true` for a forward arc and `g.oppositeArc(a)` is the other arc of the same edge. An `Arc` converts to the `Edge` it represents (so that an edge map can be read with an arc as its key), and `g.oppositeNode(w, e)` is the end of e other than `w`. In `ListGraph`, `SmartGraph` and `StaticGraph` the edge of identifier i is represented by the arcs of identifiers $2i + 1$ (forward) and $2i$ (backward), as Figure 3.2 shows. Since the other structures number their arcs in other ways, a program should usually use `direct()` and `direction()` instead of this arithmetic. A loop, i.e., an edge with $u(e) = v(e)$, also has two arcs, each of which leaves and enters the same node.

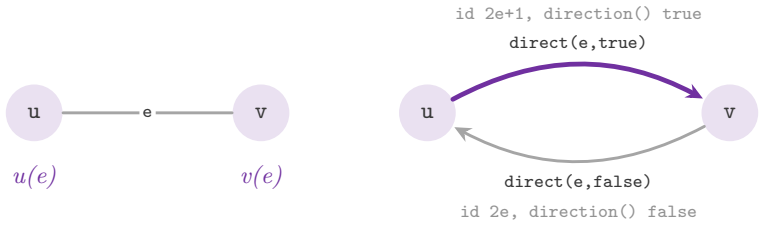


Figure 3.2. An edge e of a ListGraph, with its inherent orientation from u(e) to v(e), and the two arcs that represent it, with their identifiers.

In a bipartite graph the inherent orientation of each edge goes from its red end to its blue end, which `g.redNode(e)` and `g.blueNode(e)` return, while `g.red(v)` and `g.blue(v)` give the colour of a node and `g.asRedNode(v)` converts a `Node` to a `RedNode` (returning `INVALID` if the node is blue). Its variant `g.asRedNodeUnsafe(v)` omits the check, which makes it somewhat faster, but it is correct only on red nodes and on `INVALID`; the functions of the blue nodes are symmetric. Since an undirected graph is also a digraph (with two arcs for each edge), any algorithm written for digraphs runs on it. Furthermore, the function `undirected(g)` of `lemon/core.h` returns whether the type of `g` is undirected; its value is selected at compile time from the tag `UndirectedTag` that the undirected structures define.

3.3 Iterators and ranges

Items are visited by iterators, which are nested classes of the graph. `NodeIt`, `ArcIt` and `EdgeIt` visit all the nodes, arcs and edges; `OutArcIt` and `InArcIt` visit the arcs that leave and enter a node, `IncEdgeIt` the edges incident to a node, and, in a bipartite graph, `RedNodeIt` and `BlueNodeIt` the nodes of one colour. An iterator derives from the handle of the items it visits. Hence, we use it wherever a handle is expected, with no dereferencing, and the usual loop is

```
for( ListDigraph::OutArcIt a( g , v ) ; a != INVALID ; ++a )
 total += cost[ a ];
```

in which the iterator is constructed on the graph (and on the node, for the iterators of incident items), advanced by `++`, and compared with `INVALID`, which it becomes after the last item. Each iterator also has a constructor from the graph and an item, which continues the iteration from that item.

We call base node of an incidence iterator `it` the node on which it was constructed; then `g.baseNode(it)` is the base node and `g.runningNode(it)` the other end of the current item: for `OutArcIt` they are the source and the target of the arc, for `InArcIt` the target and the source, and for `IncEdgeIt` the node of the iterator and the opposite end of the edge. In an undirected graph `OutArcIt` and `InArcIt` visit the arcs of the incident edges in the two directions, whereas `IncEdgeIt` visits the edges. A loop is visited twice by the first two, once for each of its arcs, and also twice by `IncEdgeIt` in `ListGraph` and `SmartGraph`. Note that the order of iteration is not

specified by the concepts, and it differs between the structures (e.g., `ListDigraph` and `SmartDigraph` visit their nodes from the last added to the first); a program should not depend on it.

For the range-based `for` of C++11, we also find in the graphs functions that return a range of the same items, since a range is easier to pass to a loop or to an algorithm than a pair of iterators. These are `g.nodes()`, `g.arcs()`, `g.edges()`, `g.outArcs(v)`, `g.inArcs(v)`, `g.incEdges(v)` and, in a bipartite graph, `g.redNodes()` and `g.blueNodes()`, so that one may write `for (auto a : g.outArcs(v))`. A range holds a pair of wrapped iterators (`LemonRangeWrapper1` and `LemonRangeWrapper2` of `lemon/bits/stl_iterators.h`), which define the member types of an input iterator, a dereference that returns the item, and a postfix `++`; hence, the algorithms of the standard library that read an input range (e.g., `std::count_if` or `std::copy`) work on them too.

abLEMON. The ranges come from the repository of LEMON after its release 1.3.1, and they are distributed for the first time in abLEMON. Their iterators define the member types of an input iterator directly, instead of deriving them from `std::iterator`, which C++17 deprecated.

Writing the names of the nested types becomes tedious in a function that uses several of them, and `lemon/core.h` has macros that define them in the current scope. `DIGRAPH_TYPEDEFS(Digraph)` defines `Node`, `NodeIt`, `Arc`, `ArcIt`, `InArcIt` and `OutArcIt`, with the maps `BoolNodeMap`, `IntNodeMap`, `DoubleNodeMap`, `BoolArcMap`, `IntArcMap` and `DoubleArcMap` of that type. `GRAPH_TYPEDEFS(Graph)` adds `Edge`, `EdgeIt`, `IncEdgeIt` and three edge maps, and `BPGRAPH_TYPEDEFS(BpGraph)` adds `RedNode`, `RedNodeIt`, `BlueNode`, `BlueNodeIt` and their maps. Inside a template, where the graph type is a dependent name, one writes `TEMPLATE_DIGRAPH_TYPEDEFS(GR)` (and its undirected and bipartite counterparts), which inserts the keywords `typename` and `template` that the compiler then requires. In abLEMON the types defined by these macros are marked as possibly unused (with GCC and Clang, by the attribute `unused`); hence, a function that uses only some of them compiles without warnings. LEMON 1.3.1 turned the warning on unused local typedefs off by a pragma, for the whole compilation of the user.

3.4 Maps

A graph stores its structure and no other datum. A cost, a capacity, a flow, a label or the distance computed by an algorithm is kept in a map, i.e., in an object that has the types `Key` and `Value` and that offers `operator[]( key )` to read a value, `set( key , value )` to write one, or both (the concepts that make this precise are given in chapter 6, together with the maps of `lemon/maps.h`). Each graph structure provides the standard maps of its items, namely `NodeMap< V >`, `ArcMap< V >`, `EdgeMap< V >` and, in a bipartite graph, `RedNodeMap< V >` and `BlueNodeMap< V >`. Such a map stores a value of type `V` for each item of the graph on which it is constructed, and `map[item]` reads and writes it in $O(1)$ time. Furthermore, the map follows the changes of the graph, since the graph informs its maps of each item added or erased (cf. appendix A): a map constructed on `g` gets a slot for each item added to `g` afterwards, whose value is `V()` (not the value given to the constructor), and it drops the slot of each item erased. Of the two constructors, `NodeMap( g )` value-initializes the values and `NodeMap( g , value )` sets them all to `value`; we remark that the standard maps can be neither copied nor assigned, and the values of one map are copied into another by `mapCopy()` of chapter 6.

An algorithm reads and writes its data through maps whose types are template parameters; hence, any object with the right interface may be passed in place of a standard map, which is often (but not always) the most convenient choice. A length equal for all the arcs may be a `ConstMap`, and a cost may be computed on the fly from two other maps by an arithmetic adaptor (e.g., their difference). Also, a result may go to a map of the caller or to a `NullMap`, which discards what is written to it. Our next program shows the items of an undirected graph, the iterators, a map that follows the addition of an edge, and a function template whose graph type is checked against the concept of undirected graphs (cf. section 3.5).

```
// Items, iterators and maps (sections 3.2 to 3.4): the edges of an
// undirected graph and their two arcs, the iterators and the ranges, the
// value that a map gives to an item added after it, and a generic
// function whose graph type is checked against the Graph concept.

#include <iostream>
#include <lemon/concepts/graph.h>
#include <lemon/list_graph.h>

using namespace lemon;
```

```
// The sum of the weights of the edges incident to v, for any graph type
template< typename GR , typename WM >
typename WM::Value incidentWeight( const GR & g , const WM & w ,
                                   typename GR::Node v )
{
 checkConcept< concepts::Graph , GR >();
 typename WM::Value sum = 0;
 for( typename GR::IncEdgeIt e( g , v ) ; e != INVALID ; ++e )
  sum += w[ e ];
 return( sum );
 }

int main()
{
 int errors = 0;
 ListGraph g;
 ListGraph::Node a = g.addNode() , b = g.addNode() , c = g.addNode();
 ListGraph::Edge ab = g.addEdge( a , b ) , bc = g.addEdge( b , c ) ,
                 ca = g.addEdge( c , a );

 // the two arcs of an edge: the forward one goes from u() to v()
 ListGraph::Arc fw = g.direct( ab , true ) , bw = g.direct( ab , false );
 if( g.source( fw ) != g.u( ab ) || g.target( fw ) != g.v( ab ) )
  ++errors;
 if( g.oppositeArc( fw ) != bw || ! g.direction( fw ) || g.direction( bw ) )
  ++errors;
 if( ListGraph::Edge( bw ) != ab )
  ++errors;  // an arc converts to its edge
 if( g.direct( ab , b ) != bw )
  ++errors;  // the arc of ab leaving b
 if( g.oppositeNode( a , ab ) != b )
  ++errors;

 // a default-constructed item is INVALID in every graph of abLEMON
 ListGraph::Node none;
 if( none != INVALID )
  ++errors;

 // iterators and ranges visit the same items
 int nodes = 0 , arcs = 0 , incident = 0;
 for( ListGraph::NodeIt n( g ) ; n != INVALID ; ++n )
  ++nodes;
 for( ListGraph::Arc e : g.arcs() ) {
  ( void )e;
  ++arcs;
  }
 for( ListGraph::IncEdgeIt e( g , a ) ; e != INVALID ; ++e ) {
  if( g.oppositeNode( a , e ) == g.runningNode( e ) )
   ++incident;
  }
 if( nodes != 3 || arcs != 6 || incident != 2 )
  ++errors;

 // a map with an initial value, and an item added after the map
 ListGraph::EdgeMap< int > weight( g , 1 );
 weight[ bc ] = 5;
 weight.set( ca , 2 );
 ListGraph::Node d = g.addNode();
 ListGraph::Edge cd = g.addEdge( c , d );
 if( weight[ cd ] != 0 )
  ++errors;  // a new item gets Value(), not 1
 weight[ cd ] = 4;
 if( incidentWeight( g , weight , c ) != 11 )
  ++errors;
```

```
// ids: unique among the items of a kind, and reused after an erase
int id_d = g.id( d );
if( g.nodeFromId( id_d ) != d || id_d > g.maxNodeId() )
 ++errors;
g.erase( d );  // erases cd too
if( g.valid( d ) || g.valid( cd ) )
 ++errors;
ListGraph::Node e = g.addNode();
if( g.id( e ) != id_d )
 ++errors;  // the freed id is taken again

std::cout << "items, iterators and maps: " << errors << " errors"
          << std::endl;
return( errors == 0 ? 0 : 1 );
}
```

3.5 Concepts and their checking

A function template of the library (an algorithm, a map adaptor, a copy function) accepts any type that offers the operations it uses. These operations are written down, once for each kind of object, as a concept class in `lemon/concepts/`. A concept class is an archetype, which declares with empty bodies the types and the functions that the concept requires (e.g., `concepts::Digraph` declares `Node`, `Arc`, the iterators, `source()`, `target()`, `id()` and the maps); hence, a template that compiles with the archetype uses only what the concept guarantees. Each concept class also contains a nested template `Constraints< T >`, whose member function `constraints()` exercises those operations on a candidate type `T`. Those of the graphs (`concepts::Digraph`, `concepts::Graph` and `concepts::BpGraph`, with the components of which they are made) are described in chapter 4. Those of the maps are in chapter 6, those of the heaps in chapter 7, and the others (paths, maximum flows, minimum cost flows, minimum mean cycles) in the chapters of their families.

Checking is done by two function templates of `lemon/concept_check.h`. A call `checkConcept< Concept , T >()` takes the address of `Concept::Constraints< T >::constraints`, which forces the compiler to instantiate that function. If `T` lacks one of the operations the concept requires, the compilation fails with an error that names it; at run time the call does nothing. Its older form `function_requires< C >()` does the same for a class `C` whose member `constraints()` is to be instantiated (a form that is now rarely used). Both are empty when `NDEBUG` is defined (i.e., in an optimized build without assertions), and in a debug build they cost only compilation time. In the library, the tests check each graph structure, map and heap against its concept (e.g., `test/digraph_test.cc` calls `checkConcept< Digraph , ListDigraph >()`), and a user who writes a new structure checks it in the same way (appendix A). A generic function may also call `checkConcept` on its template parameters, as `incidentWeight()` does in the program of section 3.4, in which case a wrong argument is reported where the function is called instead of deep inside its body. Finally, the object `ignore_unused_variable_warning`, called with any arguments, silences the warnings on the variables that a function `constraints()` declares and does not use.

> **abLEMON.** `ignore_unused_variable_warning` is an object of a class with a variadic `operator()`, instead of a set of overloaded functions as in LEMON 1.3.1. Hence, an unqualified call to a function of the same name of another library is no longer made ambiguous by argument-dependent lookup on the types of `lemon`.

3.6 The algorithm classes

Most algorithms of the library are class templates that share one pattern, which we describe here on `Dijkstra` (of `lemon/dijkstra.h`, cf. chapter 10), while the chapters of Part III give the template parameters, the defaults and the queries of each class, and they say where a class departs from the pattern.

Template parameters and traits. The first parameters of the class are the types of its input; thus, `Dijkstra< GR , LEN , TR >` takes the type `GR` of the digraph (by default `ListDigraph`) and the type `LEN` of the map of the lengths (by default `GR::ArcMap< int >`). Its last parameter `TR` is a traits class, by default `DijkstraDefaultTraits< GR , LEN >`. It defines the other types used by the algorithm (the heap with its cross reference map, the maps of the predecessors, of the distances and of the processed nodes, the operations on the lengths), each with a static function that creates an object of that type; the lengths have the type `LEN::Value`, and the default heap is a `BinHeap` on them.

Named template parameters. We remark that a traits class is rarely written by hand, since each of its types can be replaced by a named template parameter. This is a nested template of the algorithm class, whose member `Create` is the class with that type replaced. For instance, `Dijkstra< GR , LEN >::SetDistMap< M >::Create` is a `Dijkstra` that writes the distances into a map of type `M`, which the caller passes by `distMap(m)`, and `SetHeap< H , CR >` replaces the heap (cf. chapter 7). Named parameters can be chained, as in `SetPredMap< P >::Create::SetDistMap< M >::Create`. A type replaced in this way is not created by the algorithm, and the caller must give the object before the first run.

Execution. A constructor takes the input objects (here the digraph and the length map) by reference, and they must outlive the algorithm, while the maps that the algorithm writes are created by it at the first run, unless the caller has given them by the setter functions `predMap()`, `distMap()`, `processedMap()` and `heap()`, each of which returns a reference to the algorithm (and the calls can thus be chained). The simplest way to execute the algorithm is `run(s)`, which computes the shortest paths from `s` to all the nodes reachable from it, while `run(s, t)` stops as soon as `t` is processed and returns whether `t` is reachable. For a finer control, `run(s)` is the sequence `init()`, `addSource(s)`, `start()` of Algorithm 3.1, and these functions may also be called one by one: `init()` resets the data of the nodes, and `addSource(s, d)` adds a source with the initial distance `d` (zero by default), and one may give several sources. Then `start()` runs the search until the heap is empty, `start(t)` stops when `t` is processed, and `start(nm)` stops at the first node `v` with `nm[v]` true that reaches the top of the heap, which it returns (or `INVALID` if there is none). Between these calls, `processNextNode()` performs one step and returns the node it processed, `nextNode()` gives the node that the next step will process, and `emptyQueue()` and `queueSize()` describe the heap.

Algorithm 3.1 (The run of an algorithm class, on Dijkstra).

```
run(s):
 1  init()                      reset the data of all the nodes
 2  addSource(s, 0)             put s in the heap with distance 0
 3  start():
 4    while not emptyQueue()
 5      v = processNextNode()   pop the node of least distance and
 6                              relax the arcs of delta+(v)
```

Queries. After a run the results are read by query functions, named after the objects of the problem: `dist(v)`, `predArc(v)`, `predNode(v)` and `path(t)` for the shortest paths, `reached(v)` and `processed(v)` for the state of a node, and `distMap()` and `predMap()` for the maps. A query is meaningful only after a run that reached the node (this is its precondition, stated in the documentation of each function), and its value remains available until the next `init()`.

Function-type interface. Some algorithms (among them `bfs()`, `dfs()` and `dijkstra()`) also have a function that returns a "wizard" object, on which the optional outputs are given by name before the run, as in

```
bool found = dijkstra( g , length ).distMap( d ).path( p ).run( s , t );
```

which writes the distances into `d` and the path from `s` to `t` into `p`. Such a wizard constructs the algorithm for the run, together with the maps that were not given, and destroys them afterwards. It is rather convenient in simple cases, whereas the class gives access to the intermediate state and to repeated runs. Our third program goes through the pattern, with a named template parameter, the separate steps of a run with one and with two sources, the queries, and the function-type interface.

```
// The pattern of the algorithm classes (section 3.6), on Dijkstra: a named
// template parameter that replaces a map of the traits, run() split into
// init(), addSource() and start(), the queries, and the function-type
// interface of the same algorithm.

#include <iostream>
#include <lemon/dijkstra.h>
#include <lemon/smart_graph.h>

using namespace lemon;

int main()
{
 int errors = 0;
```

```
SmartDigraph g;
SmartDigraph::Node s = g.addNode() , a = g.addNode() , b = g.addNode() ,
                   t = g.addNode() , u = g.addNode();
SmartDigraph::ArcMap< double > len( g );
len[ g.addArc( s , a ) ] = 2.0;
len[ g.addArc( s , b ) ] = 5.0;
len[ g.addArc( a , b ) ] = 1.0;
len[ g.addArc( a , t ) ] = 4.0;
len[ g.addArc( b , t ) ] = 2.0;
len[ g.addArc( u , s ) ] = 1.0;  // u is not reachable from s

// the distances go into a map of the caller, whose type is set by the
// named template parameter SetDistMap
typedef SmartDigraph::NodeMap< double > DistMap;
typedef Dijkstra< SmartDigraph , SmartDigraph::ArcMap< double > >::SetDistMap<
  DistMap >::Create Dij;
DistMap dist( g );
Dij dij( g , len );
dij.distMap( dist );

// run(s) is init(), addSource(s) and start(); start(t) stops at t
dij.init();
dij.addSource( s );
dij.start( t );
if( ! dij.processed( t ) || dist[ t ] != 5.0 )
 ++errors;
if( dij.predNode( t ) != b || dij.predNode( b ) != a )
 ++errors;
if( dij.reached( u ) )
 ++errors;

// two sources at different initial distances
dij.init();
dij.addSource( s , 0.0 );
dij.addSource( b , -1.0 );
dij.start();
if( dij.dist( t ) != 1.0 || dij.dist( a ) != 2.0 )
 ++errors;

// the function-type interface: the optional outputs are set by name
DistMap d2( g );
bool found = dijkstra( g , len ).distMap( d2 ).run( s , t );
if( ! found || d2[ t ] != 5.0 )
 ++errors;

std::cout << "algorithm pattern: " << errors << " errors" << std::endl;
return( errors == 0 ? 0 : 1 );
}
```

3.7 Tolerances and numeric traits

An algorithm that works on floating-point data compares numbers that may carry a rounding error, and a test such as `x == 0` or `a < b` then gives an answer that depends on the last bits of the operands. In `lemon/tolerance.h` the class template `Tolerance< T >` collects the comparisons that the algorithms need. Each of them returns `true` only if the relation holds surely, i.e., even allowing for an error up to a threshold ε, as follows:

$$\begin{array}{ll} \texttt{less(a, b)}: & a + \varepsilon < b, \\ \texttt{different(a, b)}: & \texttt{less(a, b)} \vee \texttt{less(b, a)}, \\ \texttt{positive(a)}: & \varepsilon < a, \\ \texttt{negative(a)}: & a < -\varepsilon, \\ \texttt{nonZero(a)}: & \texttt{positive(a)} \vee \texttt{negative(a)}, \end{array} \tag{3.1}$$

while `zero()` returns 0. Its general version is meant for exact types (the integers, or a rational type), and its

functions are static and compare exactly, i.e., as in (3.1) with $\varepsilon = 0$. Instead, the specializations for `float`, `double` and `long double` hold an ε in each object. It is set by the constructor (as in `Tolerance< double > tol( 1e-8 )`) or by `tol.epsilon( e )`, and read by `tol.epsilon()`; its default is 10^{-4}, 10^{-10} and 10^{-14}, respectively (the last one being the `double` value 10^{-14}, widened). The static function `defaultEpsilon( e )` changes the default for the objects constructed afterwards, everywhere in the program. Any other type gets the general template, and a user who works with another inexact type writes a specialization with the same functions.

We stress that the threshold is absolute, i.e., ε in (3.1) does not scale with the operands; hence, `Tolerance < double >` finds 10^{20} and $10^{20} + 10^5$ surely different (although their relative gap is 10^{-15}), and 10^{-12} and $2 \cdot 10^{-11}$ equal (although one is 20 times the other). Its default suits, therefore, data of "moderate" magnitude, and the algorithms that take a `Tolerance` (the maximum flow and minimum cut algorithms, among others) have a function `tolerance()` by which the caller sets one that fits the scale of the data. On the other hand, the minimum cost flow algorithms keep tolerances of a different kind, which are relative to the data unless the caller sets them (cf. chapter 13). A short program shows both behaviors of the threshold, the defaults, and the function `countNodes()` discussed at the end of the section.

```
// Tolerance (section 3.7): the exact comparisons of the general template,
// the absolute epsilon of the floating-point specializations, and the
// counting functions of core.h that read the tags of a graph.

#include <iostream>
#include <lemon/core.h>
#include <lemon/list_graph.h>
#include <lemon/smart_graph.h>
#include <lemon/tolerance.h>

using namespace lemon;

int main()
{
 int errors = 0;

 // the general template compares exactly, with static functions
 if( Tolerance< int >::less( 2 , 2 ) || ! Tolerance< int >::positive( 1 ) )
  ++errors;

 // double: epsilon 1e-10 by default, an absolute threshold
 Tolerance< double > tol;
 if( tol.epsilon() != 1e-10 )
  ++errors;
 double x = 0.1 + 0.2;  // 0.30000000000000004 in binary
 if( x == 0.3 )
  ++errors;
 if( tol.different( x , 0.3 ) || tol.nonZero( x - 0.3 ) )
  ++errors;
 if( ! tol.less( 0.3 , 0.3 + 1e-9 ) )
  ++errors;
 // the threshold does not scale with the operands
 if( ! tol.different( 1e20 , 1e20 + 1e5 ) )
  ++errors;  // relative gap 1e-15
 if( tol.different( 1e-12 , 2e-11 ) )
  ++errors;  // one is 20 times the other

 // a looser epsilon for one object, the default for the others
 Tolerance< double > loose( 1e-3 );
 if( ! loose.negative( -0.01 ) || loose.negative( -0.0001 ) )
  ++errors;
 if( Tolerance< float >::defaultEpsilon() != 1e-4f )
  ++errors;
 // the default of long double is the double 1e-14, widened
 if( Tolerance< long double >::defaultEpsilon() !=
     static_cast< long double >( 1e-14 ) )
  ++errors;
```

```
// countNodes() is O(1) on SmartDigraph (it has NodeNumTag) and a scan
// on ListDigraph; both give the same number
SmartDigraph sg;
ListDigraph lg;
for( int i = 0 ; i < 5 ; ++i ) {
 sg.addNode();
 lg.addNode();
 }
if( countNodes( sg ) != 5 || countNodes( lg ) != 5 )
 ++errors;
if( sg.nodeNum() != 5 )
 ++errors;

std::cout << "tolerances: " << errors << " errors" << std::endl;
return( errors == 0 ? 0 : 1 );
}
```

Numeric constants. The "small" header `lemon/math.h` includes `<cmath>` and defines, in the namespace `lemon`, the constants `E`, `LOG2E`, `LOG10E`, `LN2`, `LN10`, `PI`, `PI_2`, `PI_4`, `SQRT2` and `SQRT1_2`, as `long double` values written with 35 significant digits. It also defines `isNaN(v)`, which tests `v != v`, and `round(r)`, which rounds to the nearest integer with the halves away from zero. In abLEMON the constants are `inline constexpr` from C++17 on (and `const` before), so that the C++20 module `lemon` can export them.

Traits of graphs and maps. A user rarely includes `lemon/bits/traits.h` directly; it holds the templates by which the library selects an implementation at compile time from the properties of a type. Thus, `ItemSetTraits< GR , Item >` gives, for a graph type and a kind of item, the iterator (`ItemIt`), the map template (`Map< V >::Type`) and the notifier of the items, and a template can be written once for nodes, arcs and edges. Also, `MapTraits< M >` gives whether a map is a reference map and which types its `operator[]` returns (chapter 6). Other templates detect the tags that a structure may define: `NodeNumTag`, `ArcNumTag` and `EdgeNumTag` mean that the structure counts its items in $O(1)$ time, and `FindArcTag` and `FindEdgeTag` that it finds the arc between two nodes directly; also, `BuildTag` means that it is built in one call from another graph, and `UndirectedTag` that it is undirected. Typically, a counting function of `lemon/core.h` is selected by these tags; indeed, `countNodes(g)` calls `g.nodeNum()` on a structure with `NodeNumTag`, in $O(1)$ time, and scans the nodes with `NodeIt` otherwise, in $O(n)$ time, and the other utilities of chapter 4 (`countArcs()`, `findArc()`, the copies) are selected in the same way, by `enable_if` on the tag.

3.8 Exercises

Exercise 3.1. Modify the first program so that it prints the nodes of the shortest path from `s` to `t` in the order in which the path visits them, by `dijkstra.path(t)` and the path utilities of chapter 8 instead of the loop on `predNode()`. (The program `manual/examples/03-first-program.cc` gives the distance against which to check the result.)

Exercise 3.2. Write a function template that, for any undirected graph `g` and edge map `w`, returns the edge of largest weight incident to a node (or `INVALID` if the node is isolated), and check its graph type by `checkConcept`; call it on a `ListGraph` and on a `SmartGraph`. (The function `incidentWeight()` of `manual/examples/03-items-maps.cc` is a model.)

Exercise 3.3. Build a `ListGraph` with a loop and two parallel edges, and count, for one of their end nodes, the items visited by `OutArcIt`, `InArcIt` and `IncEdgeIt`. Explain the counts by the representation of Figure 3.2. (The test `test/graph_test.cc` checks the degrees of such graphs.)

Exercise 3.4. Find three `double` values a, b and c such that `Tolerance< double >` finds a equal to b and b equal to c, but a different from c; hence, the equality of (3.1) is not transitive. Discuss what this implies for an algorithm that groups values by equality. (The program `manual/examples/03-tolerance.cc` checks the threshold.)

Exercise 3.5. Replace the length map of the program of section 3.6 by a map of type `ConstMap< SmartDigraph::Arc , double >` with value 1 (cf. chapter 6), so that `Dijkstra` computes the number of arcs of the shortest paths, and check the result against a breadth-first search (chapter 9). (The test `test/bfs_test.cc` shows the breadth-first search.)

4. Graph structures

We are concerned in this chapter with the classes that store a graph, and with the utilities of `lemon/core.h` that count, find and copy its items. All of them implement the concepts of chapter 3; hence, an algorithm written for a digraph runs on any of them. They differ in what they allow after the construction (adding items, erasing them, nothing at all), in the time of their operations and in the memory they take. One usually chooses between a list-based structure, which allows any modification, a vector-based one, which only grows, a static one, which is built once and is the most compact, and an implicit one, which stores no item and computes the structure of a full, grid or hypercube graph from the indices. Section 4.1 describes the concepts and compares the structures in Table 4.1, and sections 4.2 to 4.6 give the structures family by family. Section 4.7 deals with the arc and edge sets that share the nodes of another graph, section 4.8 with the functions and classes that count the items and find the arcs between two nodes, and section 4.9 with the copies of a graph into another structure; the exercises are in section 4.10. Their representations follow the textbook ones (cf. Ahuja, Magnanti and Orlin 1993, §2.3 and Cormen et al. 2001, §22.1): adjacency lists, linked in the list-based and vector-based structures, and stored contiguously (the "forward star" representation) in the static ones.

4.1 The graph concepts

In `lemon/concepts/digraph.h` the class `concepts::Digraph` declares the items `Node` and `Arc` and the iterators `NodeIt`, `ArcIt`, `OutArcIt` and `InArcIt`, with their ranges. It also declares the functions `source()`, `target()`, `id()`, `nodeFromId()`, `arcFromId()`, `maxNodeId()`, `maxArcId()`, `oppositeNode()`, `baseNode()` and `runningNode()`, and the map templates `NodeMap< T >` and `ArcMap< T >`. Note that a digraph cannot be copied or assigned, since its copy constructor and assignment are private; it is duplicated by the copies of section 4.9. The concept `concepts::Graph`, in `lemon/concepts/graph.h`, adds the item `Edge`, the iterators `EdgeIt` and `IncEdgeIt`, the functions `u()`, `v()`, `direction()`, `direct()`, `oppositeArc()`, `edgeFromId()` and `maxEdgeId()`, the template `EdgeMap< T >`, and the tag `UndirectedTag`; an undirected graph conforms to `concepts::Digraph` too, with two arcs for each edge (cf. chapter 3). In `lemon/concepts/bpgraph.h`, `concepts::BpGraph` adds the items `RedNode` and `BlueNode`, the iterators `RedNodeIt` and `BlueNodeIt`, the maps `RedNodeMap< T >` and `BlueNodeMap< T >`, and the functions `red()`, `blue()`, `asRedNode()`, `asBlueNode()`, their unsafe variants, `redNode(e)`, `blueNode(e)`, `maxRedId()` and `maxBlueId()`. All three concepts describe only the queries; adding, erasing and clearing are not part of them, since some structures do not offer them.

These concepts are assembled from components, the classes of `lemon/concepts/graph_components.h`, which a structure or an algorithm may also require separately. Each component exists in three versions, for digraphs, graphs and bipartite graphs, and it takes as a template parameter the component on which it is built (by default the base one):

- `GraphItem< sel >`, the concept of a handle (default and copy construction, assignment, conversion from `INVALID`, `==`, `!=`, `<`), with a character `sel` that keeps the types of nodes, arcs and edges distinct.
- `BaseDigraphComponent`, `BaseGraphComponent` and `BaseBpGraphComponent`, the items with `source()` and `target()` (and `u()`, `v()`, `direction()`, `direct()`, `oppositeArc()`, or the colours).
- `IDableDigraphComponent` (and its graph and bipartite versions), the identifiers and the functions that invert them, which must be unique and immutable.
- `IterableDigraphComponent`, the iterators, built on `GraphItemIt` (the concept of `NodeIt`, `ArcIt` and `EdgeIt`) and `GraphIncIt` (that of `OutArcIt`, `InArcIt` and `IncEdgeIt`), with the functions `first()`, `next()`, `firstOut()` and the others of the base classes.
- `AlterableDigraphComponent`, the alteration notifiers, by which the structure informs its maps and other observers of each change (cf. appendix A).
- `MappableDigraphComponent`, the map templates, each of which must conform to `GraphMap`, i.e., be a

reference map constructed on the graph, with or without an initial value.
- `ExtendableDigraphComponent`, `ErasableDigraphComponent` and `ClearableDigraphComponent`, the functions `addNode()` and `addArc()` (or `addEdge()`, `addRedNode()`, `addBlueNode()`), `erase()`, and `clear()`, each of which requires the alterable component.

Thus, `concepts::Digraph` is made of the base, idable, iterable and mappable components. Besides it, the tests of the library check the components that each structure offers in addition: e.g., `test/digraph_test.cc` checks `ListDigraph` against the extendable, erasable, clearable and alterable components, and `StaticDigraph` against the clearable one only. Table 4.1 lists the structures of the chapter with what they offer and what they cost. Its memory column gives the number of `int` values that each node and each arc take in the arrays of the structure (from the private structs `NodeT` and `ArcT` of the base classes of `lemon/list_graph.h`, `lemon/smart_graph.h` and `lemon/edge_set.h`, and from the arrays of the static structures), and $d^+(v)$ and $d(v)$ are the out-degree and the degree of v.

Table 4.1. The graph structures.

Structure	Concept	Changes	Counting	Memory per node / arc	`findArc(u, v)`
`ListDigraph`	Digraph	add, erase, others	$O(n)$, $O(m)$	4 / 6	$O(d^+(u))$
`ListGraph`	Graph	add, erase, others	$O(n)$, $O(m)$	3 / 3	$O(d(u))$
`ListBpGraph`	BpGraph	add, erase, others	$O(n)$, $O(m)$	6 and a `bool` / 3	$O(d(u))$
`SmartDigraph`	Digraph	add, snapshot	$O(1)$	2 / 4	$O(d^+(u))$
`SmartGraph`	Graph	add, snapshot	$O(1)$	1 / 2	$O(d(u))$
`SmartBpGraph`	BpGraph	add, snapshot	$O(1)$	3 and a `bool` / 2	$O(d(u))$
`StaticDigraph`	Digraph	build	$O(1)$	2 / 4	$O(d^+(u))$
`StaticGraph`	Graph	build	$O(1)$	1 / 3	$O(d(u))$
`CompactDigraph`	Digraph	build	$O(1)$	1 / 1	$O(d^+(u))$
`FullDigraph`, `FullGraph`, `FullBpGraph`	Digraph, Graph, BpGraph	resize	$O(1)$	none	$O(1)$
`GridGraph`, `HypercubeGraph`	Graph	resize	$O(1)$	none	$O(1)$
`ListArcSet`, `ListEdgeSet`	Digraph, Graph	add, erase	$O(n)$, $O(m)$	2 / 6, 1 / 3	$O(d^+(u))$
`SmartArcSet`, `SmartEdgeSet`	Digraph, Graph	add	$O(n)$, $O(m)$	2 / 4, 1 / 2	$O(d^+(u))$

In the table, "others" are the changes of the end nodes, the contraction and the splitting of section 4.2, and "snapshot" means that the items added after a snapshot can be removed (section 4.3). Also, "build" means that the structure is built in one call and can be rebuilt (section 4.4), and "resize" that the implicit structure changes its size, which clears its maps. Counting is that of `countNodes()` and `countArcs()` (section 4.8), and the memory of an undirected structure is given per arc, an edge having two. For the arc and edge sets, the memory per node is that of the node map in which they keep the heads of their lists (the nodes belong to the underlying graph), and their arcs store the end nodes as handles of the underlying graph, which are counted here as one `int` each.

4.2 List graphs

In `lemon/list_graph.h` we find `ListDigraph`, `ListGraph` and `ListBpGraph`, the structures that allow any modification. A `ListDigraph` keeps its nodes and its arcs in two `std::vector`s. The nodes form a doubly linked list, and so do the arcs out of each node and the arcs into each node, through indices stored in the arcs. An erased node or arc is unlinked and put on a free list, whose items are reused by the next additions; hence, the identifiers are the positions in the vectors, bounded by the largest number of items that existed at the same time. Adding a node or an arc costs $O(1)$ (amortized, because of the growth of the vectors) and erasing an arc $O(1)$, while erasing a node costs $O(d^+(v) + d^-(v))$, since the arcs at the node are erased first. The nodes are visited from the last added to the first, and `ArcIt` follows the lists of the arcs out of each node. Since the structure keeps no count of its items, `countNodes()` and `countArcs()` take linear time.

In `ListGraph` each edge is stored as two arcs with consecutive indices (cf. Figure 3.2), and each node keeps one doubly linked list of the arcs that leave it. The arcs that enter a node are obtained from this list as the opposites of those that leave it. The class `ListBpGraph` adds to the nodes of `ListGraph` a colour, an index within the colour (`id(RedNode)`, `id(BlueNode)`), and a doubly linked list for each colour. Besides the functions of the concepts, the three classes have the following ones, which keep the iterators valid unless it is said otherwise.

- `addNode()` and `addArc(s, t)` (`addEdge(u, v)` in `ListGraph`, and `addRedNode()`, `addBlueNode()` and `addEdge(r, b)` in `ListBpGraph`), `erase(item)` for a node, an arc or an edge (which invalidates the iterators on the erased items), and `clear()`.
- `valid(item)`, which returns whether a handle identifies an item at present; an erased item may become valid again after an addition that reuses its index.
- `changeTarget(a, n)`, `changeSource(a, n)` and `reverseArc(a)` in `ListDigraph`, which move the ends of an arc and keep its identifier and its values in the maps. In the same way, `ListGraph` has `changeU(e, n)` and `changeV(e, n)`, and `ListBpGraph` has `changeRed(e, n)` and `changeBlue(e, n)`. The iterators along the lists that the arc leaves are invalidated (e.g., an `InArcIt` on `a` after `changeTarget()`).
- `contract(u, v, r = true)` in `ListDigraph` and `ListGraph`, which moves the arcs (or edges) of `v` to `u` by the changes above and erases `v`. If `r` is `true`, the arcs between `u` and `v` are erased instead of becoming loops, while the loops that `u` already had are kept.
- `split(n, connect = true)` in `ListDigraph`, which adds a node, moves to it the arcs out of `n`, and, if `connect` is `true`, adds an arc from `n` to it. Also, `split(a)` adds a node `w`, makes it the target of `a`, and adds an arc from `w` to the old target of `a`; both return the new node.
- `reserveNode(n)` and `reserveArc(m)` (`reserveEdge(m)` in the undirected classes), which reserve the memory of the vectors before a large graph is built.

Snapshots. A nested class `Snapshot` saves the state of the graph and restores it later, removing the items added in between. A snapshot is made by the constructor `Snapshot(g)` or by `save(g)` (which replaces a previous one), and it is used by `restore()`. It observes the notifiers of the graph (cf. appendix A) and records the items added. When one of them is erased, the snapshot removes it from its record, at a cost linear in the number of items recorded. An item that existed before the snapshot, instead, cannot be erased without consequences: its erasure detaches the snapshot from the graph, after which `valid()` returns `false`. Also `restore()` invalidates the snapshot, and a state cannot be restored twice unless `save()` is called again. Furthermore, the other modifications (`changeTarget()`, `reverseArc()`, `contract()`, `split()` and so on) are not recorded, and they must not be used together with a snapshot. Our next program goes through these functions on a `ListDigraph`.

```
// ListDigraph (section 4.2): the modifications that only the list-based
// graphs offer (erase, changeTarget, reverseArc, contract, split), and a
// Snapshot that removes what was added after it.

#include <iostream>
#include <lemon/list_graph.h>

using namespace lemon;

int main()
{
 int errors = 0;
 ListDigraph g;
 ListDigraph::Node a = g.addNode() , b = g.addNode() , c = g.addNode() ,
                   d = g.addNode();
 ListDigraph::Arc ab = g.addArc( a , b ) , bc = g.addArc( b , c ) ,
                  cd = g.addArc( c , d ) , da = g.addArc( d , a ) ,
                  ac = g.addArc( a , c );
 ListDigraph::ArcMap< int > cost( g , 1 );
 cost[ ac ] = 7;

 // reverseArc() and changeTarget() keep the arc, with its id and its value
 g.reverseArc( ac );  // now c -> a
 if( g.source( ac ) != c || g.target( ac ) != a || cost[ ac ] != 7 )
  ++errors;
 g.changeTarget( ac , b );  // now c -> b
 if( countInArcs( g , b ) != 2 )
  ++errors;
```

```
// contract(a, b): b is erased, its arcs are joined to a, and the loop
// a -> a made of ab is removed (r = true by default)
g.contract( a , b );
if( g.valid( b ) || g.valid( ab ) )
 ++errors;
if( g.source( bc ) != a || g.target( ac ) != a )
 ++errors;
if( countNodes( g ) != 3 || countArcs( g ) != 4 )
 ++errors;

// split(c): a new node takes the arcs out of c, joined to c by a new arc
ListDigraph::Node c2 = g.split( c );
if( g.source( cd ) != c2 || g.source( ac ) != c2 )
 ++errors;
if( countOutArcs( g , c ) != 1 || findArc( g , c , c2 ) == INVALID )
 ++errors;

// a snapshot: what is added after save() goes away at restore()
ListDigraph::Snapshot snap( g );
ListDigraph::Node e = g.addNode();
g.addArc( e , a );
g.addArc( d , e );
if( countNodes( g ) != 5 || countArcs( g ) != 7 )
 ++errors;
snap.restore();
if( countNodes( g ) != 4 || countArcs( g ) != 5 || g.valid( e ) )
 ++errors;
if( snap.valid() )
 ++errors;  // restore() invalidates the snapshot

// erasing a node erases its arcs
g.erase( d );
if( g.valid( cd ) || g.valid( da ) || countArcs( g ) != 3 )
 ++errors;

std::cout << "ListDigraph: " << errors << " errors" << std::endl;
return( errors == 0 ? 0 : 1 );
}
```

4.3 Smart graphs

In `lemon/smart_graph.h` we find `SmartDigraph`, `SmartGraph` and `SmartBpGraph`, which store their items in vectors and link the arcs of each node in singly linked lists. They cannot erase an item, and in exchange they are typically faster to build and somewhat smaller than the list-based ones (Table 4.1). Their identifiers are the positions in the vectors, i.e., the nodes and the arcs are numbered from 0 in the order of addition. The items are counted in $O(1)$ time, since the structures have the tags `NodeNumTag`, `ArcNumTag` and `EdgeNumTag` and the functions `nodeNum()`, `arcNum()` and `edgeNum()` (and `redNum()`, `blueNum()` in `SmartBpGraph`). Besides the functions of the concepts, they have `addNode()`, `addArc()` (or `addEdge()`, `addRedNode()` and `addBlueNode()`), `valid()`, `clear()`, `reserveNode()` and `reserveArc()` (or `reserveEdge()`). Also, `SmartDigraph` has `split(n , connect = true)`, as in `ListDigraph`.

Snapshots. Items can be removed from a smart graph only by its `Snapshot`, which records the numbers of nodes and arcs at `save(g)` (or at the constructor `Snapshot(g)`). Its `restore()` erases the items with larger indices, the last first, and notifies the maps of each of them; saving takes $O(1)$ time, and restoring time linear in the number of items removed. Unlike the snapshot of a list graph, this one does not observe the graph, and its validity is not checked. Hence, a `restore()` after a `split()` of `SmartDigraph`, or after a restore to an older snapshot, leaves the graph in an inconsistent state, as the header warns.

4.4 Static graphs

The headers `lemon/static_graph.h` and `lemon/compact_graph.h` define `StaticDigraph`, `StaticGraph` and `CompactDigraph`, which are built in one call and cannot be modified afterwards (but can be built again). They store the arcs out of each node one after the other, in arrays with no pointer to follow. Hence, they take less memory than the other structures, and they scan the arcs out of a node contiguously, which is often (but not always) faster in practice. Their nodes and arcs are numbered from 0, and `node(i)`, `arc(i)` (`edge(i)` in `StaticGraph`) and `index(item)` convert between items and indices, the index of an item being its identifier. Note that each class has `nodeNum()` and `arcNum()` (and `edgeNum()`), in $O(1)$ time, `clear()`, and two functions `build()`.

- `build(g, nodeRef, arcRef)` copies any digraph `g` (any graph, for `StaticGraph`). It writes into the map `nodeRef` the image of each node, which it also reads, and into `arcRef` (or `edgeRef`) the image of each arc, for which a `NullMap` may be given if it is not needed (one usually needs only the node references). The copies of section 4.9 call it, by the tag `BuildTag`.
- `build(n, begin, end)` builds the structure with `n` nodes from a range of `std::pair< int , int >`, the k-th pair (i, j) giving the arc (or edge) of index $k-1$ from the node of index i to that of index j.

Rebuilding a structure, or clearing it, clears the values of its maps.

StaticDigraph. Its arrays are those of Figure 4.1: `node_first_out`, of $n+1$ entries, gives the first arc out of each node (the arcs out of node v being those from `node_first_out[v]` to `node_first_out[v+1]-1`), and `arc_target` the target of each arc. Two more arrays, `node_first_in` and `arc_next_in`, link the arcs into each node in a list, and `arc_source` and `arc_next_out` complete the four `int` values of each arc. Its `OutArcIt` is specialized: it walks the indices from the first arc of the node to the last, with no array access per step. When the structure is built from a list, the pairs must come in nondecreasing order of their first element (a `LEMON_ASSERT` checks it, unless the assertions are disabled, cf. chapter 8), and the arcs keep the order of the list; when built from a digraph, the nodes take the indices in the order of the `NodeIt` of `g`, and the arcs out of each node are sorted (by the private comparator `ArcLess` of the class) by the index of their target. Building from a list takes $O(n+m)$ time, and from a digraph $O(n+\sum_v d^+(v)\log d^+(v))$ because of the sorting.

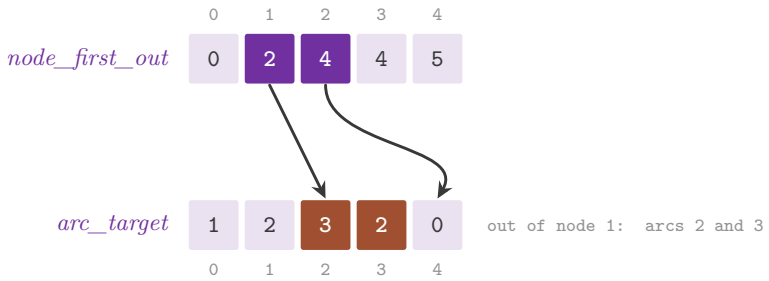


Figure 4.1. The arrays of the StaticDigraph built from the arcs (0,1), (0,2), (1,3), (1,2), (3,0): the arcs out of node 1 are those of indices 2 and 3, whose targets are 3 and 2.

StaticGraph. This undirected counterpart of `StaticDigraph` represents the edge of index i by the arcs $2i+1$, from u to v, and $2i$, from v to u, as `SmartGraph` does. Since the arcs out of each node are stored one after the other, and the arcs into a node are the opposites of those out of it, one array of positions serves both directions. It takes three `int` values per arc (its target, its position, and the arc at each position) and one per node, the first position of its arcs. Here the pairs of `build(n, begin, end)` may come in any order; the edge $k-1$ has u equal to the first element of the k-th pair and v to the second, and the arcs are placed by a counting sort on their sources (Algorithm 4.1), in $O(n+m)$ time. A copy from a graph numbers the nodes and the edges in the order of their iterators and then builds from the list of the end nodes, also in linear time.

Algorithm 4.1 (Building a StaticGraph from a list of edges).

```
build(n, ends[0..m-1]):
 1  for each edge i:  target[2i] = u_i ; target[2i+1] = v_i
 2  first[0..n] = 0
 3  for each edge i:  first[u_i + 1]++ ; first[v_i + 1]++
 4  for v = 0 .. n-1: first[v+1] += first[v]      (prefix sums)
 5  fill[v] = first[v] for each node v
 6  for each arc a = 0 .. 2m-1:
 7    p = fill[source(a)]++ ,  source(a) = target[a xor 1]
```

```
8    out[p] = a ;  pos[a] = p
```

Here the arcs out of node v are `out[first[v]]` to `out[first[v+1]-1]`, `nextOut(a)` is `out[pos[a]+1]` while it stays below `first[v+1]`, and `nextIn(a)` is the opposite of `nextOut` on the opposite arc (`lemon/static_graph.h`, `StaticGraphBase::build()`).

abLEMON. `StaticGraph` is new in abLEMON (version 1.5.0): LEMON 1.3.1 had only the static digraph. Also, the copy constructor and the assignment of `StaticDigraph` are private, as those of the other structures, while in LEMON 1.3.1 the implicit ones copied the pointers to its arrays, which were then deleted twice.

CompactDigraph. It is a `StaticDigraph` without the arrays of the incoming arcs and of the sources: it stores only `node_first_out` and `arc_target`, i.e., one `int` per node and one per arc, and an `Arc` handle carries its source together with its index. It is built as a `StaticDigraph` is (also sorting the arcs by its own `ArcLess` when it copies a digraph). Its `InArcIt` exists only to conform to `concepts::Digraph`, since with no index of the incoming arcs it scans all the arcs: visiting the arcs into one node takes $O(m)$ time, and into all the nodes $O(nm)$. Also, `arc(i)` (which is not static here) finds the source of the arc by a binary search on `node_first_out`, in $O(\log n)$ time. An algorithm that visits the incoming arcs (e.g., one run on the reverse digraph of chapter 5) should use `StaticDigraph` instead.

abLEMON. `CompactDigraph` comes from the repository of LEMON after its release 1.3.1, and it is distributed for the first time in abLEMON, whose documentation also gives the cost of its `InArcIt`.

Our next program builds the three static structures from lists, copies a `ListDigraph` into a `StaticDigraph` with one of its maps, and checks the implicit graphs of the next two sections.

```
// The static and the implicit graphs (sections 4.4 to 4.6): StaticDigraph
// and CompactDigraph built from a list of arcs sorted by source, StaticGraph
// from a list of edges in any order, a ListDigraph copied into a
// StaticDigraph with one of its maps, and the full, grid and hypercube
// graphs with the counts and the indices they compute.

#include <iostream>
#include <utility>
#include <vector>
#include <lemon/compact_graph.h>
#include <lemon/full_graph.h>
#include <lemon/grid_graph.h>
#include <lemon/hypercube_graph.h>
#include <lemon/list_graph.h>
#include <lemon/static_graph.h>

using namespace lemon;

int main()
{
 int errors = 0;

 // arcs sorted by source: arc k joins the k-th pair
 std::vector< std::pair< int , int > > arcs;
 arcs.push_back( std::make_pair( 0 , 1 ) );
 arcs.push_back( std::make_pair( 0 , 2 ) );
 arcs.push_back( std::make_pair( 1 , 3 ) );
 arcs.push_back( std::make_pair( 1 , 2 ) );
 arcs.push_back( std::make_pair( 3 , 0 ) );
 StaticDigraph sd;
 sd.build( 4 , arcs.begin() , arcs.end() );
 if( sd.nodeNum() != 4 || sd.arcNum() != 5 )
  ++errors;
 if( sd.source( sd.arc( 3 ) ) != sd.node( 1 ) ||
     sd.target( sd.arc( 3 ) ) != sd.node( 2 ) )
  ++errors;
 CompactDigraph cd;
 cd.build( 4 , arcs.begin() , arcs.end() );
 if( countInArcs( cd , cd.node( 2 ) ) != 2 )
  ++errors;  // a scan of the arcs
```

```
// edges in any order: edge k joins the k-th pair, u() first
std::vector< std::pair< int , int > > edges;
edges.push_back( std::make_pair( 2 , 0 ) );
edges.push_back( std::make_pair( 0 , 1 ) );
edges.push_back( std::make_pair( 1 , 2 ) );
StaticGraph sg;
sg.build( 3 , edges.begin() , edges.end() );
if( sg.u( sg.edge( 0 ) ) != sg.node( 2 ) ||
    sg.v( sg.edge( 0 ) ) != sg.node( 0 ) )
 ++errors;
if( countIncEdges( sg , sg.node( 0 ) ) != 2 || sg.arcNum() != 6 )
 ++errors;

// a ListDigraph copied into a StaticDigraph, with a map and the node
// references; the arcs out of each node come sorted by target
ListDigraph lg;
ListDigraph::Node x = lg.addNode() , y = lg.addNode() , z = lg.addNode();
ListDigraph::ArcMap< int > cap( lg );
cap[ lg.addArc( x , z ) ] = 3;
cap[ lg.addArc( x , y ) ] = 4;
cap[ lg.addArc( y , z ) ] = 5;
StaticDigraph st;
StaticDigraph::ArcMap< int > scap( st );
ListDigraph::NodeMap< StaticDigraph::Node > ref( lg );
digraphCopy( lg , st ).nodeRef( ref ).arcMap( cap , scap ).run();
int sum = 0 , prev = -1;
for( StaticDigraph::OutArcIt a( st , ref[ x ] ) ; a != INVALID ; ++a ) {
 sum += scap[ a ];
 if( st.index( st.target( a ) ) < prev )
  ++errors;
 prev = st.index( st.target( a ) );
 }
if( sum != 7 || st.arcNum() != 3 )
 ++errors;

// the implicit graphs: everything is computed from the indices
FullDigraph fd( 4 );
FullGraph fg( 4 );
if( fd.arcNum() != 16 || fg.edgeNum() != 6 || fg.arcNum() != 12 )
 ++errors;
if( fg.edge( fg( 1 ) , fg( 1 ) ) != INVALID )
 ++errors;  // no loops in FullGraph
if( fd.source( fd.arc( fd( 2 ) , fd( 3 ) ) ) != fd( 2 ) )
 ++errors;
GridGraph grid( 4 , 3 );  // 4 columns, 3 rows
if( grid.nodeNum() != 12 || grid.edgeNum() != 2 * 12 - 4 - 3 )
 ++errors;
if( grid.target( grid.right( grid( 1 , 2 ) ) ) != grid( 2 , 2 ) )
 ++errors;
if( grid.up( grid( 1 , 2 ) ) != INVALID )
 ++errors;  // the top row
if( grid.col( grid( 3 , 1 ) ) != 3 || grid.row( grid( 3 , 1 ) ) != 1 )
 ++errors;
HypercubeGraph cube( 3 );
if( cube.nodeNum() != 8 || cube.edgeNum() != 3 * 4 )
 ++errors;
HypercubeGraph::Edge e = findEdge( cube , cube( 5 ) ,
                                   cube( 7 ) );  // 101, 111
if( e == INVALID || cube.dimension( e ) != 1 )
 ++errors;

std::cout << "static and implicit graphs: " << errors << " errors"
          << std::endl;
return( errors == 0 ? 0 : 1 );
}
```

4.5 Full graphs

In `lemon/full_graph.h` we find `FullDigraph`, `FullGraph` and `FullBpGraph`, the complete graphs, which store only their sizes and compute all the rest from the indices; they take $O(1)$ memory, and each of their functions takes $O(1)$ time. `FullDigraph(n)` has the nodes $0, \ldots, n-1$ and an arc from each node to each node, loops included, i.e., n^2 arcs, the arc from s to t having the identifier $ns+t$. `FullGraph(n)` has an edge between each pair of distinct nodes, i.e., $n(n-1)/2$ edges and no loop, and `FullBpGraph(r, b)` has r red nodes, b blue ones, and the rb edges between them, while the default constructors give the empty graphs. Nodes are given by their index, as `g(i)` (or `g.redNode(i)` and `g.blueNode(i)` in `FullBpGraph`), and `index(v)` is the inverse; `g.arc(s, t)` and `g.edge(u, v)` give the arc and the edge between two nodes, which are `INVALID` for $u = v$ in `FullGraph`, and `findArc()` and `findEdge()` of section 4.8 use them, since the classes have the tags `FindArcTag` and `FindEdgeTag`. Finally, `resize(n)` (or `resize(r, b)`) changes the number of nodes; it notifies the maps that all the items were erased and then that the new ones were built, and hence the values of the maps are lost.

4.6 Grid and hypercube graphs

The headers `lemon/grid_graph.h` and `lemon/hypercube_graph.h` define two more implicit undirected graphs, with $O(1)$ memory, constant time counting and the tags `FindArcTag` and `FindEdgeTag`.

GridGraph. `GridGraph(w, h)` has wh nodes, the node (i, j) in column $i \in \{0, \ldots, w-1\}$ and row $j \in \{0, \ldots, h-1\}$ having the index $i + jw$, and an edge between two nodes whose positions differ by 1 in one coordinate; there are $2wh - w - h$ edges, the vertical ones first (Figure 4.2), with the origin at the bottom left corner. The node at a position is `g(i, j)` (whose range is checked by a `LEMON_DEBUG`, i.e., only when `LEMON_ENABLE_DEBUG` is defined), and `col(v)`, `row(v)` and `pos(v)`, a `dim2::Point< int >` (cf. chapter 8), give the position of a node, while `width()` and `height()` give the size. Also, the arcs `right(v)`, `left(v)`, `up(v)` and `down(v)` leave v in the four directions, and they are `INVALID` on the border. Its read maps `IndexMap`, `ColMap` and `RowMap`, returned by `indexMap()`, `colMap()` and `rowMap()`, give the position, the column and the row of each node (e.g., for the coordinates of a drawing), and `resize(w, h)` changes the size as in the full graphs (we remark that the class has no default constructor).

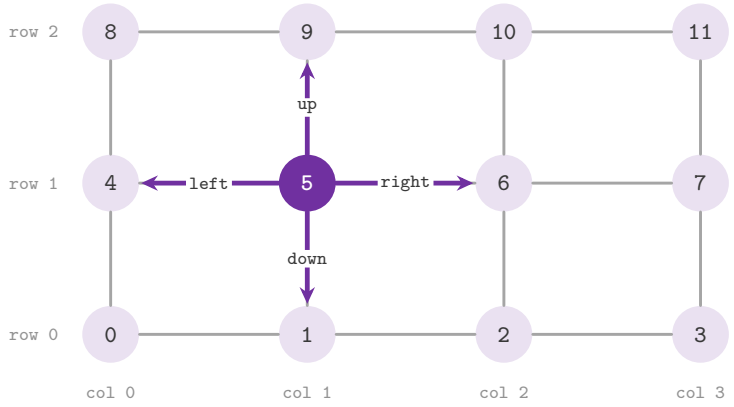


Figure 4.2. GridGraph(4, 3): the index of each node, and the arcs that right(), left(), up() and down() give for node 5, i.e., g(1, 1).

HypercubeGraph. `HypercubeGraph(d)`, for $d \geq 1$ (which a `LEMON_ASSERT` checks), has the 2^d nodes whose indices are the integers of d bits, and an edge between two nodes whose indices differ in exactly one bit, i.e., $d\,2^{d-1}$ edges. Since the indices are of type `int`, the largest dimension is 26 on a platform with 32-bit `int`. Thus, the node of index i is `g(i)` and `index(v)` the inverse; `dimension()` is d, `dimension(e)` and `dimension(a)` give the bit in which the end nodes of an edge or an arc differ, `projection(v, k)` returns whether bit k of the index of v is one, and `resize(d)` changes the dimension. Its nested read map `HyperMap< T , BF = std::plus< T > >` is constructed on the graph, a range of d values $x_0, \ldots, x_{d-1}$, an initial value `fv` (default 0) and a binary function `bf`. It gives for each node the accumulation of `bf` over the values of the bits that are one in its index, starting from `fv`; with the default function it is $\texttt{fv} + \sum_{k:\,\text{bit } k=1} x_k$, which gives, e.g., the position of each node in a drawing of the hypercube from d base vectors.

4.7 Arc sets and edge sets

The header `lemon/edge_set.h` defines four structures that have their arcs (or edges) on the nodes of another graph: `ListArcSet< GR >` and `SmartArcSet< GR >`, which conform to `concepts::Digraph`, and `ListEdgeSet < GR >` and `SmartEdgeSet< GR >`, which conform to `concepts::Graph`. Their parameter `GR` is the type of the graph whose nodes are shared, which may be a digraph or a graph. A set has the `Node` type of `GR` and the nodes of `GR` (with their identifiers, iterators and notifier), and only the arcs or edges belong to it; hence, several sets may be built on one graph (e.g., the arcs of several commodities, or a sparse subgraph whose arcs are not in the graph), and the node maps of `GR` serve all of them. A set is constructed on the graph, `ListArcSet< GR > s( g )`, and it has `addArc(s, t)` (or `addEdge(u, v)`), `clear()`, and the functions of its concept; its counting is linear.

Both `List` sets link the arcs of each node in doubly linked lists, as `ListDigraph` and `ListGraph` do, and they also have `erase(a)` (or `erase(e)`) in $O(1)$ time. They observe the nodes of the underlying graph, through a private node map of the nested class `NodesImpl`; when a node is erased from it, they erase its arcs first, at a cost linear in their number. The two `Smart` sets store the arcs in a vector with singly linked lists, which is slightly faster, and cannot erase them. They observe the nodes too, but an erased node that has an arc in the set invalidates the whole set, which can no longer be used; their function `valid()` returns whether this has happened. The program of section 4.9 builds a `ListEdgeSet` and a `SmartArcSet` on a `ListDigraph`, and checks both behaviors.

4.8 Counting and finding arcs

Counting. In `lemon/core.h`, the functions `countNodes(g)`, `countArcs(g)`, `countEdges(g)`, `countRedNodes(g )` and `countBlueNodes(g)` return the number of items of a kind. They call `nodeNum()`, `arcNum()`, `edgeNum()`, `redNum()` or `blueNum()` in $O(1)$ time if the structure defines the corresponding tag (`NodeNumTag`, `ArcNumTag`, `E dgeNumTag`; `NodeNumTag` also selects `redNum()` and `blueNum()`), and count the items with an iterator otherwise, in linear time, whereas `countItems< GR , Item >(g)` always iterates. The choice is made at compile time by the helper classes `CountNodesSelector`, `CountArcsSelector`, `CountEdgesSelector`, `CountRedNodesSelector` and `CountBlueNodesSelector` of the internal namespaces of `lemon/core.h`, specialized by `enable_if` on the tag, which a user does not call directly. Similarly, `countOutArcs(g, v)`, `countInArcs(g, v)` and `countIncEd ges(g, v)` iterate on the arcs or edges at a node, in time linear in their number.

Finding the arcs between two nodes. A call `findArc(g, u, v, prev = INVALID)` returns an arc from `u` to `v`, the first one if `prev` is `INVALID` and the one after `prev` otherwise, or `INVALID` if there is none; repeated calls, each with the previous result, enumerate the parallel arcs. On a structure with the tag `FindArcTag` (the full, grid and hypercube graphs) it calls `g.findArc()`, in $O(1)$ time, and on the others it scans the arcs out of `u`, in $O(d^+(u))$ time. An iterator `ConArcIt< GR >`, constructed as `ConArcIt< GR > a( g , u , v )`, wraps the same enumeration. For undirected graphs, `findEdge(g, u, v, prev)` and `ConEdgeIt< GR >` find the edges between `u` and `v` (in either orientation) by scanning the edges at `u`, or in $O(1)$ time with `FindEdgeTag`; each loop at `u` is returned once for $u = v$. The two functions dispatch on the tags through the internal classes `FindAr cSelector` and `FindEdgeSelector`.

Look-up structures. When many arcs are looked for in one digraph, three classes of `lemon/core.h` answer faster than the scan, at the price of some memory and of a preprocessing:

- `ArcLookUp< GR >` builds, for each node, a binary search tree of the arcs out of it, keyed by the identifier of their target (Figure 4.3). Its constructor calls `refresh()`, which builds all the trees in $O(m \log \Delta)$ time, where Δ is the largest out-degree, and `refresh(v)` rebuilds the tree of one node in $O(d^+(v) \log d^+(v))$ time. A query `lookup(s, t)` returns an arc from `s` to `t`, or `INVALID`, in $O(\log d^+(s))$ time. It finds one arc only, and the structure is static: after a change of the digraph `refresh()` must be called (or `refresh( v)`, if only the arcs out of `v` changed) before the next query.
- `AllArcLookUp< GR >` is an `ArcLookUp` that also links the arcs with the same source and target, and `look up(s, t, prev)` enumerates them: the first in $O(\log d^+(s))$ time and each of the next ones in $O(1)$ time. It is static as well, and `refresh(v)` rebuilds the data of one node.
- `DynArcLookUp< GR >` keeps a self-adjusting binary search tree, the splay tree of Sleator and Tarjan 1985, for the arcs out of each node, and it observes the arcs of the digraph, so that it follows each addition and erasure with no refresh. A query `lookup(s, t, prev)` takes $O(\log d^+(s))$ amortized time, and, by the static optimality of splay trees, the total time of a sequence of queries is within a constant factor of that of the best static search tree for the frequencies of the sequence. However, as the header remarks, its constant

factors are often larger than those of the two static classes, which are therefore preferable on a digraph that changes rarely.

In fact, the trees of `ArcLookUp` are perfectly balanced, since `refresh(v)` sorts the arcs out of `v` (by its private comparator `ArcLess`) and makes the middle one the root of each subtree (Algorithm 4.2); hence, a query follows at most $\lceil \log_2(d^+(s)+1) \rceil$ arcs of the tree.

Algorithm 4.2 (The search tree of ArcLookUp).

```
refresh(v):
 1  L = the arcs out of v, sorted by the id of their target
 2  head[v] = build(L, 0, |L|-1)          (INVALID if L is empty)
build(L, a, b):
 3  m = (a + b) / 2
 4  left[L[m]]  = build(L, a, m-1) if a < m, else INVALID
 5  right[L[m]] = build(L, m+1, b) if m < b, else INVALID
 6  return L[m]
lookup(s, t):
 7  e = head[s]
 8  while e != INVALID and target(e) != t:
 9    e = left[e] if t < target(e) else right[e]
10  return e
```

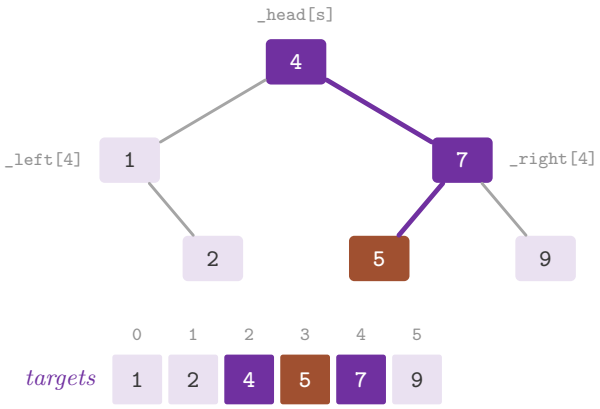


Figure 4.3. The tree of ArcLookUp for a node whose arcs go to the nodes 1, 2, 4, 5, 7 and 9; the look-up of target 5 visits the arcs to 4, 7 and 5.

We remark that `AllArcLookUp::refresh()`, the variant without arguments, does not compile in the present header (it passes an arc to `refresh(Node)`); one calls `refresh(v)` for each node `v` instead, which gives the same result. Our next program compares the four ways of finding an arc on a digraph with parallel arcs, and shows that only `DynArcLookUp` sees a new arc before a refresh.

```
// Finding the arcs between two nodes (section 4.8): findArc() and ConArcIt,
// which scan the arcs out of the source, and the three look-up structures,
// on a digraph with parallel arcs; ArcLookUp and AllArcLookUp are refreshed
// after a change, DynArcLookUp follows it by itself.

#include <iostream>
#include <lemon/core.h>
#include <lemon/smart_graph.h>

using namespace lemon;

int main()
{
 int errors = 0;
 SmartDigraph g;
 const int n = 6;
 SmartDigraph::Node v[ n ];
```

```
for( int i = 0 ; i < n ; ++i )
 v[ i ] = g.addNode();
for( int i = 0 ; i < n ; ++i )
 for( int j = 0 ; j < n ; ++j )
  if( i != j && ( i + j ) % 3 != 0 )
   g.addArc( v[ i ] , v[ j ] );
g.addArc( v[ 0 ] , v[ 1 ] );  // two more arcs parallel to v0 -> v1
g.addArc( v[ 0 ] , v[ 1 ] );

ArcLookUp< SmartDigraph > one( g );
AllArcLookUp< SmartDigraph > all( g );
DynArcLookUp< SmartDigraph > dyn( g );

// every pair: the four ways agree on whether an arc exists
for( int i = 0 ; i < n ; ++i )
 for( int j = 0 ; j < n ; ++j ) {
  SmartDigraph::Arc a = findArc( g , v[ i ] , v[ j ] );
  bool exists = ( i != j && ( i + j ) % 3 != 0 );
  if( ( a != INVALID ) != exists )
   ++errors;
  if( ( one( v[ i ] , v[ j ] ) != INVALID ) != exists )
   ++errors;
  if( ( all( v[ i ] , v[ j ] ) != INVALID ) != exists )
   ++errors;
  if( ( dyn( v[ i ] , v[ j ] ) != INVALID ) != exists )
   ++errors;
  if( one( v[ i ] , v[ j ] ) != INVALID &&
      g.target( one( v[ i ] , v[ j ] ) ) != v[ j ] )
   ++errors;
  }

// all the parallel arcs: three from v0 to v1
int by_find = 0 , by_con = 0 , by_all = 0 , by_dyn = 0;
for( SmartDigraph::Arc a = findArc( g , v[ 0 ] , v[ 1 ] ) ; a != INVALID ;
     a = findArc( g , v[ 0 ] , v[ 1 ] , a ) )
 ++by_find;
for( ConArcIt< SmartDigraph > a( g , v[ 0 ] , v[ 1 ] ) ; a != INVALID ; ++a )
 ++by_con;
for( SmartDigraph::Arc a = all( v[ 0 ] , v[ 1 ] ) ; a != INVALID ;
     a = all( v[ 0 ] , v[ 1 ] , a ) )
 ++by_all;
for( SmartDigraph::Arc a = dyn( v[ 0 ] , v[ 1 ] ) ; a != INVALID ;
     a = dyn( v[ 0 ] , v[ 1 ] , a ) )
 ++by_dyn;
if( by_find != 3 || by_con != 3 || by_all != 3 || by_dyn != 3 )
 ++errors;

// a new arc: DynArcLookUp sees it at once, ArcLookUp after refresh()
SmartDigraph::Arc a03 = g.addArc( v[ 0 ] , v[ 3 ] );
if( dyn( v[ 0 ] , v[ 3 ] ) != a03 )
 ++errors;
one.refresh( v[ 0 ] );
if( one( v[ 0 ] , v[ 3 ] ) != a03 )
 ++errors;

std::cout << "arc look-up: " << errors << " errors" << std::endl;
return( errors == 0 ? 0 : 1 );
}
```

4.9 Copying graphs

Since a graph structure cannot be copied by its copy constructor, its duplicate, possibly into a structure of another type, is made by the classes `DigraphCopy< From , To >`, `GraphCopy< From , To >` and `BpGraphCopy< From , To >` of `lemon/core.h`, or by the functions `digraphCopy(from, to)`, `graphCopy(from, to)` and

`bpGraphCopy(from, to)`, which return such an object. Before running it one can request the following data, each by a function that returns the object (and may thus be chained):

- `nodeRef(map)`, a map with the nodes of `from` as keys, into which the copy writes the image of each node, and `nodeCrossRef(map)`, a map with the nodes of `to` as keys, into which it writes the original of each node; `arcRef()`, `arcCrossRef()`, `edgeRef()`, `edgeCrossRef()`, `redRef()`, `redCrossRef()`, `blueRef()` and `blueCrossRef()` do the same for the other kinds of items;
- `nodeMap(fmap, tmap)`, which copies the values of the map `fmap` of `from` into the map `tmap` of `to`, item by item; `arcMap()`, `edgeMap()`, `redNodeMap()` and `blueNodeMap()` do the same for the other kinds of items;
- `node(n, tn)`, which writes into `tn` the image of the node `n`, and `arc()`, `edge()`, `redNode()` and `blueNode()` likewise.

Then `run()` clears `to` and copies the structure, followed by the data that were requested. Internally, each request is an object of one of the classes `MapCopy`, `ItemCopy`, `RefCopy` and `CrossRefCopy` (derived from `MapCopyBase`), and the structure is copied by `DigraphCopySelector`, `GraphCopySelector` or `BpGraphCopySelector`, which call `build()` on a target with `BuildTag`. Also, `GraphCopy` and `BpGraphCopy` compute the images of the arcs (and `BpGraphCopy` those of the nodes, from the red and the blue ones) by their private maps `ArcRefMap` and `NodeRefMap`. In `GraphCopy` the image of an arc is the arc of the image of its edge whose source is the image of its source (for a loop, the arc with the same `direction()`), so that the orientation of each arc is kept even if the target structure orders the end nodes of its edges differently.

Algorithm 4.3 (DigraphCopy::run()).

```
run():
 1  nr = node map of from, ar = arc map of from      (local references)
 2  if To has BuildTag:  to.build(from, nr, ar)
 3  else:
 4    to.clear()
 5    for each node v of from:  nr[v] = to.addNode()
 6    for each arc a of from:   ar[a] = to.addArc(nr[source(a)], nr[target(a)])
 7  for each requested node datum:  copy it through nr
 8  for each requested arc datum:   copy it through ar
```

The copy takes $O(n+m)$ time for the structure plus the time of the maps that were requested, except when the target is built by `build()`, whose cost is that of section 4.4 (e.g., $O(n+\sum_v d^+(v)\log d^+(v))$ for a `StaticDigraph`). The node references are always computed, because the arcs need them.

> **abLEMON.** `DigraphCopy`, `GraphCopy` and `BpGraphCopy` hold the objects that copy the requested data, which their destructor deletes; in abLEMON they have a move constructor and no copy constructor, which `digraphCopy()` and the other functions use to return them by value. LEMON 1.3.1 copied them, and a copy that was not elided deleted those objects twice.

Our last program copies a `ListGraph` into a `SmartGraph` with the node references, the cross references of the edges, an edge map and the image of one arc, and then builds the arc and edge sets of section 4.7.

```
// Copies and arc sets (sections 4.7 and 4.9): a ListGraph copied into a
// SmartGraph with the references of its nodes, the cross references of its
// edges, an edge map and the image of one arc; then a ListEdgeSet and a
// SmartArcSet on the nodes of a ListDigraph, and what an erased node does
// to them.

#include <iostream>
#include <lemon/edge_set.h>
#include <lemon/list_graph.h>
#include <lemon/smart_graph.h>

using namespace lemon;

int main()
{
 int errors = 0;

 ListGraph lg;
 ListGraph::Node a = lg.addNode() , b = lg.addNode() , c = lg.addNode();
```

```
ListGraph::Edge ab = lg.addEdge( a , b ) , bc = lg.addEdge( b , c );
lg.addEdge( c , c );  // a loop
ListGraph::EdgeMap< double > w( lg );
w[ ab ] = 1.5;
w[ bc ] = 2.5;

SmartGraph sg;
sg.addNode();  // the copy clears the target first
ListGraph::NodeMap< SmartGraph::Node > nref( lg );
SmartGraph::EdgeMap< ListGraph::Edge > ecross( sg );
SmartGraph::EdgeMap< double > sw( sg );
ListGraph::Arc ba = lg.direct( ab , false );
SmartGraph::Arc sba;
graphCopy( lg , sg )
  .nodeRef( nref )
  .edgeCrossRef( ecross )
  .edgeMap( w , sw )
  .arc( ba , sba )
  .run();
if( countNodes( sg ) != 3 || countEdges( sg ) != 3 )
 ++errors;
for( SmartGraph::EdgeIt e( sg ) ; e != INVALID ; ++e )
 if( sw[ e ] != w[ ecross[ e ] ] )
  ++errors;
// the image of an arc has the images of its end nodes
if( sg.source( sba ) != nref[ b ] || sg.target( sba ) != nref[ a ] )
 ++errors;

// an edge set and an arc set sharing the nodes of a ListDigraph
ListDigraph base;
ListDigraph::Node x = base.addNode() , y = base.addNode() ,
  z = base.addNode();
ListEdgeSet< ListDigraph > es( base );
ListEdgeSet< ListDigraph >::Edge xy = es.addEdge( x , y );
es.addEdge( y , z );
es.addEdge( z , x );
SmartArcSet< ListDigraph > as( base );
as.addArc( x , y );
as.addArc( x , z );
if( countEdges( es ) != 3 || countArcs( es ) != 6 || countArcs( as ) != 2 )
 ++errors;
es.erase( xy );
if( countEdges( es ) != 2 )
 ++errors;

// erasing z from the base graph erases its edges from the ListEdgeSet,
// but it invalidates the SmartArcSet, which has an arc at z
base.erase( z );
if( countEdges( es ) != 0 || countNodes( es ) != 2 )
 ++errors;
if( as.valid() )
 ++errors;

std::cout << "copies and arc sets: " << errors << " errors" << std::endl;
return( errors == 0 ? 0 : 1 );
}
```

4.10 Exercises

Exercise 4.1. Build a `ListDigraph` with 10^5 nodes and 10^6 random arcs, and measure the time of a loop that visits the arcs out of each node, on it and on the `StaticDigraph` obtained by `digraphCopy()`. Explain the difference by the storage of Figure 4.1. (The test `test/digraph_test.cc` checks that the two structures have the same arcs.)

Exercise 4.2. Show that `contract(u, v)` of `ListDigraph` with `r = true` erases the arcs between `u` and `v` but not the loops that `u` already had, and write the sequence of calls that contracts `u` and `v` and erases all the loops at `u`. (The program `manual/examples/04-list-graph.cc` contracts two nodes.)

Exercise 4.3. Prove that Algorithm 4.1 places the arcs out of each node in increasing order of their index, and that `nextIn()` visits the arcs into a node in the same order as the `nextOut()` of their opposites. (The test `test/graph_test.cc` checks a `StaticGraph` built from a random graph with loops and parallel edges.)

Exercise 4.4. Give the number of nodes, edges and arcs of `GridGraph(w, h)` and of `HypercubeGraph(d)`, prove the formulae of section 4.6, and check them against `nodeNum()`, `edgeNum()` and `arcNum()` for small sizes. (The program `manual/examples/04-static-graphs.cc` checks some of them.)

Exercise 4.5. On a digraph with n nodes and m arcs, compare the total time of q queries by `findArc()`, by `ArcLookUp` (preprocessing included) and by `DynArcLookUp`, and find the values of q above which each look-up structure becomes the faster choice. (The program `manual/examples/04-arc-lookup.cc` and the test `test/arc_look_up_test.cc` use the three structures.)

Exercise 4.6. Copy a `ListBpGraph` into a `SmartBpGraph` with `bpGraphCopy()`, requesting the references of the red nodes and a blue node map, and check that each edge keeps its red and its blue end. (The test `test/graph_copy_test.cc` copies bipartite graphs.)

5. Graph adaptors

We are concerned in this chapter with the classes of `lemon/adaptors.h`, which present a digraph or a graph to an algorithm as if it were another one, without copying it. The other structure may have fewer nodes or arcs, reversed arcs or no orientation at all, split nodes, or only the arcs of positive residual capacity of a flow. An adaptor stores a pointer to the adapted structure (and, possibly, to some maps), and it answers each query of the graph concepts of chapter 4 by one or more queries on that structure; hence, building it costs $O(1)$ time and memory, while the price is paid at each step of an iteration. Section 5.1 describes what the adaptors have in common, sections 5.2 to 5.5 describe them one family at a time (filters, orientation, residual digraph, node splitting), and section 5.6 collects the exercises.

5.1 Views of a graph

Each adaptor of the header conforms to `concepts::Digraph` or to `concepts::Graph` (cf. chapter 3). Therefore, it can be passed to the algorithms of the library and to the functions of `core.h`, and it can be adapted again: the test `test/adaptors_test.cc` builds, e.g., `Undirector< const SplitNodes< Orienter< const GridGraph > > >` and checks it against both concepts. Three properties are shared, in some form, by all the classes of the chapter, and we state them once here.

Constness and modification. The first template parameter is the type of the adapted structure, and it may be given as `const` (e.g., `ReverseDigraph< const ListDigraph >`). In that case the adaptor can be built from a constant reference, and it offers no modifying function. Otherwise, the adaptors that keep the item set of the adapted structure (`ReverseDigraph`, `SubDigraph`, `SubGraph`, `FilterNodes`, `FilterArcs`, `FilterEdges`, `Undirector` and `Orienter`) forward `addNode()`, `addArc()` (or `addEdge()`), `erase()` and `clear()` to it, and one changes the adapted structure through the adaptor. `ResidualDigraph` and `SplitNodes`, instead, take their digraph as implicitly `const`, since an arc added to a residual digraph (or a node added to a split digraph) has no meaning for the underlying one. Each adaptor also has a creator function, i.e., `reverseDigraph()`, `subDigraph()`, `subGraph()`, `filterNodes()`, `filterArcs()`, `filterEdges()`, `undirector()`, `orienter()`, `residualDigraph()` and `splitNodes()`. It returns the read-only version (the one on the `const` structure), with the filter maps `const` or not as they are passed.

Items. The `Node` and `Arc` (and `Edge`) types of an adaptor are usually convertible to and from those of the adapted structure, as the notes of the header state class by class. For instance, a node of a `SubDigraph` is a node of the digraph, and an edge of an `Undirector` is an arc of the digraph (the exceptions are given where they occur). Hence, a map of the adapted structure can be read with the items of the adaptor, which is how we use the maps in the examples below.

Maps and counting. For the adaptors that keep the item set, the `NodeMap`, `ArcMap` and `EdgeMap` of the adaptor are maps of the adapted structure, and they store a value for each of its items (the hidden ones included). `Undirector`, `ResidualDigraph` and `SplitNodes`, whose arcs are not arcs of the digraph, build their arc maps from two maps of the digraph. Counting by `countNodes()`, `countArcs()` and `countEdges()` takes constant time on `ReverseDigraph`, `Undirector`, `Orienter` and `SplitNodes` when the adapted structure counts in constant time. The filtering adaptors (and `ResidualDigraph`, which is one of them) count by iterating, in linear time; in fact, their tags `NodeNumTag` and `ArcNumTag` are `False`.

In general, the cost of an iteration on an adaptor is that of the iteration on the adapted structure plus a constant amount of work per item visited (which is rather small), with one exception which may matter in practice. A filtering adaptor skips the hidden items one by one, since it has no list of the visible ones; e.g., `firstOut()` of a node of a `SubDigraph` may visit all the arcs that leave the node in the digraph before it finds a visible one. An algorithm run on a filter that hides most of the items may therefore be (somewhat, or much) slower than the

same algorithm run on a copy made with `digraphCopy()` (cf. chapter 4). Of course, the copy costs time and memory, and it does not follow the later changes of the filter.

The classes of the header are built on base classes that a user does not instantiate, i.e., `DigraphAdaptorBase` and `GraphAdaptorBase` (which forward each query to the adapted structure), `ReverseDigraphBase`, `SubDigraphBase`, `SubGraphBase`, `UndirectorBase`, `OrienterBase` and `SplitNodesBase` (which change the queries), completed by the extenders of `lemon/bits/` (cf. appendix A); the maps of `Undirector` and `SplitNodes` are built on their private classes `ArcMapBase` and `NodeMapBase`, and the arc filters of `ResidualDigraph` are the functors `ResForwardFilter` and `ResBackwardFilter` of the namespace `_adaptor_bits`.

5.2 Reversing and filtering

`ReverseDigraph< DGR >` presents the digraph `DGR` with all its arcs reversed. Its `source()` and `target()` are exchanged, the outgoing arcs of a node are its incoming arcs in the digraph (and vice versa), `findArc(u, v)` looks for an arc from `v` to `u` in the digraph, and `addArc(u, v)` adds the arc (v, u) to it. Its constructor takes the digraph (`explicit ReverseDigraph( DGR & digraph )`), and the adaptor counts in the time of the digraph. One typically uses it to run a search backwards, e.g., to find the nodes from which a target is reachable (as the example at the end of section 5.3 does), or to compute the distances to a node by `Dijkstra` with the same length map.

The filtering adaptors hide some of the items by means of `bool` maps, called filters: an item is visible if and only if its filter value is `true`. `SubDigraph< DGR, NF, AF >` filters both the nodes and the arcs of a digraph, with a node map `NF` and an arc map `AF`, whose defaults are `DGR::NodeMap< bool >` and `DGR::ArcMap< bool >`. An arc is visible when it is enabled and both its ends are enabled. `SubGraph< GR, NF, EF >` does the same for a graph, with an edge filter `EF` (by default `GR::EdgeMap< bool >`), and an edge is visible when it and both its ends are enabled. Three specialized filters keep a constant map, i.e., a `ConstMap` with the inlined value `true` (cf. §6.3), in place of the filter they lack. `FilterNodes< GR, NF >` works on a digraph or on a graph (it chooses the concept from the `UndirectedTag` of `GR`), and it hides the nodes together with the arcs or edges incident to them. `FilterArcs< DGR, AF >` hides arcs of a digraph, and `FilterEdges< GR, EF >` hides edges of a graph (and both their arcs); neither of the two hides a node.

Their constructors take the structure and references to the filters, which the adaptor keeps by pointer, e.g., `SubDigraph( DGR & digraph, NF & node_filter, AF & arc_filter )` and `FilterArcs( DGR & digraph, AF & arc_filter )`. Thus, the filters must outlive the adaptor, and a value written directly into a filter is seen at once through the adaptor. The same can be done through the adaptor itself. `status(item, v)` sets the filter value of the item to `v` and `status(item)` reads it, while `enable(item)` and `disable(item)` are `status(item, true)` and `status(item, false)`; each of them costs one access to the filter map. Building the filters (e.g., `DGR::ArcMap< bool > f( g, true )`, which shows the whole digraph) is left to the caller, since only the caller knows which items are to be seen.

A `SubDigraph` with an all-true node filter and a `FilterArcs` show the same digraph; the difference is one of efficiency. The former checks the filter values of the two ends of each arc it visits (the template flag `ch = true` of `SubDigraphBase`), while the latter, whose node filter is the constant `true`, checks only the arc filter. Note that `findArc()` and `findEdge()` of a filter are those of the adapted structure followed by a scan of the arcs they return until a visible one is found. Their cost is, hence, that of the adapted structure times the number of hidden parallel arcs.

5.3 Changing the orientation

`Undirector< DGR >` presents a digraph as an undirected graph. Each arc of the digraph is an edge of the adaptor, with `u(e)` its source and `v(e)` its target, and, as for the other graphs of the library, each edge also gives two arcs: the forward one (`direction(a) == true`), oriented as the arc of the digraph, and the backward one. The `Edge` type of the adaptor is the `Arc` type of the digraph, while its `Arc` is a pair (arc of the digraph, direction). The identifier of an arc is $2i + 1$ for the forward arc and $2i$ for the backward one, where i is the identifier of the arc in the digraph (so that `maxArcId()` is $2\,$`maxArcId()` $+1$ of the digraph). Also, `edgeNum()` is `arcNum()` of the digraph and `arcNum()` is twice as much, both in the time of the digraph. The constructor takes the digraph, `addEdge(u, v)` adds the arc (u, v) to it, and the `ArcMap` of an `Undirector` keeps two arc maps of the digraph, one per direction.

When an algorithm on the undirected graph needs an arc map and the data are on the digraph, the nested

class `Undirector< DGR >::CombinedArcMap< FW, BK >` reads the forward arcs from the arc map `FW` and the backward ones from `BK`. It is created by the static function `combinedArcMap(forward, backward)`, its value type is that of `FW`, and it is a reference map when `FW` is one. For instance, `Undirector< const G >::combinedArcMap( cost, cost )` gives an undirected length (or capacity) taken from one arc map of the digraph.

`Orienter< GR, DM >` goes the other way, and it presents a graph as a digraph in which each edge is an arc. The arc goes from `u(e)` to `v(e)` when the direction map `DM` (a `bool` edge map, by default `GR::EdgeMap< bool >`) is `true` on the edge, and from `v(e)` to `u(e)` otherwise. Its constructor takes the graph and the direction map, which is kept by pointer, and `reverseArc(a)` negates the direction of an arc by one write to the map. Furthermore, `addArc(u, v)` adds an edge to the graph and sets its direction, so that the new arc goes from `u` to `v`. Since the arcs of the adaptor are the edges of the graph, `arcNum()` is `edgeNum()` of the graph. However, `firstOut()` and `firstIn()` scan the incident edges of the node and skip those of the wrong direction, i.e., they cost the degree of the node in the graph. The function `checkCombiningAdaptors()` of `test/adaptors_test.cc` orients a grid by an `Orienter`, splits its nodes and views the result through an `Undirector`.

The example below keeps the arcs of a digraph whose cost is at most a threshold (by `FilterArcs`), and it counts the connected components of the result through an `Undirector`, before and after enabling one more arc. It then searches the digraph backwards through a `ReverseDigraph`.

```
// Graph adaptors (section 5.2 and 5.3): the arcs of cost at most a
// threshold kept by FilterArcs, the result seen as an undirected graph by
// Undirector, and the nodes that reach a target found by a search on
// ReverseDigraph; the original digraph is never copied nor changed.

#include <iostream>
#include <lemon/adaptors.h>
#include <lemon/bfs.h>
#include <lemon/connectivity.h>
#include <lemon/list_graph.h>

using namespace lemon;

int main()
{
 ListDigraph g;
 ListDigraph::Node v[ 6 ];
 for( int i = 0 ; i < 6 ; ++i )
  v[ i ] = g.addNode();
 ListDigraph::ArcMap< int > cost( g );
 // two triangles 0-1-2 and 3-4-5 of cheap arcs, joined by an expensive one
 const int arcs[ 7 ][ 3 ] = { { 0 , 1 , 1 } , { 1 , 2 , 2 } , { 2 , 0 , 1 } ,
                              { 2 , 3 , 9 } , { 3 , 4 , 1 } , { 4 , 5 , 2 } ,
                              { 5 , 3 , 1 } };
 for( int k = 0 ; k < 7 ; ++k )
  cost[ g.addArc( v[ arcs[ k ][ 0 ] ] , v[ arcs[ k ][ 1 ] ] ) ] =
    arcs[ k ][ 2 ];

 // the arc filter: true on the arcs of cost at most 5
 ListDigraph::ArcMap< bool > cheap( g );
 for( ListDigraph::ArcIt a( g ) ; a != INVALID ; ++a )
  cheap[ a ] = cost[ a ] <= 5;

 typedef FilterArcs< ListDigraph > Cheap;
 Cheap sub( g , cheap );
 typedef Undirector< const Cheap > UCheap;
 UCheap usub( sub );

 int comps = countConnectedComponents( usub );  // 2: the two triangles
 int all = countConnectedComponents( undirector( g ) );  // 1
 std::cout << "components with the cheap arcs: " << comps
           << ", with all the arcs: " << all << std::endl;

 // enabling the expensive arc again joins the two components
 sub.enable( findArc( g , v[ 2 ] , v[ 3 ] ) );
```

```
int joined = countConnectedComponents( usub );

// the nodes that reach v[0] are those that v[0] reaches backwards
sub.disable( findArc( g , v[ 2 ] , v[ 3 ] ) );
ReverseDigraph< const ListDigraph > rg( g );
Bfs< ReverseDigraph< const ListDigraph > > bfs( rg );
bfs.run( v[ 0 ] );
int reach = 0;
for( ListDigraph::NodeIt n( g ) ; n != INVALID ; ++n )
 if( bfs.reached( n ) )
  ++reach;
std::cout << "nodes that reach node 0: " << reach << std::endl;

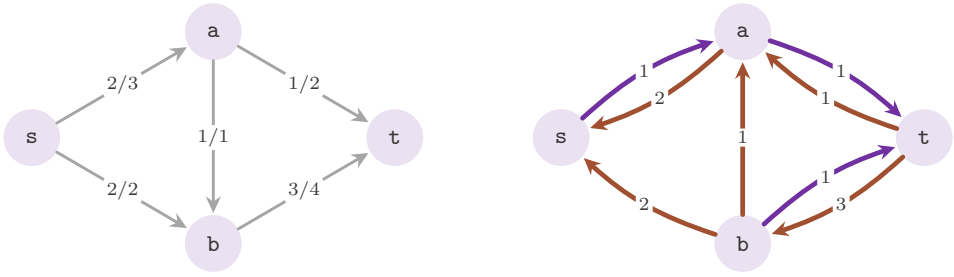
return( ( comps == 2 && all == 1 && joined == 1 && reach == 3 ) ? 0 : 1 );
}
```

5.4 The residual digraph

Let $G = (V, A)$ be a digraph, $u : A \to \mathbb{R}_+$ a capacity and x a flow on it, i.e., a vector with $0 \le x_a \le u_a$ for each arc a. The residual digraph of x (cf. Ahuja, Magnanti and Orlin 1993, §2.4) has node set V and arc set $A_f \cup A_b$, where

$$A_f = \{(i,j) : (i,j) \in A,\ x_{ij} < u_{ij}\}, \qquad A_b = \{(j,i) : (i,j) \in A,\ x_{ij} > 0\}. \tag{5.1}$$

Each forward arc $(i,j) \in A_f$ has residual capacity $u_{ij} - x_{ij}$, and each backward arc $(j,i) \in A_b$ has residual capacity x_{ij}. The union is taken with multiplicity; thus, an arc of G may give two arcs of the residual digraph, and parallel arcs may appear (Figure 5.1). A directed path from s to t in the residual digraph is called an augmenting path. Sending along it the "bottleneck", i.e., the smallest residual capacity of its arcs (i.e., increasing x on the forward arcs and decreasing it on the backward ones) gives a flow of larger value.



flow / capacity *residual digraph*

Figure 5.1. A flow x (left, flow/capacity on each arc) and its residual digraph (right): the forward arcs, in the primary colour, where x is below the capacity, and the backward arcs, in the second colour, where x is positive, each with its residual capacity.

`ResidualDigraph< DGR, CM, FM, TL >` implements (5.1) as an adaptor. `DGR` is the digraph (implicitly `const`), `CM` the capacity map (by default `DGR::ArcMap< int >`, implicitly `const`), `FM` the flow map (by default `CM`), which the adaptor may write, and `TL` the tolerance (by default `Tolerance< CM::Value >`, cf. chapter 3). The tolerance decides which arcs exist. A forward arc is there when `tolerance.positive( cap[a] - flow[a] )` holds, and a backward arc when `tolerance.positive( flow[a] )` holds; on floating-point data, an arc whose residual capacity is below the epsilon of the tolerance is therefore hidden. As the header notes, the class is a `SubDigraph` of an `Undirector` of the digraph, whose arc filter is a `CombinedArcMap` of these two tests. Hence, an arc of the adaptor is an arc of the `Undirector` (convertible to the arc of the digraph it comes from), the adaptor counts in linear time, and its iterations skip the arcs of zero residual capacity one by one. The constructor is

```
ResidualDigraph( const DGR & digraph, const CM & capacity, FM & flow,
                 const TL & tolerance = Tolerance() );
```

and the members that are specific to the residual digraph follow.

- `residualCapacity(a)` returns $u_a - x_a$ for a forward arc and x_a for a backward one.

- `augment(a, v)` adds `v` to the flow of the original arc if `a` is forward, and it subtracts `v` if `a` is backward. It writes the flow map by `set()`, and it does not check that the result stays within the bounds.
- `forward(a)` and `backward(a)` (static), on an arc of the adaptor, give its direction. On an arc of the digraph, they return the forward or the backward residual arc that corresponds to it, whether or not that arc is currently visible.
- `ResidualCapacity` is a read-only arc map of the adaptor whose value at an arc is `residualCapacity(a)`. It is created by `residualCapacity()`, and its value type is that of the capacity map.

The function `residualDigraph(digraph, capacity, flow)` returns the adaptor with the default tolerance.

We remark that the augmenting path method of Ford and Fulkerson 1956 needs only the residual digraph, and Algorithm 5.1 writes it with the classes of the library. The choice of a shortest path, i.e., of a path found by a breadth-first search, is the rule of Edmonds and Karp 1972.

Algorithm 5.1 (Augmenting paths on the residual digraph).

```
input:  digraph G, capacity u, feasible flow x (e.g., x = 0), nodes s, t
R := ResidualDigraph(G, u, x)
loop
  run Bfs on R from s, stopped at t
  if t is not reached then stop
  P := the path from s to t of the search
  delta := min { residualCapacity(a) : a in P }
  for each arc a of P: augment(a, delta)
end loop
output: x, a maximum flow; the nodes reached by the last search,
        the source side of a minimum cut
```

Clearly, the algorithm keeps x feasible, since the flow of an arc changes by at most its residual capacity. When it stops, the set S of the nodes reached by the last search contains s but does not contain t, and no arc of the residual digraph leaves it. Since S is closed in this sense, each arc of G from S to $V \setminus S$ is saturated and each arc from $V \setminus S$ to S is empty, which proves the following classical statement.

Theorem 5.1. *A flow x from s to t is maximum if and only if its residual digraph has no path from s to t. In that case, the set S of the nodes reachable from s in the residual digraph gives a cut whose capacity, $\sum_{(i,j)\in A:\, i\in S,\, j\notin S} u_{ij}$, equals the value of x.*

For a proof, see Ford and Fulkerson 1956 and Ahuja, Magnanti and Orlin 1993, §6.5. With breadth-first searches the number of augmentations is $O(nm)$ (Edmonds and Karp 1972), where $n = |V|$ and $m = |A|$. Each search costs $O(m)$ on the adaptor (the arcs of zero residual capacity being skipped at a constant cost each), and Algorithm 5.1 thus runs in $O(nm^2)$ time. The library has much faster maximum flow algorithms, `Preflow` and `EdmondsKarp` (cf. chapter 12). In our view, the adaptor is meant, rather, for the algorithms that search the residual digraph of a flow already found, e.g., for a minimum cut, for the cycles of a circulation, or for a shortest path of reduced costs. The example below runs Algorithm 5.1, and it checks the value and the cut against `Preflow`; the function `checkResidualDigraph()` of `test/adaptors_test.cc` does the same on another digraph.

```
// The residual digraph (section 5.4): a maximum flow found by augmenting
// along the shortest paths of ResidualDigraph (Algorithm 5.1), checked
// against Preflow, and the minimum cut read from the last search.

#include <iostream>
#include <limits>
#include <lemon/adaptors.h>
#include <lemon/bfs.h>
#include <lemon/list_graph.h>
#include <lemon/path.h>
#include <lemon/preflow.h>

using namespace lemon;

int main()
{
 typedef ListDigraph::ArcMap< int > IntArcMap;
 ListDigraph g;
```

```
ListDigraph::Node s = g.addNode() , a = g.addNode() , b = g.addNode() ,
                  c = g.addNode() , t = g.addNode();
IntArcMap cap( g ) , flow( g , 0 );
cap[ g.addArc( s , a ) ] = 4;
cap[ g.addArc( s , b ) ] = 3;
cap[ g.addArc( a , b ) ] = 2;
cap[ g.addArc( a , c ) ] = 2;
cap[ g.addArc( b , c ) ] = 4;
cap[ g.addArc( b , t ) ] = 2;
cap[ g.addArc( c , t ) ] = 5;

typedef ResidualDigraph< const ListDigraph , IntArcMap > Residual;
Residual res( g , cap , flow );
Residual::ResidualCapacity rcap = res.residualCapacity();

int value = 0 , augmentations = 0;
Bfs< Residual > bfs( res );
for( ; ; ) {
 bfs.run( s , t );  // a shortest augmenting path, if any
 if( ! bfs.reached( t ) )
  break;
 Path< Residual > p = bfs.path( t );
 int delta = std::numeric_limits< int >::max();
 for( Path< Residual >::ArcIt e( p ) ; e != INVALID ; ++e )
  if( rcap[ e ] < delta )
   delta = rcap[ e ];
 for( Path< Residual >::ArcIt e( p ) ; e != INVALID ; ++e )
  res.augment( e , delta );
 value += delta;
 ++augmentations;
 }

// the nodes reached by the last search are the source side of a minimum
// cut, whose capacity equals the value of the flow
int cut = 0;
for( ListDigraph::ArcIt e( g ) ; e != INVALID ; ++e )
 if( bfs.reached( g.source( e ) ) && ! bfs.reached( g.target( e ) ) )
  cut += cap[ e ];

Preflow< ListDigraph , IntArcMap > preflow( g , cap , s , t );
preflow.run();

std::cout << "flow value " << value << " after " << augmentations
          << " augmentations, cut " << cut << ", Preflow "
          << preflow.flowValue() << std::endl;
return( ( value == 7 && cut == 7 && preflow.flowValue() == 7 ) ? 0 : 1 );
}
```

5.5 Splitting the nodes

Several network problems (often, but not always, routing problems) put a bound or a cost on the nodes (the number of paths through a node, the capacity of a switch, the cost of a visit), while the algorithms of the library take them on the arcs. The standard transformation (cf. Ahuja, Magnanti and Orlin 1993, §2.4) replaces each node u by two nodes u_{in} and u_{out}, joined by the arc (u_{in}, u_{out}) which carries the data of the node. Each arc (v, u) of the digraph then enters u_{in}, and each arc (u, w) leaves u_{out} (Figure 5.2). It is easy to check (and one can see it in Figure 5.2) that a path of the new digraph through (u_{in}, u_{out}) is a path of the old one through u, and conversely.

`SplitNodes< DGR >` implements the transformation on the digraph `DGR`, which is implicitly `const`. One has $2n$ nodes and $m + n$ arcs, i.e., the m original arcs and the n bind arcs. A node of the adaptor is a node of the digraph with a flag (in or out), and it is convertible to the node of the digraph. An arc, instead, is either an arc or a node of the digraph (a `BiVariant`); hence, the arc type is not convertible to the arc of the digraph in general (it converts to it when it is an original arc). The identifiers are $2i$ for the in-node and $2i + 1$ for the

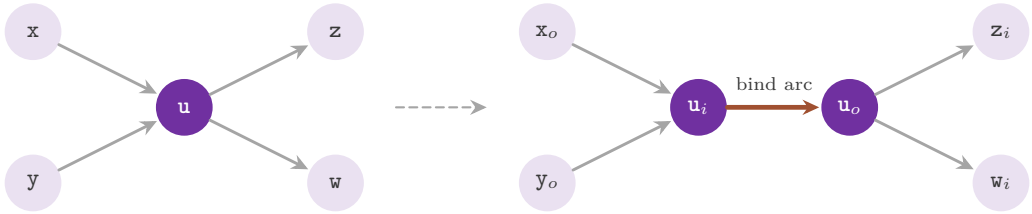


Figure 5.2. The transformation of SplitNodes: the node u becomes the in-node u_i, entered by the arcs that entered u, and the out-node u_o, left by the arcs that left u, joined by the bind arc.

out-node, and $2i$ for an original arc and $2i+1$ for the bind arc of a node, where i is the identifier in the digraph. `nodeNum()` and `arcNum()` are computed by `countNodes()` and `countArcs()` of the digraph, i.e., in the time of the digraph. The static members translate between the two structures, as follows.

- `inNode(n)` and `outNode(n)`, on a node of the adaptor, return which of the two it is; on a node of the digraph, they return its in-node or its out-node.
- `origArc(a)` and `bindArc(a)` return whether an arc of the adaptor is an original arc or a bind arc.
- `arc(n)` returns the bind arc of a node of the digraph, and `arc(a)` the arc of the adaptor that corresponds to an arc of the digraph.

One gives the data by two combining maps, nested in the class. `CombinedNodeMap< In, Out >`, created by `combinedNodeMap(in_map, out_map)`, reads the in-nodes from the node map `In` and the out-nodes from `Out`. `CombinedArcMap< AM, NM >`, created by `combinedArcMap(arc_map, node_map)`, reads the original arcs from the arc map `AM` and the bind arcs from the node map `NM`. Their value types are those of `In` and `AM`, respectively. In this way, one may turn a node capacity into the capacity of a bind arc. By Menger's theorem (Menger 1927), the largest number of s-t paths with no inner node in common is then the value of a maximum flow from s_{out} to t_{in}, when each bind arc of an inner node has capacity 1 (and each original arc a capacity of at least 1). The example below computes it by `Preflow` on the adaptor.

```
// Splitting the nodes (section 5.5): the largest number of s-t paths that
// share no inner node, as a maximum flow on SplitNodes with capacity 1 on
// the bind arcs of the inner nodes (Menger's theorem).

#include <iostream>
#include <lemon/adaptors.h>
#include <lemon/list_graph.h>
#include <lemon/preflow.h>

using namespace lemon;

int main()
{
 ListDigraph g;
 ListDigraph::Node s = g.addNode() , a = g.addNode() , b = g.addNode() ,
                   c = g.addNode() , d = g.addNode() , t = g.addNode();
 // every s-t path goes through a, b or c, and those through b and c
 // also go through d
 g.addArc( s , a );
 g.addArc( s , b );
 g.addArc( s , c );
 g.addArc( a , t );
 g.addArc( b , d );
 g.addArc( c , d );
 g.addArc( d , t );
 g.addArc( a , d );

 ListDigraph::ArcMap< int > arc_cap( g , 2 );    // arcs: no limit that binds
 ListDigraph::NodeMap< int > node_cap( g , 1 );  // inner nodes: capacity 1
 node_cap[ s ] = node_cap[ t ] = 2;

 typedef SplitNodes< ListDigraph > Split;
```

```
Split split( g );
typedef Split::CombinedArcMap< ListDigraph::ArcMap< int > ,
                               ListDigraph::NodeMap< int > >
  CapMap;
CapMap cap( arc_cap , node_cap );

// the paths leave from the out-node of s and end in the in-node of t
Preflow< Split , CapMap > preflow( split , cap , split.outNode( s ) ,
                                   split.inNode( t ) );
preflow.run();

int nodes = countNodes( split ) , arcs = countArcs( split );
std::cout << "node-disjoint paths: " << preflow.flowValue() << " (split "
          << "digraph of " << nodes << " nodes and " << arcs << " arcs)"
          << std::endl;
return( ( preflow.flowValue() == 2 && nodes == 12 && arcs == 14 ) ? 0 : 1 );
}
```

5.6 Exercises

Exercise 5.1. Write the type of the adaptor that presents the subgraph of a `ListDigraph` induced by a set of nodes, with the arcs reversed, to an algorithm that takes a `const` digraph. Say which of its operations cost more than the corresponding operation of the digraph. The functions `checkReverseDigraph()` and `checkSubDigrap h()` of `test/adaptors_test.cc` check the two adaptors separately.

Exercise 5.2. Modify `examples/05-residual-augment.cc` so that each augmentation uses a path of largest bottleneck capacity (a "widest" path) instead of a shortest one, and compare the number of augmentations on a digraph with "large" capacities. The value must still agree with `Preflow`.

Exercise 5.3. Prove that, at the end of Algorithm 5.1, the cut given by the nodes reached by the last search has capacity equal to the value of the flow (Theorem 5.1). The example `examples/05-residual-augment.cc` computes both and checks it.

Exercise 5.4. Starting from `examples/05-split-nodes.cc`, use `SplitNodes` and a minimum cost flow algorithm (cf. chapter 13) to find k paths from s to t with no inner node in common and of least total length.

Exercise 5.5. Given an undirected graph and an orientation of its edges in a `bool` edge map, use `Orienter` and an algorithm for the strongly connected components (cf. chapter 18) to decide whether the orientation is strongly connected. Then reverse one arc by `reverseArc()` and decide again, without rebuilding anything; the function `checkOrienter()` of `test/adaptors_test.cc` checks `reverseArc()`.

6. Maps

We are concerned in this chapter with the maps of the library, i.e., with the objects that associate a value with a key. Since the graphs store no data of their own, maps are the only way in which data are attached to the nodes, arcs and edges of a graph (cf. chapter 3). A map is any type that offers the types `Key` and `Value` together with an `operator[]` (to read) or a `set()` (to write); hence, besides the standard graph maps, which store a value for each item, one can pass to an algorithm a map that computes its values on demand, from other maps, from a function or from the graph. Section 6.1 gives the concepts and section 6.2 the standard graph maps, as a user sees them. Sections 6.3 to 6.6 describe the classes of `lemon/maps.h` (the basic maps, the map adaptors, the maps tied to a graph and the iterable maps), section 6.7 the functions that scan a map, and section 6.8 collects the exercises.

6.1 The map concepts

The header `lemon/concepts/maps.h` defines four concepts, as archetype classes with a nested `Constraints` template that `checkConcept()` instantiates on a candidate type (cf. chapter 3). `concepts::ReadMap< K, T >` requires the types `Key` and `Value` and the member `Value operator[]( const Key & ) const`. `concepts::WriteMap< K, T >` requires the same types and `void set( const Key &, const Value & )`, and `concepts::ReadWriteMap< K, T >` is both; finally, `concepts::ReferenceMap< K, T, R, CR >` is a read-write map whose `operator[]` returns a `Reference` (`R`), and a `ConstReference` (`CR`) on a constant map. It declares the tag `typedef True ReferenceMapTag`, by which the library recognizes it. Note that an algorithm reads a map only through `operator[]` and `set()`, and the result of `operator[]` need not be a reference (in fact, the adaptors of section 6.4 return their values by copy).

Two helpers make writing a map short. `MapBase< K, V >` is an empty base class that only defines `Key` and `Value`, and the classes of `lemon/maps.h` derive from it. `MapTraits< M >`, in `lemon/bits/traits.h`, gives the `ReferenceMapTag` of any map type and the types `ReturnValue` and `ConstReturnValue` that the map returns. These are references for a reference map and values otherwise, and the adaptors use them to forward the reference of the map they wrap.

> **abLEMON.** The archetypes of `lemon/concepts/maps.h` return a static buffer of the size and alignment of the value, instead of dereferencing the address `sizeof(Value)` as LEMON 1.3.1 did (which is undefined behavior, and which the optimizer flagged as an access out of bounds). The concepts are unchanged.

6.2 Graph maps

Each graph structure of chapter 4 has the nested templates `NodeMap< V >`, `ArcMap< V >` and, if it is undirected, `EdgeMap< V >` (a bipartite graph also has `RedNodeMap< V >` and `BlueNodeMap< V >`). They are reference maps, built by `NodeMap( const Graph & g )`, which value-initializes the values, or by `NodeMap( const Graph & g, const V & value )`. The graph keeps them up to date, since each map registers itself as an observer of the alteration notifier of the graph (cf. appendix A), and it is told of each item added or erased. A key added after the construction of the map receives the value `V()`, not the value given to the constructor, and the value of an erased key is lost. The assignment operators of the standard maps are private; one copies the values of a map into another by `mapCopy()` (section 6.7).

The storage is chosen by `DefaultMap`, in `lemon/bits/default_map.h`. For `bool`, the character types, the integer types, the floating-point types and the pointers, it is a `VectorMap`, i.e., a `std::vector< V >` indexed by the identifier of the item. For any other value type it is an `ArrayMap`, an array allocated through `std::allocator`, whose capacity doubles (up to the first power of two above the largest identifier) when an item with a larger identifier is added. In both cases a lookup costs $O(1)$ and adding an item costs $O(1)$ amortized, while the

memory is proportional to the largest identifier plus one, which may exceed the number of items after some erasures.

Each map also has three iterators, inherited from `MapExtender` (in `lemon/bits/map_extender.h`). `MapIt` iterates on the items and dereferences to a reference to their value, `ConstMapIt` dereferences to a constant reference, and `ItemIt` only iterates on the items. The test `test/maps_test.cc` uses `MapIt` and `ConstMapIt` on the maps of a `ListDigraph`.

abLEMON. `ArrayMap` constructs and destroys its values through `std::allocator_traits`, since C++20 removed the members `construct()` and `destroy()` of `std::allocator`. When it grows, it moves the values into the larger buffer by `std::move_if_noexcept()`, rather than copying them.

6.3 Basic maps

The basic maps of `lemon/maps.h` need no graph. `NullMap< K, V >` reads `V()` at each key and absorbs whatever is written to it. It is the map to pass (in place of a real one) when an algorithm requires a map whose content is not wanted (e.g., a predecessor map that the caller does not read). `ConstMap< K, V >` reads one value at each key, given to the constructor (`ConstMap( const V & v )`) or default-constructed, and it also absorbs the writes; `setAll(v)` changes the value. Its specialization `ConstMap< K, Const< V, v > >` carries the value in its type, as the template argument of the tag `Const< T, v >`, so that the compiler may fold the read. The filtering adaptors of chapter 5, e.g., use `ConstMap< Node, Const< bool, true > >` in place of the filter they lack. `TrueMap< K >` and `FalseMap< K >` are the read-only constant maps of `true` and `false`, and `IdentityMap< T >` reads the key (its key and value types are both `T`).

`RangeMap< V >` is a `std::vector< V >` with integer keys in $[0, \text{size})$. It is built from a size and a value (`RangeMap( int size = 0, const V & value = V() )`), from a `std::vector`, or from another `RangeMap` whose value type converts to `V`, and `resize(size, value)` changes its key set. It is a reference map, typically used as the cross-reference map of a heap whose items are small integers (cf. chapter 7).

`SparseMap< K, V, Comp >` is a `std::map< K, V, Comp >`, with `Comp = std::less< K >` by default, together with a default value for the keys that are not stored. Its reads and writes cost $O(\log s)$, where s is the number of stored keys. One remark is in order here: the non-constant `operator[]`, which returns a reference, inserts an absent key with the default value, while the constant one does not. `setAll(v)` changes the default value and erases the stored keys, and the class is not assignable (its copy assignment is private). Each of these classes has a creator function with the name of the class in lower case (`nullMap()`, `constMap()`, `identityMap()`, `rangeMap()`, `sparseMap()`, `trueMap()` and `falseMap()`).

`LoggerBoolMap< IT, KEY >` is a write-only `bool` map. It copies each key set to `true` to the output iterator `IT`, in the order of the writes, and it ignores the keys set to `false`; `begin()` and `end()` return the first iterator and the one after the last key written. The map is meant for the algorithms that report a set by writing `true` once per item. For instance, `dfs( g ).processedMap( loggerBoolMap( std::back_inserter( v ) ) ).run( s )` stores the nodes in the order in which the search processes them. The key type `KEY` is deduced from the iterator (from its `value_type`, or from that of its container for an insert iterator), and the iterator must either be an inserter or point to enough room. Being only writable, a `LoggerBoolMap` cannot serve where a readable map is required, e.g., as the reached map of `Bfs`, `Dfs` or `Dijkstra`.

6.4 Map adaptors and arithmetic

A map adaptor is a map whose values are computed, at each read, from one or two other maps. It stores references to the maps it combines (or copies of the constants and functors it uses) and nothing else. Hence, building it costs $O(1)$, and a read costs one or two reads of the underlying maps (which may, in turn, be adaptors).

Two remarks apply to all the adaptors. First, they keep the maps by reference, and the maps must therefore outlive them: a temporary map passed to a creator function (e.g., `addMap( m, constMap< Node, int >( 1 ) )`) is destroyed at the end of the full expression, and an adaptor that refers to it cannot be stored. Second, most adaptors are read-only, since a write to a computed value has, in general, no meaning; the exceptions are said explicitly. Table 6.1 lists them, with `m`, `m1` and `m2` the adapted maps, `x` a key, `v` a constant and `f` a functor. Each class has a creator function with the name of the class in lower case (`composeMap()`, `addMap()`, `notWriteMap()`, and so on).

Table 6.1. The map adaptors of `lemon/maps.h`: the value at a key and, when the adaptor is writable, the effect of `set(x, y)`.

Class	Value at `x`	Writes	Key and value types
`ComposeMap< M1, M2 >`	`m1[m2[x]]`	no	key of `M2`, value of `M1`
`CombineMap< M1, M2, F, V >`	`f(m1[x], m2[x])`	no	key of `M1`, value `V`
`FunctorToMap< F, K, V >`	`f(x)`	no	`K`, `V`
`MapToFunctor< M >`	`m[x]`, also as `operator()`	no	those of `M`
`ConvertMap< M, V >`	`m[x]` converted to `V`	no	key of `M`, `V`
`ForkMap< M1, M2 >`	`m1[x]`	to `m1` and `m2`	those of `M1`
`AddMap`, `SubMap`, `MulMap`, `DivMap< M1, M2 >`	`m1[x]` ∘ `m2[x]`	no	those of `M1`
`ShiftMap< M, C >`	`m[x] + v`	no	those of `M`
`ShiftWriteMap< M, C >`	`m[x] + v`	`m.set(x, y - v)`	those of `M`
`ScaleMap< M, C >`	`v * m[x]`	no	those of `M`
`ScaleWriteMap< M, C >`	`v * m[x]`	`m.set(x, y / v)`	those of `M`
`NegMap< M >`	`-m[x]`	no	those of `M`
`NegWriteMap< M >`	`-m[x]`	`m.set(x, -y)`	those of `M`
`AbsMap< M >`	`m[x]` if `m[x] >= 0`, else `-m[x]`	no	those of `M`
`AndMap`, `OrMap< M1, M2 >`	`m1[x] && m2[x]`, `m1[x] || m2[x]`	no	key of `M1`, `bool`
`NotMap< M >`	`!m[x]`	no	key of `M`, `bool`
`NotWriteMap< M >`	`!m[x]`	`m.set(x, !y)`	key of `M`, `bool`
`EqualMap`, `LessMap< M1, M2 >`	`m1[x] == m2[x]`, `m1[x] < m2[x]`	no	key of `M1`, `bool`

In the four arithmetic adaptors ∘ is `+`, `-`, `*` and `/`, and the key and value types of `M2` must convert to those of `M1`. The constant type `C` of the shift and scale adaptors defaults to the value type of `M`. `ScaleWriteMap` can be written only when the constant is not zero and `/` is defined between the value and `C`. `ComposeMap` returns its value with the `ConstReturnValue` of `M1` (by reference, when `M1` is a reference map), while the others return by value.

`FunctorToMap` takes its key and value types from the members `argument_type` and `result_type` of the functor when they are not given. Since lambdas (and most modern functors) lack these members, `K` and `V` must then be given explicitly, as in `functorToMap< int, double >( f )`; for a plain function pointer `V (*)(K)`, an overload of `functorToMap()` deduces them. Conversely, `MapToFunctor` defines `argument_type` and `result_type`, for the code that still uses them.

One passes the adaptors to the algorithms like any other map. When the algorithm is a class, its map type must be named, which the creator functions do not do; in C++11 and later, `decltype` names it. In the example below, the travel time of an arc is the quotient of a length map and a speed map, read by `Dijkstra` through a `DivMap` and never stored.

> **abLEMON.** The value type of `CombineMap` defaults to the type that the functor returns on the values of the two maps, deduced by `decltype`. LEMON 1.3.1 read the member `result_type` of the functor instead, which C++20 removed from the function objects of `<functional>` and which lambdas never had; thus, `combineMap ( m1, m2, f )` now accepts a lambda.

```
// Map adaptors (section 6.4) and map utilities (section 6.7): the travel
// time of an arc as the quotient of two arc maps, passed to Dijkstra
// without being stored; a combination by a lambda, whose value type is
// deduced; and the utilities that scan a map.

#include <iostream>
#include <lemon/dijkstra.h>
#include <lemon/list_graph.h>
#include <lemon/maps.h>

using namespace lemon;

int main()
{
```

```
typedef ListDigraph::ArcMap< double > DoubleArcMap;
ListDigraph g;
ListDigraph::Node s = g.addNode() , a = g.addNode() , b = g.addNode() ,
                  t = g.addNode();
DoubleArcMap length( g ) , speed( g );
ListDigraph::Arc sa = g.addArc( s , a ) , sb = g.addArc( s , b ) ,
                 at = g.addArc( a , t ) , bt = g.addArc( b , t );
length[ sa ] = 10;
speed[ sa ] = 5;  // 2 hours
length[ sb ] = 6;
speed[ sb ] = 1;  // 6 hours
length[ at ] = 30;
speed[ at ] = 10;  // 3 hours
length[ bt ] = 2;
speed[ bt ] = 2;  // 1 hour

// the time of an arc, computed when Dijkstra reads it
typedef DivMap< DoubleArcMap , DoubleArcMap > TimeMap;
TimeMap time( length , speed );
Dijkstra< ListDigraph , TimeMap > dijkstra( g , time );
dijkstra.run( s , t );
double fastest = dijkstra.dist( t );  // 5, through a

// the same time plus a fixed delay of one hour per arc, by a lambda: the
// value type of the CombineMap is the type that the lambda returns
auto delayed = combineMap(
  length , speed , []( double l , double v ) { return( l / v + 1 ); } );
Dijkstra< ListDigraph , decltype( delayed ) > dijkstra2( g , delayed );
dijkstra2.run( s , t );
double with_delay = dijkstra2.dist( t );  // 7, through a

// the utilities: the slowest arc, the arcs of speed below 5, a copy
ListDigraph::Arc slowest = mapMin( g , speed );
int slow = mapCountIf( g , speed , []( double v ) { return( v < 5 ); } );
DoubleArcMap copy( g );
mapCopy( g , time , copy );
bool same = mapCompare( g , copy , time );
double longest = mapMaxValue( g , time );  // 6, the arc (s, b)

std::cout << "fastest " << fastest << ", with delays " << with_delay
          << ", slow arcs " << slow << ", longest time " << longest
          << std::endl;
return( ( fastest == 5 && with_delay == 7 && slowest == sb && slow == 2 &&
           same && longest == 6 )
          ? 0
          : 1 );
}
```

6.5 Maps of the items of a graph

The maps of this section are tied to a graph, whose type `GR` is their first template parameter, and most of them take the item type `K` (`GR::Node`, `GR::Arc` or `GR::Edge`) as the second. They are of two kinds. Views compute their value from the graph (and possibly from a map) at each read, while look-up maps store a value per item together with an inverse structure that finds the items by their value.

Views. `IdMap< GR, K >` reads the identifier `id()` of an item. This is unique among the items of its kind, and it does not change while the item exists (even if other items are erased). Its `operator()(id)`, and its nested `InverseMap` (returned by `inverse()`), give the item with a given identifier, through `fromId()` of the graph. `SourceMap< GR >` and `TargetMap< GR >` read the source and the target of an arc, while `ForwardMap< GR >` and `BackwardMap< GR >` read the arcs `direct(e, true)` and `direct(e, false)` of an edge of an undirected graph. `PotentialDifferenceMap< GR, POT >` reads, at an arc (i, j), the difference $\pi_j - \pi_i$ of a node map of potentials π; for a zero cost, it is the reduced cost of the arc up to the sign convention (cf. chapter 13). Each view has a creator function, i.e., `idMap< K >( g )`, `sourceMap( g )`, `targetMap( g )`, `forwardMap( g )`, `backwardMap(`

`g )` and `potentialDifferenceMap( g, pot )`.

Degree maps. `InDegMap< GR >` and `OutDegMap< GR >` read the in-degree and the out-degree of a node of a digraph in $O(1)$. They compute the degrees at construction, in $O(n+m)$ time, and store them in a node map; then they observe the arcs of the digraph, and update the degrees of the ends of each arc added or erased. The update happens only through the notifications of the graph; hence, as the header warns, the functions that change an arc without adding or erasing it (e.g., `changeSource()`, `changeTarget()` and `reverseArc()` of `ListDigraph`) leave the degrees wrong.

Look-up maps. `RangeIdMap< GR, K >` gives each item of its kind an identifier in $[0, n_K)$, where n_K is the number of items of the kind, and it keeps the inverse in a `std::vector`. Hence, `operator()(id)` (and `InverseMap::operator[]`) return the item with a given range identifier in $O(1)$, and `size()` returns n_K. The identifiers follow the order of the items at construction, and a new item takes the next one. When an item is erased, the item with the largest identifier takes its identifier; the range thus stays contiguous, but the identifiers may change (and they are usually not those of `IdMap`). Furthermore, `swap(p, q)` exchanges the identifiers of two items, by which one can impose any order.

`CrossRefMap< GR, K, V >` stores a value for each item, and it keeps a `std::multimap` from the values to the items. Its `set(key, val)` costs $O(c \log n)$, where c is the number of items with the old value of the key, since the loop that looks for the key among them recomputes the range of the old value at each step. `operator()(val)`, or the `InverseMap`, returns one item with the value (or `INVALID`) in $O(\log n)$. Also, `count(val)` returns the number of items with a value, and `beginValue()` and `endValue()` delimit a forward iterator (`ValueIt`) on the values, with multiplicity and in increasing order. We remark that the inverse structure holds only the values written by `set()`. On a newly built map, which reads `V()` at each item, `count( V() )` is therefore zero. The map is meant for values that are all different, or nearly so; when many items share a value, `IterableValueMap` (section 6.6) is usually the better choice. Neither `CrossRefMap` nor `IterableValueMap` is a reference map, i.e., their values are written only by `set()`.

abLEMON. The iterators `ValueIt` of `CrossRefMap` and `IterableValueMap` define their member types, instead of deriving from `std::iterator`, which C++17 deprecated.

6.6 Iterable maps

An iterable map stores a value for each item of a graph. Besides the reads and writes of a map, it lists the items that have a given value, in time proportional to their number. `IterableBoolMap` and `IterableIntMap` are reference maps, whose `operator[]` returns a proxy object that writes through `set()`; `IterableValueMap` is a read-write map, and the three classes are kept up to date when items are added to or erased from the graph.

`IterableBoolMap< GR, K >` keeps the items in one vector, together with the position of each item, and the items mapped to `true` are in the positions before a separator, and those mapped to `false` after it (Figure 6.1). `set(key, value)` moves the item across the separator, in $O(1)$, by swapping it with the first item of the other side; `setAll(value)` moves only the separator, also in $O(1)$, and `trueNum()` and `falseNum()` count the two sides in $O(1)$. The iterators `TrueIt`, `FalseIt` and `ItemIt(map, value)` list the items of one side in time proportional to their number, and so do the ranges `trueKeys()`, `falseKeys()` and `items(value)` of the range-based `for` loops. The constructor `IterableBoolMap( const GR & g, bool def = false )` maps all the items to `def`, and an item added later is mapped to `false`.

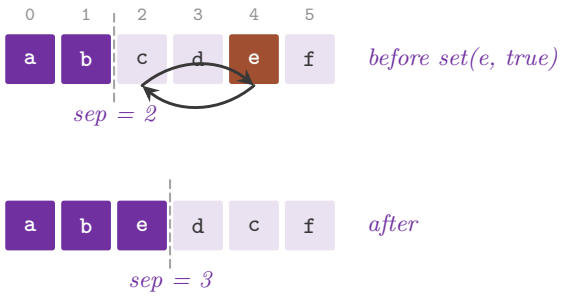


Figure 6.1. IterableBoolMap: the items in one vector, the true ones before the separator; set(e, true) swaps e with the first false item and moves the separator.

`IterableIntMap< GR, K >` stores an integer per item, and it keeps a doubly linked list of the items for each

non-negative value, with the heads in a vector indexed by the value. Thus, `set(key, value)` unlinks the item from the list of its old value and links it at the head of the list of the new one; this costs $O(1)$, apart from the changes of the vector of heads, which grows to the new value plus one when the value exceeds its size, and shrinks when the lists of the largest values become empty. `size()` returns the largest value plus one, and `ItemIt(map, value)` and `items(value)` list the items with a value. An item with a negative value is in no list, and it cannot be listed. The constructor without a value sets all the values to -1, the one with a value sets them to it, and an item added later reads -1. The memory is $O(n_K + w)$, where w is the largest value; the map is therefore meant for rather "small" values. The proxy returned by `operator[]` offers the integer operators (`++`, `--`, `+=`, `-=`, `*=`, and so on), each of which goes through `set()`.

`IterableValueMap< GR, K, V >` does the same for any value type with an `operator<`, and it keeps a linked list per value, with the heads in a `std::map< V, K >` that holds only the values in use. Hence, `set(key, value)` and `ItemIt(map, value)` cost $O(\log d)$, where d is the number of distinct values, and `beginValue()` and `endValue()` delimit a forward iterator on the distinct values, in increasing order. The constructor maps all the items to a value (by default `V()`), and an item added later reads `V()`.

abLEMON. The nodes of the lists of `IterableIntMap` and `IterableValueMap` initialize their links to `INVALID` when they are created, which LEMON 1.3.1 left indeterminate.

The records of the lists are the structures `IterableIntMapNode` and `IterableValueMapNode` of the namespace `_maps_bits` (which also holds `IteratorTraits`, by which `LoggerBoolMap` deduces its key type, and `CombineValue`), and the degree maps of section 6.5 keep their degrees in a private node map, `AutoNodeMap`, which sets the degree of a new node to zero; none of these is meant to be used directly.

The iterable integer map is the natural data structure for the buckets of degrees of the smallest-last ordering of Matula and Beck 1983. This is the order obtained by removing, one at a time, a node of least degree in the graph that remains. The largest degree met at a removal is the degeneracy of the graph, and the running maximum of these degrees gives the core number of each node, i.e., the largest k such that the node belongs to a subgraph of minimum degree k.

Algorithm 6.1 (Smallest-last ordering and core numbers by an iterable integer map).

```
input:  graph G = (V, E)
deg := IterableIntMap on V, deg[v] := degree of v
k := 0;  d := 0
repeat |V| times
  while ItemIt(deg, d) == INVALID: d := d + 1
  u := ItemIt(deg, d)                    (a node of least current degree)
  k := max(k, d);  core[u] := k;  append u to the order
  deg.set(u, -1)                         (u leaves all the lists)
  for each edge (u, w) with deg[w] > 0: deg.set(w, deg[w] - 1)
  d := max(d - 1, 0)
output: the order (reversed, the smallest-last order), core, k
```

Theorem 6.1. *Algorithm 6.1 computes the core numbers and the degeneracy k of G in $O(|V| + |E|)$ time.*

The correctness is proved in Matula and Beck 1983. As for the time, each `set()` costs $O(1)$, since the values only decrease after the first assignment and the vector of heads no longer grows, and there are $O(|V| + |E|)$ of them. Also, the least current degree drops by at most one per removal, which justifies the last step and bounds the increments of d by $O(|V|)$ plus the largest degree. The example below implements Algorithm 6.1 on a small graph, and then it marks the nodes of the largest core in an `IterableBoolMap`, whose `trueNum()` gives their number and whose `TrueIt` lists them.

```
// Iterable maps (section 6.6): the smallest-last ordering and the core
// number of each node of a graph (Algorithm 6.1), with the nodes kept in
// buckets by their current degree in an IterableIntMap, in O(n + m) time;
// an IterableBoolMap then gives the nodes of the largest core.

#include <iostream>
#include <vector>
#include <lemon/list_graph.h>
#include <lemon/maps.h>

using namespace lemon;
```

```
int main()
{
 ListGraph g;
 std::vector< ListGraph::Node > v( 7 );
 for( int i = 0 ; i < 7 ; ++i )
  v[ i ] = g.addNode();
 // a clique on 0, 1, 2, 3, the path 3 - 4 - 5 and the edge 5 - 1, and the
 // pendant node 6 on 4
 for( int i = 0 ; i < 4 ; ++i )
  for( int j = i + 1 ; j < 4 ; ++j )
   g.addEdge( v[ i ] , v[ j ] );
 g.addEdge( v[ 3 ] , v[ 4 ] );
 g.addEdge( v[ 4 ] , v[ 5 ] );
 g.addEdge( v[ 5 ] , v[ 1 ] );
 g.addEdge( v[ 4 ] , v[ 6 ] );

 // the current degree of the nodes not yet removed; -1 for the removed
 // ones, which IterableIntMap leaves out of its lists
 IterableIntMap< ListGraph , ListGraph::Node > deg( g );
 for( ListGraph::NodeIt n( g ) ; n != INVALID ; ++n ) {
  int d = 0;
  for( ListGraph::IncEdgeIt e( g , n ) ; e != INVALID ; ++e )
   ++d;
  deg.set( n , d );
  }

 ListGraph::NodeMap< int > core( g );
 std::vector< ListGraph::Node > order;  // the smallest-last order, reversed
 int k = 0 , d = 0;
 for( int removed = 0 ; removed < countNodes( g ) ; ++removed ) {
  while( IterableIntMap< ListGraph , ListGraph::Node >::ItemIt( deg , d ) ==
         INVALID )
   ++d;
  ListGraph::Node u =
    IterableIntMap< ListGraph , ListGraph::Node >::ItemIt( deg , d );
  if( d > k )
   k = d;
  core[ u ] = k;
  order.push_back( u );
  deg.set( u , -1 );
  for( ListGraph::IncEdgeIt e( g , u ) ; e != INVALID ; ++e ) {
   ListGraph::Node w = g.runningNode( e );
   if( deg[ w ] > 0 )
    deg.set( w , deg[ w ] - 1 );
   }
  if( d > 0 )
   --d;  // a degree drops by at most one per removal
  }

 // the nodes of the largest core, i.e., of core number k
 IterableBoolMap< ListGraph , ListGraph::Node > top( g , false );
 for( ListGraph::NodeIt n( g ) ; n != INVALID ; ++n )
  top[ n ] = core[ n ] == k;

 std::cout << "degeneracy " << k << ", nodes in the " << k
           << "-core: " << top.trueNum()
           << ", first removed: " << g.id( order[ 0 ] ) << std::endl;
 bool ok = k == 3 && top.trueNum() == 4 && core[ v[ 4 ] ] == 2 &&
           core[ v[ 5 ] ] == 2 && core[ v[ 6 ] ] == 1 && order[ 0 ] == v[ 6 ];
 for( IterableBoolMap< ListGraph , ListGraph::Node >::TrueIt n( top ) ;
      n != INVALID ; ++n )
  ok = ok && g.id( n ) < 4;
 return( ok ? 0 : 1 );
 }
```

6.7 Map utilities

The functions at the end of `lemon/maps.h` scan the items of one kind of a graph, the kind given by the key type of the map, and use the values of a map. Each of them costs one pass over the items, i.e., $O(n_K)$ reads of the map, plus the cost of the predicate or of the comparison.

- `mapCopy(g, from, to)` writes `from[x]` into `to` for each item `x` of the kind of `to`. An edge map can thus be copied to an arc map of the same graph, since an arc converts to its edge, but not vice versa. `mapFill( g, map, val)` writes `val` at each item.
- `mapCompare(g, m1, m2)` returns `true` if the two maps agree, by `operator==`, at each item of the kind of `m2`.
- `mapMin(g, map)` and `mapMax(g, map)` return an item of least and of largest value, or `INVALID` on an empty item set. When there are ties (which may well happen on integer data), they return the first one met in the order of the item iterator. The variants with a third argument take a comparison functor in place of `std::less< V >`. `mapMinValue()` and `mapMaxValue()` return the value, and they require a non-empty item set.
- `mapFind(g, map, val)` and `mapFindIf(g, map, pred)` return the first item whose value equals `val` (or satisfies `pred`), and `INVALID` if there is none. `mapCount(g, map, val)` and `mapCountIf(g, map, pred)` return the number of such items.

The example of section 6.4 uses `mapMin()`, `mapCountIf()`, `mapCopy()`, `mapCompare()` and `mapMaxValue()`, and the test `test/maps_test.cc` checks all of them.

6.8 Exercises

Exercise 6.1. Write a map class, derived from `MapBase`, whose value at an arc is the Euclidean length of the arc, computed from a node map of coordinates. Check that it conforms to `concepts::ReadMap` by `checkConcept()`. Then obtain the same map without writing a class, from `SourceMap`, `TargetMap`, `ComposeMap` and `CombineMap`.

Exercise 6.2. In `examples/06-map-adaptors.cc`, replace the `DivMap` by a `CombineMap` with a lambda, and the delay by a `ShiftMap` of the `DivMap`; check that the distances do not change.

Exercise 6.3. Explain why the declaration `AddMap< M, ConstMap< K, int > > m = addMap( length, constMap< K, int >( 1 ) );` compiles, while a later read of `m` has undefined behavior, and correct it.

Exercise 6.4. Modify `examples/06-degeneracy.cc` so that it colors the graph greedily in the smallest-last order, with at most $k + 1$ colors, and check the coloring.

Exercise 6.5. Use a `RangeIdMap` and its `swap()` to number the nodes of an acyclic digraph in a topological order, and check that each arc goes from a smaller to a larger range identifier. The test `test/maps_test.cc` checks `swap()` after an erasure.

7. Heaps

We are concerned in this chapter with the heaps of the library, i.e., with the implementations of the abstract data type priority queue, which the algorithms of Part III use to choose the next node to scan. A heap stores a set of items, each with a priority. It gives the item of least priority, removes it, and changes the priority of an item in the heap. The cost of these operations differs (sometimes rather markedly) from one structure to another; hence, the best choice depends on the mix of operations of the algorithm and on the type of the priorities. Section 7.1 gives the interface common to all the heaps (the concept `concepts::Heap`). Sections 7.2 to 7.4 describe the three families of implementations, i.e., the implicit d-ary heaps, the linked heaps and the heaps of integer priorities. Section 7.5 compares them and shows how an algorithm is given a heap, and section 7.6 collects the exercises.

7.1 The heap concept

The concept `concepts::Heap< PR, IM, CMP >` of `lemon/concepts/heap.h` describes the interface. `PR` is the type of the priorities, `IM` a read-write map from the items to `int`, called the cross-reference map, and `CMP` a functor that compares two priorities, by default `std::less< PR >`. The type of the items is `IM::Key`; thus, the items are whatever the map takes as keys (nodes, arcs, or integers with a `RangeMap`). Note that the heap uses the cross-reference map to find an item in its storage. While an item is in the heap, the map holds its position (a non-negative integer), and otherwise one of the states of the enum `State`. These are `PRE_HEAP` (-1), if the item has never been in the heap, and `POST_HEAP` (-2), if it has been in the heap and has left it; `IN_HEAP` (0) is the value returned by `state()` for an item in the heap. The map must hold `PRE_HEAP` for each item before the first insertion, which is the precondition of the constructors `explicit Heap( IM & map )` and `Heap( IM & map, const CMP & comp )`. Since the heap keeps a reference to the map, the map must also outlive it.

The operations are the following, where n is the number of items in the heap; Table 7.1 gives their costs.

- `size()` returns the number of items, and `empty()` returns whether it is zero.
- `push(i, p)` inserts the item `i`, which must not be in the heap, with priority `p`.
- `top()` and `prio()` return the item of least priority and its priority (the heap must not be empty). With `std::greater` as `CMP`, they return the item of largest priority.
- `pop()` removes the item of least priority, whose state becomes `POST_HEAP`, and `erase(i)` removes the item `i`, which must be in the heap.
- `operator[](i)` returns the priority of the item `i`, which must be in the heap.
- `decrease(i, p)` and `increase(i, p)` change the priority of an item in the heap to `p`, which must not be larger (respectively, smaller) than the current one under `CMP`. The heaps do not check it.
- `set(i, p)` changes the priority of `i` if it is in the heap, by `decrease()` or `increase()` as the case may be, and it inserts `i` otherwise (also when it is `POST_HEAP`).
- `state(i)` returns the state of an item. `state(i, st)`, with `st` equal to `PRE_HEAP` or `POST_HEAP`, erases the item if it is in the heap and then writes `st` into the map (`IN_HEAP` is ignored).
- `clear()` empties the heap without touching the map. To reuse the heap on the same items, the caller must also reset the map to `PRE_HEAP`.

Two remarks on the cross-reference map are in order. First, although the concept requires only a read-write map, several heaps write it through `operator[]`, e.g., `state(i, st)` in all of them, `decrease()` of `BinomialHeap`, and `push()` and `pop()` of `RadixHeap` and `BucketHeap`. They need, therefore, a reference map; the maps used in practice (a graph map of `int` or a `RangeMap< int >`) are reference maps. Second, the state `POST_HEAP` allows an algorithm such as `Dijkstra` to distinguish a scanned node from an unreached one by one read of the map, without a further map.

Besides the concept, `BinHeap`, `QuadHeap`, `DHeap`, `BucketHeap` and `SimpleBucketHeap` have `push(pair)`, which

takes a `std::pair< Item, Prio >`. Also, `BinHeap`, `QuadHeap` and `DHeap` have `replace(i, j)`, which gives the place and the priority of the item `i` (in the heap) to the item `j` (not in the heap), in $O(1)$. The test `test/heap_test.cc` checks each heap against the concept, by `checkConcept()`, and then by sorting, by increasing priorities, and inside `Dijkstra`.

7.2 Implicit d-ary heaps

A d-ary heap (Johnson 1975) is a complete d-ary tree, stored level by level in a vector, whose items satisfy the heap order: the priority of an item is not smaller than that of its parent. With positions from 0, the parent of position i is $\lfloor (i-1)/d \rfloor$, and its children are $di+1, \ldots, di+d$. The binary heap of Williams 1964 is the case $d = 2$ (Figure 7.1). The cross-reference map holds the position of each item, and each move of an item updates it.

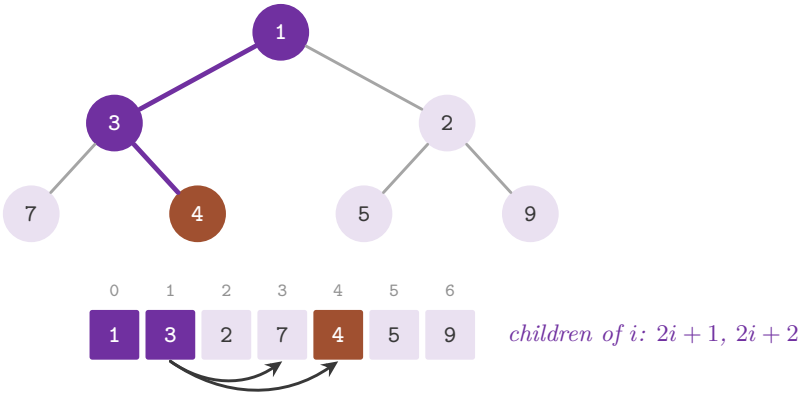


Figure 7.1. A binary heap as a tree and as the vector that stores it; a decrease of the item in position 4 moves it along the path to the root, here through the positions 1 and 0.

All the operations reduce to the two procedures of Algorithm 7.1, which move an item along a path of the tree until the heap order holds again.

Algorithm 7.1 (Sifting in a d-ary heap).

```
bubbleUp(hole, (i, p)):           (the item i with priority p goes up)
  while hole > 0 and p < prio(parent(hole)):
    move the item of parent(hole) to hole;  hole := parent(hole)
  put (i, p) at hole
bubbleDown(hole, (i, p), n):      (the item i with priority p goes down)
  while hole has children among the positions 0 .. n-1:
    c := the child of least priority
    if not prio(c) < p: break
    move the item of c to hole;  hole := c
  put (i, p) at hole
push(i, p):    append a position n; bubbleUp(n, (i, p))
pop():         mark top POST_HEAP; bubbleDown(0, last item, n - 1)
decrease(i,p): bubbleUp(position of i, (i, p))
increase(i,p): bubbleDown(position of i, (i, p), n)
erase(i):      bubbleUp the last item into the position of i, then,
               if it did not move, bubbleDown it
```

Each `move` writes the new position into the cross-reference map.

A path from the root to a leaf has $\lceil \log_d(n(d-1)+1) \rceil$ levels. Hence, `bubbleUp()` costs $O(\log_d n)$ comparisons, while `bubbleDown()` costs $O(d \log_d n)$, since it compares the d children at each level. The degree d thus trades the cost of `push()` and `decrease()` for that of `pop()`. In an algorithm with many decreases per removal, as `Dijkstra` on a dense digraph (with up to m decreases and n removals), a degree near $\max(2, m/n)$ balances the two (Johnson 1975). Of course, a larger degree also means fewer levels and fewer cache misses, which in practice often (but not always) favours $d = 4$ over $d = 2$, even when the counts of comparisons do not.

The library has three classes of this family, all with the template parameters `PR`, `IM` and `CMP` (by default `std::less< PR >`) and with both constructors of the concept.

- `BinHeap< PR, IM, CMP >`, in `lemon/bin_heap.h`, is the binary heap. It is the default heap of `Dijkstra`, `BidirDijkstra`, `AStar` and `MinCostArborescence`, and the heap of the matching algorithms.
- `QuadHeap< PR, IM, CMP >`, in `lemon/quad_heap.h`, is the 4-ary heap (the header calls it "fourary"), whose four children are compared by unrolled code.
- `DHeap< PR, IM, D, CMP >`, in `lemon/dheap.h`, is the d-ary heap, with the degree as the template parameter `D`, by default 16. The header suggests a power of two, which turns the divisions and multiplications of the positions into shifts.

abLEMON. `CapacityScaling` (cf. chapter 13) keeps the nodes of its shortest path searches in a `QuadHeap`, where LEMON 1.3.1 used a `BinHeap`.

7.3 Linked heaps

The heaps of this section store the items in heap-ordered trees, linked by indices. Two trees are joined in $O(1)$ by making the root of larger priority a child of the other. Each of the three classes keeps one record per item in a vector, indexed by the cross-reference map. The record stays after the item leaves the heap (a flag records whether it is in), and it is reused when the item comes back; hence, the memory is proportional to the number of distinct items ever pushed since the last `clear()`. The three classes have the template parameters `PR`, `IM` and `CMP` and both constructors of the concept.

Fibonacci heaps. `FibHeap< PR, IM, CMP >`, in `lemon/fib_heap.h`, is the heap of Fredman and Tarjan 1987. The roots of its trees form a circular list, with a pointer to a root of least priority. `push()` adds a root, and `decrease()` cuts the item from its parent when the heap order breaks. The parent, if it had already lost a child, is cut in turn (the cascading cut), and so on upwards. `pop()` removes the minimum, moves its children to the list of the roots, and then consolidates the roots, linking two roots of equal degree until all the degrees differ. This bounds the degree of a node by $\log_\phi n$, with $\phi = (1+\sqrt{5})/2$; in fact, the code sizes its array of degrees by $\lfloor 2.08 \ln N \rfloor + 1$, with N the number of records, since $1/\ln\phi \approx 2.078$. Furthermore, `erase(i)` cuts the item, makes it the minimum (in the words of a comment of the code, "as if its prio would be -infinity"), and pops it, and `increase()` is `erase()` followed by `push()`.

Algorithm 7.2 (Decrease and pop in a Fibonacci heap).

```
decrease(i, p):
  prio(i) := p;  q := parent(i)
  if q exists and p < prio(q):
    cut(i, q)                       (i becomes a root, unmarked)
    while q is not a root:          (cascading cut)
      if q is unmarked: mark q; stop
      r := parent(q);  cut(q, r);  q := r
  if p < prio(min): min := i
pop():
  remove min from the roots and add its children to them
  A[0 .. maxdeg] := none
  for each root r:                   (consolidation)
    while A[degree(r)] is a root s:
      link the one of larger priority under the other, call the result r
      A[degree(r) - 1] := none
    A[degree(r)] := r
  min := the root of least priority
```

Theorem 7.1. *In a Fibonacci heap, `push()`, `top()` and `decrease()` cost $O(1)$ amortized time, and `pop()` and `erase()` cost $O(\log n)$ amortized time.* See Fredman and Tarjan 1987. The class follows the analysis there, except that `increase()` is an erasure followed by an insertion; it therefore costs $O(\log n)$ amortized. Accordingly, the header warns that a heap with many increases and erasures is better a binary one.

Pairing heaps. `PairingHeap< PR, IM, CMP >`, in `lemon/pairing_heap.h`, is the self-adjusting heap of Fredman, Sedgewick, Sleator and Tarjan 1986, i.e., a single heap-ordered tree in which each node keeps its children in a list. `push()` links the new item with the root, and `decrease()` cuts the subtree of the item and links it with the root. `pop()` removes the root and joins its subtrees in two passes: it first links them in pairs from left to

right, and then it links the results from right to left into one tree (the two-pass pairing). The amortized costs of `push()` and `pop()` are $O(1)$ and $O(\log n)$ (Fredman et al. 1986). That of `decrease()` is $o(\log n)$, namely $O(2^{2\sqrt{\log\log n}})$ (Pettie 2005), and it cannot be $O(1)$ for this kind of structure (Fredman 1999). Yet, the pairing heap is often reported to be as fast as a binary heap (or faster) on the decreases of the shortest path algorithms.

Binomial heaps. `BinomialHeap< PR, IM, CMP >`, in `lemon/binomial_heap.h`, is the heap of Vuillemin 1978. It is a list of binomial trees of distinct degrees, in increasing order of degree, like the binary digits of n. `push()` merges a tree of one node into the list, linking the trees of equal degree as in a binary addition, and then it scans the roots for the minimum, in $O(\log n)$. `pop()` merges the children of the minimum, reversed, into the list, also in $O(\log n)$. `decrease()` moves the item up its tree by exchanging it with its parent, in $O(\log n)$, and it updates the cross-reference map of each item it moves. The item of least priority is kept, so that `top()` and `prio()` cost $O(1)$.

> **abLEMON.** `BinomialHeap` compares the priorities of the roots as priorities, where LEMON 1.3.1 compared them as `int` and truncated the fractional ones, taking for the minimum a root that was not; and `push()` looks for the minimum among the roots after the merge, since the merge may make the former minimum the child of a root of equal priority, after which `pop()` of LEMON 1.3.1 looped for ever (trac #646). The test `test/heap_test.cc` gives every comparison heap floating-point priorities with many ties, and the case of the ticket.

A limitation of erase(). In `PairingHeap` and `BinomialHeap`, `erase(i)` decreases the priority of the item to `prio() - 1` and pops it, and `increase()` calls `erase()`. This requires a numeric priority type. Also, it removes the right item only when `prio() - 1` compares smaller than `prio()` under `CMP`, which is not the case with `std::greater` as `CMP`, nor with floating-point priorities so large that $x - 1 = x$ (e.g., above 2^{53} in `double`); in these cases the minimum is removed instead of the item. `FibHeap` does not have this limitation, and neither do `BinHeap`, `QuadHeap` and `DHeap`, which erase by position.

7.4 Heaps of integer priorities

When the priorities are integers and the sequence of the minima removed does not decrease (as in `Dijkstra` with non-negative integer lengths), an array indexed by the priority may do better than a comparison heap. The library has, indeed, three such heaps. Their priority type is `int`, they have no comparison functor, and their only constructor takes the map (plus the initial minimum and capacity, for `RadixHeap`).

Radix heaps. `RadixHeap< IM >`, in `lemon/radix_heap.h`, is the one-level radix heap of Ahuja, Mehlhorn, Orlin and Tarjan 1990. The items are kept in boxes, each a doubly linked list, whose ranges of priorities start at the current minimum m and have widths $1, 1, 2, 4, 8, \ldots$ (Figure 7.2). Thus, box $k \geq 1$ covers 2^{k-1} priorities, and $O(\log C)$ boxes cover a span of C priorities. An item enters the box of its priority, and it then moves only towards lower boxes (unless it is increased). When box 0 is empty, `pop()` (and also `top()` and `prio()`) takes the first non-empty box and finds its minimum m'. It gives the boxes up to that one new ranges starting at m', and it redistributes the items of that box into them. Since that box covered at most as many priorities as all the boxes below it, each of its items moves to a lower box.

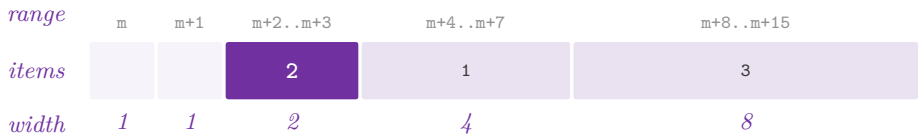


Figure 7.2. The boxes of a radix heap from the minimum m, of widths 1, 1, 2, 4, 8; the first non-empty box (here box 2) is the one that is redistributed when box 0 is empty.

Algorithm 7.3 (Removal from a radix heap).

```
pop():
  b := the first non-empty box                    (findFirst())
  if b > 0:                                       (moveDown())
    m' := the least priority in box b;  e := the end of box b
    for k = 0 .. b: min(k) := min(m' + (1 + 1 + 2 + ... up to k-1), e)
    move each item of box b down to the box of its priority
  remove an item of box 0 (all its items have priority m')
push(i, p) and decrease(i, p):
  extend the boxes upwards while p is beyond the last one
```

```
put i in the box of p, searching downwards; if p is below the
start of box 0, throw PriorityUnderflowError
```

Each box is a private record `RadixBox` (the start of its range, its width and the head of its list), and each item a record with its links, its box and its priority. The constructor `RadixHeap( IM & map, int minimum = 0, int capacity = 0 )` creates the boxes for the priorities from `minimum` to `minimum + capacity - 1`, and the heap adds boxes when a larger priority arrives; `clear(minimum, capacity)` empties it and creates the boxes again. The priorities must not go below the start of box 0. This is `minimum` at first and, once `top()`, `prio()` or `pop()` has been called, the least priority in the heap at that moment. A `push()`, `set()` or `decrease()` below it throws the exception `RadixHeap< IM >::PriorityUnderflowError` (which the comments of the header call `UnderFlowPriorityException`), even if no item has been removed yet. In `Dijkstra` with integer lengths in $[0, C]$, the priorities in the heap lie in $[d, d + C]$, where d is the last distance removed. For this case, the analysis of Ahuja et al. 1990 gives the bound $O(m + n \log(nC))$ on the running time, m and n being the numbers of arcs and nodes.

> **abLEMON.** The boxes of a `RadixHeap` stay contiguous: in LEMON 1.3.1, `moveDown()` gave the boxes before the first non-empty one ranges that could reach past the start of the next box, which still held its items, so that after an `increase()` an item could sit in a box before one with a smaller priority and be popped first (trac #681). On 20000 pseudo-random monotone sequences of operations the old code gave a wrong minimum in 18892, the present one in none; `test/heap_test.cc` has the case of the ticket and such a sequence checked against a sorted set.

Bucket heaps. `BucketHeap< IM, MIN >`, in `lemon/bucket_heap.h`, keeps a doubly linked list of the items of each priority, with the heads in a vector indexed by the priority (the buckets of Dial 1969), together with a pointer to the bucket of the minimum. The priorities must be non-negative, since they index the vector, which grows up to the largest priority plus one. `push()`, `erase()`, `decrease()` and `increase()` cost $O(1)$, apart from the growth of the vector. `top()`, `prio()` and `pop()` advance the pointer over the empty buckets, and a `push()` or a `decrease()` below the pointer moves it back. Hence, on a sequence of operations whose removed minima do not decrease, the removals cost $O(n + C)$ in all, where C is the largest priority.

With `MIN = false` the heap gives the item of largest priority, and the pointer moves downwards; e.g., `MaxCardinalitySearch` uses `BucketHeap< IM, false >` when its capacity map is the constant map `ConstMap< Arc, Const< int, 1 > >`. The header states that the class "is not intended to use as a Dijkstra heap". It works there (the test runs `Dijkstra` with it), but its time and memory grow with the largest distance, rather than with the number of nodes.

The items are private records `BucketItem` (the item, its priority and its two links), kept contiguous in a vector: an erased record is replaced by the last one, whose position is then updated in the cross-reference map.

`SimpleBucketHeap< IM, MIN >` is a reduced bucket heap, with singly linked lists and a free list of records. It supports only `push()`, `top()`, `prio()`, `pop()`, `size()`, `empty()`, `clear()` and `state(i)`; it has no `erase()`, `decrease()`, `increase()` or `set()`, and it does not conform to the concept. Its `operator[](i)` scans all the buckets, i.e., it costs $O(n + C)$, and it returns -1 for an item that is not in the heap.

7.5 Choosing a heap

Table 7.1 summarizes the costs, as they follow from the code and from the analyses cited above; n is the number of items in the heap, d the degree of a `DHeap`, and C the span of the priorities.

Table 7.1. The costs of the heaps (am. = amortized; "−" = not available).

Heap	`push`	`top`	`pop`	`decrease`	`increase`, `erase`
`BinHeap`	$O(\log n)$	$O(1)$	$O(\log n)$	$O(\log n)$	$O(\log n)$
`QuadHeap`	$O(\log_4 n)$	$O(1)$	$O(4 \log_4 n)$	$O(\log_4 n)$	$O(4 \log_4 n)$
`DHeap`	$O(\log_d n)$	$O(1)$	$O(d \log_d n)$	$O(\log_d n)$	$O(d \log_d n)$
`FibHeap`	$O(1)$	$O(1)$	$O(\log n)$ am.	$O(1)$ am.	$O(\log n)$ am.
`PairingHeap`	$O(1)$	$O(1)$	$O(\log n)$ am.	$o(\log n)$ am.	$O(\log n)$ am.
`BinomialHeap`	$O(\log n)$	$O(1)$	$O(\log n)$	$O(\log n)$	$O(\log n)$
`RadixHeap`	$O(\log C)$	$O(\log C)$ am.	$O(\log C)$ am.	$O(1)$ am.	$O(\log C)$, $O(1)$
`BucketHeap`	$O(1)$	$O(C)$	$O(C)$	$O(1)$	$O(1)$
`SimpleBucketHeap`	$O(1)$	$O(C)$	$O(C)$	−	−

For the radix heap, the amortized costs charge the downward movements of an item between boxes (at most

$O(\log C)$ for each item, between two increases) to its insertion, and the scans of the boxes to the removal. An increase, instead, moves the item upwards by at most $O(\log C)$ boxes. For the bucket heaps, the costs of `top()` and `pop()` are worst case; on a monotone sequence of operations, they add up to $O(n+C)$ in all (cf. section 7.4).

These bounds suggest a choice, which measurements on the instances of interest should confirm. A binary or 4-ary heap is the safe default, since it is simple, it has small constants and good locality, and it supports each operation in logarithmic time with any priority type and comparison. A Fibonacci or pairing heap may pay off when the decreases far outnumber the removals and the heap is large; the counts of comparisons then favour it, but the constants often do not. A radix heap or a bucket heap fits integer priorities with a monotone sequence of minima and a "small" span C. Finally, one should not use a pairing or a binomial heap for priorities without `-`, or with a comparison other than `std::less`, when items are erased or increased (section 7.3).

The algorithms of the library take the type of their heap from their traits class, and they accept another one through a named template parameter (cf. chapter 3 and chapter 10). `SetStandardHeap< H, CR >` gives the heap type `H` and the cross-reference type `CR` (by default `Digraph::NodeMap< int >`), and the algorithm then creates both objects, as `CR( digraph )` and `H( cr )`. `SetHeap< H, CR >` gives only the types, and the objects are passed by the function `heap(h, cr)` before `run()`. For instance,

```
Dijkstra< ListDigraph, ListDigraph::ArcMap< int > >
  ::SetStandardHeap< RadixHeap< ListDigraph::NodeMap< int > > >
  ::Create dijkstra( g, length );
```

runs `Dijkstra` on a radix heap. The example below solves one shortest path problem with each heap of the library in this way, and compares the distances with those of the default binary heap.

```
// The heap of Dijkstra (section 7.5): the same shortest path problem on a
// grid digraph solved with each heap of the library through the named
// parameter SetStandardHeap, and the distances compared with those of the
// default binary heap.

#include <iostream>
#include <lemon/bin_heap.h>
#include <lemon/binomial_heap.h>
#include <lemon/bucket_heap.h>
#include <lemon/dheap.h>
#include <lemon/dijkstra.h>
#include <lemon/fib_heap.h>
#include <lemon/pairing_heap.h>
#include <lemon/quad_heap.h>
#include <lemon/radix_heap.h>
#include <lemon/smart_graph.h>

using namespace lemon;

typedef SmartDigraph Digraph;
typedef Digraph::ArcMap< int > LengthMap;
typedef Digraph::NodeMap< int > CrossRef;

// true if Dijkstra with the heap H gives the distances dist from s
template< typename H >
bool agrees( const Digraph & g , const LengthMap & len , Digraph::Node s ,
             const Digraph::NodeMap< int > & dist )
{
 typename Dijkstra< Digraph ,
                    LengthMap >::template SetStandardHeap< H >::Create
   dijkstra( g , len );
 dijkstra.run( s );
 for( Digraph::NodeIt n( g ) ; n != INVALID ; ++n )
  if( dijkstra.dist( n ) != dist[ n ] )
   return( false );
 return( true );
 }

int main()
{
```

```
// a 30 x 30 grid, with arcs in both directions and lengths from 1 to 9
const int W = 30;
Digraph g;
Digraph::Node node[ W ][ W ];
for( int i = 0 ; i < W ; ++i )
 for( int j = 0 ; j < W ; ++j )
  node[ i ][ j ] = g.addNode();
LengthMap len( g );
unsigned seed = 12345;
for( int i = 0 ; i < W ; ++i )
 for( int j = 0 ; j < W ; ++j ) {
  if( i + 1 < W ) {
   seed = seed * 1103515245u + 12345u;
   len[ g.addArc( node[ i ][ j ] , node[ i + 1 ][ j ] ) ] =
     1 + ( seed >> 16 ) % 9;
   seed = seed * 1103515245u + 12345u;
   len[ g.addArc( node[ i + 1 ][ j ] , node[ i ][ j ] ) ] =
     1 + ( seed >> 16 ) % 9;
   }
  if( j + 1 < W ) {
   seed = seed * 1103515245u + 12345u;
   len[ g.addArc( node[ i ][ j ] , node[ i ][ j + 1 ] ) ] =
     1 + ( seed >> 16 ) % 9;
   seed = seed * 1103515245u + 12345u;
   len[ g.addArc( node[ i ][ j + 1 ] , node[ i ][ j ] ) ] =
     1 + ( seed >> 16 ) % 9;
   }
  }
Digraph::Node s = node[ 0 ][ 0 ];

Dijkstra< Digraph , LengthMap > reference( g , len );  // a BinHeap
reference.run( s );
Digraph::NodeMap< int > dist( g );
for( Digraph::NodeIt n( g ) ; n != INVALID ; ++n )
 dist[ n ] = reference.dist( n );

bool ok = agrees< QuadHeap< int , CrossRef > >( g , len , s , dist ) &&
          agrees< DHeap< int , CrossRef > >( g , len , s , dist ) &&
          agrees< FibHeap< int , CrossRef > >( g , len , s , dist ) &&
          agrees< PairingHeap< int , CrossRef > >( g , len , s , dist ) &&
          agrees< BinomialHeap< int , CrossRef > >( g , len , s , dist ) &&
          agrees< RadixHeap< CrossRef > >( g , len , s , dist ) &&
          agrees< BucketHeap< CrossRef > >( g , len , s , dist );

std::cout << "distance to the far corner: " << dist[ node[ W - 1 ][ W - 1 ] ]
          << ( ok ? ", the same with every heap" : ", a heap disagrees" )
          << std::endl;
return( ok ? 0 : 1 );
}
```

abLEMON. `BidirDijkstra` and `AStar` (which LEMON 1.3.1 did not have) and `MinCostArborescence` take the type of their heap from their traits, and they accept another one through the named parameter `SetHeap`.

The second example uses the heaps directly. Each of them sorts one sequence of priorities after some decreases and increases; then a `BinHeap` with `std::greater` and a `BucketHeap` with `MIN = false` give the largest priority first, and a `RadixHeap` throws on a priority below the last one removed.

```
// The heaps (chapter 7): each heap of the library sorts the same sequence
// of priorities, after some of them are decreased and increased; a binary
// heap with std::greater and a bucket heap with MIN = false give the
// largest first; a radix heap refuses a priority below the last one
// removed.

#include <algorithm>
#include <functional>
```

```
#include <iostream>
#include <vector>
#include <lemon/bin_heap.h>
#include <lemon/binomial_heap.h>
#include <lemon/bucket_heap.h>
#include <lemon/dheap.h>
#include <lemon/fib_heap.h>
#include <lemon/maps.h>
#include <lemon/pairing_heap.h>
#include <lemon/quad_heap.h>
#include <lemon/radix_heap.h>

using namespace lemon;

const int N = 12;
const int prio[ N ] = { 9 , 4 , 17 , 4 , 0 , 23 , 8 , 15 , 1 , 12 , 6 , 30 };

// pushes the items 0 .. N-1, decreases item 5 to 2 and increases item 4 to
// 20, then pops them all: true if the priorities come out sorted and the
// states are those of the concept
template< typename Heap >
bool sortTest()
{
 RangeMap< int > cross( N , Heap::PRE_HEAP );
 Heap heap( cross );
 std::vector< int > v( prio , prio + N );
 for( int i = 0 ; i < N ; ++i )
  heap.push( i , v[ i ] );
 heap.decrease( 5 , 2 );
 v[ 5 ] = 2;
 heap.increase( 4 , 20 );
 v[ 4 ] = 20;
 heap.set( 11 , 3 );
 v[ 11 ] = 3;  // set() decreases here
 std::sort( v.begin() , v.end() );
 bool ok = heap.size() == N && heap.state( 7 ) == Heap::IN_HEAP;
 for( int k = 0 ; k < N ; ++k ) {
  ok = ok && heap.prio() == v[ k ] && heap[ heap.top() ] == v[ k ];
  int i = heap.top();
  heap.pop();
  ok = ok && heap.state( i ) == Heap::POST_HEAP;
  }
 return( ok && heap.empty() );
 }

int main()
{
 typedef RangeMap< int > IM;
 bool ok =
   sortTest< BinHeap< int , IM > >() && sortTest< QuadHeap< int , IM > >() &&
   sortTest< DHeap< int , IM > >() && sortTest< DHeap< int , IM , 3 > >() &&
   sortTest< FibHeap< int , IM > >() && sortTest< PairingHeap< int ,
                                                               IM > >() &&
   sortTest< BinomialHeap< int , IM > >() && sortTest< RadixHeap< IM > >() &&
   sortTest< BucketHeap< IM > >();

 // the largest priority first
 IM c1( N , -1 ) , c2( N , -1 );
 BinHeap< int , IM , std::greater< int > > maxheap( c1 );
 BucketHeap< IM , false > maxbucket( c2 );
 for( int i = 0 ; i < N ; ++i ) {
  maxheap.push( i , prio[ i ] );
  maxbucket.push( i , prio[ i ] );
  }
 ok = ok && maxheap.prio() == 30 && maxbucket.prio() == 30;
```

```
// SimpleBucketHeap: push and pop only
IM c3( N , -1 );
SimpleBucketHeap< IM > simple( c3 );
for( int i = 0 ; i < N ; ++i )
 simple.push( i , prio[ i ] );
int last = -1;
while( ! simple.empty() ) {
 ok = ok && simple.prio() >= last;
 last = simple.prio();
 simple.pop();
 }

// RadixHeap: no priority below the last one removed
IM c4( N , -1 );
RadixHeap< IM > radix( c4 );
radix.push( 0 , 10 );
radix.push( 1 , 12 );
radix.pop();          // removes 10
radix.push( 2 , 11 );  // allowed: not below 10
bool thrown = false;
try {
 radix.push( 3 , 9 );
 }
catch( const RadixHeap< IM >::PriorityUnderflowError & ) {
 thrown = true;
 }
ok = ok && thrown;

std::cout << ( ok ? "all the heaps agree" : "a heap is wrong" ) << std::endl;
return( ok ? 0 : 1 );
}
```

7.6 Exercises

Exercise 7.1. Prove that a path from the root to a leaf of a d-ary heap of n items has $\lceil \log_d(n(d-1)+1) \rceil$ levels, and derive the costs of the first three rows of Table 7.1.

Exercise 7.2. Extend `examples/07-heap-sort.cc` with `DHeap` of degrees 2, 8 and 32, and with `std::greater` as the comparison for each comparison heap except `PairingHeap` and `BinomialHeap`; then add these two, call `erase()` on an item that is not the maximum, and observe the limitation of section 7.3.

Exercise 7.3. On grid digraphs of growing size, time `examples/07-dijkstra-heaps.cc` with each heap and with lengths in $[1, 9]$ and in $[1, 10^6]$, and explain the ranking of `BucketHeap` and `RadixHeap` in the two cases. The test `test/heap_test.cc` checks the distances of each heap (function `dijkstraHeapTest()`).

Exercise 7.4. Show by an example that a `RadixHeap` throws `PriorityUnderflowError` on a `push()` below the current minimum after a call of `top()` with no removal, and explain why `Dijkstra` never does it.

Exercise 7.5. Write the cross-reference map of a heap whose items are the integers $0, \ldots, k-1$ as a `RangeMap< int >`, and reuse one heap for several sorts, resetting the map between them with `state(i, PRE_HEAP)` instead of a new map. The function `heapSortTest()` of `test/heap_test.cc` sorts with a `RangeMap`.

8. Paths and auxiliary structures

This chapter collects the data structures and the tools that the algorithms of Part III use internally, and that a program often needs around them. They are the paths of `lemon/path.h` (with the concept of `lemon/concepts/path.h`), the union-find structures of `lemon/unionfind.h`, the elevator of `lemon/elevator.h`, the plane geometry of `lemon/dim2.h` and the radix sorts of `lemon/radix_sort.h`; then come the random numbers of `lemon/random.h`, the counters and timers of `lemon/counter.h` and `lemon/time_measure.h`, the colours of `lemon/color.h`, and the exceptions, assertions and parser of the command line of `lemon/error.h`, `lemon/assert.h` and `lemon/arg_parser.h`. The first three are data structures whose costs matter for the algorithms built on them, and we give them most of the chapter. We describe the others rather more briefly, but with all their members, since the reader of a program that uses them should find each call here.

8.1 Paths

A path of a digraph $G = (N, A)$ is a sequence of arcs $(a_1, \ldots, a_k)$ such that the target of a_i is the source of a_{i+1} for $i = 1, \ldots, k-1$. Its length, in the sense of the path classes, is the number k of its arcs (the length under a cost map is computed by the user, cf. §10.1). The four path classes of abLEMON store the sequence of arcs and no other data, i.e., they do not store the nodes, they cannot give the source of a path of no arcs, and they do not check that consecutive arcs are joined (which is left to the function `checkPath()`). All four take the digraph type as their only template parameter `GR`, and define the types `Digraph` and `Arc`.

Their common interface is made of a default constructor, a constructor and an assignment from a path of any type (both implemented by `pathCopy()`), `length()`, `empty()` and `clear()`. The arcs are read by `nth(n)` and by its alias `operator[]`, by `front()` and `back()`, and by the iterator `ArcIt`, which converts to `Arc` and runs from the first arc to `INVALID`; `nthIt(n)` places an `ArcIt` on the n-th arc, and `arcs()` wraps `ArcIt` in a range for the range-based `for` of C++11. The classes differ in how the arcs are stored, which decides what can be done and at which cost (Table 8.1).

Table 8.1. The path classes of `lemon/path.h` and the cost of their operations, for a path of k arcs ("am." stands for amortized, "no" for an operation that the class does not have).

class	storage	`nth(i)`	`addFront`, `eraseFront`	`addBack`, `eraseBack`	`length()`	other
`Path`	two vectors	$O(1)$	$O(1)$ am.	$O(1)$ am.	$O(1)$	
`SimplePath`	one vector	$O(1)$	no	$O(1)$ am.	$O(1)$	
`ListPath`	doubly linked list	$O(i)$	$O(1)$	$O(1)$	$O(k)$	`splice`, `split` in $O(1)$
`StaticPath`	one array	$O(1)$	no	no	$O(1)$	read only

In `Path` the arcs added at the front are kept in one vector, in reverse order, and those added at the back in another. When one of the two vectors is empty and an arc must be erased from its end, half of the other vector is moved into it; hence, `eraseFront()` and `eraseBack()` cost $O(1)$ amortized over any sequence of operations. `SimplePath` keeps one vector and grows only at the back, which makes it somewhat faster than `Path` when the arcs are only added at the back. A `ListPath` allocates one node per arc; hence, `nth()` walks i nodes and `length()` walks the whole list; however, it is the only class that can be cut and joined in constant time, which is indeed the reason to prefer it. Its member `spliceBack(tpath)` moves all the arcs of `tpath` to the back of the path, and `spliceFront(tpath)` to its front, leaving `tpath` empty; `splice(it, tpath)` inserts them before the arc of the iterator `it` (at the back if `it` is `INVALID`). Conversely, `split(it, tpath)` moves the arcs from `it` on into `tpath`, after clearing it (the clearing being the only part that may cost more than $O(1)$). Finally, a `Static`

`Path` is an array of exactly k arcs, which can only be built from another path or assigned; it is the most compact of the four, and usually the one to choose for storing many paths.

The concept `concepts::Path` of `lemon/concepts/path.h` states this common interface (a path type, the copy from another path, `length()`, `empty()`, `clear()`, `ArcIt` and `arcs()`), and the four classes conform to it. A second concept, `concepts::PathDumper`, describes the objects that the search algorithms of chapters 9 and 10 return from their queries `path()`. A path dumper has `length()`, `empty()` and either an `ArcIt` or, when its type `RevPathTag` is `True`, a `RevArcIt` that lists the arcs from the last to the first. In the library, the dumpers are `PredMapPath` and `PredMatrixMapPath` of `lemon/bits/path_dump.h`, which follow a map of predecessor arcs back from a node, and therefore list the arcs in reverse. They are not meant to be kept, but to be assigned to a path, as in `Path<ListDigraph> p = dijkstra.path(t);` (note that an `ArcIt` built on the temporary that such a conversion creates would point to a destroyed object).

All constructors and assignments of a path call `pathCopy(from, to)`. This function checks at compile time that `from` is a path dumper, and then chooses among four ways of copying. If `from` lists its arcs backwards (`RevPathTag`), the arcs are added to the front of `to`, else to its back; and if `to` defines the type `BuildTag` (as the four classes do), its member `build()` or `buildRev()` fills it in one pass, after reserving the memory, in place of a sequence of `addBack()` or `addFront()`. This choice is made by the helpers `RevPathTagIndicator`, `BuildTagIndicator`, `PathCopySelector`, `PathCopySelectorForward` and `PathCopySelectorBackward` of the namespace `_path_bits`, which are not part of the interface. Also, `copyPath(to, from)` is the same copy with the arguments in the opposite order, kept only for compatibility with older programs.

Four other utilities read a path together with its digraph. The predicate `checkPath(g, p)` is true if the target of each arc of `p` is the source of the next one (an empty path being consistent), and `pathSource(g, p)` and `pathTarget(g, p)` return the source of the first arc and the target of the last one, or `INVALID` for an empty path. Next, the iterator `PathNodeIt<Path>` lists the $k + 1$ nodes of a consistent path from its source to its target, given the digraph and the path in its constructor; a third argument gives the node to start from, which is needed only for a path of no arcs. Also, `pathNodes(g, p)` wraps this iterator in a range, and `for (auto u : pathNodes(g, p))` then visits the nodes in order.

> **abLEMON.** The allocator of `ListPath` constructs and destroys its nodes through `std::allocator_traits`, since C++20 removed the members `construct()` and `destroy()` of `std::allocator` that LEMON 1.3.1 called; the paths therefore compile from C++11 to C++23. The alias `operator[]` of `nth()` and the ranges of `arcs()` and `pathNodes()` come from the development of LEMON after its release 1.3.1, which did not distribute them.

In the following program we build a path at both ends and copy it into the other three classes. Then we split and splice a `ListPath`, list the nodes of a path, and copy the path dumper of a search of Dijkstra (cf. §10.2) into a `Path`.

```
// The path structures of section 8.1: a path built at both ends, copied
// into the other three types, split and spliced, walked node by node, and
// the path of a Dijkstra search (a path dumper) copied into a Path.

#include <iostream>
#include <vector>
#include <lemon/list_graph.h>
#include <lemon/path.h>
#include <lemon/dijkstra.h>

using namespace lemon;

int main()
{
 typedef ListDigraph::Node Node;
 typedef ListDigraph::Arc Arc;
 ListDigraph g;
 std::vector< Node > v;
 for( int i = 0 ; i < 5 ; ++i )
  v.push_back( g.addNode() );
 std::vector< Arc > a;  // the chain v0 -> v1 -> v2 -> v3 -> v4
 for( int i = 0 ; i < 4 ; ++i )
  a.push_back( g.addArc( v[ i ] , v[ i + 1 ] ) );
 Arc shortcut = g.addArc( v[ 0 ] , v[ 3 ] );
```

```
int bad = 0;

// a Path grows at both ends and is indexed in O(1)
Path< ListDigraph > p;
p.addBack( a[ 2 ] );
p.addFront( a[ 1 ] );
p.addFront( a[ 0 ] );
p.addBack( a[ 3 ] );
if( p.length() != 4 || p.nth( 0 ) != a[ 0 ] || p[ 3 ] != a[ 3 ] )
 ++bad;
if( ! checkPath( g , p ) || pathSource( g , p ) != v[ 0 ] ||
    pathTarget( g , p ) != v[ 4 ] )
 ++bad;

// the other three types are built from it by their copy constructors
SimplePath< ListDigraph > sp( p );
StaticPath< ListDigraph > stp( p );
ListPath< ListDigraph > lp( p );
if( sp.length() != 4 || stp.back() != a[ 3 ] || lp.nth( 2 ) != a[ 2 ] )
 ++bad;

// a ListPath is split before its third arc and spliced back in O(1)
ListPath< ListDigraph > tail;
lp.split( lp.nthIt( 2 ) , tail );
if( lp.length() != 2 || tail.length() != 2 || tail.front() != a[ 2 ] )
 ++bad;
lp.spliceBack( tail );
if( lp.length() != 4 || ! tail.empty() || ! checkPath( g , lp ) )
 ++bad;

// the nodes of a path, its source included
int count = 0;
for( Node u : pathNodes( g , p ) ) {
 if( u != v[ count ] )
  ++bad;
 ++count;
 }
if( count != 5 )
 ++bad;

// Dijkstra gives a path dumper, read backwards, copied into a Path
ListDigraph::ArcMap< int > len( g , 1 );
len[ shortcut ] = 10;
Dijkstra< ListDigraph > dijkstra( g , len );
dijkstra.run( v[ 0 ] );
Path< ListDigraph > q = dijkstra.path( v[ 4 ] );
if( q.length() != 4 || q.front() != a[ 0 ] || q.back() != a[ 3 ] )
 ++bad;

std::cout << "path of " << q.length() << " arcs from " << g.id( v[ 0 ] )
          << " to " << g.id( v[ 4 ] ) << ", " << bad << " errors"
          << std::endl;
return( bad == 0 ? 0 : 1 );
}
```

8.2 Union-find structures

A union-find structure maintains a partition of a finite set of items into classes under two operations: the union of the classes of two items, and the find of the class of an item, whose answer is a representative of the class, the same for all its items. It is the structure behind Kruskal's algorithm (§11.1), where the classes are the components of the forest grown so far, and behind the blossoms of the weighted matching algorithms of chapter 17. Each class is stored as a tree whose root is the representative and whose other items point to their parents. A find walks from the item to the root, and a union makes one root the child of the other. Two rules keep the

trees shallow: the union by size (or by rank) puts the root of the smaller tree under the root of the larger, so that the depth of a tree of s items is at most $\log_2 s$, and the path compression makes the items met during a find point to the root (Figure 8.1).

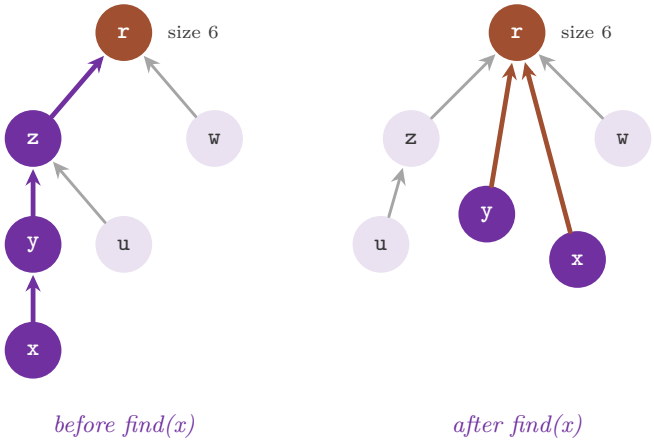


Figure 8.1. A class of a union-find structure as a tree, the root r carrying the size of the class: find(x) follows the path x, y, z, r (left), and the compression of that path makes x and y children of r (right).

Theorem 8.1. *With the union by size (or by rank) and the path compression, any sequence of m finds and unions on n items takes $O(m\,\alpha(m,n))$ time, where α is the inverse of Ackermann's function, which is at most 4 for any n that can be stored.*

We refer to Tarjan 1975 for the proof. In practice, the cost of an operation is usually a small constant, and one may take it as such when choosing an algorithm.

All the classes of `lemon/unionfind.h` take as template parameter the type `IM` of a map from items to integers. The items are the keys of `IM` (typically the nodes, arcs or edges of a graph), and the map, given to the constructor by reference, stores the index of each item in the internal vectors. From then on the map belongs to the structure: writing into it corrupts the structure, and each item must be added with `insert()` before it is used in another operation.

`UnionFind<IM>` is the plain structure, with four operations and `clear()`. Here `insert(a)` makes `a` a singleton class and returns its index, and `find(a)` returns the index of the representative of the class of `a`, an integer between 0 and the number of the items inserted, minus one. A call `join(a, b)` returns `false` if `a` and `b` are already in the same class, and otherwise unites the two classes and returns `true`; `size(a)` is the number of items of the class of `a`. Algorithm 8.1 gives `find()` and `join()` as the code has them. We remark that the documentation of the class speaks of the rank heuristic, while the code compares the sizes of the classes (which it stores, negated, at the roots); the bound of Theorem 8.1 holds for both rules.

Algorithm 8.1 (Find and union in `UnionFind`).

```
find(x):
1  r := x
2  while r is not a root: r := parent(r)
3  while x != r: next := parent(x); parent(x) := r; x := next
4  return r

join(a, b):
1  ra := find(a); rb := find(b)
2  if ra = rb: return false
3  if size(ra) > size(rb): parent(rb) := ra; size(ra) := size(ra) + size(rb)
4  else:                   parent(ra) := rb; size(rb) := size(rb) + size(ra)
5  return true
```

Next, `UnionFindEnum<IM>` adds the enumeration of the classes and of the items of each class, at the price of some memory and time per operation: each class keeps a doubly linked list of its items, and the classes are linked in a second list. Its classes are identified by integer ids rather than by representatives. Thus, `insert(item)` makes a new class and returns its id, `insert(item, cls)` adds the item to the class `cls`, `find(item`

`)` returns the id of the class of the item, and `join(a, b)` returns the id of the united class, or -1 if the two items were already together. Also, `size(cls)` is the number of items of a class, and `item(cls)` returns one of its items. This structure can also undo: `split(cls)` breaks a class into singletons, and `erase(item)` removes an item from its class, deleting the class if it becomes empty, in time proportional to the size of the class (as the documentation warns). Finally, `eraseClass(cls)` removes a whole class, and `clear()` all of them. Its iterator `ClassIt` lists the ids of the classes, and `ItemIt`, constructed from the structure and a class id, lists the items of that class.

`ExtendFindEnum<IM>` is the simplification of `UnionFindEnum` in which the classes are never united. It has the same two forms of `insert()`, `find()`, `item()`, `erase()`, `eraseClass()`, `clear()`, `ClassIt` and `ItemIt`, and no `join()`, a class therefore growing only by insertion; in exchange, `find()` reads the class of an item in $O(1)$.

`HeapUnionFind<V, IM, Comp>` attaches a priority of type `V` to each item and keeps the item of least priority of each class, where `Comp` (by default `std::less<V>`) decides what least means. It is stored as a $(2, 16)$-tree whose leaves are the items, with a tournament for the minimum on the internal nodes. Besides the union, it supports the split of a class into the classes it was made of, which the blossoms of the weighted matching algorithms need. Its documentation gives the costs: $O(1)$ for `insert(item, prio)`, which returns the id of the new class, and $O(\log n)$ for `find(item)` and for the changes of a priority, where n is the number of the items of the class. A call `join(begin, end)` unites the k classes whose ids are in the range (at least two) and returns the id of the new class, in $O(k \log n)$ time; `split(cls, out)` restores the k classes of which `cls` was made, at the same cost, and writes their ids into the output iterator `out`.

Priorities are read by `operator[](item)` and written by `set(item, prio)`, `decrease(item, prio)` and `increase(item, prio)`. The least priority of a class and its item are given by `classPrio(cls)` and `classTop(cls)`, and `classRep(cls)` gives an item of the class that does not depend on the priorities. Furthermore, `alive(cls)` is true if a class is not nested into another one (only alive classes are returned by `find()`), and `trivial(cls)` if it holds a single item directly. Its iterators are `ItemIt`, which lists the items of a class, and `ClassIt`, which lists the alive classes, or, constructed with a class id, the classes of which `cls` is made. Finally, the structures `ItemT`, `ClassT`, `ItemNode` and `ClassNode` of the header are the internal records of these classes.

In the following program we compute the components of a small graph with `UnionFind`, counting the edges that close a cycle, and enumerate them with `UnionFindEnum`; then we follow the minimum of a class of `HeapUnionFind` through a join, a decrease and a split.

```
// The union-find structures of section 8.2: the components of a graph by
// UnionFind, the classes enumerated by UnionFindEnum, and the minimum of
// each class kept by HeapUnionFind through a join and a split.

#include <iostream>
#include <iterator>
#include <vector>
#include <lemon/list_graph.h>
#include <lemon/unionfind.h>

using namespace lemon;

int main()
{
 typedef ListGraph::Node Node;
 ListGraph g;
 std::vector< Node > v;
 for( int i = 0 ; i < 6 ; ++i )
  v.push_back( g.addNode() );
 // two components, {0, 1, 2} and {3, 4}, and the isolated node 5
 g.addEdge( v[ 0 ] , v[ 1 ] );
 g.addEdge( v[ 1 ] , v[ 2 ] );
 g.addEdge( v[ 2 ] , v[ 0 ] );
 g.addEdge( v[ 3 ] , v[ 4 ] );

 int bad = 0;

 // UnionFind: one join per edge, false on the edge that closes a cycle
 ListGraph::NodeMap< int > index( g );
```

```
UnionFind< ListGraph::NodeMap< int > > uf( index );
for( ListGraph::NodeIt n( g ) ; n != INVALID ; ++n )
 uf.insert( n );
int cycles = 0;
for( ListGraph::EdgeIt e( g ) ; e != INVALID ; ++e )
 if( ! uf.join( g.u( e ) , g.v( e ) ) )
  ++cycles;
if( cycles != 1 || uf.find( v[ 0 ] ) != uf.find( v[ 2 ] ) ||
    uf.find( v[ 0 ] ) == uf.find( v[ 3 ] ) || uf.size( v[ 1 ] ) != 3 ||
    uf.size( v[ 5 ] ) != 1 )
 ++bad;

// UnionFindEnum: the same classes, enumerated with their items
typedef UnionFindEnum< ListGraph::NodeMap< int > > UFE;
ListGraph::NodeMap< int > index2( g );
UFE ufe( index2 );
for( ListGraph::NodeIt n( g ) ; n != INVALID ; ++n )
 ufe.insert( n );
for( ListGraph::EdgeIt e( g ) ; e != INVALID ; ++e )
 ufe.join( g.u( e ) , g.v( e ) );
int classes = 0 , items = 0;
for( UFE::ClassIt c( ufe ) ; c != INVALID ; ++c ) {
 ++classes;
 for( UFE::ItemIt i( ufe , c ) ; i != INVALID ; ++i )
  ++items;
 }
if( classes != 3 || items != 6 )
 ++bad;

// HeapUnionFind: priorities 5, 3, 8, 1 on v0 .. v3; join three classes,
// read the minimum, change a priority, split back
typedef HeapUnionFind< int , ListGraph::NodeMap< int > > HUF;
ListGraph::NodeMap< int > index3( g );
HUF huf( index3 );
std::vector< int > cls;
const int prio[] = { 5 , 3 , 8 , 1 };
for( int i = 0 ; i < 4 ; ++i )
 cls.push_back( huf.insert( v[ i ] , prio[ i ] ) );
int joined = huf.join( cls.begin() , cls.begin() + 3 );  // v0, v1, v2
if( huf.classPrio( joined ) != 3 || huf.classTop( joined ) != v[ 1 ] ||
    huf.find( v[ 2 ] ) != joined )
 ++bad;
huf.decrease( v[ 2 ] , 2 );
if( huf.classPrio( joined ) != 2 || huf.classTop( joined ) != v[ 2 ] )
 ++bad;
std::vector< int > parts;
huf.split( joined , std::back_inserter( parts ) );
if( parts.size() != 3 || huf.classPrio( huf.find( v[ 0 ] ) ) != 5 ||
    huf.classPrio( huf.find( v[ 3 ] ) ) != 1 )
 ++bad;

std::cout << classes << " classes, " << cycles << " cycle edge, " << bad
          << " errors" << std::endl;
return( bad == 0 ? 0 : 1 );
}
```

8.3 The elevator

Push-relabel algorithms of maximum flow (Goldberg and Tarjan 1988; see chapter 12) keep a label on each node, i.e., a lower bound on its distance to the sink in the residual digraph, and a set of active nodes, those with a positive excess. At each step they take an active node of highest label (in the variant that we consider here), push its excess along the admissible arcs, and lift its label when no arc is admissible. Furthermore, the gap heuristic of Cherkassky and Goldberg 1997 moves to the top all the nodes above a level that has become empty, since none of them can reach the sink any longer. An elevator is the data structure that supports these

operations, each in (amortized) constant time. Two of them are in `lemon/elevator.h`, `Elevator` and `LinkedElevator`, with the same interface; `Preflow` and `Circulation` take the type of theirs from their traits class (`Elevator` by default, another one by the named parameters `SetElevator` and `SetStandardElevator`), and the fractional matchings of chapter 17 use a `LinkedElevator`.

Both classes take as template parameters the type `GR` of the graph and the type `Item` of the labelled items (`GR::Node`, `GR::Arc` or `GR::Edge`), and the labels are the integers from 0 to `maxLevel()`. Its constructor `Elevator(graph, max_level)` sets the largest label, and `Elevator(graph)` takes it equal to the number of the items of the graph, as the flow algorithms do. An `Elevator` stores the items in one array, ordered by label, with the active items of each level before the inactive ones, and with two pointers per level, to its first item and to its last active item (Figure 8.2). Hence, activating, deactivating and lifting by one level are swaps of two places of the array, and `onLevel(l)`, `emptyLevel(l)`, `aboveLevel(l)` and `activesOnLevel(l)` are differences of pointers, computed in $O(1)$. A `LinkedElevator` keeps one doubly linked list per level instead; it needs no array of all the items, but it counts (somewhat slowly) the items of a level, or above a level, by walking the lists.

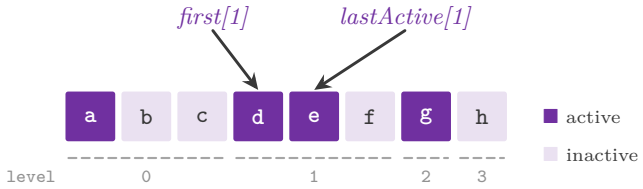


Figure 8.2. The array of an Elevator: the items in the order of their levels, the active items of each level first, and the two pointers of level 1.

The interface of the two classes has four groups of members. First, the initialization lists the items level by level: `initStart()` begins it, `initAddItem(i)` puts an item on the current level (level 0 at first), `initNewLevel()` passes to the next level, and `initFinish()` ends it, the items not listed being put on the top level, all inactive. Afterwards, the level of an item is read by `operator[](i)` and its state by `active(i)`, which `activate(i)` and `deactivate(i)` change (with the preconditions that the item is inactive, or active, before the call).

Note that the highest active item, given by `highestActive()` (or `INVALID`), has its level given by `highestActiveLevel()` (or -1); it is lifted by one level by `liftHighestActive()`, to a given higher level by `liftHighestActive(l)`, or to the top level, where it is deactivated, by `liftHighestActiveToTop()`. Three similar operations exist for the active item that `activeOn(l)` returns on a given level, namely `liftActiveOn(l)`, `liftActiveOn(l, new_level)` and `liftActiveToTop(l)`, and `lift(i, new_level)` lifts any active item. Finally, `liftToTop(l)` moves all the items on level `l` and above to the top level and deactivates them (the gap heuristic), and `activeFree(l)` is true if a level has no active item. Also, `dirtyTopButOne(i)` moves an inactive item from the top level to the level below it without updating the structure, and the documentation reserves it to those who know what it is for. In an `Elevator`, a lift from a level l to a level l' costs $O(l' - l)$, since one place is shifted on each level in between; in a `LinkedElevator` it costs $O(1)$.

abLEMON. `Elevator` allocates one place more than the items of the graph, so that it may be constructed on a graph without items; in LEMON 1.3.1, `Circulation` read past its storage on an empty digraph through it.

We remark that the constructor with a given largest label also takes the number of the items to be that label (the code initializes both with `max_level`). Hence, `Elevator(graph, max_level)` gives correct counts only when `max_level` equals the number of the items, as it does in the flow algorithms of the library. Also, the member `lift(i, new_level)` of `LinkedElevator`, which no algorithm of the library calls, updates the ends of the list of the new level, where it should update those of the level that the item leaves, when the item is the first or the last of its list; it is therefore better avoided, the other lifts being correct.

8.4 Two-dimensional geometry

In `lemon/dim2.h`, the namespace `dim2` holds the plane vectors and boxes that the drawing function `graphToEps()` of chapter 21, the planar drawings of chapter 18 and the generators of geometric instances use. A `dim2::Point<T>` is a vector with the public coordinates `x` and `y` of type `T`. It is constructed from two coordinates (the default constructor value-initializes them) or converted from a `Point` of another coordinate type, and it has `size()` (always 2), `operator[]` (0 for `x`, 1 for `y`) and `normSquare()`. Its operators are the sum, the difference and the negation of vectors, the product by a scalar on either side and the division by a scalar, the scalar product `operator*` of two vectors, the comparisons `==` and `!=`, and the forms `+=`, `-=`, `*=` and `/=`. Free functions `makePoint(`

`x, y)`, `rot90(p)`, `rot180(p)` and `rot270(p)` (the rotations in the positive direction) complete it, together with the stream operators, which write `(x,y)` and read the same form, with the parentheses and the comma optional.

A `dim2::Box<T>` is an axis-parallel rectangle, possibly empty. Its default constructor makes an empty box, while the box of a point, of two corners (bottom left and top right) or of four sides (left, bottom, right, top) is not empty. The calls `add(p)` and `add(box)` enlarge it to contain a point or another box, `operator&` intersects two boxes (the result being empty if they do not overlap), and `inside(p)` is true if a point is in the closed box (never for an empty box); `empty()` and `clear()` read and reset the emptiness. Its corners and sides are read by `bot tomLeft()`, `topRight()`, `bottomRight()`, `topLeft()`, `bottom()`, `top()`, `left()` and `right()`, and set by the members of the same names that take the new value; `width()` and `height()` give the sizes. Values of an empty box are undefined, and the setters require a box that is not empty; note that the stream operator `>>`, which reads `((l,b),(r,t))`, sets the corners by these setters and therefore leaves an empty box empty. Finally, the maps `XMap` and `YMap` (read and write) and `ConstXMap`, `ConstYMap` and `NormSquareMap` (read only) turn a map whose values are points into a map of their coordinates or of their squared norm, and are made by `xMap()`, `yMap ()` and `normSquareMap()`.

8.5 Radix sort

Both radix sorts of `lemon/radix_sort.h` sort a range of items by an integer key, given by a function object or a function `functor` applied to each item (the identity if it is omitted). Their time is linear in the number of items for a fixed size of the keys; hence, they are often worth trying in place of `std::sort` when the keys are integers of a few bytes. Its key may be of any signed or unsigned integer type, and its type is the one that `funct or` returns.

A call `radixSort(first, last, functor)` runs a quicksort whose pivots are the powers of two. For a signed type, it first separates the items with a negative key from the others. Then it finds the highest bit b of the keys of each part, partitions the part on bit b by the swaps of the partition of quicksort, then each half on bit $b-1$, and so on (Algorithm 8.2). Its time is $O(n \log c)$ and its extra space $O(\log c)$ (the recursion), where n is the number of items and c the largest absolute value of a key; it is not stable, and it needs bidirectional iterators.

Algorithm 8.2 (Radix sort by bit partitions, `radixSort()`).

```
radixSort(R):                        R is a range, key(x) an integer
1  split R into R- (key < 0) and R+ (key >= 0) by swaps
2  for each part P in (R-, R+):
3     b := the highest bit in which the keys of P differ from 0 (from 1 in R-)
4     sort(P, b)

sort(P, b):
1  while b >= 0 and P has at least two items:
2     split P into P0 (bit b of the key is 0) and P1 (bit b is 1) by swaps
3     sort(P0, b - 1); P := P1; b := b - 1
```

A call `stableRadixSort(first, last, functor)` sorts by the bytes of the keys, from the least significant to the most significant, with one counting pass per byte. Each pass counts the items of each of the 256 values of the byte, computes where the items of each value begin, and copies the items in that order into a second buffer, which keeps the relative order of the items with the same byte; on a signed type, the last pass puts the negative keys first. This sort is stable and takes $O(b\,(n+256))$ time, where b is the number of bytes of the type of the key (whatever the largest key), and it allocates a buffer of $2n$ items. Both functions have overloads for a plain function pointer, and both accept a lambda; the signed or unsigned version of each is chosen at compile time, from the type of the key, by the helpers `RadixSortSelector` and `StableRadixSortSelector` of the namespace `_radix_sort_bits`.

> **abLEMON.** The type of the key is deduced with `decltype` from the call of `functor`, rather than read from its member `result_type`, which C++20 removed from the function objects of the standard library; a lambda or any other callable may therefore be passed, which LEMON 1.3.1 did not accept.

We remark that `radixSort()` builds the mask of the highest bit as the `int` expression `1 << max_digit`, which overflows for the keys of a 64-bit type that use more than 31 bits. For such keys, `stableRadixSort()` is the safe choice.

8.6 Random numbers

In `lemon/random.h`, abLEMON implements the Mersenne Twister of Matsumoto and Nishimura 1998, a generator of period $2^{19937} - 1$ that is equidistributed in 623 dimensions for 32-bit numbers. It comes in a 32-bit and a 64-bit version, which give different sequences from the same seed. A class template `_random_bits::Random<Word>` holds the interface, and three types instantiate it: `Random32` (the 32-bit version), `Random64` (the 64-bit one) and `Random`, whose word is `unsigned long`, i.e., 64 bits on Linux and macOS and 32 bits on Windows. Thus, a program that must give the same numbers on all platforms should use `Random32` or `Random64`. Its global object `rnd`, of type `Random`, is the generator that the library and most programs use; it is constructed with a fixed seed, and therefore gives the same sequence at each run until it is seeded again. Other names of the namespace `_random_bits` are the implementation: `RandomCore` holds the state and twists it, and `RealConversion`, `IntConversion`, `BoolConversion`, `Mapping`, `Masker`, `Shifting`, `ShiftMultiplier`, `Initializer` and `BoolProducer` turn words into numbers of a given type.

A generator is seeded by its constructor or by `seed()`, from one number or from a range of numbers, each converted to the word of the generator. Without arguments, `seed()` reads four words from `/dev/urandom` by `seedFromFile()` (except on Windows) and, if that fails, calls `seedFromTime()`, which seeds from the process id and the time of day; both return `true` on success. A copy, by construction or by assignment, copies the state, and the copy then gives the same sequence from that point on. However, the constructor from a number is a template on the type of the number, which also matches a generator of the other word size: `Random64 r(r32)` does not copy `r32` but fails to compile, since the state can only be copied between generators of the same type.

Uniform numbers are given by several members. The call `real<T>()` returns a number of type `T` (`double` by default) in $[0, 1)$, and `operator()()`, `operator()(b)` and `operator()(a, b)` a `double` in $[0, 1)$, $[0, b)$ and $[a, b)$. The call `integer(b)` and its alias `operator[](b)` return an integer in $\{0, \dots, b-1\}$, of the type of `b`, and `integer(a, b)` one in $\{a, \dots, b-1\}$. Also, `integer<T>()` and `uinteger<T>()` return an integer uniform over the whole range of `T`, or over its nonnegative part (`int` and `unsigned int` by default), and `boolean()` returns a fair bit, taken from a buffer of the bits of one word.

Several other distributions are available (cf. the test `test/random_test.cc`). The call `boolean(p)` is `true` with probability p, and `gauss()` and `gauss(mean, std_dev)` are normal, by the polar form of the method of Box and Muller 1958 given by Marsaglia and Bray 1964. The call `lognormal(n_mean, n_std_dev)` (or `lognormal(pair)`) returns the exponential of a normal variable of the given parameters, and `lognormalMD(mean, std_dev)` a lognormal variable of the given mean and standard deviation, whose parameters `lognormalParamsFromMD()` computes. Next come `exponential(lambda)` (of mean $1/\lambda$, with $\lambda = 1$ by default), `gamma(k)` for an integer shape (a sum of k exponentials), `gamma(k, theta)` for any positive shape (by rejection), `weibull(k, lambda)`, `pareto(k, x_min)` and `poisson(lambda)`; the last is the method of Knuth 1969, whose time is linear in the value returned. In the plane, `disc()` is uniform on the unit disc, `gauss2()` gives the two (dependent) normal coordinates of one step of the polar method, and `exponential2()` is a rotationally symmetric distribution whose coordinates are, conditionally on the other being zero, exponential.

abLEMON. The packages of abLEMON declare, next to the Boost Software License, the BSD-3-Clause notice of the Mersenne Twister that the header carries, and `rnd` is exported when the library is built as a DLL on Windows.

8.7 Counting and timing

In `lemon/counter.h`, a few classes help to count events in a program under development, e.g., the operations of an algorithm. A `Counter` has a title and an output stream (`std::cerr` by default); it is changed by `++`, `--`, `+=` and `-=`, read by its conversion to `int` and reset by `reset(c)`, and it prints its title and value when it is destroyed. A `Counter::SubCounter`, constructed on a parent counter with a title and a stream, counts in the same way, prints its value when it is destroyed, and adds it to the parent at that moment. A `Counter::NoSubCounter`, instead, prints no report and passes each change to its parent at once; thus, replacing one with the other turns the report of a phase on or off without touching the code that counts. Subcounters can have subcounters, and `NoCounter` has the interface of `Counter` but counts nothing; one can thus compile the counting of a program away by changing one type.

Time is measured by `lemon/time_measure.h`. A `TimeStamp` holds five times, i.e., the real time, the user and system times of the process and those of its children, which `stamp()` reads from the operating system. Its constructor `TimeStamp(NULL)` makes a stamp of the present moment, and the default constructor one of zeros. Stamps can be added, subtracted and scaled, `ellapsed()` gives the time passed since the stamp, and `realTim`

`e()`, `userTime()`, `systemTime()`, `cUserTime()` and `cSystemTime()` read the five values (the last two are not computed on Windows). Its stream operator prints `u: ...s, s: ...s, cu: ...s, cs: ...s, real: ...s`; after `TimeStamp::format(TimeStamp::SHORT)`, a setting global to all stamps, it prints the real time and its relative difference from the total processor time.

A `Timer` starts at its construction (unless it is constructed with `false`), and is controlled by `start()`, `stop()`, `halt()`, `reset()` and `restart()`. It counts the calls of `start()` and stops only after as many calls of `stop()`, which allows one to time a recursive function, while `halt()` stops it at once; `running()` returns the number of calls of `stop()` still needed. Its times are read by the same five members as those of a stamp, or by its conversion to `TimeStamp`, which printing it uses. A `TimeReport` is a `Timer` that prints a title and its times when it is destroyed (unless `active(false)` has been called), and `NoTimeReport` is the version that measures no time. The granularity of the clocks is typically 10 ms (as the documentation warns), which makes a single measure of a short function rather unreliable. For a short function, `runningTimeTest(f, min_time, num, full_time)` calls `f` 1, 2, 4, ... times until the total real time exceeds `min_time` seconds (10 by default), and returns the average time of one call; the number of calls and the total time are written into the two optional pointers.

8.8 Colors

The colours that `graphToEps()` uses are those of `lemon/color.h`. A `Color` holds three components in $[0, 1]$, read and set by `red()`, `green()` and `blue()`, and set together by `set(r, g, b)`. Fifteen constants are defined in the library: `WHITE`, `BLACK`, `RED`, `GREEN`, `BLUE`, `YELLOW`, `MAGENTA`, `CYAN`, `GREY`, `DARK_RED`, `DARK_GREEN`, `DARK_BLUE`, `DARK_YELLOW`, `DARK_MAGENTA` and `DARK_CYAN`. A `Palette` is a map from `int` to `Color` that cycles through a list of colours. Its constructor takes `have_white` (white first, `false` by default) and `num` (the number of colours, -1 by default), and fills the list with the 26 colours of its default list (27 with white), cut to `num` colours or repeated up to them. Then `operator[](i)` and `set(i, c)` read and write the colour of `i` modulo `size()`, and `add(c)` and `resize(s)` change the list. Finally, `distantColor(c)` returns the corner of the colour cube farthest from `c`, and `distantBW(c)` returns black for a light colour and white for a dark one, by the luminance of `c`.

8.9 Errors, assertions and the command line

All exceptions of the library derive from `Exception` (`lemon/error.h`), a class derived from `std::exception`, whose `what()` returns `"lemon::Exception"`. An `IoError` is thrown when a file cannot be read or written; it carries a message and a file name, set by its constructors or by `message()` and `file()` and read by the members of the same names. A `FormatError` is thrown by the readers of chapter 21 when the input is malformed, and carries a line number too (`line()`, zero if unknown). The `what()` of both composes the message, the file and the line, e.g., `lemon:FormatError: unknown section (in file 'g.lgf' at line 12)`. Other headers derive further exceptions from `Exception`, e.g., the `ArgParserException` described below.

In `lemon/assert.h`, the macro `LEMON_ASSERT(exp, msg)` checks the preconditions of the library, and the user may use it as well. If `exp` is false, it calls a handler with the file, the line, the function, the message and the text of the expression; the default handler, `assert_fail_abort()`, prints them on `std::cerr` and calls `std::abort()`. Assertions are enabled by default, and disabled by `LEMON_DISABLE_ASSERTS` or by the standard macro `NDEBUG`. Moreover, `LEMON_ASSERT_ABORT` selects the default handler explicitly, and `LEMON_ASSERT_CUSTOM` the function named by `LEMON_CUSTOM_ASSERT_HANDLER`, which must have the signature of `assert_fail_abort()`. Defining both modes, or one of them together with `NDEBUG` or `LEMON_DISABLE_ASSERTS`, is a compilation error, and the mode may be changed within a translation unit by redefining the macros and including the header again. Finally, `LEMON_DEBUG(exp, msg)` checks the internal invariants of the algorithms; it works as `LEMON_ASSERT`, but it is disabled unless `LEMON_ENABLE_DEBUG` is defined, since some of its checks are expensive.

`ArgParser` (`lemon/arg_parser.h`) parses the command line of the tools of chapter 22, and of any program. It is constructed on `argc` and `argv`, and the options are declared without their leading `-`. Calls of `intOption()`, `doubleOption()`, `boolOption()` and `stringOption()` declare an option with a help string, a default value and a flag that makes it mandatory, and `refOption()` declares one whose value is written directly into a variable of the program. In the same way, `other(name, help)` documents the arguments that are not options, `synonym(syn, opt)` adds another name of an option, and `optionGroup(group, opt)` puts options into a group, for which `onlyOneGroup(group)` and `mandatoryGroup(group)` require at most one, or at least one, of its options. All these calls return the parser, and can therefore be chained.

Then `parse()`, or its synonym `run()`, reads the arguments. An argument that starts with `-` (and is not `-` alone) must be a declared option; a `bool` option is set to `true` by its presence, and any other option takes the next

argument as its value, which must be read in full as a number for a numeric option. Other arguments are collected in `files()`. Afterwards, `given(name)` is true if an option appeared, and `operator[](name)` returns its value through the proxy `RefType`, which converts to `int`, `double`, `bool` or `std::string` after checking the type of the option with `LEMON_ASSERT` (an integer option also converts to `double`); `commandName()` returns `argv[0]`. Options `-help`, `--help` and `-h` print a summary of the options. On a request of help, an unknown option, a missing or malformed value, or a violated condition on the mandatory options and groups, the parser prints a message and calls `exit(1)`. If `throwOnProblems()` has been called, it throws instead an `ArgParserException`, whose `reason()` is `HELP`, `UNKNOWN_OPT` or `INVALID_OPT`. Its records `GroupData` and `OtherArg` of the header hold the groups and the help of the other arguments.

The last program of the chapter uses an elevator, the two radix sorts, two generators seeded alike and a box; it also uses a counter that writes into a string, a timer, and a parser of a command line given as an array.

```
// The auxiliary tools of sections 8.3 to 8.9: an elevator of labels, the
// two radix sorts, the random number generator, points and boxes, a
// counter, a timer and a parser of the command line.

#include <iostream>
#include <sstream>
#include <vector>
#include <lemon/list_graph.h>
#include <lemon/elevator.h>
#include <lemon/radix_sort.h>
#include <lemon/random.h>
#include <lemon/dim2.h>
#include <lemon/counter.h>
#include <lemon/time_measure.h>
#include <lemon/arg_parser.h>

using namespace lemon;

int key( int x )
{
 return( x % 10 );
 }  // the last decimal digit

int main()
{
 int bad = 0;

 // Elevator: four nodes on levels 0, 0, 1 and 2 (out of 0 .. 4)
 ListDigraph g;
 ListDigraph::Node n[ 4 ];
 for( int i = 0 ; i < 4 ; ++i )
  n[ i ] = g.addNode();
 Elevator< ListDigraph , ListDigraph::Node > lev( g , 4 );
 lev.initStart();
 lev.initAddItem( n[ 0 ] );
 lev.initAddItem( n[ 1 ] );
 lev.initNewLevel();
 lev.initAddItem( n[ 2 ] );
 lev.initNewLevel();
 lev.initAddItem( n[ 3 ] );
 lev.initFinish();
 lev.activate( n[ 1 ] );
 lev.activate( n[ 2 ] );
 if( lev.highestActive() != n[ 2 ] || lev.highestActiveLevel() != 1 ||
     lev.onLevel( 0 ) != 2 || lev.activesOnLevel( 0 ) != 1 )
  ++bad;
 lev.liftHighestActive( 3 );  // n2 from level 1 to level 3
 if( lev[ n[ 2 ] ] != 3 || ! lev.emptyLevel( 1 ) || lev.aboveLevel( 0 ) != 2 )
  ++bad;

 // radix sorts: by value, and stably by the last digit
```

```
std::vector< int > x = { 42 , -7 , 15 , 0 , 32 , -12 , 15 , 5 };
std::vector< int > y = x;
radixSort( x.begin() , x.end() );
for( std::size_t i = 1 ; i < x.size() ; ++i )
 if( x[ i - 1 ] > x[ i ] )
  ++bad;
std::vector< int > z = { 42 , 15 , 32 , 5 , 12 };
stableRadixSort( z.begin() , z.end() , key );
if( z != std::vector< int >( { 42 , 32 , 12 , 15 , 5 } ) )
 ++bad;

// random numbers: one seed, one sequence
Random32 r1( 2026 ) , r2( 2026 );
for( int i = 0 ; i < 100 ; ++i ) {
 int a = r1[ 6 ] + 1 , b = r2.integer( 1 , 7 );
 double u = r1();
 r2();
 if( a != b || a < 1 || a > 6 || u < 0 || u >= 1 )
  ++bad;
 }
Random32 saved( r1 );  // the state is copied
if( saved.integer< int >() != r1.integer< int >() )
 ++bad;

// points and boxes
dim2::Box< double > box;
box.add( dim2::Point< double >( 1 , 2 ) );
box.add( dim2::Point< double >( -1 , 5 ) );
if( box.width() != 2 || box.height() != 3 ||
    ! box.inside( dim2::Point< double >( 0 , 3 ) ) ||
    box.inside( dim2::Point< double >( 2 , 3 ) ) )
 ++bad;
dim2::Point< int > p( 3 , 4 );
if( p.normSquare() != 25 || rot90( p ) != dim2::Point< int >( -4 , 3 ) )
 ++bad;

// a counter that prints its value into a string at its destruction
std::ostringstream report;
{
 Counter ops( "operations: " , report );
 Counter::NoSubCounter cmp( ops );
 for( int i = 0 ; i < 10 ; ++i )
  ++cmp;
 ops += 5;
 }
if( report.str() != "operations: 15\n" )
 ++bad;

// a timer stopped and restarted, whose times do not decrease
Timer t;
double first = t.realTime();
t.stop();
t.start();
if( t.realTime() < first || t.running() != 1 )
 ++bad;

// a command line parsed, with an option of each kind
const char * argv[] = {
 "prog" , "-n" , "7" , "-eps" , "1e-3" , "-v" , "input.lgf"
 };
ArgParser ap( 7 , argv );
int nodes = 0;
ap.refOption( "n" , "number of nodes" , nodes )
  .doubleOption( "eps" , "tolerance" , 1e-6 )
  .boolOption( "v" , "verbose" )
```

```
  .other( "file" , "the input" );
ap.throwOnProblems();
ap.parse();
double eps = ap[ "eps" ];
bool verbose = ap[ "v" ];
if( nodes != 7 || eps != 1e-3 || ! verbose || ap.files().size() != 1 ||
    ap.files()[ 0 ] != "input.lgf" || ! ap.given( "n" ) )
 ++bad;

std::cout << bad << " errors" << std::endl;
return( bad == 0 ? 0 : 1 );
}
```

8.10 Exercises

Exercise 8.1. Prove the amortized bound of `Path` of Table 8.1: show that any sequence of q calls of `addFront()`, `addBack()`, `eraseFront()` and `eraseBack()` on an empty `Path` takes $O(q)$ time, although an erasure from an empty end moves half of the other vector. (The test `test/path_test.cc` checks the four path classes.)

Exercise 8.2. Write a function that reverses a path of a digraph into a path of the adaptor `ReverseDigraph` of chapter 5, using `ListPath::spliceFront()`, and check the result with `checkPath()`; compare it with a copy by `pathCopy()` and the iterator `ArcIt`.

Exercise 8.3. Compute the connected components of a graph with `UnionFindEnum`, print the items of each class with `ClassIt` and `ItemIt`, and compare the classes with those of `connectedComponents()` of chapter 18. (The example `examples/08-union-find.cc` and the test `test/unionfind_test.cc` use the same calls.)

Exercise 8.4. Measure with `runningTimeTest()` the time of `radixSort()`, `stableRadixSort()` and `std::sort` on 10^6 random keys drawn by `rnd.integer<int>()`, and then on keys drawn in $\{0, \ldots, 255\}$; explain the two rankings by the costs of §8.5. (The test `test/radix_sort_test.cc` checks the two sorts.)

Exercise 8.5. Draw 10^5 numbers by `gamma(2.5, 2.0)` and by `pareto(3.0, 1.0)`, and compare their sample means with the means $k\theta$ and $kx_{\min}/(k-1)$ of the two distributions. (The test `test/random_test.cc` checks the generator.)

Exercise 8.6. Use a `Counter` with two subcounters to count the comparisons and the swaps of an insertion sort, and verify that the parent includes the counts of a `SubCounter` only after it is destroyed, and those of a `NoSubCounter` at once. (The test `test/counter_test.cc` exercises the counters.)

9. Graph search

We are concerned in this chapter with the visit of the nodes of a digraph $G = (N, A)$, with $n = |N|$ and $m = |A|$, that can be reached along the arcs from a given set $S \subseteq N$ of sources. The breadth-first search of `lemon/bfs.h` visits them in the order of their distance from S, i.e., of the least number of arcs of a path from a source. Instead, the depth-first search of `lemon/dfs.h` goes as deep as it can along each path before it backtracks. Both take $O(n + m)$ time, and both are usually the building blocks of many algorithms of the following chapters (e.g., connectivity, matchings and flows). Finally, the maximum cardinality search of `lemon/max_cardinality_search.h` is a third order of visit, driven by a priority queue; it recognizes chordal graphs, and it underlies the minimum cut algorithm of Nagamochi and Ibaraki of chapter 15. All three classes follow the pattern of the algorithm classes of chapter 3 (traits, named template parameters, `run()`, `init()` and `start()`, queries), which we recall in §9.1 for the first two.

9.1 The common interface

`Bfs` and `Dfs` have the same shape. Their template parameters are the digraph type `GR` (`ListDigraph` by default) and a traits class `TR`, by default `BfsDefaultTraits<GR>` and `DfsDefaultTraits<GR>`. These traits give the types of four maps on the nodes, namely `PredMap`, the arc by which each node was reached (by default a `NodeMap<Arc>`), `DistMap`, its distance or depth (a `NodeMap<int>`), `ReachedMap` (a `NodeMap<bool>`) and `ProcessedMap`. By default, the last one is a `NullMap`, i.e., a map that discards what is written into it, since most programs do not need it; the named parameter `SetStandardProcessedMap` makes it a `NodeMap<bool>`.

Named parameters `SetPredMap<T>`, `SetDistMap<T>`, `SetReachedMap<T>` and `SetProcessedMap<T>` set the type of a map to `T`. With them, the algorithm does not allocate the map: one of type `T` must be given by the setter of the same name (`predMap()`, `distMap()`, `reachedMap()` or `processedMap()`) before `init()`, or an assertion fails. With the default types, a map that is not given is allocated at `init()` and deleted by the destructor. A map that is given, on the other hand, typically stays the property of the caller, and keeps the results after the algorithm object is gone.

In the simplest use, one calls `run(s)`, which visits the nodes reachable from `s`, and then reads the results by the queries. For more control, `init()` allocates the maps and marks all the nodes unreached, `addSource(s)` adds a source (several may be added), and one of the functions `start()` performs the search. One can also run the search in steps of one node (`processNextNode()` of `Bfs`) or of one arc (`processNextArc()` of `Dfs`), while `emptyQueue()`, `queueSize()` and `nextNode()` or `nextArc()` give what is left to do. A search may be stopped when a target node is reached (`start(t)` and `run(s, t)`) or when a condition given by a map is met. Finally, `run()` visits all the nodes, starting a new search from each node that the previous ones have not reached.

Its queries are `reached(v)`, `dist(v)`, `predArc(v)` and `predNode(v)` (`INVALID` for a source and for a node not reached), `distMap()` and `predMap()`. Moreover, `path(t)` returns a path dumper (cf. §8.1) of the path from a source to `t` along the arcs of the predecessor map. We remark that `dist(v)` is undefined for a node not reached, since `init()` does not write the distances.

Each class also has a function-type interface, `bfs()` and `dfs()` (§9.4), and a version with a visitor, `BfsVisit` and `DfsVisit`, which calls the members of an object of the user at each event of the search in place of filling maps (§9.5).

9.2 Breadth-first search

Let $d(v)$ be the distance of v from S, i.e., the least number of arcs of a path from a node of S to v, with $d(v) = \infty$ if there is no such path. A breadth-first search computes $d(v)$ for all the nodes, together with a shortest path

tree (a forest, if there are several sources). It keeps a queue of the nodes that are reached but not yet processed, and processes them in the order in which they entered it; hence, all the nodes at distance k are processed before any node at distance $k+1$. Algorithm 9.1 gives the steps as `Bfs` implements them.

Algorithm 9.1 (Breadth-first search, `Bfs`).

```
init():
1  for each node v: pred(v) := INVALID; reached(v) := false; processed(v) := false
2  Q := empty queue
addSource(s):
3  if not reached(s): reached(s) := true; pred(s) := INVALID; d(s) := 0; push s on Q
start():
4  while Q is not empty:
5     u := pop the first node of Q; processed(u) := true
6     for each arc a = uv out of u:
7        if not reached(v): reached(v) := true; pred(v) := a; d(v) := d(u) + 1
8                           push v on Q
```

Theorem 9.1. *When `start()` returns, the nodes reached are those with $d(v) < \infty$, and `dist(v)` equals $d(v)$ for each of them. The arcs `predArc(v)` form a forest in which the path from a source to v has $d(v)$ arcs, and the search takes $O(n+m)$ time.*

We prove it by induction on d, using that the distances of the nodes in the queue never decrease and differ by at most one (cf. Cormen et al. 2001, §22.2). Each node enters the queue at most once and each arc is scanned once, from its source, and therefore the bound holds; the queue is a vector of n places, allocated by `init()`. Note that the class does not read $d(u)$ to compute $d(u)+1$: it counts the levels by remembering where the nodes of the next level begin in the queue, and the distance of a node is written once, when the node is reached.

Otherwise, the interface is the one of §9.1, and a call of `processNextNode()` processes the first node of the queue and returns it (the queue must not be empty). Its form `processNextNode(target, reach)` also sets `reach` to `true` if `target` is reached in the step, and `processNextNode(nm, rnode)` sets `rnode`, if it was `INVALID`, to the first node reached in the step whose value in the `bool` node map `nm` is true. Accordingly, `start(t)` processes nodes until `t` is reached, i.e., until it enters the queue (its distance and path being then final), and `start(nm)` processes them until a node with `nm` true is reached, which it returns (or `INVALID`). Thus, `run(s, t)` is `init()`, `addSource(s)` and `start(t)`, and returns `reached(t)`. We remark that the documentation of `run(s, t)` states that the search stops when `t` is processed, while the code stops it as soon as `t` is reached; this saves the processing of the rest of its level (usually a somewhat small saving), and gives the same results for `t`. Finally, `nextNode()` returns the first node of the queue (or `INVALID`), `queueSize()` the number of nodes in it, and `emptyQueue()` whether there are none.

Figure 9.1 shows, on the left, the tree and the distances of a breadth-first search. On the right, the tree of a depth-first search of the same digraph is the subject of the next section.

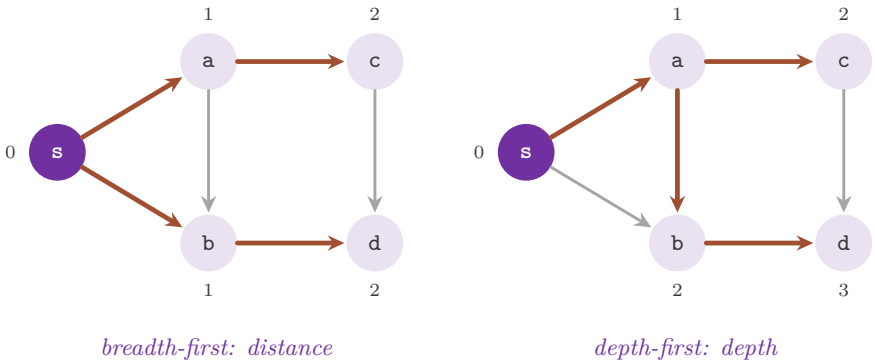


Figure 9.1. A breadth-first search (left) and a depth-first search (right) of the same digraph from s, the out-arcs of each node scanned from top to bottom: the arcs of the two trees, the distance of each node in the first and its depth in the tree in the second.

9.3 Depth-first search

The depth-first search advances from the last node reached along an arc not yet scanned, and backtracks to the node it came from when the node has no such arc left. `Dfs` implements it without recursion, by a stack that holds, for each node of the current path from the source, the next out-arc to scan from it (Algorithm 9.2). A node is processed, in the sense of the `ProcessedMap`, when the search backtracks from it, i.e., when all the nodes reachable from it through unreached nodes have been visited. In `Dfs`, the value `dist(v)` is the depth of v in the tree of the search, i.e., the number of arcs of the path of the tree from the source; it is the height of the stack when v is reached, and in general it is not a distance in the digraph.

Algorithm 9.2 (Depth-first search with a stack of arcs, `Dfs`).

```
addSource(s):                                    the stack must be empty
1  if not reached(s): reached(s) := true; pred(s) := INVALID
2     if s has an out-arc: push its first out-arc; depth(s) := 0
3     else: processed(s) := true; depth(s) := 0
processNextArc():
4  a = uv := the arc on the top of the stack
5  if not reached(v): reached(v) := true; pred(v) := a
6                     push the first out-arc of v (INVALID if none); depth(v) := height - 1
7  else: replace the top of the stack by the next out-arc of u
8  while the top of the stack is INVALID:
9     processed(node of the top) := true; pop
10    if the stack is not empty: replace its top by the next out-arc of its source
11 return a
```

Each arc is pushed once and each node is reached once, which gives $O(n + m)$ time; the stack is a vector of n places allocated by `init()`. This finishing order of the nodes, i.e., the order in which they are processed, has the properties for which the depth-first search is used (Tarjan 1972). In particular, an arc uv examined while v is still on the stack closes a directed cycle; conversely, if the digraph is acyclic, each arc uv has v finished before u.

Lemma 9.1. *If G is acyclic, the reverse of the order in which `run()` processes the nodes is a topological order, i.e., the source of each arc precedes its target.*

A proof is in Cormen et al. 2001, §22.4. The example at the end of §9.5 computes such an order, by recording the nodes as they are processed.

Otherwise, the interface follows §9.1, with three differences. First, the unit of work is an arc: `processNextArc()` scans the arc on the top of the stack and returns it, and `nextArc()` returns the next arc to be scanned, as an `OutArcIt` (`INVALID` if the stack is empty), while `queueSize()` is the height of the stack. Second, `addSource()` requires an empty stack (checked by `LEMON_DEBUG`), since a source added to a search in progress would make the previous sources lose one of their arcs, and their depths would be wrong; this is why `run()` adds each new source only after the previous search has finished. Third, the stopping condition of `start(am)` is an arc map, not a node map: the search stops before scanning an arc a with `am[a]` true and returns it, or returns `INVALID` if the search ends first. As in `Bfs`, `start(t)` and `run(s, t)` stop as soon as `t` is reached, and `run(s, t)` returns `reached(t)`.

9.4 Function-type interfaces

The functions `bfs(g)` and `dfs(g)` return a `BfsWizard` and a `DfsWizard` on the digraph `g`. Their members set the maps of the algorithm and the outputs, each returning a new wizard, and their `run()` runs a `Bfs` or a `Dfs` with them. Named function parameters `predMap(m)`, `distMap(m)`, `reachedMap(m)` and `processedMap(m)` pass the maps that the algorithm fills. Their types are deduced from the arguments, so that any writable map may be passed, e.g., a `LoggerBoolMap` (cf. chapter 6) that records the nodes in the order of processing. For a search between two nodes, `path(p)` gives a path into which the path to the target is copied, and `dist(d)` a variable that receives its distance or depth.

Three runs are available, `run(s)`, `run(s, t)`, which returns whether `t` was reached and fills the path and the distance, and `run()`, which visits all the nodes. Maps not given are allocated by the algorithm for the duration of the run, and the path given to `path()` may be of any path type that a path dumper can be assigned to (by default a `Path`). Also, the classes `BfsWizardBase` and `DfsWizardBase` hold the pointers to the parameters, and are not used directly. Two typical calls are the following.

```
bool found = bfs(g).path(p).dist(d).run(s, t);
dfs(g).processedMap(loggerBoolMap(std::back_inserter(order))).run();
```

9.5 Visitors

`BfsVisit<GR, VS, TR>` and `DfsVisit<GR, VS, TR>` perform the same searches as `Bfs` and `Dfs`. However, in place of filling a predecessor and a distance map, they call a member of a visitor object of type `VS` at each event of the search; the visitor is given to the constructor together with the digraph. They keep only a `ReachedMap` (a `NodeMap<bool>` by default, set by `reachedMap()` or by the named parameter `SetReachedMap<T>`). Their execution control is that of `Bfs` and `Dfs` (`init()`, `addSource()`, `processNextNode()` or `processNextArc()`, `nextNode()` or `nextArc()`, `emptyQueue()`, `queueSize()`, the three `start()` and the three `run()`), and their only query is `reached(v)`. In the library, the visitors `BfsVisitor<GR>` and `DfsVisitor<GR>` define the events as members with an empty body. A visitor of the user may derive from them, and redefine only the members it needs.

`BfsVisit` calls `start(s)` for each source and `reach(v)` when a node is reached for the first time (a source included). It calls `process(u)` when a node is taken from the queue, `discover(a)` when an arc reaches a node not yet reached (followed by `reach()` of its target), and `examine(a)` when an arc is scanned whose target was already reached. `DfsVisit` calls `start(s)` and `stop(s)` when the search from the source `s` begins and ends, and `reach(v)`, `discover(a)` and `examine(a)` as `BfsVisit` does. It also calls `leave(v)` when the search backtracks from a node (the moment at which `Dfs` marks it processed), and `backtrack(a)` when the search returns along the arc `a` of the tree to its source. Thus, `reach()` and `leave()` delimit the time during which a node is on the stack, and an `examine(a)` whose target is on the stack reveals a directed cycle, which the program below exploits. Note that `DfsVisit::nextArc()` returns an `Arc`, while `Dfs::nextArc()` returns an `OutArcIt`.

In the following program we compute the distances of a breadth-first search, a shortest path by `bfs()`, and a topological order by the finishing order of `dfs()` (Lemma 9.1). Then we find a directed cycle by a visitor of `DfsVisit`.

```
// The searches of sections 9.2 to 9.5: the distances of a breadth-first
// search, a path by the function-type interface, a topological order by
// the finishing order of a depth-first search, and a directed cycle found
// by a visitor of DfsVisit.

#include <iostream>
#include <iterator>
#include <vector>
#include <lemon/list_graph.h>
#include <lemon/bfs.h>
#include <lemon/dfs.h>
#include <lemon/maps.h>
#include <lemon/path.h>

using namespace lemon;

typedef ListDigraph::Node Node;
typedef ListDigraph::Arc Arc;

// a visitor that keeps the nodes on the stack of the search and records
// an arc that enters one of them, i.e., an arc that closes a cycle
struct CycleVisitor : public DfsVisitor< ListDigraph >
{
 const ListDigraph & g;
 ListDigraph::NodeMap< bool > & onStack;
 Arc & back;
 CycleVisitor( const ListDigraph & gr , ListDigraph::NodeMap< bool > & s ,
               Arc & b )
     : g( gr ) , onStack( s ) , back( b ) { }
 void reach( const Node & n ) { onStack[ n ] = true; }
 void leave( const Node & n ) { onStack[ n ] = false; }
 void examine( const Arc & a ) {
  if( onStack[ g.target( a ) ] && back == INVALID )
   back = a;
```

```
  }
 };

int main()
{
 // a DAG on six nodes: 0 -> 1, 0 -> 2, 1 -> 3, 2 -> 3, 3 -> 4, 2 -> 5
 ListDigraph g;
 std::vector< Node > v;
 for( int i = 0 ; i < 6 ; ++i )
  v.push_back( g.addNode() );
 const int arcs[][ 2 ] = { { 0 , 1 } , { 0 , 2 } , { 1 , 3 } ,
                           { 2 , 3 } , { 3 , 4 } , { 2 , 5 } };
 for( const auto & a : arcs )
  g.addArc( v[ a[ 0 ] ] , v[ a[ 1 ] ] );

 int bad = 0;

 // breadth-first search: the number of arcs of a shortest path
 Bfs< ListDigraph > bfs( g );
 bfs.run( v[ 0 ] );
 const int expected[] = { 0 , 1 , 1 , 2 , 3 , 2 };
 for( int i = 0 ; i < 6 ; ++i )
  if( ! bfs.reached( v[ i ] ) || bfs.dist( v[ i ] ) != expected[ i ] )
   ++bad;

 // the function-type interface, stopped at the target
 Path< ListDigraph > p;
 int d = -1;
 bool found = lemon::bfs( g ).path( p ).dist( d ).run( v[ 0 ] , v[ 4 ] );
 if( ! found || d != 3 || p.length() != 3 || ! checkPath( g , p ) )
  ++bad;

 // depth-first search over all the nodes: the order in which they are
 // processed (finished), reversed, is a topological order
 std::vector< Node > finished;
 dfs( g )
   .processedMap( loggerBoolMap( std::back_inserter( finished ) ) )
   .run();
 ListDigraph::NodeMap< int > pos( g );
 for( int i = 0 ; i < int( finished.size() ) ; ++i )
  pos[ finished[ i ] ] = int( finished.size() ) - 1 - i;
 for( ListDigraph::ArcIt a( g ) ; a != INVALID ; ++a )
  if( pos[ g.source( a ) ] >= pos[ g.target( a ) ] )
   ++bad;
 if( finished.size() != 6 )
  ++bad;

 // the DAG has no cycle; the arc 4 -> 1 makes one
 for( int round = 0 ; round < 2 ; ++round ) {
  if( round == 1 )
   g.addArc( v[ 4 ] , v[ 1 ] );
  ListDigraph::NodeMap< bool > onStack( g , false );
  Arc back = INVALID;
  CycleVisitor visitor( g , onStack , back );
  DfsVisit< ListDigraph , CycleVisitor > visit( g , visitor );
  visit.run();
  if( ( back != INVALID ) != ( round == 1 ) )
   ++bad;
  // the arc found closes the cycle 1 -> 3 -> 4 -> 1, which one of its
  // three arcs depends on the order of the out-arcs
  if( round == 1 ) {
   int s = g.id( g.source( back ) ) , t = g.id( g.target( back ) );
   if( ! ( ( s == 1 && t == 3 ) || ( s == 3 && t == 4 ) ||
           ( s == 4 && t == 1 ) ) )
    ++bad;
```

```
  }
 }

std::cout << "topological order:";
for( int i = int( finished.size() ) - 1 ; i >= 0 ; --i )
 std::cout << ' ' << g.id( finished[ i ] );
std::cout << "; " << bad << " errors" << std::endl;
return( bad == 0 ? 0 : 1 );
}
```

9.6 Maximum cardinality search

The maximum cardinality search orders the nodes of a graph so that each node, when it is chosen, has the largest number of neighbours among those already chosen. In abLEMON, `MaxCardinalitySearch<GR, CAP, TR>` extends it to capacities on the arcs. The cardinality of an unprocessed node u is

$$\kappa(u) \;=\; \sum_{uv \in A,\ v \text{ processed}} c(uv), \tag{9.1}$$

and at each step the algorithm processes an unprocessed node of largest cardinality (Algorithm 9.3), recording its cardinality at that moment. On a digraph the sum (9.1) is over the arcs from u to the processed nodes, since the algorithm scans the in-arcs of the node it processes. On an undirected graph, whose arcs are the two directions of the edges, it is over the edges that join u to processed nodes; with unit capacities, $\kappa(u)$ is then the number of processed neighbours of u. When the heap becomes empty, i.e., when no unprocessed node is joined to a processed one, `run()` takes a new source among the nodes not reached.

Algorithm 9.3 (Maximum cardinality search, `MaxCardinalitySearch`).

```
1  H := empty heap of maximum priority; push the source s with priority 0
2  while H is not empty:
3     u := the node of largest priority in H; cardinality(u) := its priority; pop u
4     processed(u) := true
5     for each arc wu entering u:
6        if w was never in H: push w with priority c(wu)
7        else if w is in H: increase the priority of w by c(wu)
```

With unit capacities, the reverse of the order of a maximum cardinality search of a graph is a perfect elimination order (one in which the neighbours of each node that follow it form a clique) if and only if the graph is chordal, i.e., if every cycle of four or more nodes has a chord (Tarjan and Yannakakis 1984). Such a test can be done in linear time, and the example at the end of this section implements a simple version of it. With general capacities, the order is a maximum adjacency order, which the algorithm of Nagamochi and Ibaraki (chapter 15) needs.

Its template parameters are the graph type `GR`, the type `CAP` of the map of the capacities, by default `ConstMap<GR::Arc, Const<int, 1>>` (unit capacities), and the traits class `TR`, by default `MaxCardinalitySearchDefaultTraits<GR, CAP>`. Its traits give the `CardinalityMap` (a `NodeMap` of the value type of the capacities), the `ProcessedMap` (a `NullMap` by default), the `HeapCrossRef` and the `Heap`. Note that the heap is a `BucketHeap` of maximum priority for the unit capacities, and a `BinHeap` with `std::greater`, i.e., again of maximum priority, otherwise; the internal templates `HeapSelector` and `Selector` of the traits make this choice. Hence, the search takes $O(n+m)$ time with unit capacities, and $O(m \log n)$ with the binary heap. Named parameters `SetCapacityMap<T>`, `SetCardinalityMap<T>`, `SetProcessedMap<T>` and `SetHeap<H, CR>` change these types and require the map, or the heap, to be given by the corresponding setter; `SetStandardHeap<H, CR>` changes the type of the heap, which the algorithm then allocates with its cross reference map.

Its constructor takes the graph and the capacity map, or only the graph, in which case the map of unit capacities is created. Its setters are `capacityMap(m)`, `cardinalityMap(m)`, `processedMap(m)` and `heap(hp, cr)`, and each has a reader without arguments (the comment of the reader `processedMap()` calls it the cardinality map, but it returns the processed map). Its execution control is made of `init()`, `addSource(s, capacity)`, which pushes `s` with the given priority (0 by default) if it was never in the heap, `processNextNode()`, `nextNode()`, `emptyQueue()` and `emptySize()`, which, despite its name, is the number of nodes in the heap. Searches are run by `start()`, `run(s)`, `run()`, and `start(dest)` and `start(nm)`; the last two stop when `dest`, or a node with `nm` true, is on the top of the heap, and write its cardinality (and `true` into the `ProcessedMap`) without removing it from the heap.

Its queries are `cardinality(v)`, i.e., the cardinality of v when it was processed (undefined for a node never processed), `currentCardinality(v)`, its priority in the heap (for a node reached but not processed), and `cardinalityMap()`. Also, `reached(v)` is true if v has ever been in the heap, and `processed(v)` if it has left it.

In the following program we test two graphs for chordality: we run the search step by step, recording the order of the nodes, and we check that the neighbours of each node chosen before it form a clique, and that their number is the cardinality given by the algorithm.

```
// Maximum cardinality search (section 9.6) as a test of chordality: the
// search order of a chordal graph, reversed, is a perfect elimination
// order, i.e., the neighbours of each node searched before it form a
// clique; on a chordless cycle of four nodes the test fails.

#include <iostream>
#include <vector>
#include <lemon/list_graph.h>
#include <lemon/max_cardinality_search.h>

using namespace lemon;

typedef ListGraph::Node Node;

// true if the order of the search is a perfect elimination order reversed
bool chordal( const ListGraph & g , int & bad )
{
 MaxCardinalitySearch< ListGraph > mcs( g );
 std::vector< Node > order;
 mcs.init();
 for( ListGraph::NodeIt n( g ) ; n != INVALID ; ++n )
  if( ! mcs.reached( n ) ) {
   mcs.addSource( n );
   while( ! mcs.emptyQueue() )
    order.push_back( mcs.processNextNode() );
   }
 ListGraph::NodeMap< int > rank( g );
 for( int i = 0 ; i < int( order.size() ) ; ++i )
  rank[ order[ i ] ] = i;
 bool ok = true;
 for( int i = 0 ; i < int( order.size() ) ; ++i ) {
  std::vector< Node > before;  // the neighbours searched before
  for( ListGraph::IncEdgeIt e( g , order[ i ] ) ; e != INVALID ; ++e ) {
   Node w = g.oppositeNode( order[ i ] , e );
   if( rank[ w ] < i )
    before.push_back( w );
   }
  // with unit capacities the cardinality is the number of them
  if( mcs.cardinality( order[ i ] ) != int( before.size() ) )
   ++bad;
  for( std::size_t a = 0 ; a < before.size() ; ++a )
   for( std::size_t b = a + 1 ; b < before.size() ; ++b )
    if( findEdge( g , before[ a ] , before[ b ] ) == INVALID )
     ok = false;
  }
 return( ok );
 }

int main()
{
 int bad = 0;

 // a triangulated hexagon: the cycle 0 .. 5 and the chords 0-2, 0-3, 0-4
 ListGraph g;
 std::vector< Node > v;
 for( int i = 0 ; i < 6 ; ++i )
  v.push_back( g.addNode() );
```

```
for( int i = 0 ; i < 6 ; ++i )
 g.addEdge( v[ i ] , v[ ( i + 1 ) % 6 ] );
for( int i = 2 ; i <= 4 ; ++i )
 g.addEdge( v[ 0 ] , v[ i ] );
if( ! chordal( g , bad ) )
 ++bad;

// the cycle of four nodes, with no chord
ListGraph c;
std::vector< Node > w;
for( int i = 0 ; i < 4 ; ++i )
 w.push_back( c.addNode() );
for( int i = 0 ; i < 4 ; ++i )
 c.addEdge( w[ i ] , w[ ( i + 1 ) % 4 ] );
if( chordal( c , bad ) )
 ++bad;

std::cout << bad << " errors" << std::endl;
return( bad == 0 ? 0 : 1 );
}
```

9.7 Exercises

Exercise 9.1. Run `Bfs` from several sources added with `addSource()` and show that `dist(v)` is the distance from the nearest source; compare with a single search from a new node joined by an arc to each source. (The test `test/bfs_test.cc` exercises the interface of `Bfs`.)

Exercise 9.2. Use `Bfs` on an undirected graph to test whether it is bipartite, by checking that no edge joins two nodes at the same distance from the source of their component; compare the answer with `bipartite()` of chapter 18.

Exercise 9.3. Write a visitor of `DfsVisit` that numbers the nodes in preorder and in postorder, and use the two numbers to classify the arcs into tree, back, forward and cross arcs; check that a digraph is acyclic if and only if there is no back arc. (The example `examples/09-search.cc` detects the back arcs; the test `test/dfs_test.cc` exercises `Dfs` and `DfsVisit`.)

Exercise 9.4. Stop a `Dfs` by `start(am)` at the first arc whose target is on the current path, with an arc map computed on the fly from the `ReachedMap` and the `ProcessedMap`; explain why a node map would not suffice for this, which is why `start()` of `Dfs` takes an arc map.

Exercise 9.5. Run `MaxCardinalitySearch` with the capacities of the edges of a graph, and verify that the cardinality of the last node processed, t, equals the capacity of a minimum cut between t and the node s processed just before it (the lemma behind the algorithm of Nagamochi and Ibaraki), by comparing it with `Preflow` of chapter 12. (The test `test/max_cardinality_search_test.cc` checks the cardinalities on a small digraph.)

10. Shortest paths

We are concerned in this chapter with the shortest paths of a digraph $G = (N, A)$, with $n = |N|$ nodes and $m = |A|$ arcs, whose arcs have lengths $\ell : A \to \mathbb{R}$. The library has five classes for them. `Dijkstra` (`lemon/dijkstra.h`) computes the shortest paths from a set of sources when the lengths are nonnegative, and `BidirDijkstra` and `AStar` (`lemon/bidir_dijkstra.h`, `lemon/astar.h`) compute a shortest path between two nodes, usually scanning much less of the digraph. `BellmanFord` (`lemon/bellman_ford.h`) accepts negative lengths and detects the negative cycles, and `Suurballe` (`lemon/suurballe.h`) finds k arc-disjoint paths of least total length between two nodes. We first state the problem and the optimality conditions that all of them rely on (§10.1), and then describe the classes in this order.

10.1 The problem and its optimality conditions

We define the length of a path $P = (a_1, \dots, a_k)$ as $\ell(P) = \sum_{i=1}^{k} \ell(a_i)$. Given a source s, the distance of a node v is

$$d(v) \;=\; \min\{\, \ell(P) \;:\; P \text{ is a path from } s \text{ to } v \,\}, \tag{10.1}$$

with $d(v) = +\infty$ if v cannot be reached from s. This minimum exists (and is attained by a path without repeated nodes) if no cycle of negative length can be reached from s; otherwise, the distance of the nodes reachable from such a cycle is $-\infty$, and a shortest walk does not exist. With several sources S, each with an initial distance $d_0(s)$, the distance of v is the least value of $d_0(s) + \ell(P)$ over the sources s and the paths P from s to v. All the classes of this chapter find shortest paths by maintaining tentative distances d and a predecessor arc for each node, and stop when the following conditions hold.

Theorem 10.1. *Let d be finite on the nodes reachable from s, with $d(s) = 0$, and let it be the length of some path from s for each of them. Then d is the distance (10.1) if and only if*

$$d(v) \;\le\; d(u) + \ell(uv) \qquad \text{for each arc } uv \text{ with } u \text{ reachable.} \tag{10.2}$$

If, moreover, each node $v \neq s$ has a predecessor arc uv with $d(v) = d(u) + \ell(uv)$, the predecessor arcs form a tree of shortest paths.

We refer to Ahuja, Magnanti and Orlin 1993, §5.2 for the proof: summing (10.2) along any path from s to v shows that no path is shorter than $d(v)$. Equivalently, (10.2) means that the reduced lengths

$$\ell_\pi(uv) \;=\; \ell(uv) + \pi(u) - \pi(v) \tag{10.3}$$

are nonnegative for the potential $\pi = d$. Since $\ell_\pi(P) = \ell(P) + \pi(u) - \pi(v)$ for any path P from u to v, a change of the lengths into reduced lengths changes the length of all the paths between two given nodes by the same amount, and keeps the shortest ones; we shall use this fact for A* (§10.3) and for Suurballe's algorithm (§10.5). Also, the shortest path problem is the minimum cost flow problem of chapter 13 in which one unit of flow is sent from s to each other node, and d is an optimal potential of its dual.

Table 10.1. The classes of this chapter, with the lengths they accept and the cost of a run on a digraph of n nodes and m arcs (k is the number of paths of `Suurballe`).

class	lengths	answers	time with the default heap
`Dijkstra`	≥ 0	all the distances from a set of sources	$O((n+m)\log n)$
`BidirDijkstra`	≥ 0	one path between two nodes	$O((n+m)\log n)$, usually much less
`AStar`	≥ 0	one path between two nodes, with a heuristic	$O((n+m)\log n)$ with a consistent heuristic

class	lengths	answers	time with the default heap
`BellmanFord`	any	all the distances, or a negative cycle	$O(nm)$
`Suurballe`	≥ 0	k arc-disjoint paths between two nodes	$O(k\,(n+m)\log n)$

10.2 Dijkstra's algorithm

In the algorithm of Dijkstra 1959, the reached nodes that are not yet processed are kept in a heap, keyed by their tentative distance. At each step it removes a node of least tentative distance, declares its distance final, and relaxes its out-arcs, i.e., it decreases the tentative distance of each target v of an arc uv to $d(u)+\ell(uv)$ if this is smaller (Algorithm 10.1). With nonnegative lengths, the nodes are processed in nondecreasing order of distance, and a node removed from the heap cannot be reached later by a shorter path.

Algorithm 10.1 (Dijkstra's algorithm, `Dijkstra`).

```
init():
1  for each node v: pred(v) := INVALID; state(v) := PRE_HEAP
addSource(s, d0):
2  push s on H with priority d0
start():
3  while H is not empty:
4     u := the node of least priority in H; d(u) := its priority; pop u   [processed]
5     for each arc a = uv out of u:
6        if v was never in H: push v with priority d(u) + l(a); pred(v) := a
7        else if v is in H and d(u) + l(a) < priority(v):
8           decrease the priority of v to d(u) + l(a); pred(v) := a
```

Theorem 10.2. *If the lengths are nonnegative, each node processed by Algorithm 10.1 has its distance from the sources as its priority when it leaves the heap, and the predecessor arcs of the processed nodes form a forest of shortest paths. The algorithm performs at most n pushes and pops and at most m decreases of a priority.*

A proof is in Cormen et al. 2001, §24.3; the bound on the operations holds since each node enters the heap once and each arc is relaxed once, from its source. Hence, the time depends on the heap (cf. chapter 7): with the `BinHeap` of the default traits, all the operations take $O(\log n)$ and the search takes $O((n+m)\log n)$, while a `FibHeap` gives $O(m+n\log n)$ by its decreases in amortized $O(1)$ (Fredman and Tarjan 1987). Note that the Fibonacci bound is better only in theory for most digraphs of practical size, the binary heap (or the 4-ary `QuadHeap`) being usually faster; with small integer lengths, a `RadixHeap` or a `BucketHeap` is perhaps worth a try.

Template parameters and traits. `Dijkstra<GR, LEN, TR>` takes the digraph type `GR` (`ListDigraph` by default), the type `LEN` of the map of the lengths (by default `GR::ArcMap<int>`), whose `Value` is the type of the distances, and the traits class `TR`, by default `DijkstraDefaultTraits<GR, LEN>`. These traits give `OperationTraits`, i.e., the operations on the distances (by default `DijkstraDefaultOperationTraits<Value>`, with the static `zero()` and the members `plus()` and `less()` that are the sum and the comparison of the type), and `HeapCrossRef` and `Heap`, i.e., a `NodeMap<int>` and a `BinHeap` with `std::less`. They also give the three maps `PredMap` (a `NodeMap<Arc>`), `DistMap` (a `NodeMap` of the distances) and `ProcessedMap` (a `NullMap`). Named parameters `SetPredMap<T>`, `SetDistMap<T>`, `SetProcessedMap<T>` and `SetStandardProcessedMap` change the maps as in §9.1. Moreover, `SetHeap<H, CR>` sets the types of the heap and of its cross reference map, both of which must then be given by `heap()`; `SetStandardHeap<H, CR>` sets them for a heap that the algorithm can allocate (one whose constructor takes the cross reference map, which takes the digraph). Finally, `SetOperationTraits<T>` replaces the operations.

Operations on the distances. In the algorithm, the priority of a node is computed by `plus()` and two priorities are compared by `less()`, the sources starting from `zero()`. Other operations give other problems, provided that the length of a path does not decrease when an arc is appended to it, which is the property that Theorem 10.2 needs. For instance, `plus()` defined as the maximum gives the paths of least bottleneck, i.e., the paths whose longest arc is as short as possible, which the example at the end of this section computes. Note that the heap still orders the priorities by the comparison of its type (`std::less` in the default traits), which must agree with `less()`.

abLEMON. `Dijkstra` (and `BellmanFord`, §10.4) keep an object of their operation traits class, default constructed or given by `operationTraits(ops)`, and call its members `plus()` and `less()`; hence, the operations may depend on the instance, e.g., on a tolerance stored in the object. In LEMON 1.3.1 they were

static functions of the traits class.

Interface. Its constructor takes the digraph and the length map, and its setters are `lengthMap(m)`, `operationTraits(ops)` (with the reader `operationTraits()`), `predMap(m)`, `distMap(m)`, `processedMap(m)` and `heap(hp, cr)`; the maps and the heap not given are allocated by `init()` and deleted by the destructor. Its execution control is made of `init()`, `addSource(s, dst)`, `processNextNode()`, `nextNode()`, `emptyQueue()` and `queueSize()`, the three forms of `start()`, and `run(s)` and `run(s, t)`.

A call `addSource(s, dst)` adds a source with the initial distance `dst` (`zero()` by default). It pushes `s` if it is not in the heap; if `s` is already in the heap with a priority less than `dst`, it sets the priority to `dst` and the predecessor of `s` to `INVALID`. This is the behavior that the documentation describes, and it matters only if a source is added twice; we remark that it raises the priority, and that a node already processed is pushed again. The sources are therefore better added once each, after `init()` and before `start()`. Then `start()` processes the nodes until the heap is empty, while `start(t)` processes them until `t` is on the top of the heap, then writes the distance of `t`, marks it processed and pops it without relaxing its arcs; `run(s, t)` does `init()`, `addSource(s)` and `start(t)`, and returns whether `t` was processed. Similarly, `start(nm)` stops when a node with `nm` true is on the top of the heap, writes its distance and marks it processed (in the `ProcessedMap`, while it stays in the heap), and returns it, or returns `INVALID` if the heap empties first.

Its queries are `dist(v)`, `predArc(v)`, `predNode(v)`, `path(t)` (a path dumper, cf. §8.1), `distMap()`, `predMap()`, `reached(v)` (whether v has been in the heap) and `processed(v)` (whether it has left it). The distance is written into the `DistMap` only when a node is processed; hence, `dist(v)` is undefined for a node that is reached but not processed, and for such a node, `currentDist(v)` returns its priority in the heap, i.e., an upper bound on its distance that may still decrease.

Function-type interface. Finally, `dijkstra(g, length)` returns a `DijkstraWizard`, whose named function parameters are `predMap(m)`, `distMap(m)`, `processedMap(m)`, `path(p)` and `dist(d)`, and whose runs are `run(s)` and `run(s, t)`. The latter returns whether `t` is reached and fills the path and the distance, as in §9.4; the base class `DijkstraWizardBase` holds the parameters.

The following program, first, runs Dijkstra's algorithm on a grid and checks the distances against (10.2). Then it uses two sources, the function-type interface and the operations of the least bottleneck. Finally, it compares the results with those of the two point-to-point searches of the next section.

```
// Dijkstra's algorithm and the two point-to-point searches of sections
// 10.2 and 10.3 on a grid: the distances checked against the optimality
// conditions (10.2), two sources, the function-type interface, a path of
// least bottleneck by other operation traits, and BidirDijkstra and AStar
// with the Manhattan distance as the heuristic.

#include <cstdlib>
#include <iostream>
#include <vector>
#include <lemon/list_graph.h>
#include <lemon/dijkstra.h>
#include <lemon/bidir_dijkstra.h>
#include <lemon/astar.h>
#include <lemon/maps.h>
#include <lemon/path.h>

using namespace lemon;

typedef ListDigraph::Node Node;
typedef ListDigraph::Arc Arc;
typedef ListDigraph::ArcMap< int > LengthMap;

// the operations of a path of least bottleneck: the "length" of a path is
// its longest arc, which is nondecreasing along the path as a sum of
// nonnegative lengths is
struct BottleneckOps
{
 typedef int Value;
 static Value zero() { return( 0 ); }
 Value plus( const Value & l , const Value & r ) const {
  return( l < r ? r : l );
```

```
  }
 bool less( const Value & l , const Value & r ) const { return( l < r ); }
 };

int main()
{
 const int K = 12;  // a K x K grid, with an arc each way between neighbours
 ListDigraph g;
 std::vector< Node > v( K * K );
 for( int i = 0 ; i < K * K ; ++i )
  v[ i ] = g.addNode();
 LengthMap len( g );
 ListDigraph::NodeMap< int > row( g ) , col( g );
 for( int r = 0 ; r < K ; ++r )
  for( int c = 0 ; c < K ; ++c ) {
   row[ v[ r * K + c ] ] = r;
   col[ v[ r * K + c ] ] = c;
   // the lengths are at least 1, so that the Manhattan distance to the
   // target is a consistent heuristic
   if( c + 1 < K ) {
    len[ g.addArc( v[ r * K + c ] , v[ r * K + c + 1 ] ) ] =
      1 + ( r * 7 + c * 3 ) % 5;
    len[ g.addArc( v[ r * K + c + 1 ] , v[ r * K + c ] ) ] =
      1 + ( r * 3 + c * 5 ) % 4;
    }
   if( r + 1 < K ) {
    len[ g.addArc( v[ r * K + c ] , v[ ( r + 1 ) * K + c ] ) ] =
      1 + ( r * 5 + c * 7 ) % 6;
    len[ g.addArc( v[ ( r + 1 ) * K + c ] , v[ r * K + c ] ) ] =
      1 + ( r + c ) % 3;
    }
   }
 Node s = v[ 0 ] , t = v[ K * K - 1 ];
 int bad = 0;

 // one source: the distances satisfy (10.2), tight on the tree arcs
 Dijkstra< ListDigraph , LengthMap > dijkstra( g , len );
 dijkstra.run( s );
 for( ListDigraph::ArcIt a( g ) ; a != INVALID ; ++a ) {
  int red =
    len[ a ] + dijkstra.dist( g.source( a ) ) - dijkstra.dist( g.target( a ) );
  if( red < 0 || ( dijkstra.predArc( g.target( a ) ) == a && red != 0 ) )
   ++bad;
  }
 int d_st = dijkstra.dist( t );

 // two sources at distance 0: the distance from the nearer one
 Dijkstra< ListDigraph , LengthMap > multi( g , len );
 multi.init();
 multi.addSource( s );
 multi.addSource( t );
 multi.start();
 Dijkstra< ListDigraph , LengthMap > from_t( g , len );
 from_t.run( t );
 for( int i = 0 ; i < K * K ; ++i ) {
  int a = dijkstra.dist( v[ i ] ) , b = from_t.dist( v[ i ] );
  if( multi.dist( v[ i ] ) != ( a < b ? a : b ) )
   ++bad;
  }

 // the function-type interface, stopped when t is processed
 Path< ListDigraph > p;
 int d = 0;
 if( ! lemon::dijkstra( g , len ).path( p ).dist( d ).run( s , t ) ||
     d != d_st || ! checkPath( g , p ) || pathTarget( g , p ) != t )
```

```
 ++bad;

// a path whose longest arc is as short as possible
Dijkstra< ListDigraph , LengthMap >::SetOperationTraits<
  BottleneckOps >::Create bottleneck( g , len );
bottleneck.run( s );
int longest = 0;
Path< ListDigraph > bp = bottleneck.path( t );
for( Path< ListDigraph >::ArcIt a( bp ) ; a != INVALID ; ++a )
 if( len[ a ] > longest )
  longest = len[ a ];
if( longest != bottleneck.dist( t ) || longest > 3 )
 ++bad;

// the two point-to-point searches agree with Dijkstra
BidirDijkstra< ListDigraph , LengthMap > bidir( g , len );
if( ! bidir.run( s , t ) || bidir.dist() != d_st ||
    ! checkPath( g , bidir.path() ) || pathSource( g , bidir.path() ) != s )
 ++bad;
ListDigraph::NodeMap< int > manhattan( g );
for( int i = 0 ; i < K * K ; ++i )
 manhattan[ v[ i ] ] = std::abs( row[ v[ i ] ] - row[ t ] ) +
                       std::abs( col[ v[ i ] ] - col[ t ] );
typedef ListDigraph::NodeMap< int > HeuristicMap;
AStar< ListDigraph , LengthMap , HeuristicMap > astar( g , len , manhattan );
typedef ConstMap< Node , int > ZeroMap;
ZeroMap zero( 0 );
AStar< ListDigraph , LengthMap , ZeroMap > blind( g , len , zero );
if( ! astar.run( s , t ) || astar.dist() != d_st || ! blind.run( s , t ) ||
    blind.dist() != d_st )
 ++bad;
// a consistent heuristic scans each node at most once
if( astar.scanned() > blind.scanned() || blind.scanned() > K * K )
 ++bad;

std::cout << "distance " << d_st << ", A* scanned " << astar.scanned()
          << " nodes, with h = 0 " << blind.scanned() << "; " << bad
          << " errors" << std::endl;
return( bad == 0 ? 0 : 1 );
}
```

10.3 Shortest paths between two nodes

When only the path from a source s to a target t is wanted, `Dijkstra::run(s, t)` stops when t is processed, but it still processes all the nodes closer to s than t. Both classes of this section usually scan far fewer nodes. Both have nonnegative lengths, both answer any number of queries with one object, each by a call `run(s, t)` that returns whether t is reachable from s, and both give the result by `dist()` and `path()` (a `lemon::Path`, empty when $s = t$). Their node maps are allocated once, by the constructor; each run resets only the nodes that the previous run touched, and its time is therefore proportional to the part of the digraph that it scans (times the logarithm of the size of the heap). Their heap is a `BinHeap` by default. The named parameter `SetHeap<H, CR>` replaces it by a heap of type `H` with a cross reference map of type `CR` (a `NodeMap<int>` by default), as in `SetHeap<FibHeap<int, IntNodeMap>>::Create`.

> **abLEMON.** `BidirDijkstra` and `AStar` are new in abLEMON (from trac #249 and #376 of LEMON), and take the type of their heap from a traits class (`BidirDijkstraDefaultTraits`, `AStarDefaultTraits`) through `SetHeap`.

Bidirectional search. `BidirDijkstra<GR, LEN, TR>` runs two searches of Dijkstra (Pohl 1971), a forward one from s along the arcs and a backward one from t against them, with tentative distances d_f and d_b. At each step it advances the search whose least priority is smaller. Each arc uv scanned by one search whose other end has been seen by the other search gives a path from s to t, of length $d_f(u) + \ell(uv) + d_b(v)$, and the algorithm keeps the shortest one found, of length μ. It stops as soon as

$$\min_{H_f} d_f \;+\; \min_{H_b} d_b \;\geq\; \mu, \tag{10.4}$$

where H_f and H_b are the heaps of the two searches (Algorithm 10.2).

Algorithm 10.2 (Bidirectional Dijkstra, `BidirDijkstra::run(s, t)`).

```
1  if s = t: return true with the empty path
2  push s on Hf and t on Hb with priority 0; mu := +inf
3  while Hf and Hb are not empty:
4     if a path was found and min(Hf) + min(Hb) >= mu: stop
5     choose the search X with the smaller least priority, and Y the other one
6     u := pop the top of X
7     for each arc a of u (out-arcs if X is forward, in-arcs if backward), w its other end:
8        relax w in X with d_X(u) + l(a), as in Algorithm 10.1
9        if w has been seen by Y and d_X(u) + l(a) + d_Y(w) < mu:
10          mu := d_X(u) + l(a) + d_Y(w); meet := a
11 if no path was found: return false
12 path := the forward tree to meet, meet, the backward tree from meet
```

Theorem 10.3. *With nonnegative lengths, when (10.4) holds the value μ is the distance from s to t, and the path built at line 12 has length μ.*

Its proof is short. Let $\delta(x, y)$ be the distance from x to y, and suppose that a path P from s to t has $\ell(P) < \mu$ when (10.4) holds. Then each node x of P has $\delta(s, x) < \min_{H_f} d_f$ or $\delta(x, t) < \min_{H_b} d_b$, i.e., it has been processed by one of the two searches, with its final distance. Hence, P has an arc uv with u processed by the forward search and v by the backward one; the search that processed the second of them scanned uv and set $\mu \le d_f(u) + \ell(uv) + d_b(v) \le \ell(P)$ at line 10, a contradiction. In practice, the two searches meet roughly halfway; on a digraph in which the number of nodes within distance r grows quickly with r (as in a road network, roughly as r^2), they scan far fewer nodes than one search from s. Its template parameters are `GR`, `LEN` (by default `GR::ArcMap<int>`) and `TR` (by default `BidirDijkstraDefaultTraits<GR, LEN>`), and the backward search needs the in-arcs of the digraph. Its internal record `Search` holds the distances, the predecessors, the cross reference map and the heap of one search.

Search guided by a heuristic. `AStar<GR, LEN, HM, TR>` runs a search of Dijkstra from s in which the priority of a node is its tentative distance plus a heuristic value $h(v)$, an estimate of its distance $\delta(v, t)$ to t, given by a node map of type `HM` with the value type of the lengths (Hart, Nilsson and Raphael 1968). The search stops when t is removed from the heap (Algorithm 10.3). Such a heuristic is admissible if

$$h(v) \;\le\; \delta(v, t) \qquad \text{for each node } v, \tag{10.5}$$

and consistent if

$$h(u) \;\le\; \ell(uv) + h(v) \qquad \text{for each arc } uv. \tag{10.6}$$

A consistent heuristic with $h(t) = 0$ is admissible, and with it the reduced lengths $\ell(uv) - h(u) + h(v)$ (i.e., (10.3) with $\pi = -h$) are nonnegative. Hence, A* is then Dijkstra's algorithm on the reduced lengths, which keep the shortest paths, and it scans each node at most once. With a heuristic that is everywhere zero, it is Dijkstra's algorithm stopped at t. The closer h is to $\delta(\cdot, t)$, the fewer nodes it scans; on a road network, the straight-line distance to t is a typical choice.

Algorithm 10.3 (A*, `AStar::run(s, t)`).

```
1  g(s) := 0; push s on H with priority h(s)
2  while H is not empty:
3     u := pop the top of H; scanned := scanned + 1
4     if u = t: return true with the path of the predecessor arcs from s to t
5     for each arc a = uv out of u:
6        if v was never seen: g(v) := g(u) + l(a); pred(v) := a; push v with g(v) + h(v)
7        else if g(u) + l(a) < g(v):
8           g(v) := g(u) + l(a); pred(v) := a
9           decrease v to g(v) + h(v) if it is in H, else push it again
10 return false
```

Theorem 10.4. *If the lengths are nonnegative and the heuristic is admissible, $g(t)$ is the distance from s to t when t is removed from the heap. If the heuristic is consistent, no node is pushed again at line 9.*

We refer to Hart, Nilsson and Raphael 1968 for the first statement; the second follows from the remark on the reduced lengths above. With an admissible heuristic that is not consistent, a node whose distance improves after

its scan is pushed again (line 9) and scanned again. Its query `scanned()` returns the number of scans of the last run, a node scanned twice being counted twice. Note that the heuristic must be admissible for the target of each run; since the map is read through a reference, its values may be changed between two runs with different targets. We remark that `AStar` has no default for the type of the length map, and that its heap is ordered by the priorities $g + h$.

10.4 The Bellman-Ford algorithm

In the algorithm of Bellman 1958 and Ford 1956, negative lengths are allowed. It proceeds by rounds, each of which relaxes the out-arcs of the nodes whose distance changed in the previous round, the active nodes (Algorithm 10.4); with no negative cycle reachable from the sources, at most $n - 1$ rounds change a distance. Thus, the algorithm takes $O(nm)$ time in the worst case, and much less in practice when the active nodes are few. If a negative cycle is reachable from the sources, the rounds never end, and a cycle of negative length appears among the predecessor arcs.

Algorithm 10.4 (Bellman-Ford by rounds of the active nodes, `BellmanFord`).

```
init(value):
1  for each node v: pred(v) := INVALID; d(v) := value
2  active := all the nodes if value < infinity, else none
addSource(s, d0):
3  d(s) := d0; add s to active
processNextWeakRound():
4  next := empty
5  for each node u in active, for each arc a = uv out of u:
6     if d(u) + l(a) < d(v): d(v) := d(u) + l(a); pred(v) := a; add v to next
7  active := next; return (active is empty)
start():         repeat processNextWeakRound() at most n - 1 times, until it returns true
checkedStart():  the same at most n times; return (active is empty)
```

`processNextWeakRound()` reads the current distance $d(u)$ at line 6, which may already have been decreased in the same round. In `processNextRound()`, instead, the distances of the active nodes are copied at the beginning of the round and read from the copy, which keeps the rounds exact in the following sense.

Theorem 10.5. *After k calls of `processNextRound()` from a single source s, $d(v)$ is the least length of a path from s to v with at most k arcs, for each node v; after k calls of `processNextWeakRound()`, $d(v)$ is at most that length. If no negative cycle is reachable from s, the active set is empty after at most n rounds of either kind, and d is then the distance (10.1).*

A proof is in Ahuja, Magnanti and Orlin 1993, §5.4. Hence, `start()` performs at most $n - 1$ weak rounds and stops early when one changes no distance, and `checkedStart()` performs at most n and returns `false` if some node is still active, which means that a negative cycle is reachable from the sources. On the other hand, `limitedStart(num)` performs at most `num` exact rounds, and computes the distances of the paths of at most `num` arcs (the predecessor arcs, however, do not form such paths in general, as the documentation warns). Also, `run(s)` and `run(s, num)` are `init()` and `addSource(s)` followed by `start()` and `limitedStart(num)`.

Its other members are those of `Dijkstra`, with a few differences. Its template parameters `GR`, `LEN` and `TR` default to `ListDigraph`, `GR::ArcMap<int>` and `BellmanFordDefaultTraits<GR, LEN>`, and the named parameters are `SetPredMap<T>`, `SetDistMap<T>` and `SetOperationTraits<T>`. Its default operation traits `BellmanFordDefaultOperationTraits<V>` have, besides `zero()`, `plus()` and `less()`, a static `infinity()`, the infinity of `V` if it has one and its largest value otherwise; in the latter case `plus()` returns the largest value if one of its terms is that value, and an unreached node thus keeps its distance. The call `init(value)` gives all the nodes the distance `value` (`infinity()` by default). If `value` is finite, all the nodes are active; this computes the least lengths of the paths that may start anywhere, and is the usual way of looking for a negative cycle anywhere in the digraph (e.g., `init(0)` followed by `checkedStart()`). Then `addSource(s, dst)` sets the distance of `s` to `dst` (`zero()` by default) and makes it active.

Its queries are `dist(v)`, `predArc(v)`, `predNode(v)`, `path(t)`, `distMap()`, `predMap()` and `reached(v)`, which is true if the distance of v is not `infinity()`. Furthermore, the iterator `ActiveIt`, and the range `activeNodes()`, list the active nodes after the last round. Finally, `negativeCycle()` follows the predecessor arcs back from the active nodes and returns, as a `lemon::Path`, the first cycle of predecessor arcs that it meets, or an empty path if it meets none. A cycle of predecessor arcs created by the relaxations always has negative length (Ahuja,

Magnanti and Orlin 1993, §5.5). It may be called at any time, e.g., after each round, and returns a cycle as soon as the predecessor arcs reached from the active nodes contain one. The function `bellmanFord(g, length)` returns a `BellmanFordWizard`, with the named function parameters `predMap(m)`, `distMap(m)`, `path(p)` and `dist(d)`. Its runs are `run(s)` and `run(s, t)`, the latter running `run(s)` in full and then reading the path and the distance of `t`.

abLEMON. `BellmanFord` keeps the maps it allocates in a `std::unique_ptr`, apart from the pointers through which it reads them, and a map given by the user is therefore never deleted; LEMON 1.3.1 could delete one. Like `Dijkstra`, it keeps an object of its operation traits, set by `operationTraits(ops)` and read by `operationTraits()`; the test `test/bellman_ford_test.cc` uses one whose comparison has a tolerance, for which an improvement within the tolerance is no improvement.

In the following program we compute the distances with negative lengths and check them against (10.2). Then we compute the distances of the paths of at most two arcs, and find a negative cycle.

```
// The Bellman-Ford algorithm of section 10.4: distances with negative
// lengths and no negative cycle, the distances of the paths of at most k
// arcs, and a negative cycle detected and returned.

#include <iostream>
#include <vector>
#include <lemon/list_graph.h>
#include <lemon/bellman_ford.h>
#include <lemon/path.h>

using namespace lemon;

typedef ListDigraph::Node Node;
typedef ListDigraph::Arc Arc;
typedef ListDigraph::ArcMap< int > LengthMap;

int main()
{
 // s -> a (4), s -> b (2), b -> a (-3), a -> c (2), b -> c (5), c -> d (-1)
 ListDigraph g;
 Node s = g.addNode() , a = g.addNode() , b = g.addNode() , c = g.addNode() ,
      d = g.addNode();
 LengthMap len( g );
 len[ g.addArc( s , a ) ] = 4;
 len[ g.addArc( s , b ) ] = 2;
 len[ g.addArc( b , a ) ] = -3;
 len[ g.addArc( a , c ) ] = 2;
 len[ g.addArc( b , c ) ] = 5;
 len[ g.addArc( c , d ) ] = -1;
 int bad = 0;

 // the distances, and the conditions (10.2) they satisfy
 BellmanFord< ListDigraph , LengthMap > bf( g , len );
 bf.init();
 bf.addSource( s );
 if( ! bf.checkedStart() )  // no negative cycle
  ++bad;
 if( bf.dist( a ) != -1 || bf.dist( c ) != 1 || bf.dist( d ) != 0 ||
     bf.predNode( a ) != b )
  ++bad;
 for( ListDigraph::ArcIt e( g ) ; e != INVALID ; ++e )
  if( len[ e ] + bf.dist( g.source( e ) ) < bf.dist( g.target( e ) ) )
   ++bad;
 if( bf.path( d ).length() != 4 )  // s -> b -> a -> c -> d
  ++bad;

 // the paths of at most two arcs: d is not reached, c by s -> a -> c
 BellmanFord< ListDigraph , LengthMap > two( g , len );
 two.run( s , 2 );
 if( two.reached( d ) || two.dist( c ) != 6 || two.dist( a ) != -1 )
```

```
 ++bad;

// the function-type interface
int dist_d = 0;
if( ! bellmanFord( g , len ).dist( dist_d ).run( s , d ) || dist_d != 0 )
 ++bad;

// the arc d -> b (1) closes the cycle b -> a -> c -> d -> b of length -1
len[ g.addArc( d , b ) ] = 1;
BellmanFord< ListDigraph , LengthMap > neg( g , len );
neg.init();
neg.addSource( s );
if( neg.checkedStart() )
 ++bad;
Path< ListDigraph > cycle = neg.negativeCycle();
int total = 0;
for( Path< ListDigraph >::ArcIt e( cycle ) ; e != INVALID ; ++e )
 total += len[ e ];
if( cycle.empty() || ! checkPath( g , cycle ) || total >= 0 ||
    pathSource( g , cycle ) != pathTarget( g , cycle ) )
 ++bad;

std::cout << "negative cycle of " << cycle.length() << " arcs, length "
          << total << "; " << bad << " errors" << std::endl;
return( bad == 0 ? 0 : 1 );
}
```

10.5 Arc-disjoint paths of least total length

Given two nodes $s \neq t$ and an integer k, the problem is to find k paths from s to t that have no arc in common and the least total length. It is the minimum cost flow problem

$$\min\Big\{\sum_{a\in A}\ell(a)\,x_a \;:\; \sum_{a\in\delta^+(v)}x_a-\sum_{a\in\delta^-(v)}x_a=k\,[v=s]-k\,[v=t]\quad\forall v\in N,\quad x\in\{0,1\}^A\Big\},\tag{10.7}$$

where $\delta^+(v)$ and $\delta^-(v)$ are the arcs out of and into v, and $[\cdot]$ is 1 if the condition holds and 0 otherwise. An optimal x is the union of k arc-disjoint paths (and possibly of cycles of length zero, which the decomposition below does not use). Note that the shortest path followed by a second shortest path that avoids its arcs is not optimal in general; in Figure 10.1, the shortest path s, a, b, t leaves no second path at all. Thus, the algorithm of Suurballe 1974, in the form of Suurballe and Tarjan 1984, is the successive shortest path algorithm for (10.7). At iteration i it finds a shortest path from s to t in the residual digraph of the flow x of $i-1$ paths, which has

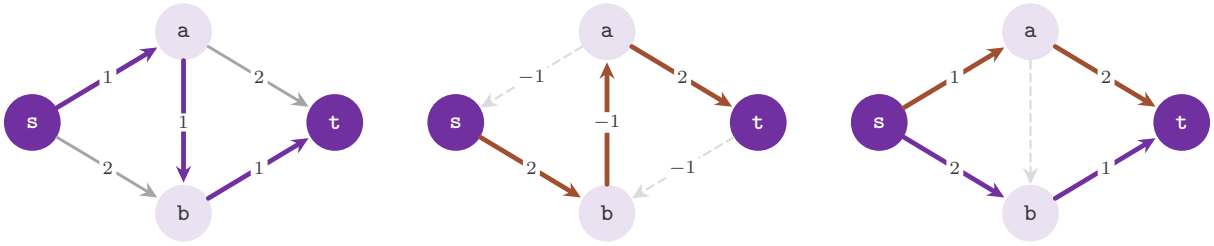
$$\text{an arc } uv \text{ of length } \ell(uv) \text{ for each } uv \text{ with } x_{uv}=0, \qquad \text{an arc } vu \text{ of length } -\ell(uv) \text{ for each } uv \text{ with } x_{uv}=1,\tag{10.8}$$

and sends one unit of flow along it, i.e., it sets x to 1 on its forward arcs and to 0 on the arcs whose reverse it uses (Algorithm 10.5).

This residual digraph has negative lengths, but with the potentials π kept by the algorithm the reduced lengths (10.3) of its arcs are nonnegative, and each path can therefore be found by Dijkstra's algorithm. After each search, the potential of each node processed by it is increased by its distance minus the distance of t, which keeps the reduced lengths of the residual arcs nonnegative and those of the arcs of the new path zero.

Algorithm 10.5 (Successive shortest paths with potentials, `Suurballe::findFlow(t, k)`).

```
1  x := 0; pi := 0; i := 0
2  while i < k:
3     run Dijkstra from s on the residual digraph (10.8) with the reduced lengths (10.3),
      stopping when t leaves the heap
4     if t is not reached: stop
5     for each node v processed: pi(v) := pi(v) + dist(v) - dist(t)
6     push one unit along the path found: x := 1 on its forward arcs, 0 on the reversed ones
7     i := i + 1
8  return i
```



first path, length 3 *residual: second path, length 3* *two paths, length 6*

Figure 10.1. Suurballe's algorithm on a digraph whose shortest path leaves no second arc-disjoint path: the first path, the second one in the residual digraph through the reverse of a-b (of length -1), and the two paths of least total length that result.

Theorem 10.6. *After i iterations, x is an optimal solution of (10.7) with $k = i$, and*

$$x_{uv} = 0 \;\Rightarrow\; \ell_\pi(uv) \geq 0, \qquad x_{uv} = 1 \;\Rightarrow\; \ell_\pi(uv) \leq 0 \qquad \text{for each arc } uv. \tag{10.9}$$

If the loop stops at line 4, there are no $i+1$ arc-disjoint paths from s to t.

Conditions (10.9) are the complementary slackness conditions of (10.7), which prove the optimality of x (cf. chapter 13); for the proof that the algorithm maintains them, see Suurballe and Tarjan 1984 or Ahuja, Magnanti and Orlin 1993, §9.7. Each iteration is one search of Dijkstra (rather cheap, since it may stop at t), which gives $O(k\,(n+m)\log n)$ time with the binary heap; the first one runs on the lengths, since all the potentials are zero, and the potentials of the nodes that it does not process stay zero. Then the paths are extracted from x by `findPaths()`, which follows from s the arcs of unit flow not yet used, in $O(km)$ time at most. Also, the node-disjoint version of the problem is solved by the same class on the digraph of the adaptor `SplitNodes` of chapter 5, in which each node is split into an arc of capacity one.

Interface. `Suurballe<GR, LEN, TR>` takes the digraph type, the type of the length map (by default `GR::ArcMap<int>`) and the traits class, by default `SuurballeDefaultTraits<GR, LEN>`. These traits give `FlowMap` (an `ArcMap<int>`), `PotentialMap` (a `NodeMap` of the lengths), `Path` (a `lemon::Path`, or any path type with `addBack()`), and the `HeapCrossRef` and the `Heap` (a `BinHeap`) of the internal searches; the named parameters `SetFlowMap<T>`, `SetPotentialMap<T>`, `SetPath<T>` and `SetHeap<H, CR>` change them. Its constructor takes the digraph and the lengths, which must be nonnegative. The setters `flowMap(m)` and `potentialMap(m)` set the maps of the flow and of the potentials (allocated otherwise).

A call `run(s, t, k)` is `init(s)` followed by `start(t, k)`, with $k = 2$ by default, and returns the number of paths found, i.e., k if there are at least k arc-disjoint paths, and their largest number otherwise. In turn, `start(t, k)` is `findFlow(t, k)` (Algorithm 10.5) followed by `findPaths()`. If the paths from one source to many targets are needed, `fullInit(s)` replaces `init(s)`: it runs one complete search of Dijkstra from `s` on the lengths, which then serves as the first iteration of each subsequent `start(t, k)`; each of them thus runs only $k-1$ searches. We remark that, after `fullInit(s)`, `findFlow(t, k)` takes the first path from that search without checking that `t` was reached; hence, it should be called only for targets reachable from `s`.

Its queries are `totalLength()` (the length of the flow, computed in $O(m)$), `flow(a)` and `flowMap()`, `potential(v)` and `potentialMap()` (an optimal dual solution, i.e., potentials that satisfy (10.9) with the flow), `pathNum()` and `path(i)` for i from 0 to `pathNum()` -1.

In the last program of the chapter we solve the instance of Figure 10.1, check that the two paths are arc-disjoint and that the flow and the potentials satisfy (10.9), and look for a third path, which does not exist.

```
// Suurballe's algorithm of section 10.5 on the digraph of Figure 10.1: the
// shortest path s -> a -> b -> t blocks any second arc-disjoint path, and
// the two paths of least total length do not use a -> b; the flow and the
// potentials satisfy the conditions (10.9).

#include <iostream>
#include <lemon/list_graph.h>
#include <lemon/suurballe.h>
```

```
using namespace lemon;

typedef ListDigraph::Node Node;
typedef ListDigraph::Arc Arc;
typedef ListDigraph::ArcMap< int > LengthMap;

int main()
{
 ListDigraph g;
 Node s = g.addNode() , a = g.addNode() , b = g.addNode() , t = g.addNode();
 LengthMap len( g );
 len[ g.addArc( s , a ) ] = 1;
 Arc ab = g.addArc( a , b );
 len[ ab ] = 1;
 len[ g.addArc( b , t ) ] = 1;
 len[ g.addArc( s , b ) ] = 2;
 len[ g.addArc( a , t ) ] = 2;
 int bad = 0;

 Suurballe< ListDigraph , LengthMap > suurballe( g , len );
 if( suurballe.run( s , t ) != 2 )  // k = 2 by default
  ++bad;
 if( suurballe.totalLength() != 6 || suurballe.flow( ab ) != 0 )
  ++bad;

 // the two paths share no arc and each goes from s to t
 ListDigraph::ArcMap< int > used( g , 0 );
 for( int i = 0 ; i < suurballe.pathNum() ; ++i ) {
  const Suurballe< ListDigraph , LengthMap >::Path & p = suurballe.path( i );
  if( ! checkPath( g , p ) || pathSource( g , p ) != s ||
      pathTarget( g , p ) != t )
   ++bad;
  for( Suurballe< ListDigraph , LengthMap >::Path::ArcIt e( p ) ;
       e != INVALID ;
       ++e )
   ++used[ e ];
  }
 for( ListDigraph::ArcIt e( g ) ; e != INVALID ; ++e )
  if( used[ e ] > 1 || used[ e ] != suurballe.flow( e ) )
   ++bad;

 // complementary slackness: reduced length >= 0 on the arcs without flow,
 // <= 0 on those with flow
 for( ListDigraph::ArcIt e( g ) ; e != INVALID ; ++e ) {
  int red = len[ e ] + suurballe.potential( g.source( e ) ) -
            suurballe.potential( g.target( e ) );
  if( ( suurballe.flow( e ) == 0 && red < 0 ) ||
      ( suurballe.flow( e ) == 1 && red > 0 ) )
   ++bad;
  }

 // a third path does not exist: two are returned
 if( suurballe.run( s , t , 3 ) != 2 )
  ++bad;

 std::cout << suurballe.pathNum() << " paths of total length "
           << suurballe.totalLength() << "; " << bad << " errors" << std::endl;
 return( bad == 0 ? 0 : 1 );
 }
```

10.6 Exercises

Exercise 10.1. Show by an example that Dijkstra's algorithm may give a wrong distance when one arc has a negative length, although no cycle is negative, and check the example against `BellmanFord`. (The test `test/di`

jkstra_test.cc exercises the interface of Dijkstra.)

Exercise 10.2. Compute the paths of least bottleneck of a digraph with the operation traits of the example examples/10-dijkstra.cc, and prove that Theorem 10.2 holds for them; then show that the maximum of the arc lengths along a path is not decreased by appending an arc, which is the property used.

Exercise 10.3. Compare the number of nodes scanned by Dijkstra::run(s, t), by BidirDijkstra and by AStar with the Manhattan distance on a grid of 100×100 nodes with random lengths at least 1, for random pairs of nodes. (The test test/point_to_point_test.cc checks the two point-to-point classes against Dijkstra, with heuristics that are zero, exact, half the exact one, and admissible but not consistent.)

Exercise 10.4. Find a negative cycle anywhere in a digraph by init(0) and checkedStart() of BellmanFord, and explain why init(0) is equivalent to adding a new source joined to each node by an arc of length zero. (The example examples/10-bellman-ford.cc and the test test/bellman_ford_test.cc find negative cycles.)

Exercise 10.5. Use Suurballe on the digraph of SplitNodes to find two node-disjoint paths of least total length, and verify that they share no inner node. (The test test/suurballe_test.cc checks the flow, the potentials and the paths.)

Exercise 10.6. Show that the potentials returned by Suurballe after k paths satisfy (10.9), and that $\pi(t) - \pi(s)$ is the length, in the residual digraph (10.8), of the last path found, by running the example examples/10-suurballe.cc with $k = 1$ and $k = 2$.

11. Spanning trees and arborescences

We are concerned in this chapter with two problems of choosing a cheapest set of arcs that connects the nodes of a graph. The first one is the minimum spanning tree of an undirected graph, which `kruskal()` of `lemon/kruskal.h` solves by Kruskal's algorithm on a union-find structure (§8.2). Its directed counterpart, the second one, is the minimum cost arborescence of a digraph rooted at a given node; `MinCostArborescence` of `lemon/min_cost_arborescence.h` solves it by the algorithm of Chu, Liu and Edmonds, and also gives an optimal dual solution, which proves the optimality of the arborescence. The two problems look alike; however, the greedy choice that solves the first is not enough for the second, whose algorithm has to contract cycles (cf. Figure 11.2).

11.1 Minimum spanning trees

Let $G = (N, E)$ be an undirected graph with costs $c : E \to \mathbb{R}$ on the edges. A spanning tree is a set $T \subseteq E$ of $n-1$ edges that contains no cycle; if G is connected, it therefore connects all the nodes. We consider the problem

$$\min \Big\{ c(T) = \sum_{e \in T} c(e) \ : \ T \text{ is a spanning tree of } G \Big\}. \tag{11.1}$$

If G is not connected, the same algorithm gives a spanning forest of least cost, i.e., a minimum spanning tree of each connected component. For a spanning tree T and an edge $f \notin T$, let $P_T(f)$ be the path of T between the ends of f. For an edge $e \in T$, let $C_T(e)$ be the set of the edges between the two components of $T \setminus \{e\}$. Since the optimal trees are characterized by two equivalent conditions, one can check a tree in either way.

Theorem 11.1. *A spanning tree T is optimal for (11.1) if and only if*

$$c(f) \ \geq \ c(e) \qquad \text{for each } f \notin T \text{ and each } e \in P_T(f), \tag{11.2}$$

or, equivalently, if and only if

$$c(e) \ \leq \ c(f) \qquad \text{for each } e \in T \text{ and each } f \in C_T(e). \tag{11.3}$$

A proof is in Ahuja, Magnanti and Orlin 1993, §13.3. In short, an edge that violates (11.2) or (11.3) gives a cheaper tree by an exchange; conversely, a tree that satisfies them can be turned into any optimal tree by exchanges that do not increase its cost.

Kruskal's algorithm (Kruskal 1956) builds a tree that satisfies (11.2) greedily. It scans the edges in nondecreasing order of cost, and keeps an edge if it joins two different components of the forest of the edges kept so far, i.e., if it closes no cycle (Algorithm 11.1). Thus, an edge that is discarded closes a cycle of edges kept before it, none of which costs more, and (11.2) holds at the end.

Algorithm 11.1 (Kruskal's algorithm, `kruskal()`).

```
1  if the costs are given as a map: sort the edges by nondecreasing cost
2  make a union-find structure with one class per node; total := 0
3  for each edge e = uv in the order of the costs:
4     if join(u, v):                    [u and v were in different classes]
5        out(e) := true; total := total + c(e)
6     else: out(e) := false
7  return total
```

Sorting takes $O(m \log m)$ time, and the m unions and finds take $O(m\,\alpha(m, n))$ by Theorem 8.1, which is usually the dominating cost when the edges are given already sorted. Note that the loop scans all the edges even after

the tree is complete, since the output receives a value for each of them; the work after the last useful edge is, however, a union-find operation per edge.

Interface. Function `kruskal(g, in, out)` accepts various types for its arguments, and returns the cost of the tree (or forest) found, of the type of the costs. Its graph `g` may be undirected, with the costs on its edges, or directed, in which case the arcs are taken as edges and their direction is disregarded. Its costs `in` may be given either (i) as a readable map on the edges (or arcs), whose values are the costs, or (ii) as an STL container of pairs `std::pair<GR::Edge, C>` (or `std::pair<GR::Arc, C>`), where `C` is the type of the costs. In case (i), the function collects the pairs of each item and its cost into a vector, and sorts it by cost with `std::sort`; in case (ii), the pairs must be already sorted by nondecreasing cost, and no sort is done.

Its output `out` may also be given in two ways. It may be (i) a writable map on the edges with `bool` values, which receives `true` for the edges of the tree and `false` for the others, each edge being written once. Alternatively, it may be (ii) an output iterator, e.g., `std::back_inserter(tree)`, into which the edges of the tree are copied in the order in which they are found (the function wraps it in a `LoggerBoolMap`, cf. chapter 6). A second overload takes `out` by constant reference, so that a temporary iterator may be passed.

This choice among the forms is made at compile time by the helper templates of the namespace `_kruskal_bits`, which are not part of the interface. They are `KruskalValueSelector`, `KruskalInputSelector`, `KruskalOutputSelector`, `SequenceInputIndicator`, `MapInputIndicator`, `SequenceOutputIndicator`, `MapOutputIndicator`, `RemoveConst` and `PairComp`; they recognize a sequence by a `value_type` with a `first_type`, and a map by its `Value`.

Figure 11.1 shows the tree of the example below. Its edges, sorted by cost, are kept in the order 1, 2, 3, 5, 6, while the edges of cost 4, 7, 8 and 9 close cycles.

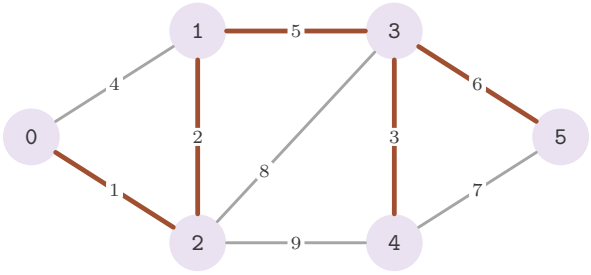


Figure 11.1. The minimum spanning tree found by Kruskal's algorithm, of total cost 17, with the cost of each edge.

In the following program we compute the tree twice, with a map of costs in and a map out, and then with a sorted vector of pairs in and an output iterator out. We also check (11.2), by a search of the path of the tree between the ends of each edge not in the tree, on the graph of the adaptor `FilterEdges` (chapter 5).

```
// Kruskal's algorithm of section 11.1 on the graph of Figure 11.1, with a
// cost map in and a bool map out, then with a sorted sequence in and an
// output iterator out; the tree is checked by the cycle condition (11.2).

#include <algorithm>
#include <iostream>
#include <iterator>
#include <utility>
#include <vector>
#include <lemon/list_graph.h>
#include <lemon/kruskal.h>
#include <lemon/bfs.h>
#include <lemon/adaptors.h>

using namespace lemon;

typedef ListGraph::Node Node;
typedef ListGraph::Edge Edge;

int main()
```

```
{
 ListGraph g;
 Node v[ 6 ];
 for( int i = 0 ; i < 6 ; ++i )
  v[ i ] = g.addNode();
 ListGraph::EdgeMap< int > cost( g );
 const int edges[][ 3 ] = { { 0 , 1 , 4 } , { 0 , 2 , 1 } , { 1 , 2 , 2 } ,
                            { 1 , 3 , 5 } , { 2 , 3 , 8 } , { 2 , 4 , 9 } ,
                            { 3 , 4 , 3 } , { 3 , 5 , 6 } , { 4 , 5 , 7 } };
 for( const auto & e : edges )
  cost[ g.addEdge( v[ e[ 0 ] ] , v[ e[ 1 ] ] ) ] = e[ 2 ];
 int bad = 0;

 // a cost map in, a bool map out
 ListGraph::EdgeMap< bool > tree( g );
 int total = kruskal( g , cost , tree );
 int in_tree = 0;
 for( ListGraph::EdgeIt e( g ) ; e != INVALID ; ++e )
  in_tree += tree[ e ];
 if( total != 17 || in_tree != 5 )
  ++bad;

 // (11.2): no edge out of the tree costs less than an edge of the tree
 // path between its ends
 FilterEdges< ListGraph > t( g , tree );
 for( ListGraph::EdgeIt e( g ) ; e != INVALID ; ++e ) {
  if( tree[ e ] )
   continue;
  Bfs< FilterEdges< ListGraph > > bfs( t );
  bfs.run( g.u( e ) , g.v( e ) );
  for( Node n = g.v( e ) ; n != g.u( e ) ; n = bfs.predNode( n ) )
   if( cost[ bfs.predArc( n ) ] > cost[ e ] )
    ++bad;
 }

 // a sequence of (edge, cost) pairs in increasing order of cost in, an
 // output iterator out
 std::vector< std::pair< Edge , int > > seq;
 for( ListGraph::EdgeIt e( g ) ; e != INVALID ; ++e )
  seq.push_back( std::make_pair( Edge( e ) , cost[ e ] ) );
 std::sort(
   seq.begin() , seq.end() ,
   []( const std::pair< Edge , int > & l ,
       const std::pair< Edge , int > & r ) {
    return( l.second < r.second );
    } );
 std::vector< Edge > chosen;
 if( kruskal( g , seq , std::back_inserter( chosen ) ) != 17 ||
     chosen.size() != 5 )
  ++bad;
 for( std::size_t i = 0 ; i < chosen.size() ; ++i )
  if( ! tree[ chosen[ i ] ] )
   ++bad;

 std::cout << "tree of cost " << total << " with " << chosen.size()
           << " edges; " << bad << " errors" << std::endl;
 return( bad == 0 ? 0 : 1 );
 }
```

11.2 Minimum cost arborescences

Let $G = (N, A)$ be a digraph with costs $c : A \to \mathbb{R}$ and a root r, and let $R \subseteq N$ be the set of the nodes reachable from r. An arborescence rooted at r is a set B of arcs in which each node of $R \setminus \{r\}$ has one entering arc, and from which each node of R can be reached from r; in other words, it is a spanning tree of R whose arcs are

directed away from r. We look for one of least cost $c(B) = \sum_{a \in B} c(a)$. With several sources $s_1, \dots, s_q$ in place of r, it is to find the cheapest union of arborescences rooted at the sources that reaches all the nodes reachable from them. It can be written as the linear program

$$\min \Big\{ \sum_{a \in A} c(a)\, x_a \;:\; \sum_{a \in \delta^-(X)} x_a \geq 1 \quad \forall\, \emptyset \neq X \subseteq R \setminus \{r\}, \quad x \geq 0 \Big\}, \tag{11.4}$$

where $\delta^-(X)$ is the set of the arcs entering X; its optimal vertices are the incidence vectors of the minimum cost arborescences (Edmonds 1967). Its dual has a variable $y_X \geq 0$ for each set X. By complementary slackness, an arborescence B is optimal if there is a $y \geq 0$ with

$$\sum_{X \,:\, a \in \delta^-(X)} y_X \;\leq\; c(a) \quad \text{for each arc } a, \text{ with equality for each } a \in B, \tag{11.5}$$

and with $y_X > 0$ only for sets X entered by one arc of B; then $c(B) = \sum_X y_X$.

Choosing the cheapest arc entering each node, which would mirror the greedy algorithm of the undirected case, may give cycles: in Figure 11.2, the cheapest arcs entering a and b form the cycle a, b, a. Thus, the algorithm of Chu and Liu 1965 and Edmonds 1967 handles them by contraction (Algorithm 11.2). Each node, and later each contracted set X, chooses the arc of least reduced cost that enters it. That reduced cost is then subtracted from the reduced costs of all the arcs entering X, and it becomes the dual variable y_X. A cycle of chosen arcs, all of which have reduced cost zero at that point, is contracted into a new set, which chooses in its turn. When the chosen arcs reach the part of the digraph that is already connected to a source, the contractions are expanded: the arc that enters a contracted set from outside replaces the chosen arc of the node it enters, and the other chosen arcs of the set stay.

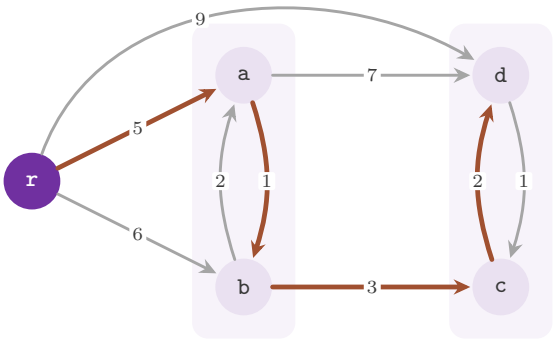


Figure 11.2. A minimum cost arborescence rooted at r: the cheapest arcs entering a and b, and those entering c and d, form two cycles, each contracted (the tinted sets) and then entered by one arc of the arborescence.

Algorithm 11.2 (Minimum cost arborescence by contractions, `MinCostArborescence`).

```
addSource(r):    mark the nodes reachable from r; r is processed
processNextNode():                    v := the last node reached; skip it if processed
1  X := {v}; path := (X)
2  loop:
3     a = uw := an arc of least reduced cost c'(a) entering X from outside X
4     new dual variable y_X := c'(a); c'(b) := c'(b) - c'(a) for each arc b entering X
5     if u is processed: break
6     if u is in a set on the path: X := the union of the sets of the path from that set
                                    to the top (the cycle is contracted)
7     else: X := {u}; push X on the path
8  expand: from a, give each node of the path its entering arc among the chosen ones,
          taking them in the order in which they were chosen; mark the nodes processed
```

Theorem 11.2. *When* `start()` *returns, the arcs* `pred(v)` *form a minimum cost arborescence of the nodes reachable from the sources, and the dual variables* y_X *satisfy (11.5) with it; hence,* `dualValue()` *equals* `arborescenceCost()`. *The algorithm takes* $O(n^2 + m)$ *time.*

Optimality is proved in Edmonds 1967 (see also Schrijver 2003), and the bound on the time is the one stated by the documentation of the class. Faster implementations, in $O(m + n \log n)$, exist (Gabow et al. 1986), but the

library does not use them. Note that the dual solution makes the result somewhat easier to check, as the example below does: (11.5) can be verified arc by arc, and the sum of the y_X must equal the cost of the arborescence.

Interface. `MinCostArborescence<GR, CM, TR>` takes the digraph type `GR`, the type `CM` of the cost map (by default `GR::ArcMap<int>`), and the traits class `TR`, by default `MinCostArborescenceDefaultTraits<GR, CM>`. Its traits give `CostMap`, `Value` (the type of the costs), `ArborescenceMap` and `PredMap`. The `ArborescenceMap` (by default an `ArcMap<bool>`) receives `false` on each arc at `init()`, and then `true` once on each arc of the arborescence; the `PredMap` is a `NodeMap<Arc>`. These traits also give `HeapCrossRef` and `Heap`, i.e., a `NodeMap<int>` and a `BinHeap` on integer priorities, which the expansion uses. Named parameters `SetArborescenceMap<T>` and `SetPredMap<T>` set the types of the two maps, which must then be given by `arborescenceMap(m)` and `predMap(m)`. Similarly, `SetHeap<H, CR>` sets the type of the heap and of its cross reference map, which the algorithm constructs on the digraph with -1 as the value of each node.

abLEMON. `MinCostArborescence` takes the type of its heap from its traits class, with the named parameter `SetHeap`; LEMON 1.3.1 used a `BinHeap` that could not be changed.

Its constructor takes the digraph and the cost map, and the setters are `arborescenceMap(m)` and `predMap(m)`; the maps not given are allocated by `init()` and deleted by the destructor. Its execution control is made of `init()`, `addSource(s)`, `processNextNode()`, `queueSize()`, `emptyQueue()`, `start()` and `run(s)`, which is `init()`, `addSource(s)` and `start()`. A call `addSource(s)` marks the nodes reachable from `s`, and may be repeated for several sources. Despite the word "queue" of the documentation, the nodes waiting to be processed are taken in the reverse of the order in which they were reached.

Its queries on the arborescence are `arborescence(a)`, which is true if the arc a is the predecessor of its target, `arborescenceMap()`, `pred(v)`, `predMap()`, and `arborescenceCost()`, computed in $O(m)$ from the predecessors. Also, `reached(v)` is true if v is reachable from a source, and `processed(v)` if it has been processed. Its queries on the dual solution are `dualNum()`, the number of the dual variables (some of which may have value zero), and `dualValue(k)` and `dualSize(k)`, the value and the number of nodes of the k-th one; `dualValue()` is the sum of all of them. An iterator `DualIt(mca, k)` lists the nodes of the k-th set, and is compared with `INVALID` to stop. Also, the internal records `CostArc`, `StackLevel` and `DualVariable` hold the arcs of least reduced cost, the levels of the path and the dual variables. Finally, the function `minCostArborescence(g, cost, source, arborescence)` runs the algorithm from one source, writes the arborescence into the given `bool` arc map, and returns its cost.

In the following program we solve the instance of Figure 11.2 and read the dual solution. Then we check (11.5) arc by arc and the equality of the primal and dual values, and call the function-type interface.

```
// The minimum cost arborescence of section 11.2 on the digraph of Figure
// 11.2: the cycle a -> b -> a of cheap arcs is contracted, and the dual
// solution read from the algorithm proves the arborescence optimal by the
// conditions (11.5).

#include <iostream>
#include <set>
#include <vector>
#include <lemon/list_graph.h>
#include <lemon/min_cost_arborescence.h>

using namespace lemon;

typedef ListDigraph::Node Node;
typedef ListDigraph::Arc Arc;
typedef ListDigraph::ArcMap< int > CostMap;

int main()
{
 ListDigraph g;
 Node r = g.addNode() , a = g.addNode() , b = g.addNode() , c = g.addNode() ,
      d = g.addNode();
 CostMap cost( g );
 const struct
 {
  Node u , v;
  int w;
```

```
 } arcs[] = { { r , a , 5 } , { r , b , 6 } , { a , b , 1 } , { b , a , 2 } ,
              { b , c , 3 } ,
             { c , d , 2 } , { a , d , 7 } , { d , c , 1 } , { r , d , 9 } };
for( const auto & e : arcs )
 cost[ g.addArc( e.u , e.v ) ] = e.w;
int bad = 0;

typedef MinCostArborescence< ListDigraph , CostMap > MCA;
MCA mca( g , cost );
mca.run( r );
// r -> a, a -> b, b -> c, c -> d: 5 + 1 + 3 + 2
if( mca.arborescenceCost() != 11 || mca.pred( b ) == INVALID ||
    g.source( mca.pred( b ) ) != a || mca.pred( r ) != INVALID )
 ++bad;

// the dual: a value y_X >= 0 on each set X of nodes, the root outside;
// an arc pays the y_X of the sets it enters, at most its cost, exactly
// its cost if it is in the arborescence, and the y_X sum to its cost
std::vector< std::set< Node > > sets( mca.dualNum() );
for( int k = 0 ; k < mca.dualNum() ; ++k ) {
 for( MCA::DualIt it( mca , k ) ; it != INVALID ; ++it )
  sets[ k ].insert( it );
 if( mca.dualValue( k ) < 0 || sets[ k ].count( r ) ||
     int( sets[ k ].size() ) != mca.dualSize( k ) )
  ++bad;
 }
for( ListDigraph::ArcIt e( g ) ; e != INVALID ; ++e ) {
 int paid = 0;
 for( int k = 0 ; k < mca.dualNum() ; ++k )
  if( sets[ k ].count( g.target( e ) ) && ! sets[ k ].count( g.source( e ) ) )
   paid += mca.dualValue( k );
 if( paid > cost[ e ] || ( mca.arborescence( e ) && paid != cost[ e ] ) )
  ++bad;
 }
if( mca.dualValue() != mca.arborescenceCost() )
 ++bad;

// the function-type interface
ListDigraph::ArcMap< bool > arb( g );
if( minCostArborescence( g , cost , r , arb ) != 11 )
 ++bad;

std::cout << "arborescence of cost " << mca.arborescenceCost() << ", "
          << mca.dualNum() << " dual variables; " << bad << " errors"
          << std::endl;
return( bad == 0 ? 0 : 1 );
}
```

11.3 Exercises

Exercise 11.1. Prove that the tree returned by Kruskal's algorithm satisfies (11.3) as well as (11.2), and check both conditions on random graphs by the method of the example `examples/11-kruskal.cc`. (The test `test/kruskal_test.cc` runs `kruskal()` with each form of input and output.)

Exercise 11.2. Compute a maximum spanning tree with `kruskal()`, through a map that negates the costs (e.g., `negMap()` of chapter 6), and explain why the greedy algorithm remains correct.

Exercise 11.3. Run `kruskal()` on a digraph and on the graph of the adaptor `Undirector` of the same digraph, and verify that the two trees have the same cost.

Exercise 11.4. Give a digraph on which choosing the cheapest arc entering each node, and then breaking each cycle at its most expensive arc, does not give a minimum cost arborescence, and check the answer of `MinCostArborescence` on it. (The test `test/min_cost_arborescence_test.cc` checks the dual solution as the example `examples/11-arborescence.cc` does.)

Exercise 11.5. Use two sources with `addSource()` and verify that the result is the cheapest union of two arborescences rooted at them; compare it with a single run from a new root joined to the two sources by arcs of cost zero.

12. Maximum flows and circulations

We are concerned in this chapter with the classes of `lemon/preflow.h`, `lemon/edmonds_karp.h` and `lemon/circulation.h`. Of these, the first two compute a flow of maximum value between two nodes of a digraph, together with a cut that proves it maximum, and share the interface of the concept in `lemon/concepts/max_flow.h`. The third one finds a flow within lower and upper bounds that meets given supplies, or a set of nodes that proves that no such flow exists. Section 12.1 states the problem and the duality on which the results of the chapter rest, sections 12.2 and 12.3 describe the two maximum flow algorithms, section 12.4 the circulation algorithm, and section 12.5 the tolerances and the concept that they share. We treat the minimum cost version of the circulation problem in chapter 13, and the minimum cuts of a graph with no source and no target in chapter 15.

12.1 The problem and its dual

Let $G = (V, A)$ be a digraph with n nodes and m arcs, let $cap : A \to \mathbf{R}_{\geq 0}$ be the capacities of the arcs, and let $s \neq t$ be two nodes, the source and the target. In the maximum flow problem one looks for a flow $f : A \to \mathbf{R}$ that solves

$$\begin{aligned} \max \quad & \sum_{sv \in A} f(sv) - \sum_{vs \in A} f(vs) \\ & \sum_{uv \in A} f(uv) = \sum_{vu \in A} f(vu) \qquad \forall u \in V \setminus \{s, t\} \\ & 0 \leq f(uv) \leq cap(uv) \qquad \forall uv \in A \end{aligned} \tag{12.1}$$

where the objective, i.e., the net flow out of s, is the *value* of the flow, and the equalities impose the conservation of the flow at the nodes other than s and t. A *cut* is a set $S \subset V$ with $s \in S$ and $t \notin S$, whose capacity

$$cap(S) = \sum_{uv \in A,\ u \in S,\ v \notin S} cap(uv) \tag{12.2}$$

bounds from above the value of any feasible flow, since that value equals the flow out of S less the flow into it. Given a flow f, the *residual digraph* has an arc uv wherever $f(uv) < cap(uv)$ and an arc vu wherever $f(uv) > 0$, with the residual capacities

$$r_f(uv) = cap(uv) - f(uv), \qquad r_f(vu) = f(uv). \tag{12.3}$$

A path from s to t in it is an *augmenting path*, along which the flow may be increased by the smallest residual capacity of its arcs. All the results of this chapter rest on the following classical theorem.

Theorem 12.1. *(Max-flow min-cut.) A feasible flow f is maximum if and only if the residual digraph of f has no path from s to t. Then the set S of the nodes that s reaches in it is a cut with $cap(S)$ equal to the value of f, and therefore a minimum cut.* For a proof see Ahuja, Magnanti and Orlin 1993, chapter 6.

Thus, the answer of the two maximum flow classes can be checked by the user. Both give, besides the flow and its value, a minimum cut through `minCut()` and `minCutMap()`, and the capacity (12.2) of that cut, computed on the data, must equal the value of the flow. Of course, the minimum cut need not be unique, and one may have to choose among several. In fact, the set reached from s in the residual digraph is the smallest source side of a minimum cut, and the complement of the set of the nodes that reach t is the largest one. The two classes need not give the same one (cf. sections 12.2.2 and 12.3).

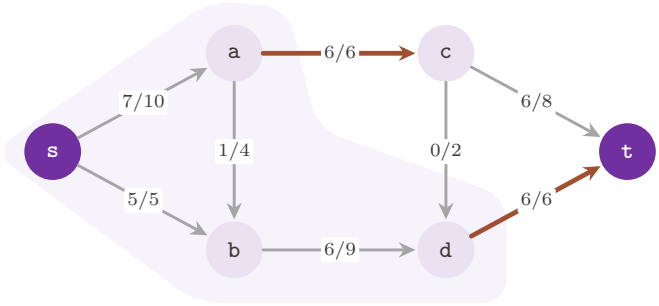


Figure 12.1. The network of the examples of sections 12.2 and 12.3: on each arc its flow in a maximum flow and its capacity; the source side of the minimum cut, {s, a, b, d}, of capacity 12, is on a tint, and the two arcs that leave it are saturated.

12.2 The push-relabel algorithm: Preflow

12.2.1 Preflows and labels

We begin with the class `Preflow`, which implements the push-relabel algorithm of Goldberg and Tarjan (Goldberg and Tarjan 1988; see also Cormen et al. 2001 and Ahuja, Magnanti and Orlin 1993, chapter 7). This algorithm does not keep a feasible flow during its course. It keeps instead a *preflow*, i.e., a map f within the bounds of (12.1) whose *excess* $e_f(u) = \sum_{vu \in A} f(vu) - \sum_{uv \in A} f(uv)$ is nonnegative at the nodes other than s, together with integer *labels* $d(u)$ (the levels of an `Elevator`, see chapter 8) such that

$$d(u) \le d(v) + 1 \qquad \text{for each arc } uv \text{ of the residual digraph.} \tag{12.4}$$

When $d(t) = 0$, condition (12.4) implies that $d(u)$ never exceeds the number of arcs of a shortest residual path from u to t. A node of positive excess is *active*, and an arc uv of the residual digraph with $d(u) = d(v) + 1$ is *admissible*. A *push* moves along an admissible arc the smaller of $e_f(u)$ and $r_f(uv)$; a *relabel* of an active node with no admissible arc raises $d(u)$ to one more than the least label of the heads of its residual arcs, which preserves (12.4). When no node is active, the preflow is a flow, and (12.4) proves it maximum by Theorem 12.1, since no residual path can then lead from s (whose label is n) to t.

The implementation runs in two phases, which are rather different. In the first one, the excess is pushed towards t only, and the nodes from which t is no longer reachable are lifted to the top level of the elevator (the level n, which marks the nodes that cannot reach t). At its end the excess of t is the maximum flow value, and the nodes at the top level form the source side of a minimum cut; however, some excess may still be stranded at the nodes that cannot reach t. Hence, a second phase sends that excess back to s, with labels computed from s in the same way, and it leaves a maximum flow. In both phases the active node of highest label is processed first. Also, a level that becomes empty lifts the nodes above it to the top, which is the *gap* heuristic: by (12.4), a node above an empty level cannot reach t through a residual path. In the words of the header, the result is "a mixture of the highest label and the bound decrease heuristics". In the code, the second rule is the processing of the active nodes from the current level downwards, without looking for the highest one again, for $20n$ steps after each sweep of n steps of the first kind.

Algorithm 12.1 (Preflow, first phase).

```
1  f(sv) := cap(sv) for the arcs sv whose head reaches t; f := 0 elsewhere
2  d(u) := BFS distance from u to t in the residual digraph (n if none)
3  while some node u != t with d(u) < n has e(u) > 0:
4      u := an active node of highest label
5      for each residual arc uv with d(v) < d(u), while e(u) > 0:
6          push min{ e(u), r(uv) } along uv
7      if e(u) > 0:
8          d(u) := min{ n, 1 + min{ d(v) : uv residual } }
9          if the old level of u is now empty:
10             lift the nodes above it to level n
11 return e(t) as the value, { u : d(u) = n } as the source side of the cut
```

Steps 1 and 2 are `init()`, which builds the labels by a breadth first search backwards from t over the arcs of positive capacity. Note that it saturates only the arcs out of s whose head can reach t. Steps 3 to 10 are `startFirstPhase()`; step 5 pushes along any residual arc to a lower level, not only to the level just below, which still preserves (12.4).

As stated in the header, the worst case running time is $O(n^2\sqrt{m})$. In practice the algorithm is usually the fastest maximum flow method of the library, and the documentation of the module recommends it "in most cases". We remark that the header warns against infinite or very large capacities (e.g., the largest value of the type), since the initial pushes of step 1 and the sums of the excesses at a node must fit in the value type.

12.2.2 The interface

`Preflow<GR, CAP, TR>` takes the type `GR` of the digraph, the type `CAP` of the capacity map (by default `GR::ArcMap< int >`), and a traits class `TR`, by default `PreflowDefaultTraits< GR , CAP >`. These traits define the value type `Value` (that of `CAP`), the type `FlowMap` of the flow map (`GR::ArcMap< Value >`), the type `Elevator` of the labels (`Elevator< GR , GR::Node >`, whose alternative is `LinkedElevator`), the functions that create the last two, and the `Tolerance< Value >` used in the comparisons. One rarely sets the traits directly, and we do not recommend it. Instead, the named template parameters `SetFlowMap< T >`, `SetElevator< T >` and `SetStandardElevator< T >` replace one type each, through their member `Create`. With `SetFlowMap` and `SetElevator` the object must be given by `flowMap()` or `elevator()` before `init()`, since the traits they make cannot create it; `SetStandardElevator`, instead, makes a type that the class still allocates, from the digraph and the largest level.

The constructor `Preflow(digraph, capacity, source, target)` stores the data by reference, and the setters `capacityMap()`, `flowMap()`, `source()`, `target()`, `elevator()` and `tolerance()` change them, each returning the object; a flow map or an elevator that is not given is allocated by `init()` and released by the destructor. The execution follows the pattern of chapter 3:

- `run()` is `init()`, `startFirstPhase()` and `startSecondPhase()`, and `runMinCut()` is `init()` and `startFirstPhase()`, which suffice for the value and for the minimum cut;
- `init()` starts from the zero flow, and `init(flowMap)` from a given map, which must be at least a preflow: after copying the map, it returns `false` if the excess of a node other than s is negative beyond the tolerance, and otherwise it saturates the residual arcs out of s (and empties the arcs into it) towards the nodes that reach t;
- `flowValue()` returns the excess of t, i.e., the maximum value already after the first phase, while `flow(arc)` and `flowMap()` give a maximum flow only after the second phase;
- `minCut(node)` returns `true` if a node is on the source side of the minimum cut found, and `minCutMap(map)` writes that side into a writable `bool` node map in $O(n)$ time.

After the first phase, `minCut()` gives the nodes at the top level, i.e., those that do not reach t in the residual digraph, which form the largest source side of a minimum cut. After the second phase it gives the nodes whose label is below the top, and the header remarks that the two answers may differ slightly with inexact computation.

> **abLEMON.** With a floating-point `Value`, `init(flowMap)` judges the excess of a node relative to the flow through the node, rather than against the absolute epsilon of the tolerance. Hence, the maximum flow of another `Preflow` on large capacities (e.g., of the order of 10^9) is taken as a preflow, while LEMON 1.3.1 refused it because the rounding errors of the sums exceed that epsilon (trac #625). A program may then start from the maximum flow of a previous run after a change of the capacities, provided that the old flow is still a preflow for the new ones (e.g., after some capacities have grown).

We give below a program that solves the network of Figure 12.1, and that checks the value against the capacity of the cut. Then it gives the maximum flow to a second `Preflow`, after the capacity of the arc (d, t) has grown from 6 to 10; the second run starts from a flow of value 12 and reaches 15.

```
// The maximum flow of section 12.2 and 12.3: the network of Figure 12.1,
// solved by Preflow and by EdmondsKarp, the minimum cut of both checked
// against the flow value, and a second Preflow started from the maximum
// flow of the first after the capacity of one arc has grown.

#include <iostream>
#include <lemon/list_graph.h>
#include <lemon/preflow.h>
#include <lemon/edmonds_karp.h>
```

```
using namespace lemon;

typedef ListDigraph::ArcMap< int > CapMap;

// The capacity of the arcs that leave the source side of a cut
template< typename CutMap >
int cutCapacity( const ListDigraph & g , const CapMap & cap ,
                 const CutMap & side )
{
 int c = 0;
 for( ListDigraph::ArcIt a( g ) ; a != INVALID ; ++a )
  if( side[ g.source( a ) ] && ! side[ g.target( a ) ] )
   c += cap[ a ];
 return( c );
 }

int main()
{
 ListDigraph g;
 ListDigraph::Node s = g.addNode() , a = g.addNode() , b = g.addNode() ,
                   c = g.addNode() , d = g.addNode() , t = g.addNode();
 CapMap cap( g );
 cap[ g.addArc( s , a ) ] = 10;
 cap[ g.addArc( s , b ) ] = 5;
 cap[ g.addArc( a , b ) ] = 4;
 cap[ g.addArc( a , c ) ] = 6;
 cap[ g.addArc( b , d ) ] = 9;
 cap[ g.addArc( c , d ) ] = 2;
 cap[ g.addArc( c , t ) ] = 8;
 ListDigraph::Arc dt = g.addArc( d , t );
 cap[ dt ] = 6;
 int errors = 0;

 // Preflow: both phases, then the cut of the first phase
 Preflow< ListDigraph > pf( g , cap , s , t );
 pf.run();
 ListDigraph::NodeMap< bool > cut( g );
 pf.minCutMap( cut );
 std::cout << "Preflow: value " << pf.flowValue() << ", cut capacity "
           << cutCapacity( g , cap , cut ) << std::endl;
 if( pf.flowValue() != 12 || cutCapacity( g , cap , cut ) != 12 )
  ++errors;
 if( ! ( cut[ s ] && cut[ a ] && cut[ b ] && cut[ d ] && ! cut[ c ] &&
         ! cut[ t ] ) )
  ++errors;

 // EdmondsKarp, one augmenting path at a time
 EdmondsKarp< ListDigraph > ek( g , cap , s , t );
 ek.init();
 int paths = 0;
 while( ek.augment() )
  ++paths;
 ListDigraph::NodeMap< bool > cut2( g );
 ek.minCutMap( cut2 );
 std::cout << "EdmondsKarp: value " << ek.flowValue() << " by " << paths
           << " augmenting paths" << std::endl;
 if( ek.flowValue() != 12 || cutCapacity( g , cap , cut2 ) != 12 )
  ++errors;

 // A larger capacity of the arc (d,t): the old maximum flow is a feasible
 // flow of the new network, from which a second Preflow starts
 CapMap cap2( g );
 for( ListDigraph::ArcIt e( g ) ; e != INVALID ; ++e )
  cap2[ e ] = cap[ e ];
```

```
cap2[ dt ] = 10;
Preflow< ListDigraph > pf2( g , cap2 , s , t );
if( ! pf2.init( pf.flowMap() ) )
 ++errors;
pf2.startFirstPhase();
std::cout << "Preflow from the old flow: value " << pf2.flowValue()
          << std::endl;
if( pf2.flowValue() != 15 )
 ++errors;

// A map that is no flow of the network is refused by EdmondsKarp, which
// checks the conservation and the bounds (the flow is set anyway)
CapMap bad( g , 0 );
bad[ dt ] = 1;
if( ek.init( bad ) )
 ++errors;
if( ! ek.init( pf.flowMap() ) || ek.flowValue() != 12 || ek.augment() )
 ++errors;

return( errors );
}
```

12.3 Augmenting paths: EdmondsKarp

Next, the class `EdmondsKarp` implements the algorithm of Edmonds and Karp 1972, i.e., the augmenting path method of Ford and Fulkerson in which each augmenting path is a shortest one (in the number of arcs) of the residual digraph. Since the distance from s to a node in the residual digraph never decreases, while some arc of the path is saturated at each step, at most $nm/2$ augmentations are made, each of which costs $O(m)$ for a breadth first search. Thus, the running time is $O(nm^2)$, which the header gives together with the advice to use `Preflow` when only the maximum flow is wanted.

Algorithm 12.2 (Edmonds and Karp).

```
1  f := 0 (or a given feasible flow)
2  repeat:
3      find by BFS a path P from s to t with the fewest arcs,
       along the residual arcs uv with r(uv) > 0
4      if there is none: stop, f is maximum
5      delta := min{ r(uv) : uv in P }
6      increase f along P by delta (decrease it on the backward arcs)
```

Steps 3 to 6 are one call of `augment()`, which returns `false` when no path is found. Then `start()` repeats it, and `run()` is `init()` followed by `start()`. In our view, the advantage of the class over `Preflow` lies in this control: between two calls of `augment()` the flow is a feasible flow, whose value `flowValue()` returns. One can then stop early, look at the intermediate flows, or interleave the augmentations with some other work, as a teaching or a debugging program may want.

Its template parameters are those of `Preflow`, with the traits `EdmondsKarpDefaultTraits< GR , CAP >` (the value type, the type of the flow map and its creator, and the tolerance) and the single named parameter `SetFlowMap< T >`. The constructor `EdmondsKarp(digraph, capacity, source, target)` asserts that $s \neq t$; the setters `capacityMap()`, `flowMap()`, `source()`, `target()` and `tolerance()` return the object, and the queries `flowValue()`, `flow(arc)`, `flowMap()`, `minCut(node)` and `minCutMap(map)` are as in `Preflow`. Here the cut is the one of the last search, i.e., the set of the nodes reached from s in the residual digraph when `augment()` failed, which by Theorem 12.1 is the smallest source side of a minimum cut. In the example of Figure 12.1 the minimum cut is unique, and the three augmenting paths found by the program of section 12.2.2 lead to the same set $\{s, a, b, d\}$ as `Preflow`. Finally, `init(flowMap)` starts from a given flow, and `checkedInit(flowMap)` is kept as another name of the same function.

abLEMON. `init(flowMap)` returns whether the map is a feasible flow, with the tolerance of the algorithm (conservation at the nodes other than s and t, and $0 \leq f \leq cap$ on the arcs), as `Preflow::init()` does; the flow is set in any case. In LEMON 1.3.1 it returned nothing, and only `checkedInit()` made the same checks.

12.4 Feasible circulations: Circulation

12.4.1 The problem and its certificate

Let $lower : A \to \mathbf{R}$ and $upper : A \to \mathbf{R} \cup \{\infty\}$, with $lower \le upper$, be bounds on the flow of the arcs, and let $sup : V \to \mathbf{R}$ be signed supplies (a node with $sup(u) > 0$ supplies $sup(u)$ units, one with $sup(u) < 0$ demands $-sup(u)$). In the *network circulation problem* of the class `Circulation` one looks for any f such that

$$\sum_{uv \in A} f(uv) - \sum_{vu \in A} f(vu) \ge sup(u) \quad \forall u \in V, \qquad lower(uv) \le f(uv) \le upper(uv) \quad \forall uv \in A. \tag{12.5}$$

The system can be feasible only if $\sum_{u \in V} sup(u) \le 0$, since its left-hand sides sum to zero; when the supplies sum to zero, the inequalities hold as equalities, and (12.5) defines the feasible flows with supplies and demands. One obtains the opposite inequalities (all the demands met, some supply left) by reversing the arcs and negating the supplies, as the header suggests, e.g., with the adaptor `ReverseDigraph` of chapter 5 and the map `NegMap` of chapter 6. As a certificate of infeasibility the class gives a *barrier*, i.e., a set $B \subseteq V$ whose supply exceeds what can leave it:

$$\sum_{uv \in A,\ u \in B,\ v \notin B} upper(uv) - \sum_{uv \in A,\ u \notin B,\ v \in B} lower(uv) < \sum_{v \in B} sup(v). \tag{12.6}$$

Theorem 12.2. *(Hoffman.) The system (12.5) has a solution if and only if no set $B \subseteq V$ satisfies (12.6).* This is the circulation theorem of Hoffman 1960, which in the form with supplies is the feasibility theorem of Gale 1957. Both follow from Theorem 12.1, applied to the network that joins a source to the supply nodes and the demand nodes to a target.

12.4.2 The algorithm and its interface

`Circulation` is a push-relabel algorithm written directly for (12.5). In it, the excess of a node is $sup(u)$ plus the flow into it less the flow out of it; hence, a node of positive excess is one whose inequality is violated. Its flow starts at the lower bounds (`init()`) or at a greedy choice (`greedyInit()`), and all the nodes start at level 0. The greedy choice sets each arc, in the order of the arcs, to the deficit of its head clipped to the bounds of the arc. Then `start()` pushes from the active node of highest level along the residual arcs to lower levels, and it relabels as `Preflow` does. It ends with `false` when a node cannot be lifted (no residual arc leaves it) or when a lift leaves a level empty; the nodes at or above that level form a barrier, since no residual arc leaves them and their excess is positive. A comment in `init()` records that a global relabeling was tried and gave somewhat worse running times on random digraphs; we know of no bound on the running time stated in the header, and we give none.

Algorithm 12.3 (Circulation).

```
1  set f by the greedy rule; e(u) := sup(u) + f(in(u)) - f(out(u))
2  d(u) := 0 for each node u
3  while some node has e(u) > 0:
4      u := an active node of highest level
5      push from u along residual arcs uv with d(v) < d(u)
6      if e(u) > 0:
7          if no residual arc leaves u: L := n; return false
8          d(u) := 1 + min{ d(v) : uv residual }
9          if the old level of u is now empty: L := old level; return false
10 return true            (the barrier is { u : d(u) >= L } after false)
```

Its template parameters are `GR`, the lower bound map `LM` (by default `GR::ArcMap< int >`), the upper bound map `UM` (by default `LM`), the supply map `SM` (by default `GR::NodeMap< UM::Value >`), whose value type is the `Value` of the algorithm, and the traits `CirculationDefaultTraits< GR , LM , UM , SM >`. Its named parameters `SetFlowMap< T >`, `SetElevator< T >` and `SetStandardElevator< T >` are those of `Preflow`, and its constructor `Circulation(graph, lower, upper, supply)` takes the three maps, which `lowerMap()`, `upperMap()` and `supplyMap()` change, while `flowMap()`, `elevator()` and `tolerance()` are as in `Preflow`. `run()` is `greedyInit()` followed by `start()`, and it returns whether a feasible circulation was found. In that case `flow(arc)` and `flowMap()` give it; otherwise `barrier(node)` and `barrierMap(map)` give the barrier (which is

empty after a feasible run). Two checkers verify an answer on the data: `checkFlow()` tests the bounds and the inequalities of (12.5), the latter up to the tolerance, and `checkBarrier()` tests (12.6) on the barrier found, an infinite upper bound out of the set making the test fail.

Our second program solves a network of four nodes, and then it lowers the capacity of the arc (b, d) from 4 to 1. The node d, which demands 3 units, can then receive at most 2, `run()` returns `false`, and the program checks (12.6) on the barrier.

```
// The feasible circulation of section 12.4: a network with lower and
// upper bounds and supplies, solved by Circulation, and the same network
// with two capacities lowered, for which Circulation gives a barrier whose
// inequality (12.6) is checked here on the data.

#include <iostream>
#include <lemon/list_graph.h>
#include <lemon/circulation.h>

using namespace lemon;

typedef ListDigraph::ArcMap< int > ArcMap;
typedef ListDigraph::NodeMap< int > NodeMap;

int main()
{
 ListDigraph g;
 ListDigraph::Node a = g.addNode() , b = g.addNode() , c = g.addNode() ,
                   d = g.addNode();
 ArcMap lower( g ) , upper( g );
 NodeMap supply( g );
 ListDigraph::Arc ab = g.addArc( a , b ) , ac = g.addArc( a , c ) ,
                  bc = g.addArc( b , c ) , bd = g.addArc( b , d ) ,
                  cd = g.addArc( c , d );
 lower[ ab ] = 1;
 upper[ ab ] = 3;
 lower[ ac ] = 0;
 upper[ ac ] = 2;
 lower[ bc ] = 0;
 upper[ bc ] = 2;
 lower[ bd ] = 0;
 upper[ bd ] = 4;
 lower[ cd ] = 0;
 upper[ cd ] = 1;
 supply[ a ] = 4;
 supply[ b ] = 0;
 supply[ c ] = -1;
 supply[ d ] = -3;
 int errors = 0;

 Circulation< ListDigraph > circ( g , lower , upper , supply );
 bool feasible = circ.run();
 std::cout << "first network: " << ( feasible ? "feasible" : "infeasible" )
           << std::endl;
 if( ! feasible || ! circ.checkFlow() )
  ++errors;
 for( ListDigraph::ArcIt e( g ) ; e != INVALID ; ++e )
  std::cout << "  flow on arc " << g.id( e ) << ": " << circ.flow( e )
            << std::endl;

 // d can receive at most 2 units, but it needs 3
 upper[ bd ] = 1;
 Circulation< ListDigraph > circ2( g , lower , upper , supply );
 feasible = circ2.run();
 ListDigraph::NodeMap< bool > bar( g );
 circ2.barrierMap( bar );
 // (12.6): what can leave the barrier is less than its supply
```

```
int out = 0 , sup = 0;
for( ListDigraph::NodeIt n( g ) ; n != INVALID ; ++n )
 if( bar[ n ] )
  sup += supply[ n ];
for( ListDigraph::ArcIt e( g ) ; e != INVALID ; ++e ) {
 if( bar[ g.source( e ) ] && ! bar[ g.target( e ) ] )
  out += upper[ e ];
 else if( ! bar[ g.source( e ) ] && bar[ g.target( e ) ] )
  out -= lower[ e ];
 }
std::cout << "second network: " << ( feasible ? "feasible" : "infeasible" )
          << ", barrier of supply " << sup << " and outflow at most " << out
          << std::endl;
if( feasible || ! circ2.checkBarrier() || ! ( out < sup ) )
 ++errors;

return( errors );
}
```

abLEMON. On a digraph without nodes `Circulation` read the first of no items of its elevator in LEMON 1.3.1; now the elevator holds one item more than the digraph has, and the empty digraph has the empty circulation.

12.5 Tolerances and the common interface

All three classes compare the values of the flows through a `Tolerance< Value >` (see chapter 3), which is set and read by `tolerance()`. A residual capacity counts as positive only beyond the epsilon of the tolerance, and so does an excess; also, a push saturates an arc unless its residual capacity exceeds the excess by more than the epsilon. For an integer `Value` the comparisons are exact. For a floating-point one the default epsilon of `Tolerance` (10^{-10} for `double`) is absolute, which usually suits capacities of moderate size, and a program with data of a rather different scale should set another one. Indeed, the only comparison that is relative to the data is the one of `Preflow::init(flowMap)` (cf. section 12.2.2).

`Preflow` and `EdmondsKarp` meet the concept `concepts::MaxFlow< GR , CAP >` of `lemon/concepts/max_flow.h`, against which a class is checked by `checkConcept` (see chapter 3). The concept requires the member `Create` and the named parameter `SetFlowMap`, the constructor from the digraph, the capacities, the source and the target, and the setters `capacityMap()`, `flowMap()`, `source()`, `target()` and `tolerance()`. It also requires the two `init()` functions and `run()`, and the queries `flowValue()`, `flow()`, `flowMap()`, `minCut()` and `minCutMap()`; a function template written against these members accepts either class.

abLEMON. The concepts of the maximum flow, minimum cost flow and minimum mean cycle classes are in `lemon/concepts` (`MaxFlow`, `MinCostFlow` and `MinMeanCycle`), where the classes of the library are held to them; in LEMON 1.3.1 they were defined inside the tests.

12.6 Exercises

Exercise 12.1. Modify the program of section 12.2.2 so that it stops `EdmondsKarp` after its first augmenting path and prints the residual capacities (12.3) of the arcs. Then check by hand that the value of the flow and the number of paths agree with Algorithm 12.2. (The program `manual/examples/12-max-flow.cc` counts the paths.)

Exercise 12.2. Show that the set of the nodes from which t is not reachable in the residual digraph of a maximum flow is a minimum cut, and that it contains the cut of Theorem 12.1. On the network of Figure 12.1, add an arc that makes the two sets differ, and compare `minCutMap()` of `Preflow` after `runMinCut()` with that of `EdmondsKarp`.

Exercise 12.3. Give `Preflow::init(flowMap)` a map that is not a preflow (e.g., one unit more out of a node than into it), and check that it returns `false`. Then repeat with capacities of the order of 10^9 in `double`, where the absolute epsilon of the tolerance would refuse a true preflow. (Solved by `checkInitPreflowLarge()` in `test/max_flow_test.cc`.)

Exercise 12.4. Write a function template that takes a class meeting `concepts::MaxFlow`, runs it, and returns the capacity (12.2) of its cut minus its value; instantiate it with both `Preflow` and `EdmondsKarp`. (The concept

is checked in `test/max_flow_test.cc`.)

Exercise 12.5. Solve a problem with the opposite inequalities of (12.5), i.e., all the demands met and some supply left, by `Circulation` on a `ReverseDigraph` with a `NegMap` of the supplies, and verify the answer with `checkFlow()`. (The class is tested in `test/circulation_test.cc`.)

Exercise 12.6. Prove Theorem 12.2 from Theorem 12.1, by joining a source to the supply nodes and the demand nodes to a target, and by replacing each lower bound with a shift of the supplies. Then check (12.6) on the barrier of the second network of `manual/examples/12-circulation.cc`.

13. Minimum cost flows

We are concerned in this chapter with the four minimum cost flow algorithms of the library, `NetworkSimplex` (`lemon/network_simplex.h`), `CostScaling` (`lemon/cost_scaling.h`), `CapacityScaling` (`lemon/capacity_scaling.h`) and `CycleCanceling` (`lemon/cycle_canceling.h`), and with the concept `concepts::MinCostFlow` of `lemon/concepts/min_cost_flow.h` that they meet. We also describe two internal headers that a user meets through their effects, i.e., the default tolerances of `lemon/bits/mcf_tolerance.h`, and the starting flow and the bound on the infinite capacities of `lemon/bits/mcf_feasible_flow.h`. Section 13.1 states the problem in the forms that the library takes, together with its optimality conditions; section 13.2 describes the interface that the four classes share, and section 13.3 the way in which they handle floating-point data. Sections 13.4 to 13.8 describe the algorithms one by one, and section 13.9 gives some advice on the choice among them. In chapter 14 we treat the certificates that the four classes give when a problem has no optimal flow, and the classes that enumerate several flows.

13.1 The problem, its forms and its dual

Let $G = (V, A)$ be a digraph with n nodes and m arcs, and let $lower : A \to \mathbf{R}$ and $upper : A \to \mathbf{R} \cup \{+\infty\}$, with $lower \le upper$, be bounds on the flow of the arcs. Let also $cost : A \to \mathbf{R}$ be the cost of a unit of flow on each arc, and $sup : V \to \mathbf{R}$ the signed supplies of the nodes (a supply if positive, a demand if negative). Following Ahuja, Magnanti and Orlin 1993 and the page `doc/min_cost_flow.dox` of the documentation, the library solves the problem in the form with "greater or equal" supply constraints (GEQ):

$$\begin{aligned} \min \quad & \sum_{uv \in A} cost(uv)\, f(uv) \\ & \sum_{uv \in A} f(uv) - \sum_{vu \in A} f(vu) \ge sup(u) \qquad \forall u \in V \\ & lower(uv) \le f(uv) \le upper(uv) \qquad \forall uv \in A. \end{aligned} \tag{13.1}$$

Since the left-hand sides sum to zero, (13.1) can be feasible only if $\sum_{u \in V} sup(u) \le 0$. Thus, the supplies must all leave their nodes, while some demand may be left unmet. When the supplies sum to zero, the constraints hold as equalities, and (13.1) is the usual equality form of the problem; a user who wants that form must therefore make the supplies sum to zero. In addition, `NetworkSimplex` takes the opposite form with "less or equal" constraints (LEQ),

$$\sum_{uv \in A} f(uv) - \sum_{vu \in A} f(vu) \le sup(u) \qquad \forall u \in V, \tag{13.2}$$

which needs $\sum_{u \in V} sup(u) \ge 0$, and in which the demands must be met while some supply may stay at its node. The other three classes take only the GEQ form; one can bring the LEQ form to it by reversing the arcs and negating the supplies, e.g., with `ReverseDigraph` (chapter 5) and `NegMap` (chapter 6).

In the dual problem the variables are the *potentials* $\pi : V \to \mathbf{R}$ of the nodes, and the *reduced cost* of an arc under them is

$$cost^{\pi}(uv) = cost(uv) + \pi(u) - \pi(v). \tag{13.3}$$

Hence, the potentials are the opposite of the prices of the textbook convention, in which $cost^\pi(uv) = cost(uv) - \pi(u) + \pi(v)$; we keep the sign of the library throughout. For the GEQ form, the optimality conditions are the following complementary slackness conditions of linear programming.

Theorem 13.1. *A feasible flow f of (13.1) is optimal if and only if there are potentials π such that, for each arc uv and each node u,*

$$\begin{aligned} &cost^\pi(uv) > 0 \Rightarrow f(uv) = lower(uv), \qquad cost^\pi(uv) < 0 \Rightarrow f(uv) = upper(uv), \\ &\pi(u) \le 0, \qquad \textstyle\sum_{uv\in A} f(uv) - \sum_{vu\in A} f(vu) \ne sup(u) \Rightarrow \pi(u) = 0. \end{aligned} \tag{13.4}$$

For the LEQ form the same holds with $\pi(u) \ge 0$ in place of $\pi(u) \le 0$. This is the duality theorem of linear programming applied to (13.1), whose constraint matrix is totally unimodular; see Ahuja, Magnanti and Orlin 1993, chapter 9. We shall often use a consequence of it, which needs the residual network of a flow f. This network has an arc uv of residual capacity $upper(uv) - f(uv)$ and cost $cost(uv)$ wherever $f(uv) < upper(uv)$, and an arc vu of residual capacity $f(uv) - lower(uv)$ and cost $-cost(uv)$ wherever $f(uv) > lower(uv)$.

Theorem 13.2. *(Negative cycle optimality.) A feasible flow of the equality form is optimal if and only if its residual network has no directed cycle of negative cost. Then (13.4) holds for the potentials d, where $d(u) \le 0$ is the cost of a shortest residual path to u from an extra node joined to all the others by arcs of cost zero.* This is due to Klein 1967, and it underlies the cycle canceling methods of section 13.8 and the final potentials of cost scaling.

Three outcomes are possible, and `run()` returns them as the constants of the nested enum `ProblemType` of each class. With `OPTIMAL` a flow and potentials meeting (13.4) were found; with `INFEASIBLE` no flow satisfies the constraints; and with `UNBOUNDED` the problem is feasible, while a directed cycle of negative cost, whose arcs have infinite upper bounds, lets the cost decrease without limit. An infinite upper bound is the constant `INF` of the class, i.e., the infinity of `Value` if the type has one and its largest value otherwise. Integer data have integer optimal flows, since the matrix of (13.1) is totally unimodular, and on integer data the four algorithms return one of them.

13.2 The common interface

All four classes take as template parameters the type `GR` of the digraph, the number type `V` of the flows, of the bounds and of the supplies (by default `int`), and the number type `C` of the costs and of the potentials (by default `V`); both must be signed, as the constructors assert. `CostScaling` and `CapacityScaling` take a fourth parameter, a traits class, which we describe with them. The constructor takes the digraph, which must not change afterwards unless `reset()` is called. Then the parameters of the problem are given by functions that copy the data at once (hence, the maps need not outlive the call), each returning the object:

- `lowerMap(map)`, the lower bounds, zero on the arcs if it is not called;
- `upperMap(map)`, the upper bounds, `INF` on the arcs if it is not called;
- `costMap(map)`, the costs, 1 on the arcs if it is not called;
- `supplyMap(map)`, the supplies, zero if neither it nor `stSupply()` is called, and `stSupply(s, t, k)`, which gives k to s, $-k$ to t and zero to the other nodes;
- `supplyType(type)`, with `GEQ` (the default) or `LEQ`, in `NetworkSimplex` only.

Then `run()` solves the problem and returns its `ProblemType`, with arguments that differ from class to class (Table 13.1). After a run, the queries are `totalCost()`, i.e., the cost of the flow in $O(m)$ time, `flow(arc)` and `flowMap(map)`, which give the flow, and `potential(node)` and `potentialMap(map)`, which give the potentials. With a template argument, e.g., `totalCost< double >()`, the sum is made in another type, which helps when it may overflow `Cost`. We describe the certificates `infeasibleCut()`, `infeasibleCuts(k)`, `disjointInfeasibleCuts()`, `unboundedCycle()` and `unboundedCycles(k)`, which are meaningful after a run that returned `INFEASIBLE` or `UNBOUNDED`, in chapter 14, and the tolerances in section 13.3.

One may call `run()` again: the parameters are kept, and only those that changed have to be given anew. `resetParams()` sets the parameters back to their defaults (and the supply type to `GEQ`), while `reset()` also rebuilds the internal copy of the digraph, as is needed when the digraph itself was modified after the construction or after the last `reset()`. In any case a second run solves the problem from scratch, whatever changed.

Table 13.1. The four classes: the arguments of `run()` with their defaults, the worst case running time stated in each header (U is the largest supply or capacity, K the largest cost), the supply types, and the template parameters beyond `GR`, `V` and `C`.

Class	run()	Worst case	Forms	Traits
NetworkSimplex	pivot_rule = BLOCK_SEARCH	not stated	GEQ, LEQ	none
CostScaling	method = PARTIAL_AUGMENT, factor = 16	$O(n^2 m \log(nK))$	GEQ	LargeCost
CapacityScaling	factor = 4	$O(m \log U\,(n+m) \log n)$	GEQ	Heap
CycleCanceling	method = CANCEL_AND_TIGHTEN	$O(n^2 m^2 \log n)$	GEQ	none

All four meet the concept concepts::MinCostFlow< GR , V , C > of lemon/concepts/min_cost_flow.h . It requires the constructor from the digraph, reset(), resetParams(), the five setters of the parameters (without supplyType()), run() with its defaults, the queries above, the tolerance functions of section 13.3, and the single certificates infeasibleCut() and unboundedCycle(). A function template written against these members accepts any of the four, as does the function optimal() of the program below. This program solves a transshipment problem of seven nodes with the four classes, and it checks that their costs agree and that the flow and the potentials of each meet (13.4). Then it changes the cost of one arc and runs NetworkSimplex again. Finally it gives two more units of supply to one node; the problem is then infeasible in the GEQ form, but it has an optimal flow in the LEQ form, with nonnegative potentials.

```
// The minimum cost flow of section 13.2: a small transshipment problem
// solved by the four algorithms, whose total costs must agree and whose
// potentials must meet the optimality conditions (13.4) with the flow of
// each; then the same problem with the LEQ supply type and more supply
// than demand, which only NetworkSimplex takes.

#include <iostream>
#include <lemon/list_graph.h>
#include <lemon/network_simplex.h>
#include <lemon/cost_scaling.h>
#include <lemon/capacity_scaling.h>
#include <lemon/cycle_canceling.h>

using namespace lemon;

typedef ListDigraph::ArcMap< int > ArcMap;
typedef ListDigraph::NodeMap< int > NodeMap;

// Whether flow and potentials of an algorithm meet the complementary
// slackness conditions (13.4) of the GEQ form, and the flow the bounds
// and the supplies (as equalities, since the supplies sum to zero)
template< typename MCF >
bool optimal( const ListDigraph & g , const MCF & mcf , const ArcMap & upper ,
              const ArcMap & cost , const NodeMap & supply )
{
 for( ListDigraph::ArcIt a( g ) ; a != INVALID ; ++a ) {
  const int f = mcf.flow( a );
  const int rc = cost[ a ] + mcf.potential( g.source( a ) ) -
                 mcf.potential( g.target( a ) );
  if( f < 0 || f > upper[ a ] )
   return( false );
  if( ( rc > 0 && f != 0 ) || ( rc < 0 && f != upper[ a ] ) )
   return( false );
  }
 for( ListDigraph::NodeIt n( g ) ; n != INVALID ; ++n ) {
  int net = 0;
  for( ListDigraph::OutArcIt a( g , n ) ; a != INVALID ; ++a )
   net += mcf.flow( a );
  for( ListDigraph::InArcIt a( g , n ) ; a != INVALID ; ++a )
   net -= mcf.flow( a );
  if( net != supply[ n ] || mcf.potential( n ) > 0 )
   return( false );
  }
 return( true );
```

```
 }

int main()
{
 ListDigraph g;
 ListDigraph::Node s1 = g.addNode() , s2 = g.addNode() , m1 = g.addNode() ,
                   m2 = g.addNode() , t1 = g.addNode() , t2 = g.addNode() ,
                   t3 = g.addNode();
 ArcMap upper( g ) , cost( g );
 NodeMap supply( g );
 struct
 {
  ListDigraph::Node u , v;
  int cap , cost;
  } arcs[] = { { s1 , m1 , 4 , 2 } , { s1 , m2 , 3 , 4 } ,
               { s2 , m1 , 3 , 3 } ,
              { s2 , m2 , 4 , 1 } , { m1 , m2 , 2 , 1 } , { m1 , t1 , 3 , 2 } ,
              { m1 , t2 , 3 , 5 } , { m2 , t2 , 4 , 2 } , { m2 , t3 , 3 , 3 } ,
              { s1 , t1 , 1 , 3 } };
 ListDigraph::Arc id[ 10 ];
 for( int i = 0 ; i != 10 ; ++i ) {
  id[ i ] = g.addArc( arcs[ i ].u , arcs[ i ].v );
  upper[ id[ i ] ] = arcs[ i ].cap;
  cost[ id[ i ] ] = arcs[ i ].cost;
  }
 supply[ s1 ] = 5;
 supply[ s2 ] = 4;
 supply[ m1 ] = 0;
 supply[ m2 ] = 0;
 supply[ t1 ] = -3;
 supply[ t2 ] = -3;
 supply[ t3 ] = -3;
 int errors = 0;

 NetworkSimplex< ListDigraph > ns( g );
 ns.upperMap( upper ).costMap( cost ).supplyMap( supply );
 CostScaling< ListDigraph > cs( g );
 cs.upperMap( upper ).costMap( cost ).supplyMap( supply );
 CapacityScaling< ListDigraph > cap( g );
 cap.upperMap( upper ).costMap( cost ).supplyMap( supply );
 CycleCanceling< ListDigraph > cc( g );
 cc.upperMap( upper ).costMap( cost ).supplyMap( supply );

 if( ns.run() != NetworkSimplex< ListDigraph >::OPTIMAL ||
     cs.run() != CostScaling< ListDigraph >::OPTIMAL ||
     cap.run() != CapacityScaling< ListDigraph >::OPTIMAL ||
     cc.run() != CycleCanceling< ListDigraph >::OPTIMAL )
  return( 1 );
 std::cout << "total costs: " << ns.totalCost() << ' ' << cs.totalCost() << ' '
           << cap.totalCost() << ' ' << cc.totalCost() << std::endl;
 if( ns.totalCost() != 36 || cs.totalCost() != 36 || cap.totalCost() != 36 ||
     cc.totalCost() != 36 )
  ++errors;
 if( ! optimal( g , ns , upper , cost , supply ) ||
     ! optimal( g , cs , upper , cost , supply ) ||
     ! optimal( g , cap , upper , cost , supply ) ||
     ! optimal( g , cc , upper , cost , supply ) )
  ++errors;

 // A second run after a change of one cost, the other parameters being
 // kept: (m2,t3) is the only arc into t3, whose 3 units cost 10 more each
 const ListDigraph::Arc first = id[ 8 ];
 cost[ first ] += 10;
 ns.costMap( cost );
 if( ns.run() != NetworkSimplex< ListDigraph >::OPTIMAL ||
```

```
    ! optimal( g , ns , upper , cost , supply ) || ns.totalCost() != 66 )
 ++errors;
std::cout << "after the change of a cost: " << ns.totalCost() << std::endl;
cost[ first ] -= 10;

// More supply than demand: under LEQ the demands are met exactly and
// some supply may stay where it is
supply[ s2 ] = 6;
ns.costMap( cost ).supplyMap( supply ).supplyType(
  NetworkSimplex< ListDigraph >::LEQ );
if( ns.run() != NetworkSimplex< ListDigraph >::OPTIMAL )
 ++errors;
std::cout << "LEQ with two more units at s2: " << ns.totalCost() << std::endl;
for( ListDigraph::NodeIt n( g ) ; n != INVALID ; ++n )
 if( ns.potential( n ) < 0 )
  ++errors;
// under GEQ the same data are infeasible
ns.supplyType( NetworkSimplex< ListDigraph >::GEQ );
if( ns.run() != NetworkSimplex< ListDigraph >::INFEASIBLE )
 ++errors;

return( errors );
}
```

abLEMON. The four classes answer `OPTIMAL`, with the empty flow, on a digraph without nodes, which LEMON 1.3.1 reported as infeasible. They answer `UNBOUNDED` on the same problems, i.e., the feasible ones with a cycle of negative cost made of arcs of infinite upper bound: in LEMON 1.3.1 `CostScaling` and `CycleCanceling` gave that answer on any arc of negative cost and infinite upper bound, `CapacityScaling` gave a wrong answer on feasible problems that its finite replacement of the infinite bounds made infeasible, and `NetworkSimplex` did not check feasibility first. The function `runWarm()`, which versions 1.4.0 to 1.5.0 added to re-optimize from the last basis, flow or potentials, was removed in 1.6.0: a re-optimization belongs to whoever knows what has changed, and a second `run()` solves the changed problem.

13.3 Tolerances and floating-point data

With integer types the data must be integer, and the comparisons are exact. With floating-point types the data need not be integer; however, a comparison of a residual capacity or of a reduced cost with an exact zero would make the algorithms pivot or push round-off for ever, or find a feasible problem infeasible. Instead, the four classes compare against two tolerances, one on the flows and one on the reduced costs, defined once in `lemon/bits/mcf_tolerance.h`. Unless the caller sets them, they are zero for an exact type and otherwise

$$\epsilon_f = 10^6\,\varepsilon\,\max\{1,\ |sup(u)|,\ |lower(uv)|,\ |upper(uv)| < \infty\}, \qquad \epsilon_c = 10^6\,\varepsilon\,\max\{1,\ |cost(uv)|\}, \tag{13.5}$$

where ε is the machine epsilon of the type (about $2.2 \cdot 10^{-16}$ for `double`), and the maxima run over the nodes and the arcs, with the infinite upper bounds left out. In other words, the tolerances are 10^6 units in the last place of the largest datum, or of 1 when the data are smaller. An excess, a deficit or a residual capacity within ϵ_f is taken as zero, and a reduced cost above $-\epsilon_c$ as nonnegative. Also, a sum of the supplies within ϵ_f of zero is taken as zero, since supplies such as 0.1, 0.2 and −0.3, which sum to zero in decimal, in binary sum to a few units in the last place of either sign.

A call of `tolerances(flow, cost)` sets the two tolerances, and a negative value, the default, leaves them to (13.5). In turn, `flowTolerance()` and `costTolerance()` return the tolerances in use, as a `Tolerance< Value >` and a `Tolerance< Cost >` (types named `FlowTolerance` and `CostTolerance` in each class) whose epsilon is zero for an exact type. We remark that `NetworkSimplex` computes them from its data at any time, while the other three return those that the last `run()` used, and zero before the first one. A tolerance of zero reproduces the behavior of LEMON 1.3.1, in which rounding errors may keep the algorithm going for ever; hence, one should set it only on data known to be exact in binary.

Our next program solves with the four classes a problem whose supplies are 0.1, 0.2 and −0.3, and whose capacities and costs are fractional. It checks the optimal cost, 0.14, and the tolerances (13.5), which here are $10^6\varepsilon$ on the flows (the data being below 1) and $1.1 \cdot 10^6\varepsilon$ on the costs. Finally it sets other tolerances, which `flowTolerance()` then returns.

```
// Floating-point data (section 13.3): supplies of 0.1, 0.2 and -0.3,
// which do not sum to zero in binary, and fractional capacities and
// costs, solved by the four algorithms with the tolerances they derive
// from the data; the total costs must agree within the tolerance on the
// costs, and the tolerances in use are read back.

#include <cmath>
#include <iostream>
#include <limits>
#include <lemon/smart_graph.h>
#include <lemon/network_simplex.h>
#include <lemon/cost_scaling.h>
#include <lemon/capacity_scaling.h>
#include <lemon/cycle_canceling.h>

using namespace lemon;

typedef SmartDigraph::ArcMap< double > ArcMap;
typedef SmartDigraph::NodeMap< double > NodeMap;

int main()
{
 SmartDigraph g;
 SmartDigraph::Node u = g.addNode() , v = g.addNode() , w = g.addNode() ,
                    z = g.addNode();
 ArcMap upper( g ) , cost( g );
 NodeMap supply( g );
 SmartDigraph::Arc a;
 a = g.addArc( u , w );
 upper[ a ] = 0.25;
 cost[ a ] = 1.1;
 a = g.addArc( u , z );
 upper[ a ] = 0.15;
 cost[ a ] = 0.7;
 a = g.addArc( v , w );
 upper[ a ] = 0.3;
 cost[ a ] = 0.2;
 a = g.addArc( z , w );
 upper[ a ] = 0.1;
 cost[ a ] = 0.3;
 a = g.addArc( v , z );
 upper[ a ] = 0.2;
 cost[ a ] = 0.4;
 supply[ u ] = 0.1;
 supply[ v ] = 0.2;
 supply[ w ] = -0.3;
 supply[ z ] = 0.0;
 std::cout << "sum of the supplies: " << ( 0.1 + 0.2 - 0.3 ) << std::endl;

 typedef NetworkSimplex< SmartDigraph , double > NS;
 typedef CostScaling< SmartDigraph , double > CS;
 typedef CapacityScaling< SmartDigraph , double > CAP;
 typedef CycleCanceling< SmartDigraph , double > CC;
 NS ns( g );
 CS cs( g );
 CAP cap( g );
 CC cc( g );
 ns.upperMap( upper ).costMap( cost ).supplyMap( supply );
 cs.upperMap( upper ).costMap( cost ).supplyMap( supply );
 cap.upperMap( upper ).costMap( cost ).supplyMap( supply );
 cc.upperMap( upper ).costMap( cost ).supplyMap( supply );
 int errors = 0;
 if( ns.run() != NS::OPTIMAL || cs.run() != CS::OPTIMAL ||
     cap.run() != CAP::OPTIMAL || cc.run() != CC::OPTIMAL )
  return( 1 );
```

```
// 0.1 along u, z, w at a cost of 1.0 and 0.2 along (v,w): 0.1 + 0.04
const double best = 0.14;
const double ceps = ns.costTolerance().epsilon();
std::cout << "total costs: " << ns.totalCost() << ' ' << cs.totalCost() << ' '
          << cap.totalCost() << ' ' << cc.totalCost() << std::endl;
std::cout << "tolerances of NetworkSimplex: flows "
          << ns.flowTolerance().epsilon() << ", costs " << ceps << std::endl;
const double tc[] = { ns.totalCost() , cs.totalCost() , cap.totalCost() ,
                      cc.totalCost() };
for( int i = 0 ; i != 4 ; ++i )
 if( std::abs( tc[ i ] - best ) > 1e-9 )
  ++errors;

// The default tolerances are 10^6 units in the last place of the largest
// datum, taken as 1 if it is smaller: 1 for the flows, whose data are
// below 1, and the largest cost, 1.1, for the costs
const double ulp = std::numeric_limits< double >::epsilon();
if( ns.flowTolerance().epsilon() != 1.0 * ulp * 1e6 ||
    ceps != 1.1 * ulp * 1e6 ||
    cs.flowTolerance().epsilon() != 1.0 * ulp * 1e6 )
 ++errors;

// Tolerances set by the caller are those in use
ns.tolerances( 1e-12 , 1e-12 );
if( ns.run() != NS::OPTIMAL || ns.flowTolerance().epsilon() != 1e-12 ||
    std::abs( ns.totalCost() - best ) > 1e-9 )
 ++errors;

return( errors );
}
```

abLEMON. The four algorithms solve problems with floating-point costs, capacities and supplies. In LEMON 1.3.1 `NetworkSimplex` compared reduced costs and residual flows with an exact zero, and typically pivoted for ever or reported infeasibility on such data, while the other three required integer data and answered `INFEASIBLE` on supplies that sum to zero in decimal but not in binary. The default tolerances have one definition for the four (version 1.9.0), those of `NetworkSimplex` on the flows counting the finite bounds too, and `tolerances()`, `flowTolerance()` and `costTolerance()` are new.

13.4 The network simplex method

13.4.1 Bases, pivots and potentials

`NetworkSimplex` implements the primal network simplex method (Dantzig 1963; Ahuja, Magnanti and Orlin 1993, chapter 11; Kelly and O'Neill 1991), i.e., the simplex method of linear programming specialized to (13.1), in which a basis is a spanning tree. After moving the lower bounds into the supplies, the class adds an artificial root node and, for each node, an artificial arc between it and the root, oriented by the sign of its supply. The initial basis is the tree of these arcs, which carry the supplies. Depending on the sign of the sum of the supplies, a second set of artificial arcs lets the nodes keep the unmet demand (GEQ) or the unused supply (LEQ) at no cost. The artificial arcs whose flow would leave a constraint violated have a large cost, larger than the cost of any path: for an exact `Cost` it is half of the largest value of the type plus one, and otherwise $(1 + \max_{uv} |cost(uv)|)\, n$. Thus, the method is the "big-M" variant of the simplex method, and the problem is infeasible exactly when some flow (beyond ϵ_f) is left at the end on the artificial arcs that may carry none.

Each nonbasic arc is at its lower or at its upper bound, and the potentials are those that give a zero reduced cost to the arcs of the tree. A nonbasic arc is *eligible* if its reduced cost (13.3) is negative at its lower bound, or positive at its upper bound, since sending flow around the cycle that it closes with the tree then decreases the cost. A *pivot* chooses an eligible arc by the pivot rule, and finds its cycle through the *join* node, i.e., the first common ancestor of its ends in the tree. Then it sends around the cycle the largest amount that the bounds allow, and it replaces by the entering arc the first arc of the cycle that reaches a bound (Figure 13.1). When no arc is eligible, the flow and the potentials satisfy (13.4). An amount of zero (a *degenerate* pivot) changes the basis but not the flow, while an unbounded amount (a cycle of arcs with infinite upper bounds and negative

cost) proves the problem unbounded, provided it is feasible.

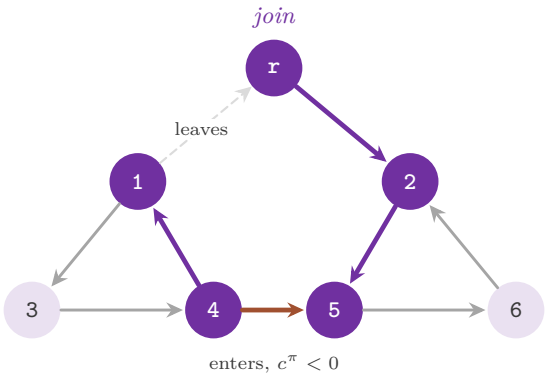


Figure 13.1. A pivot of the network simplex method: the tree of the basis hung from the root r (in the primary colour), the entering arc (4,5) of negative reduced cost, which closes a cycle with the tree path from 5 through the join node r back to 4, and the leaving arc (1,r), the first arc of the cycle to reach a bound.

Algorithm 13.1 (Network simplex).

```
1  move the lower bounds into the supplies; if the sum of the supplies has
   the wrong sign for the supply type: return INFEASIBLE
2  T := tree of the artificial arcs from and to the root; set the flows,
   the potentials and the thread of T
3  make the heuristic initial pivots (see below)
4  while the pivot rule finds an eligible arc e:
5      w := join node of the ends of e in T
6      delta := largest amount around the cycle of e and T;
       f := first arc of the cycle to reach a bound
7      if delta is infinite: go to 12
8      send delta around the cycle; if f != e: T := T - f + e
9      update the potentials of the smaller part that f splits off
10 if some artificial arc carries more than eps_f: return INFEASIBLE
11 put the lower bounds back, shift the potentials; return OPTIMAL
12 solve the problem again with zero costs: if it is infeasible, return
   INFEASIBLE; otherwise solve it again with its costs: return UNBOUNDED
```

Step 9 is `updatePotential()`. When the subtree that the pivot moves holds more than half of the nodes, it changes instead the potentials of the rest of the tree, in the opposite direction; this leaves the reduced costs as they are, and moves the potential of the root away from zero (within a limit) until a final shift brings it back. When the supplies sum to zero, step 11 shifts the potentials so that their largest value is at most zero under GEQ, and their smallest at least zero under LEQ, as (13.4) requires. Finally, step 12 is `feasibleUnbounded()`, by which the answer `UNBOUNDED` implies a feasible problem.

In step 3, the initial pivots are a heuristic of the original code. With a single supply node and a single demand node, they are made on the arcs found by a search backwards from the demand node over the arcs whose capacity is at least the total supply; with more nodes, on the cheapest arc into each demand node (or, under GEQ with less supply than demand, the cheapest arc out of each supply node). Each of these arcs enters the basis if it is eligible. Also, the constructor takes a flag `arc_mixing`, `true` by default, which stores the arcs in an interleaved order (with a stride of $\max\{m/n, 3\}$) rather than in the order of the digraph. According to the header, this usually performs like the original order, but it makes the method more robust and, in special cases, significantly faster.

The header states no bound on the number of pivots, and the rules of the class are not known to be polynomial. Each pivot costs the search of the rule, plus time proportional to the length of the cycle and to the size of the smaller part of the tree. In practice the method is usually the fastest one of the library on graphs of up to some thousands of nodes and on dense graphs (cf. section 13.9).

abLEMON. The potentials are updated on the smaller side of the tree at each pivot (version 1.4.0). In floating point, the leaving arc goes to the bound it is nearer to, its flow being set to that bound, rather than to the upper bound for a flow of, e.g., 10^{-16} (version 1.9.0), which may change the sequence of pivots. A problem

is answered `UNBOUNDED` only after a check of its feasibility (version 1.4.1).

13.4.2 Pivot rules

The argument of `run()` is the rule that chooses the entering arc, i.e., a constant of the nested enum `PivotRule`. The arcs are scanned in a wraparound fashion, and an arc is eligible when its reduced cost, signed by its state, is below $-\epsilon_c$. Let m' be the number of arcs searched, i.e., those of the digraph and, under GEQ or LEQ with unbalanced supplies, also the second artificial arcs; the five rules are then as follows.

- `FIRST_ELIGIBLE` takes the next eligible arc after the last one taken.
- `BEST_ELIGIBLE` takes the most violating arc among all of them.
- `BLOCK_SEARCH`, the default, examines blocks of $\max\{\lfloor\sqrt{m'}\rfloor, 10\}$ arcs and takes the best eligible arc of the first block that has one.
- `CANDIDATE_LIST` builds, in a major iteration, a list of at most $\max\{\lfloor 0.25\sqrt{m'}\rfloor, 10\}$ eligible arcs, and then takes the best arc of the list in at most $\max\{\lfloor 0.1\,\ell\rfloor, 3\}$ minor iterations, ℓ being the largest length of the list.
- `ALTERING_LIST` keeps the $\max\{\lfloor 0.01\,b\rfloor, 3\}$ best candidates of the previous iteration and extends them with a block of $b = \max\{\lfloor\sqrt{m'}\rfloor, 10\}$ arcs at each iteration.

Each rule is a private class of `NetworkSimplex` (`FirstEligiblePivotRule`, `BestEligiblePivotRule`, `BlockSearchPivotRule`, `CandidateListPivotRule` and `AlteringListPivotRule`), which `run()` instantiates for the chosen rule, and whose constants are those given above. According to the header, `BLOCK_SEARCH` and `ALTERING_LIST` were the most efficient rules on the instances on which they were tested, and `BLOCK_SEARCH`, being simpler and somewhat more robust, was made the default.

13.5 The starting flow and the infinite bounds

In contrast, the other three classes work on a residual network, in which each arc of the digraph appears forward and backward. An extra root node is joined to the nodes by a pair of arcs, which under GEQ take the unmet demand, with no cost and a capacity of one more than the deficit of the supplies. Two functions of `lemon/bits/mcf_feasible_flow.h` prepare that network.

The first one, `feasibleFlow()`, finds the flow that `CostScaling` and `CycleCanceling` start from, i.e., a flow within the bounds that meets the supplies. It runs the push-relabel algorithm of `Circulation` (chapter 12), with the active node of highest level first, all the nodes at level zero at the start, a push towards any lower level, and the first empty level as the proof that no such flow exists (Goldberg and Tarjan 1988; Cherkassky and Goldberg 1997). Its starting point is the greedy flow of `Circulation`, in which each arc takes the deficit of its head within its bounds. When it fails, the nodes above the empty level, together with the node being lifted, form a set that no residual arc leaves and whose excess is positive, which proves the problem infeasible; chapter 14 uses that set as a certificate. In floating point, an excess or a residual capacity within ϵ_f counts as zero, and at the end the residual capacities within it are moved to the reverse arcs, lest the algorithms send round-off around.

The second function, `infiniteBound()`, gives the bound that replaces an infinite upper bound in `CostScaling`, `CapacityScaling` and `CycleCanceling`. Suppose that no cycle of negative cost is made of arcs of infinite upper bound. Then some optimal flow sends on each arc at most

$$B = 1 + \sum_{u\in V} |sup(u)| + \sum_{uv:\ upper(uv)<\infty} |upper(uv)| + \sum_{uv\in A} |lower(uv)| \tag{13.6}$$

beyond its lower bound. Indeed, a flow decomposes into paths, which carry at most the supplies, and cycles, each of which contains an arc of finite bound (the others having a nonnegative cost, and hence being of no use). An arc that reaches B then keeps a nonnegative reduced cost in an optimal solution; hence, the potentials are optimal for the infinite bounds too. If necessary, the bound is reduced until the sums of the flows over the replaced arcs fit in `Value`. Whether such a cycle exists is decided beforehand by `negativeInfiniteCycle()`, a Bellman-Ford algorithm restricted to the arcs of infinite upper bound, which runs only if one of them has a negative cost; if such a cycle exists, a feasible problem is answered `UNBOUNDED`.

13.6 Cost scaling

`CostScaling` implements the cost scaling algorithm of Goldberg and Tarjan (Goldberg and Tarjan 1990; Goldberg 1997; Bünnagel, Korte and Vygen 1998), a primal-dual method that one may regard as the generalization to costs of the push-relabel algorithm of `Preflow`. A flow f and potentials π are *ε-optimal* when

$$cost^{\pi}(uv) \geq -\varepsilon \qquad \text{for each arc } uv \text{ of the residual network of } f, \tag{13.7}$$

and $\varepsilon = 0$ gives back (13.4). With integer costs multiplied by $n+1$, an ε-optimal flow with $\varepsilon < 1$ is optimal for the original costs. The algorithm starts from ε equal to the largest scaled cost divided by the scaling factor α, i.e., by the argument `factor` of `run()` (16 by default, and at least 2), and it divides ε by α after each phase. In a phase it *refines* the flow. First it saturates the residual arcs of negative reduced cost, which makes the flow ε-optimal but creates excesses and deficits; then it removes them by pushes along the *admissible* arcs (those of negative reduced cost) and by relabels, each of which lowers a potential by the least amount that makes an arc admissible, plus ε.

Algorithm 13.2 (Cost scaling).

```
1  find a feasible flow f (section 13.5); if none: return INFEASIBLE
2  if a cycle of infinite capacity has negative cost: return UNBOUNDED
3  scale the costs by (n+1) alpha; eps := largest scaled cost / alpha;
   pi := 0
4  while eps >= eps_min:
5      price refinement (heuristic); if it proves f eps-optimal: skip to 9
6      saturate the residual arcs of negative reduced cost
7      while some node u has positive excess (FIFO order):
           push or augment along admissible arcs, or relabel u;
           every few relabels, make a global update of the potentials
8      (the flow is now eps-optimal)
9      eps := eps / alpha, or eps_min if eps / alpha would fall below it
10 divide the potentials by (n+1) alpha; if some residual arc has a
   reduced cost below -eps_c, correct them by Bellman-Ford
11 shift the potentials to at most zero; return OPTIMAL
```

The nested enum `Method` selects the operation of step 7. With `PUSH` the flow moves on one admissible arc at a time, with a look-ahead that avoids pushing into a node that would only send the flow back. With `AUGMENT` it moves along admissible paths from a node with excess to a node with deficit. With `PARTIAL_AUGMENT`, the default and, according to the header, the most efficient and robust one, it moves along admissible paths of at most 4 arcs.

We remark that two heuristics of Goldberg 1997 are applied. The price refinement of step 5, tried at each phase from the second one on, computes potentials that may prove the current flow ε-optimal before any push. The global update of step 7 recomputes the potentials by a bucket-based search from the deficits, after a number of relabels proportional to $n + s^2$, s being the number of supply nodes.

For the internal computations the class uses the type `LargeCost` of the traits class `CostScalingDefaultTraits< GR , V , C >`, which is `long long` for an integer `Cost` (or `long` without `LEMON_HAVE_LONG_LONG`) and `double` otherwise; the named parameter `SetLargeCost< T >` changes it, provided that `Cost` is convertible to `T`. The last value ε_{min} of the scaling is 1 for an integer `LargeCost`, which proves the flow optimal, and otherwise $\epsilon_c\,(n+1)\,\alpha$, i.e., the tolerance on the reduced costs in the scaled units. As stated in the header, the worst case running time is $O(n^2 m \log(nK))$, where K is the largest arc cost. In practice the method is typically the most efficient one of the library on large sparse graphs (cf. section 13.9).

> **abLEMON.** `CostScaling` finds its starting flow on its residual network (version 1.4.0), where LEMON 1.3.1 ran `Circulation` on the digraph, and it takes floating-point costs, capacities and supplies (version 1.6.0). The price refinement of LEMON 1.3.1 could write past the end of its buckets, which is fixed.

13.7 Capacity scaling

`CapacityScaling` implements the capacity scaling version of the successive shortest path algorithm (Edmonds and Karp 1972; Ahuja, Magnanti and Orlin 1993, chapter 10). It is a dual method, which keeps potentials under

which the residual arcs of capacity at least Δ have nonnegative reduced costs, and which sends flow from the nodes with excess to the nodes with deficit along shortest paths. In a phase with a given Δ, the arcs of negative reduced cost and residual capacity at least Δ are saturated first. Then, while some node has an excess and some node a deficit of at least Δ, the algorithm of Dijkstra on the reduced costs, over the residual arcs of capacity at least Δ, finds a shortest path from a node with excess to a node with deficit. The flow is augmented along it by at least Δ, and the potentials of the nodes that the search has settled are updated by their distances. Afterwards Δ is divided by the scaling factor (the argument `factor` of `run()`, 4 by default), down to the smallest unit.

Algorithm 13.3 (Capacity scaling).

```
1  saturate the arcs of negative cost (an infinite bound replaced by B,
   section 13.5); Delta := largest power of 2 (times the unit) not above
   the smallest of the largest excess, deficit and residual capacity
2  loop:
3      saturate the residual arcs with r >= Delta and reduced cost < -eps_c
4      while a node s has e(s) >= Delta and some node has e <= -Delta:
5          Dijkstra from s on the arcs with r >= Delta, until a node t
           with e(t) <= -Delta is settled
6          if none is reached: (at the last Delta) return INFEASIBLE
7          augment along the path by min{ e(s), -e(t), its capacities }
8          update the potentials of the settled nodes
9      if Delta = unit: return OPTIMAL (or UNBOUNDED, section 13.5)
10     Delta := max{ unit, Delta / factor }
```

Here the unit is 1 for an exact `Value` and ϵ_f otherwise; hence, in floating point the last phase sends any amount above the tolerance. With a factor of at most 1 the scaling is disabled, and the method becomes the plain successive shortest path algorithm, which is usually the fastest choice when the supplies and the capacities are small. In the searches the class uses the heap type `Heap` of the traits class `CapacityScalingDefaultTraits< GR , V , C >`, by default `QuadHeap< Cost , RangeMap< int > >`, whose fewer levels make the searches cheaper than on a binary heap (Johnson 1975). The named parameter `SetHeap< T >` replaces it by any heap that meets the concept of chapter 7 with the priority type `Cost` and the cross reference map `RangeMap< int >`. As stated in the header, the worst case running time is $O(m \log U\,(n+m) \log n)$, where U is the largest supply or capacity.

abLEMON. The default heap is a 4-ary heap (version 1.4.0), the class compiles on value types narrower than `int`, and it takes floating-point data (version 1.6.0).

13.8 Cycle canceling

`CycleCanceling` implements three cycle canceling methods (Klein 1967; Goldberg and Tarjan 1989), which start from the feasible flow of section 13.5 and cancel cycles of negative cost of the residual network until none is left, i.e., until the flow is optimal by Theorem 13.2. The nested enum `Method` selects one of them.

- `SIMPLE_CYCLE_CANCELING` finds negative cycles by the algorithm of Bellman and Ford on the residual network, with a limit on its rounds that starts at 2 and grows by a factor of 1.5 when no cycle is found, and cancels node-disjoint cycles of the predecessor graph.
- `MINIMUM_MEAN_CYCLE_CANCELING` cancels a cycle of minimum mean cost at each step, found by Howard's algorithm with a limit of $\max\{n, 5\}$ iterations and, if the limit is reached, by the strongly polynomial algorithm of Hartmann and Orlin (chapter 16); the header gives the bound $O(n^2 m^3 \log n)$.
- `CANCEL_AND_TIGHTEN`, the default, which the header calls by far the most efficient and robust, is the improved version of the previous method of Goldberg and Tarjan 1989, with the bound $O(n^2 m^2 \log n)$.

In Cancel-and-Tighten an arc is admissible when its reduced cost is negative. A phase cancels, by a depth first search, the cycles of admissible arcs, until the admissible network is acyclic; then it *tightens* the potentials along a topological order of that network, and it recomputes ε as the largest violation of (13.4) by a residual arc. Every $\max\{\lfloor\sqrt{n+1}\rfloor, 5\}$ phases, instead, ε is set to the opposite of the minimum cycle mean of the residual network, and the potentials are recomputed from it by the algorithm of Bellman and Ford.

Algorithm 13.4 (Cancel-and-Tighten).

```
1  find a feasible flow f (section 13.5); if none: return INFEASIBLE
2  if a cycle of infinite capacity has negative cost: return UNBOUNDED
3  pi := 0; eps := largest -cost(uv) over the residual arcs
```

```
4  while eps * (n+1) >= 1 (exact costs), or eps > eps_c (floating point):
5      cancel the cycles of admissible arcs (cost^pi < 0) by DFS
6      if this is not the k-th phase:
7          tighten pi along a topological order of the admissible arcs;
           eps := largest -cost^pi(uv) over the residual arcs
8      else:
9          eps := - (minimum cycle mean of the residual network);
           pi := potentials for eps, by Bellman-Ford
10 compute the final potentials by Bellman-Ford; return OPTIMAL
```

During the phases the potentials are `double`, fractional even with an exact `Cost`, and the test of admissibility uses a tolerance of 10^{-6} then, as in LEMON 1.3.1, and ϵ_c with a floating-point `Cost`. After the last phase of the two methods based on mean cycles, the potentials are computed by the algorithm of Bellman and Ford on the residual network (in `SIMPLE_CYCLE_CANCELING` its distances already give them). Its operation traits count an improvement only beyond ϵ_c, since in floating point the round-off of the sums can make a cycle of cost zero (e.g., an arc and its reverse) look negative for ever. The class has no traits parameter, and the header warns that, in practice, the method is typically orders of magnitude slower than the scaling algorithms and `NetworkSimplex`.

abLEMON. `CycleCanceling` finds its starting flow on its residual network (version 1.4.0), takes floating-point data (version 1.6.0), and compares with zero only through the tolerances, its inner minimum mean cycle searches included (version 1.9.0).

13.9 Choosing an algorithm

In the documentation of the module, which follows the experimental study of Király and Kovács 2012, `NetworkSimplex` and `CostScaling` are recommended as the most efficient classes in general. `NetworkSimplex` is usually the fastest on relatively small graphs (up to several thousands of nodes) and on dense graphs, while `CostScaling` is typically more efficient on large graphs (e.g., hundreds of thousands of nodes or more), especially sparse ones. Other classes may be faster in special cases; e.g., when the total supply or the capacities are rather small, `CapacityScaling` without scaling is usually the fastest. In our view, `CycleCanceling` is mostly of interest for its strongly polynomial bound and for teaching. We remark that these are statements about typical instances, and that the ranking on a given family of problems is best found by running the four classes on it, which their common interface makes easy (cf. the program of section 13.2). Also, note that only `NetworkSimplex` takes the LEQ form directly.

13.10 Exercises

Exercise 13.1. Show that a flow of (13.1) with a cycle of negative cost in its residual network is not optimal, and derive the potentials of Theorem 13.2 from shortest path distances. Then check (13.4) on the four answers of `manual/examples/13-min-cost-flow.cc`, which does it in its function `optimal()`.

Exercise 13.2. Transform a problem in the LEQ form (13.2) into the GEQ form with `ReverseDigraph` and `NegMap`, solve it with `CostScaling`, and compare the cost with that of `NetworkSimplex` with `supplyType(LEQ)`. (The supply types are checked by `checkSupplyTypes()` in `test/min_cost_flow_test.cc`.)

Exercise 13.3. Solve the problem of `manual/examples/13-floating-point.cc` with `tolerances(0, 0)` on supplies that are exact in binary (e.g., 0.25, 0.5 and -0.75), and then on 0.1, 0.2 and -0.3 with the default tolerances; explain the difference by (13.5). (The tolerances are checked by `checkTolerances()` and `checkToleranceGetters()` in `test/min_cost_flow_test.cc`.)

Exercise 13.4. Time the five pivot rules of `NetworkSimplex` on random instances of a few thousand nodes, on an idle machine and with repeated runs, and compare the default with `BEST_ELIGIBLE`; repeat with `arc_mixing` set to `false`.

Exercise 13.5. Run `CostScaling` with the three methods and with the factors 2, 16 and 64, and `CapacityScaling` with the factors 1 and 4, on the same instances, and check that the costs agree. (The battery `test/min_cost_flow_battery_test.cc` runs every algorithm and method on limit cases and random instances, with integer and floating-point data.)

Exercise 13.6. Compute the bound B of (13.6) for the problem of `manual/examples/13-min-cost-flow.cc` with all the upper bounds infinite, and verify that an optimal flow of that problem sends less than B on each arc.

14. Certificates and the k best flows

We now turn to what the library gives besides one optimal flow. When a minimum cost flow problem has no optimal flow, the four algorithms of chapter 13 give a certificate of the answer, i.e., a set of nodes that proves it infeasible or a cycle that proves it unbounded, which a program can check on the data. They can also give several such certificates, in a definite order. When the problem has an optimal flow, the class `KBestFlows` of `lemon/k_best_flows.h` gives the k integer flows of least cost, and the class `KBestExtremeFlows` of `lemon/k_best_extreme_flows.h` gives the k vertices of least cost of the polyhedron of the flows, together with distinct optimal potentials. Section 14.1 treats the certificates of infeasibility and section 14.2 those of unboundedness, both implemented in `lemon/bits/mcf_certificate.h`; sections 14.3 and 14.4 describe the two classes of the k best flows, and section 14.5 the alternative optimal potentials. We keep the notation of section 13.1: the GEQ form (13.1), the reduced costs (13.3), and the residual network of a flow.

abLEMON. The content of this chapter is new with respect to LEMON 1.3.1, which gave no certificate and no other flow than an optimal one: `infeasibleCut()` and `unboundedCycle()` appeared in version 1.6.0, `infeasibleCuts()`, `disjointInfeasibleCuts()`, `unboundedCycles()`, `KBestFlows` and `KBestExtremeFlows` in version 1.7.0, the alternative optimal potentials in version 1.8.0, and their method `PIVOTS`, the cuts of `NetworkSimplex` taken from its last tree, the ties of `infeasibleCuts()` and the single run of Howard's algorithm in `unboundedCycles()` in version 1.9.0.

14.1 Certificates of infeasibility

14.1.1 Violated sets

For a set $N' \subseteq V$ of nodes, the *violation* of N' under GEQ is its supply less what can leave it:

$$viol(N') = \sum_{u \in N'} sup(u) - \sum_{uv \in A,\ u \in N',\ v \notin N'} upper(uv) + \sum_{uv \in A,\ u \notin N',\ v \in N'} lower(uv), \tag{14.1}$$

where no arc out of N' may have an infinite upper bound (such a set would prove no infeasibility). Under LEQ the same holds with $-sup$ in place of sup and the arcs taken backwards, i.e., the demand of N' less what can enter it. A set of positive violation clearly proves (13.1) infeasible, since its supply cannot leave it, and the converse holds too.

Theorem 14.1. *(Gale, Hoffman.) Problem (13.1) is infeasible if and only if some set N' has $viol(N') > 0$. Moreover, let sup' be the supplies net of the lower bounds, with P the sum of their positive values. Consider the network that joins a source σ to each node u with $sup'(u) > 0$ by an arc of capacity $sup'(u)$, each node u with $sup'(u) < 0$ to a target τ by an arc of capacity $-sup'(u)$, and that gives each arc uv the capacity $upper(uv) - lower(uv)$. Then, for each cut $\{\sigma\} \cup N'$ of this network, $viol(N') = P - cap(\{\sigma\} \cup N')$; hence, the largest violation is P minus the value of a maximum flow, and the source side of a minimum cut, without σ, is a set of largest violation.* Its first part is the feasibility theorem of Gale 1957 and Hoffman 1960 (Theorem 12.2 in chapter 12); the identity follows by expanding the capacity of the cut, in which the lower bounds of the arcs inside N' cancel.

14.1.2 The interface

After a `run()` that returned `INFEASIBLE`, the four classes give three kinds of certificates, all as sets of nodes of the digraph, and all empty after any other answer.

- `infeasibleCut()` returns one set N' with $viol(N') > 0$.

- `infeasibleCuts(k)` returns the k sets of largest violation, in the order of their violations, at most k and only those that prove the problem infeasible.
- `disjointInfeasibleCuts()` returns sets of positive violation that are pairwise disjoint, with no arc between any two of them.

Each set is checked on the data as they were given, by (14.1), before it is returned. In floating point the check requires the violation to exceed both the tolerance on the flows and the round-off of the sum, i.e., the default epsilon of `Tolerance` times one plus the sum of the absolute values of the terms of (14.1). When the violation does not exceed them, no set is returned; hence, a set that only round-off makes violated is never given as a certificate. `NetworkSimplex` gives its sets under the supply type, i.e., by (14.1) with the arcs reversed and the supplies negated under LEQ; the other three classes use the GEQ form.

If it proves the answer, the set of `infeasibleCut()` is the one that the algorithm has at hand when it finds no feasible flow, at no further cost. It is the set of all the nodes when the supplies sum to the wrong sign (positive under GEQ, negative under LEQ). Otherwise, it depends on the class:

- `NetworkSimplex` takes it from its last tree, whose potentials under the costs of the first phase (one on the artificial arcs that should carry no flow, zero on the others) are an optimal solution of the dual of the first phase; so is the indicator of each level set of them, i.e., the nodes whose potential is at least, or at most, a value, and the first level set that proves the answer is kept;
- `CostScaling` and `CycleCanceling` take the nodes above the level that the push-relabel algorithm of their starting flow leaves empty, together with the node it was lifting, from which no residual arc leaves (section 13.5);
- `CapacityScaling` takes the nodes that its last search for a shortest augmenting path reached from a node with excess without finding a deficit, from which no residual arc leaves.

If that set does not prove the answer (which may happen, although rarely, in floating point), the class falls back on a set of largest violation, by one maximum flow of `Preflow` on the network of Theorem 14.1. Thus, `infeasibleCut()` usually costs no more than the run, and one maximum flow at most; we remark that one can therefore call it without concern even on "large" problems.

In turn, the sets of `disjointInfeasibleCuts()` are the weakly connected components of the set of `infeasibleCut()` whose violation is positive, in the order of their violations. Since no arc joins two of them, their violations sum to at least that of the set. Note that they depend on the set that the class has at hand. In the example of section 14.1.4, `NetworkSimplex` gives a set of violation 4, whose two components are violated, while the other three classes give a set of violation 2, which is a single component.

14.1.3 The k most violated sets

We find the sets of `infeasibleCuts(k)` by the partition of Lawler 1972, applied to the cuts of the network of Theorem 14.1 as in Hamacher, Picard and Queyranne 1984. Here the space of the sets is split into *parts*, in each of which some nodes are fixed in the set and some out of it. As in Theorem 14.1, the best set of a part is found by a maximum flow in which a fixed node is joined to the source or to the target by an arc that no minimum cut may contain, and its violation bounds from above those of all the sets of the part. Once the best set of a part is given, the part is split by its free nodes in turn. The j-th child keeps the first $j-1$ of them as they are in the set, and takes the j-th one out of the set if the set has it, in otherwise.

Algorithm 14.1 (The k most violated sets).

```
root part: no node fixed; solve it by one maximum flow (Theorem 14.1)
give the sets that tie for the largest violation, from the residual
digraph of that flow (the closed sets, see below), up to k of them
while fewer than k sets are given:
    p := the part of largest bound (its own violation, or that of its
         parent if it is not solved yet)
    if p is not solved: solve it, from the maximum flow of its parent;
         if its best set is not violated, drop p; continue
    give the best set of p (unless already given); remove p
    for each free node u_j of p, j = 1, 2, ...:
        add the child that fixes u_1 .. u_{j-1} as in the set of p and
        u_j opposite to it, with the bound of p, not solved
```

Two devices keep the number of maximum flows rather small. First, a child waits in the list with the violation of

its parent as a bound, and its best set is computed only when it is the most promising part. Hence, a maximum flow is made only for the parts that might hold one of the k sets, rather than for each child of each set given. Second, the maximum flow of a part is a preflow of each of its children, whose capacities are only larger (one more fixed node adds capacity on one arc from the source or to the target). Hence, `Preflow::init(flowMap)` starts each child from the flow of its parent (chapter 12). In the worst case, as the header states, a set costs at most n maximum flows, although typically far fewer are made, since most children are never solved.

In step 2 the ties come from a result of Picard and Queyranne 1980: the minimum cuts of a network are the sets closed in the residual digraph of any maximum flow (no residual arc leaves them) that contain the source but not the target. In other words, they are the unions of closures of strongly connected components of that digraph. Also, the function `closedSets()` finds them from the maximum flow of the root, by adding the closure of one component at a time, breadth first, starting from the closure of the source. The partition then skips them instead of finding each of them again by a maximum flow of its own.

14.1.4 An example

Figure 14.1 shows a problem of five nodes, in which the node a must send 4 units on an arc of capacity 2, and d must send 3 units on one of capacity 1, while b, with no supply, can only feed a. The sets $\{a\}$, $\{a, b\}$ and $\{d\}$ have violation 2, and the sets $\{a, d\}$ and $\{a, b, d\}$ have violation 4, the largest; hence, the four best sets of `infeasibleCuts(4)` have the violations 4, 4, 2 and 2.

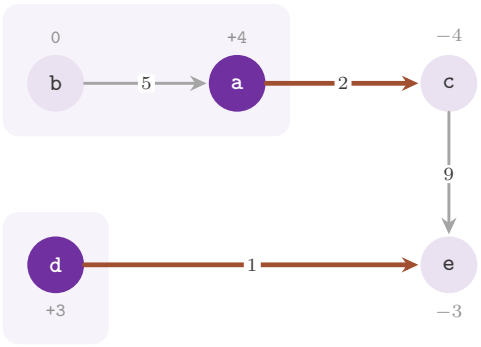


Figure 14.1. An infeasible problem: the supply of each node next to it, the capacity on each arc; the sets {a, b} and {d}, each of violation 2, are disjoint and joined by no arc, and their union, of violation 4, is a set of largest violation.

Our program solves the problem of Figure 14.1, and the unbounded problem of section 14.2, with the four classes, and it checks each certificate against the data. For `infeasibleCuts()` it checks positive violations (14.1) in a nonincreasing order and distinct sets, and for `disjointInfeasibleCuts()` pairwise disjoint sets with no arc between them. In the unbounded case it checks closed cycles of negative cost, made of arcs of infinite capacity.

```
// The certificates of chapter 14: an infeasible problem, for which each of
// the four algorithms gives sets of nodes whose violation (14.1) is
// positive, and an unbounded one, for which each gives cycles of negative
// cost made of arcs of infinite capacity; every certificate is checked
// here on the data, not taken on trust.

#include <iostream>
#include <set>
#include <vector>
#include <lemon/list_graph.h>
#include <lemon/network_simplex.h>
#include <lemon/cost_scaling.h>
#include <lemon/capacity_scaling.h>
#include <lemon/cycle_canceling.h>

using namespace lemon;

typedef ListDigraph::Node Node;
typedef ListDigraph::Arc Arc;
```

```
typedef ListDigraph::ArcMap< int > ArcMap;
typedef ListDigraph::NodeMap< int > NodeMap;

// The violation (14.1) of a set of nodes under GEQ, with no lower bounds:
// its supply less the capacity of the arcs that leave it
int violation( const ListDigraph & g , const ArcMap & upper ,
               const NodeMap & supply , const std::vector< Node > & set )
{
 ListDigraph::NodeMap< bool > in( g , false );
 int v = 0;
 for( std::size_t i = 0 ; i != set.size() ; ++i ) {
  in[ set[ i ] ] = true;
  v += supply[ set[ i ] ];
  }
 for( ListDigraph::ArcIt a( g ) ; a != INVALID ; ++a )
  if( in[ g.source( a ) ] && ! in[ g.target( a ) ] )
   v -= upper[ a ];
 return( v );
 }

template< typename MCF >
int checkInfeasible( const char * name , const ListDigraph & g ,
                     const ArcMap & upper , const ArcMap & cost ,
                     const NodeMap & supply )
{
 MCF mcf( g );
 mcf.upperMap( upper ).costMap( cost ).supplyMap( supply );
 if( mcf.run() != MCF::INFEASIBLE )
  return( 1 );
 int errors = 0;
 const std::vector< Node > cut = mcf.infeasibleCut();
 std::cout << name << ": a cut of violation "
           << violation( g , upper , supply , cut );
 if( cut.empty() || violation( g , upper , supply , cut ) <= 0 )
  ++errors;
 // the k most violated sets, in order and distinct
 const std::vector< std::vector< Node > > best = mcf.infeasibleCuts( 4 );
 std::set< std::set< int > > seen;
 std::cout << ", the best " << best.size() << ":";
 for( std::size_t j = 0 ; j != best.size() ; ++j ) {
  const int v = violation( g , upper , supply , best[ j ] );
  std::cout << ' ' << v;
  if( v <= 0 || ( j > 0 && v > violation( g , upper , supply ,
                                         best[ j - 1 ] ) ) )
   ++errors;
  std::set< int > ids;
  for( std::size_t h = 0 ; h != best[ j ].size() ; ++h )
   ids.insert( g.id( best[ j ][ h ] ) );
  if( ! seen.insert( ids ).second )
   ++errors;
  }
 // disjoint violated sets, with no arc between two of them
 const std::vector< std::vector< Node > > dis = mcf.disjointInfeasibleCuts();
 ListDigraph::NodeMap< int > owner( g , -1 );
 for( std::size_t j = 0 ; j != dis.size() ; ++j ) {
  if( violation( g , upper , supply , dis[ j ] ) <= 0 )
   ++errors;
  for( std::size_t h = 0 ; h != dis[ j ].size() ; ++h ) {
   if( owner[ dis[ j ][ h ] ] >= 0 )
    ++errors;
   owner[ dis[ j ][ h ] ] = int( j );
   }
  }
 for( ListDigraph::ArcIt a( g ) ; a != INVALID ; ++a )
  if( owner[ g.source( a ) ] >= 0 && owner[ g.target( a ) ] >= 0 &&
```

```
      owner[ g.source( a ) ] != owner[ g.target( a ) ] )
   ++errors;
 std::cout << ", " << dis.size() << " disjoint" << std::endl;
 return( errors );
 }

// Whether the arcs make a directed cycle of negative cost, each of
// infinite upper bound
bool negativeCycle( const ListDigraph & g , const ArcMap & cost ,
                    const std::vector< bool > & inf ,
                    const std::vector< Arc > & cycle )
{
 if( cycle.empty() )
  return( false );
 int c = 0;
 for( std::size_t h = 0 ; h != cycle.size() ; ++h ) {
  const Arc next = cycle[ ( h + 1 ) % cycle.size() ];
  if( g.target( cycle[ h ] ) != g.source( next ) ||
      ! inf[ g.id( cycle[ h ] ) ] )
   return( false );
  c += cost[ cycle[ h ] ];
  }
 return( c < 0 );
 }

template< typename MCF >
int checkUnbounded( const char * name , const ListDigraph & g ,
                    const ArcMap & upper , const ArcMap & cost ,
                    const std::vector< bool > & inf )
{
 MCF mcf( g );
 ArcMap up( g );
 for( ListDigraph::ArcIt a( g ) ; a != INVALID ; ++a )
  up[ a ] = inf[ g.id( a ) ] ? mcf.INF : upper[ a ];
 mcf.upperMap( up ).costMap( cost );
 if( mcf.run() != MCF::UNBOUNDED )
  return( 1 );
 int errors = 0;
 if( ! negativeCycle( g , cost , inf , mcf.unboundedCycle() ) )
  ++errors;
 const std::vector< std::vector< Arc > > cycles = mcf.unboundedCycles( 3 );
 std::set< int > used;
 for( std::size_t j = 0 ; j != cycles.size() ; ++j ) {
  if( ! negativeCycle( g , cost , inf , cycles[ j ] ) )
   ++errors;
  for( std::size_t h = 0 ; h != cycles[ j ].size() ; ++h )
   if( ! used.insert( g.id( cycles[ j ][ h ] ) ).second )
    ++errors;
  }
 std::cout << name << ": " << cycles.size() << " arc-disjoint negative cycles"
           << std::endl;
 if( cycles.size() != 2 )
  ++errors;
 return( errors );
 }

int main()
{
 int errors = 0;
 {
  // Infeasible: a sends 4 units on an arc of capacity 2, d sends 3 on
  // one of capacity 1, and b, with no supply, can only feed a
  ListDigraph g;
  Node a = g.addNode() , b = g.addNode() , c = g.addNode() , d = g.addNode() ,
       e = g.addNode();
```

```
  ArcMap upper( g ) , cost( g , 1 );
  NodeMap supply( g );
  upper[ g.addArc( a , c ) ] = 2;
  upper[ g.addArc( d , e ) ] = 1;
  upper[ g.addArc( b , a ) ] = 5;
  upper[ g.addArc( c , e ) ] = 9;
  supply[ a ] = 4;
  supply[ b ] = 0;
  supply[ c ] = -4;
  supply[ d ] = 3;
  supply[ e ] = -3;
  errors += checkInfeasible< NetworkSimplex< ListDigraph > >(
    "NetworkSimplex" , g , upper , cost , supply );
  errors += checkInfeasible< CostScaling< ListDigraph > >(
    "CostScaling" , g , upper , cost , supply );
  errors += checkInfeasible< CapacityScaling< ListDigraph > >(
    "CapacityScaling" , g , upper , cost , supply );
  errors += checkInfeasible< CycleCanceling< ListDigraph > >(
    "CycleCanceling" , g , upper , cost , supply );
  }
 {
  // Unbounded: the cycles p, q, r (cost -1) and r, s (cost -2) of
  // infinite capacity, and an arc (q,p) of cost -10 but finite capacity
  ListDigraph g;
  Node p = g.addNode() , q = g.addNode() , r = g.addNode() , s = g.addNode();
  ArcMap upper( g , 0 ) , cost( g );
  std::vector< bool > inf;
  struct
  {
   Node u , v;
   int cost;
   bool inf;
   } arcs[] = { { p , q , -2 , true } , { q , r , 1 , true } ,
                { r , p , 0 , true } ,
               { r , s , -3 , true } , { s , r , 1 , true } ,
               { q , p , -10 , false } };
  for( int i = 0 ; i != 6 ; ++i ) {
   Arc x = g.addArc( arcs[ i ].u , arcs[ i ].v );
   cost[ x ] = arcs[ i ].cost;
   upper[ x ] = 5;
   inf.push_back( arcs[ i ].inf );
   }
  errors += checkUnbounded< NetworkSimplex< ListDigraph > >(
    "NetworkSimplex" , g , upper , cost , inf );
  errors += checkUnbounded< CostScaling< ListDigraph > >( "CostScaling" , g ,
                                                          upper , cost , inf );
  errors += checkUnbounded< CapacityScaling< ListDigraph > >(
    "CapacityScaling" , g , upper , cost , inf );
  errors += checkUnbounded< CycleCanceling< ListDigraph > >(
    "CycleCanceling" , g , upper , cost , inf );
  }
 return( errors );
 }
```

14.2 Certificates of unboundedness

A run returns `UNBOUNDED` only on a feasible problem with a directed cycle whose arcs have infinite upper bounds and whose cost is negative; along such a cycle the flow may grow without limit while the cost decreases (section 13.1). After such a run the four classes give two kinds of certificates, both empty after any other answer.

- `unboundedCycle()` returns one such cycle, as the vector of its arcs in the direction of the flow.
- `unboundedCycles(k)` returns at most k such cycles, pairwise arc-disjoint, in the order of their mean costs.

In `NetworkSimplex` the cycle is the one of the pivot that it could not perform, i.e., its entering arc and the path of the tree between the two ends of it, unless that path goes through an artificial arc. In that case, and

always in the other three classes, the cycle is found by the algorithm of Bellman and Ford on the arcs of infinite upper bound, run from a virtual node joined to all the others. After each round the graph of the predecessors is searched for a cycle, which has a negative cost if it exists (Tarjan 1983), and the search gives up after $4n + 4$ rounds. In floating point the cycle is returned only if its cost is negative beyond the round-off.

One might think of the cycle of least total cost as the natural choice for several cycles, but it is NP-hard to find, since it would decide whether a digraph is Hamiltonian (cf. Exercise 14.3). Instead, the cycles of `unboundedCycles(k)` are those of minimum mean cost, which are the most negative per arc. Howard's algorithm (class `HowardMmc`, chapter 16), run on the arcs of infinite upper bound, ends on each strongly connected component with a policy whose cycle has the minimum mean of the component (Dasdan 2004); the class gives these cycles through `componentCycleNum()` and `componentCycle()`.

Algorithm 14.2 (Arc-disjoint negative cycles).

```
1  C := empty; T := the arcs of infinite upper bound
2  while |C| < k and T is not empty:
3      run Howard's algorithm on the arcs of T
4      for each strongly connected component, take its cycle of minimum
       mean if that mean is negative beyond the round-off
5      if none: stop
6      add them to C by increasing mean, up to k; remove their arcs from T
7  sort C by mean; return C
```

Note that the cycles of one run are node-disjoint, and those of later runs use only the arcs not yet taken; hence, the cycles are arc-disjoint, and the first one has the minimum mean of all. A cycle of a later run may have a smaller mean than some of an earlier one (never than the first one), which is why step 7 sorts them. Since one run of Howard's algorithm gives the cycles of all the components, the algorithm runs again only when fewer than k cycles were found.

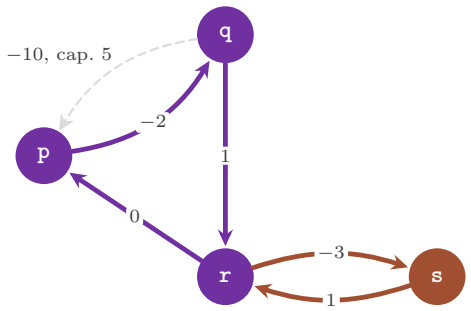


Figure 14.2. An unbounded problem: the cost on each arc; the arcs of infinite capacity are drawn full, and the arc (q,p), of cost -10 but capacity 5, dashed. The cycle (r,s,r), of mean -1, is the one of minimum mean; the cycle (p,q,r,p), of mean -1/3, is found by the second run on the arcs left.

In the problem of Figure 14.2, the four nodes form one strongly connected component of the arcs of infinite capacity, whose cycle of minimum mean is (r, s, r), of cost -2 and mean -1. A second run on the arcs left finds (p, q, r, p), of cost -1 and mean $-1/3$, and `unboundedCycles(3)` gives these two cycles, as the program of section 14.1.4 checks. The arc (q, p) closes with (p, q) a cycle of cost -12, but its capacity is finite; hence, it bounds the flow on that cycle, and it never appears in a certificate.

14.3 The k best integer flows: KBestFlows

14.3.1 The algorithm

We now describe the class `KBestFlows`, which finds the k integer flows of least cost of (13.1), in the order of their costs: first an optimal flow, then the other optimal flows, if any, and then the suboptimal ones. Its algorithm is the one of Hamacher 1995, which rests on the following property of the integer flows.

Theorem 14.2. *(Hamacher.) Let f be a flow of least cost in a set of integer flows defined by integer bounds on the arcs. Then a second best flow of the set is obtained from f by sending one unit along a cycle of least cost of the residual network of f (an arc together with its reverse not being a cycle).* Indeed, the difference between f

and a second best flow is an integer circulation of the residual network, which decomposes into cycles, each of nonnegative cost; one unit along any of them gives a flow of the set that is no costlier than the second best one. For a proof see Hamacher 1995.

Then the set of all the integer flows is split as in the partition of Lawler 1972, on an arc where the best flow and the second best flow differ, i.e., the first arc of the cycle. On one side the flow of the arc keeps its value in the best flow, which remains the best of that side; on the other it takes the value of the second best flow, which becomes the best of that side. Hence, the next flow of the sequence is the cheapest of the second best flows of the sides.

Algorithm 14.3 (K best integer flows).

```
1  f_1 := an optimal flow (NetworkSimplex, or initFlow()); if none: return
2  S := { (bounds of the problem, f_1) }; compute the second best of it
3  for i = 2, ..., k:
4      (B, f) := the set of S whose second best flow g is the cheapest;
       if no set has one: stop
5      f_i := g; let a be the first arc of the cycle from f to g
6      B' := B with lower(a) := f(a) + 1 (if g(a) > f(a)),
             or with upper(a) := f(a) - 1 (if g(a) < f(a))
7      B  := B with the flow of a fixed to f(a)
8      add (B', g) to S; compute the second best flows of (B, f), (B', g)
```

In `secondBest()`, the second best flow of a set is found as follows. First, the potentials of the residual network of the best flow of the set are computed by the algorithm of Bellman and Ford, from a virtual node joined to all the others. Since the best flow of the set leaves no cycle of negative cost, the reduced costs are nonnegative. Then, for each node v, the algorithm of Dijkstra computes the shortest paths from v on the reduced costs, and for each residual arc (u, v) the cycle of least cost through it is the arc and a shortest path from v back to u. When that path is just the reverse of the arc, which is not a cycle, one more search is made without the reverse arc. Hence, as stated in the header, the k-th flow takes $O(nm \log n)$ time after the $(k-1)$-th, and $O(m^2 \log n)$ at worst. Of course, the sequence does not depend on the algorithm that found the first flow, but for the order of the flows of equal cost.

14.3.2 The interface

`KBestFlows<GR, V, C>` takes the type of the digraph, the number type `V` of the flows, of the bounds and of the supplies (by default `int`), and the number type `C` of the costs (by default `V`). Its constructor takes the digraph, whose nodes and arcs it numbers at once; the digraph must not change afterwards. Until they are given, the bounds are 0 and infinite, the costs 1 and the supplies 0. Its setters, each returning the object, are the following ones:

- `lowerMap(map)`, `upperMap(map)`, `costMap(map)` and `supplyMap(map)`, which copy the data as in chapter 13, an upper bound that is infinite or the largest value of `Value` meaning no bound;
- `initFlow(map)`, an optimal flow to start from, which must be optimal for the data given, and after which no minimum cost flow algorithm is run;
- `tolerances(flow, cost)`, the tolerances of section 13.3 given to the `NetworkSimplex` that finds the optimal flow, a negative value leaving them to the data; `flowTolerance()` and `costTolerance()` return those in use.

Then `run(k)` finds the flows and returns their number, which is less than k if the problem has fewer integer flows, and zero if it has no optimal flow (it is infeasible or unbounded). Its queries are `flowNum()`, the number of flows of the last run, `totalCost(i)`, `flow(i, arc)` and `flowMap(i, map)`, for the i-th flow, where the first one is the flow 0. The constraints are of the GEQ form: by an arc of no cost and no upper bound into each node, an extra node supplies the demand that the flow leaves unmet. Both the bounds and the supplies must be integer, also with a floating-point `V` (whose values are then integer ones), since the flows differ from each other by whole units. Also, an infinite upper bound is allowed only if no cycle of negative cost is made of arcs of infinite upper bound, or the problem has no optimal flow. We remark that the comparisons of the search for the second best flows are exact on such data.

14.4 The k best extreme flows: KBestExtremeFlows

14.4.1 Vertices, bases and the ranking of Murty

Next, the class `KBestExtremeFlows` finds the k vertices of least cost of the polyhedron of the flows of (13.1), in the order of their costs: first an optimal vertex, then the other optimal ones, if any, and then those whose cost exceeds the optimum by at most a given gap. As the header remarks, these are the flows that a Lagrangian or a Benders decomposition may want several of, since each vertex gives a different (sub)gradient, while a flow inside a face gives a convex combination of those of its vertices. A flow is a vertex if and only if the arcs strictly within their bounds (the *free* arcs) form a forest; a *basis* of a vertex is a spanning tree that contains its free arcs, the other arcs being at a bound. A vertex is *degenerate* when it has more than one basis, i.e., when its free arcs do not span the nodes.

Its algorithm is the ranking of the extreme points of Murty 1968, which rests on the fact that the next vertex in the order of cost is adjacent to one of the vertices already given. Adjacency means here one pivot of the network simplex method away from one of their bases, i.e., along the cycle that a nonbasic arc closes with the tree, saturated. Candidates wait in a priority queue by their costs, and the bases of a degenerate vertex, which the pivots of zero amount reach, are all expanded.

Algorithm 14.4 (Ranking of the vertices).

```
1  x_1 := an optimal flow, made a vertex; T_1 := a basis of it
2  Q := { (cost(x_1), the basis T_1) }
3  while Q is not empty and fewer than k vertices are given:
4      take from Q the candidate of least cost; stop if beyond the gap
5      make its basis (one pivot from the expanded basis it came from)
6      skip it if the basis was already expanded
7      if the basis limit is reached: exact := false; stop
8      give its vertex if it is new
9      for each nonbasic residual arc r of the basis:
10         c := the cost after the pivot of r (if its cycle is bounded)
11         put (c, r) in Q, unless c exceeds the gap
12 if not exact: sort the vertices given by cost
```

A candidate is kept in the queue only as its cost, with the index of the expanded basis and of the arc of its pivot; the basis it leads to is made only when it leaves the queue. Thus, the memory is $O(n + m)$ for each basis expanded. Each pivot takes $O(n)$ time, and each basis $O(nm)$ for all its pivots (or $O(n(n + m))$, as the documentation of `basisLimit()` puts it).

As in `KBestFlows`, the first vertex is the optimal flow that `NetworkSimplex` finds, or the one given by `initFlow()`, which need not be a vertex. It is made a vertex by `purify()`: while the free arcs close a cycle, the cycle is saturated in a direction in which it is bounded, the direction in which the cost does not grow being tried first. At an optimal flow such a cycle has a reduced cost of zero, and the cost of the flow does not change.

14.4.2 Degeneracy and the limits on the bases

A degenerate vertex may have very many bases, and network problems are often highly degenerate. For instance, a vertex of a shortest path problem, i.e., a path whose other basic arcs carry no flow, has as many bases as there are spanning trees containing the path, and the ranking expands all those it meets. Two limits bound this work, and `exact()` returns whether a run stayed within them.

- `basisLimit(limit)` sets the largest number of bases a run expands, by default 1000. When it is reached the run stops; the flows given are still distinct vertices in the order of their costs, but they may not be the k best ones.
- `vertexBasisLimit(limit)` sets the largest number of bases of one vertex that a run expands, by default no limit (a value of 0). A basis counts only if one of its pivots leads to another vertex, so that the bases whose pivots are all degenerate do not take the place of one that is not. With a positive limit, the bases of a vertex beyond it are not expanded, and the degenerate pivots of its last basis are not taken. The flows given are then distinct vertices in the order of their costs, the first one optimal and the others adjacent to the bases expanded, but they may not be the k best ones.

When a limit has acted, a vertex reached only through a basis left out may have come after one of larger cost; hence, the flows are sorted by their costs at the end (step 12). Thus, `exact()` returns `false` when the last run

stopped at `basisLimit()`, or left out a basis or a pivot for `vertexBasisLimit()`.

14.4.3 The cycles of least reduced cost

With `method(CYCLES)` the class enumerates no basis, and thus avoids the expansion of the many bases of a degenerate vertex. Starting from the optimal vertex and from potentials optimal for it, it first finds the other optimal vertices. A residual arc of zero reduced cost closes a cycle of zero reduced cost if and only if its ends are in the same strongly connected component of the open residual arcs of zero reduced cost, and the cycle is then found from its head along those arcs. Each such cycle, saturated and made a vertex, gives an optimal vertex, from which the same is done again. Then, from each optimal vertex and for each arc at one of its bounds, the cycle of least reduced cost through the arc, saturated and made a vertex, gives a vertex of larger cost. Only the arcs at a bound are tried, since the free arcs of a vertex make no cycle; they are tried in the order of their reduced costs, which bound from below those of their cycles, until one reaches the reduced cost of the last cycle kept.

Algorithm 14.5 (Vertices by the cycles of least reduced cost).

```
1  x := an optimal vertex; pi := optimal potentials (Bellman-Ford)
2  O := (x); for each vertex y of O, in turn, while fewer than k:
3      for each arc r at a bound of y with reduced cost 0 whose ends lie
       in one strongly connected component of the zero arcs:
4          close r by a path of zero reduced cost (Dijkstra on zero arcs)
5          saturate the cycle, make it a vertex z; if new, give z, add to O
6  K := empty (kept candidates, at most 2 (k - given))
7  for each vertex y of O, for each arc r at a bound of y,
       by increasing reduced cost while it is below the worst kept:
8      c := rc(r) + shortest path from head(r) to tail(r) (Dijkstra,
            stopped at the end or at the bound)
9      if c > 0: saturate the cycle, make it a vertex z; keep (c, z)
10 give the vertices of K in the order of their costs, up to k
```

With this method, too, the flows are distinct vertices in the order of their costs, the optimal ones first, but those of larger cost are only some of the vertices one cycle away from an optimal one; hence, `exact()` is always `false` with this method. According to the header, each optimal vertex costs at most $O(nm \log n)$ time, and the method needs $O(km + pn)$ memory for the p trees of shortest paths made.

abLEMON. Since version 1.8.1 the method `CYCLES` tries only the arcs at a bound of a vertex, finds the cycles of zero reduced cost through the strongly connected components of the arcs of zero reduced cost, and tries the other cycles by increasing reduced cost of their arc, each path being looked for by a Dijkstra's algorithm stopped at its end or at the bound. The vertices of larger cost kept are those of the cycles of least reduced cost, while in version 1.7.0 they were those of least cost.

14.4.4 The interface

`KBestExtremeFlows<GR, V, C>` has the template parameters, the constructor, the setters `lowerMap()`, `upperMap()`, `costMap()`, `supplyMap()` and `tolerances()`, and the queries `flowTolerance()`, `costTolerance()`, `flowNum()`, `totalCost(i)`, `flow(i, arc)` and `flowMap(i, map)` of `KBestFlows`; its `initFlow(map)` takes an optimal flow that need not be a vertex. Its tolerances are given to the `NetworkSimplex` that finds the optimal flow, and they are also used in its own comparisons. A flow within the tolerance on the flows of a bound is at the bound, and a reduced cost within the tolerance on the costs of zero is zero. The method is set by `method(m)`, with `RANKING` (the default) or `CYCLES`, and the limits by `basisLimit()` and `vertexBasisLimit()`, and `run(k, gap)` finds at most k vertices, and a negative `gap`, the default, sets no limit on the excess of their cost over the optimum. It returns the number of vertices found, which is less than k if the problem has fewer vertices within the gap, and zero if it has no optimal flow; `exact()` returns whether the run was exact. Its constraints are of the GEQ form, with an extra node as in `KBestFlows`.

Our last program solves a problem of four nodes and five arcs, small enough to enumerate all its integer flows. It has 5 integer flows, of costs 5, 6, 6, 7 and 8, of which 4 are vertices (the flow of cost 7, the midpoint of the vertices of costs 6 and 8, has four free arcs, which make a cycle). `KBestFlows` gives the 5 costs in order, and `RANKING` gives the 4 costs of the vertices with `exact()` true. Then `CYCLES` gives the optimal vertex and two more, and finally the program checks the potentials of section 14.5.

```
// The k best flows of sections 14.3 to 14.5 on a network small enough to
// enumerate all its integer flows: KBestFlows must give the k cheapest of
// them in order, KBestExtremeFlows the k cheapest vertices (the flows
// whose arcs strictly within their bounds make no cycle), by the ranking
// and by the cycles of least reduced cost, and runPotentials() optimal
// potentials of the optimal flow, by both of its methods.

#include <algorithm>
#include <iostream>
#include <vector>
#include <lemon/list_graph.h>
#include <lemon/k_best_flows.h>
#include <lemon/k_best_extreme_flows.h>

using namespace lemon;

typedef ListDigraph::ArcMap< int > ArcMap;
typedef ListDigraph::NodeMap< int > NodeMap;

const int M = 5;
const int SRC[ M ] = { 0 , 0 , 1 , 1 , 2 } , TGT[ M ] = { 1 , 2 , 2 , 3 , 3 };
const int CAP[ M ] = { 2 , 2 , 1 , 2 , 2 } , COST[ M ] = { 1 , 2 , 0 , 3 , 1 };
const int SUP[ 4 ] = { 2 , 0 , 0 , -2 };

// Whether the arcs strictly within their bounds make no cycle
bool vertex( const std::vector< int > & f )
{
 int p[ 4 ] = { 0 , 1 , 2 , 3 };
 for( int i = 0 ; i != M ; ++i ) {
  if( f[ i ] == 0 || f[ i ] == CAP[ i ] )
   continue;
  int a = SRC[ i ] , b = TGT[ i ];
  while( p[ a ] != a )
   a = p[ a ];
  while( p[ b ] != b )
   b = p[ b ];
  if( a == b )
   return( false );
  p[ a ] = b;
  }
 return( true );
 }

int main()
{
 ListDigraph g;
 std::vector< ListDigraph::Node > node;
 for( int u = 0 ; u != 4 ; ++u )
  node.push_back( g.addNode() );
 ArcMap upper( g ) , cost( g );
 NodeMap supply( g );
 std::vector< ListDigraph::Arc > arc;
 for( int i = 0 ; i != M ; ++i ) {
  arc.push_back( g.addArc( node[ SRC[ i ] ] , node[ TGT[ i ] ] ) );
  upper[ arc[ i ] ] = CAP[ i ];
  cost[ arc[ i ] ] = COST[ i ];
  }
 for( int u = 0 ; u != 4 ; ++u )
  supply[ node[ u ] ] = SUP[ u ];
 int errors = 0;

 // all the integer flows, and the costs of those and of the vertices
 std::vector< int > all , vertices;
 std::vector< int > f( M , 0 );
 for( int code = 0 ; code != 3 * 3 * 2 * 3 * 3 ; ++code ) {
```

```
 int c = code;
 for( int i = 0 ; i != M ; ++i ) {
  f[ i ] = c % ( CAP[ i ] + 1 );
  c /= CAP[ i ] + 1;
  }
 int net[ 4 ] = { 0 , 0 , 0 , 0 } , total = 0;
 for( int i = 0 ; i != M ; ++i ) {
  net[ SRC[ i ] ] += f[ i ];
  net[ TGT[ i ] ] -= f[ i ];
  total += COST[ i ] * f[ i ];
  }
 if( net[ 0 ] != SUP[ 0 ] || net[ 1 ] != SUP[ 1 ] || net[ 2 ] != SUP[ 2 ] ||
     net[ 3 ] != SUP[ 3 ] )
  continue;
 all.push_back( total );
 if( vertex( f ) )
  vertices.push_back( total );
 }
std::sort( all.begin() , all.end() );
std::sort( vertices.begin() , vertices.end() );

// the k best integer flows
KBestFlows< ListDigraph > kbf( g );
kbf.upperMap( upper ).costMap( cost ).supplyMap( supply );
const int k = kbf.run( 100 );
std::cout << k << " integer flows (" << all.size() << " enumerated):";
for( int i = 0 ; i != k ; ++i )
 std::cout << ' ' << kbf.totalCost( i );
std::cout << std::endl;
if( k != int( all.size() ) )
 ++errors;
for( int i = 0 ; i != k && i != int( all.size() ) ; ++i )
 if( kbf.totalCost( i ) != all[ i ] )
  ++errors;

// the k best vertices, by the ranking of Murty and by the cycles
KBestExtremeFlows< ListDigraph > kef( g );
kef.upperMap( upper ).costMap( cost ).supplyMap( supply );
const int kv = kef.run( 100 );
std::cout << kv << " vertices (" << vertices.size() << " enumerated):";
for( int i = 0 ; i != kv ; ++i )
 std::cout << ' ' << kef.totalCost( i );
std::cout << std::endl;
if( ! kef.exact() || kv != int( vertices.size() ) )
 ++errors;
for( int i = 0 ; i != kv && i != int( vertices.size() ) ; ++i )
 if( kef.totalCost( i ) != vertices[ i ] )
  ++errors;
kef.method( KBestExtremeFlows< ListDigraph >::CYCLES );
const int kc = kef.run( 3 );
std::cout << "by the cycles:";
for( int i = 0 ; i != kc ; ++i ) {
 std::cout << ' ' << kef.totalCost( i );
 if( kef.totalCost( i ) < vertices[ 0 ] ||
     ( i > 0 && kef.totalCost( i ) < kef.totalCost( i - 1 ) ) )
  ++errors;
 }
std::cout << std::endl;
if( kc == 0 || kef.totalCost( 0 ) != vertices[ 0 ] || kef.exact() )
 ++errors;

// optimal potentials: every residual arc of the optimal flow has a
// nonnegative reduced cost, and no node a positive potential (GEQ)
for( int method = 0 ; method != 2 ; ++method ) {
 kef.potentialMethod( method == 0
```

```
                        ? KBestExtremeFlows< ListDigraph >::PATHS
                        : KBestExtremeFlows< ListDigraph >::PIVOTS );
  const int kp = kef.runPotentials( 10 );
  std::cout << kp << " optimal potentials by "
            << ( method == 0 ? "PATHS" : "PIVOTS" ) << std::endl;
  if( kp == 0 )
   ++errors;
  for( int j = 0 ; j != kp ; ++j ) {
   for( int i = 0 ; i != M ; ++i ) {
    const int x = kbf.flow( 0 , arc[ i ] );
    const int rc = COST[ i ] + kef.potential( j , node[ SRC[ i ] ] ) -
                   kef.potential( j , node[ TGT[ i ] ] );
    if( ( x < CAP[ i ] && rc < 0 ) || ( x > 0 && rc > 0 ) )
     ++errors;
    }
   for( int u = 0 ; u != 4 ; ++u )
    if( kef.potential( j , node[ u ] ) > 0 )
     ++errors;
   }
  }
 return( errors );
 }
```

14.5 Alternative optimal potentials

By Theorem 13.1, potentials are optimal if and only if, together with an optimal flow f, they satisfy (13.4); for the GEQ form with the extra node, this means that the reduced cost of each open residual arc of f is nonnegative. These potentials form a polyhedron (up to a constant, which the class fixes by making the potential of the extra node zero), which has, in general, many vertices: an optimal flow that is degenerate in the primal has several optimal potentials. A program may want several of them, e.g., distinct dual solutions for a sensitivity analysis, or different cuts in a decomposition (somewhat as for the extreme flows of section 14.4). `runPotentials(k )` finds at most k distinct vertices of that polyhedron, and returns their number (zero if the problem has no optimal flow); `potentialNum()` and `potential(i, node)` read them, and the first ones are the potentials 0. Two potentials that differ by a constant are the same, and two within the tolerance on the costs of each other are the same too.

One sets the method by `potentialMethod(m)`. With `PATHS`, the default, the potentials are computed from potentials π optimal for the optimal vertex of section 14.4. They are $\pi + d_s$ and $\pi - d'_s$, for each node s in turn, where d_s are the distances from s and d'_s the distances to s in the residual network of the flow on the reduced costs (one Dijkstra's algorithm each), the nodes not reached taking the largest distance. Each of them is the vertex of the dual polyhedron given by the tree of the shortest paths, whose arcs have zero reduced cost under it.

With `PIVOTS` the potentials are the vertices reached from that of the optimal vertex by the pivots of the dual simplex method, breadth first. In a tree of arcs of zero reduced cost, an arc whose opposite residual arc is closed (an arc at a bound, i.e., a degenerate arc of the primal) leaves. Then the side of its tail is raised until an arc into it reaches a zero reduced cost and enters, and the potentials change if the step is not zero.

Algorithm 14.6 (Optimal potentials by dual pivots).

```
1  pi := optimal potentials for the optimal vertex (Bellman-Ford)
2  while the tree of zero reduced cost from the root misses some nodes:
       raise the nodes it misses by the least reduced cost of an arc
       entering them
3  P := (pi); for each pi' of P, in turn, while fewer than k:
4      build the tree of zero reduced cost under pi'
5      for each tree arc r whose opposite residual arc is closed:
6          raise the side of the tail of r by the least reduced cost of
           an open arc entering it
7          if the step is positive: add the new potentials to P if new
```

Step 2 first makes π a vertex: the arcs from the extra node to the nodes are always open, so that one of them enters any set of missed nodes. According to the header, each vertex costs $O(nm)$ time with this method.

abLEMON. `runPotentials()` appeared in version 1.8.0 with the method `PATHS`, and `potentialMethod(PIVOTS)` in version 1.9.0.

14.6 Exercises

Exercise 14.1. Prove the identity of Theorem 14.1 by expanding the capacity of the cut $\{\sigma\} \cup N'$, and use it to compute by hand the largest violation in the problem of Figure 14.1. (The program `manual/examples/14-certificates.cc` checks the violations of the sets it receives.)

Exercise 14.2. Enumerate all the subsets of the nodes of a random infeasible problem with eight nodes, compute their violations by (14.1), and compare the k largest with `infeasibleCuts(k)` of the four classes. (Solved by the battery `test/min_cost_flow_battery_test.cc`, which checks the k best sets against the enumeration of all the subsets.)

Exercise 14.3. Show that the cycle of least total cost among the cycles of negative cost decides whether a digraph has a Hamiltonian cycle (give each arc the cost -1 and look for a cycle of cost $-n$), and explain why Algorithm 14.2 uses the mean cost instead. (The certificates are checked by `checkCertificates()` in `test/min_cost_flow_test.cc`.)

Exercise 14.4. Change the capacities of the example of section 14.4.4 and check, by the enumeration of the program, that `KBestFlows` still gives all the integer flows in order. (Solved by `checkAgainstEnumeration()` in `test/k_best_flows_test.cc`.)

Exercise 14.5. Build a shortest path problem on a complete digraph of six nodes, as a minimum cost flow problem with one unit of supply, and run `KBestExtremeFlows` with `basisLimit(10)`; observe `exact()`, and then raise the limit or set `vertexBasisLimit(1)`. (The extreme flows are checked against an enumeration by `checkExtremeAgainstEnumeration()`, and the limit cases by `checkLimitCases()`, in `test/k_best_flows_test.cc`.)

Exercise 14.6. Verify that the potentials of `runPotentials()` satisfy (13.4) with any optimal flow, not only with the one they were computed from, and that both methods give vertices of the polyhedron of the optimal potentials. (Solved by `checkPotentials()` in `test/k_best_flows_test.cc`, and in part by `manual/examples/14-k-best.cc`.)

15. Minimum cuts

We are concerned in this chapter with the cuts of minimum capacity of a graph taken as a whole, i.e., with the nonempty proper subsets of the nodes whose boundary has the least capacity, when no source and no sink are fixed in advance. Between two given nodes, a minimum cut is the dual of a maximum flow, and we refer to chapter 12 for it. abLEMON has three classes for these problems, all in the group Minimum Cut Algorithms: `HaoOrlin` (`lemon/hao_orlin.h`) for a digraph, `NagamochiIbaraki` (`lemon/nagamochi_ibaraki.h`) for an undirected graph, and `GomoryHu` (`lemon/gomory_hu.h`), which builds a tree from which the minimum cut between any two nodes of an undirected graph can be read. Section 15.1 states the problems and the two facts on which the algorithms rest, sections 15.2 to 15.4 describe the algorithms and their classes, and section 15.5 compares them.

15.1 The problems

Let $D = (V, A)$ be a digraph with $n = |V| \geq 2$ nodes, $m = |A|$ arcs and nonnegative capacities $c : A \to \mathbb{R}_+$. For $X \subseteq V$ we write $\delta^+(X)$ for the arcs that leave X and $c(\delta^+(X))$ for the sum of their capacities. A *cut* is a set X with $\emptyset \neq X \subsetneq V$, and the *minimum cut* of D is the value

$$\lambda(D) \;=\; \min\{\, c(\delta^+(X)) \;:\; \emptyset \neq X \subsetneq V \,\} \tag{15.1}$$

together with a set that attains it. We call such a set a minimum cut as well. For two distinct nodes s and t, the minimum s-t cut $\lambda(s,t)$ is the same minimum restricted to the sets with $s \in X$ and $t \notin X$; by the max-flow min-cut theorem it equals the value of a maximum flow from s to t (cf. Ahuja, Magnanti and Orlin 1993, chapter 6). For an undirected graph $G = (V, E)$ with capacities on the edges, $\delta(X)$ is the set of the edges with one end in X and the other outside, and $\lambda(G)$ is defined as in (15.1). In this case the cut function is symmetric, i.e., $c(\delta(X)) = c(\delta(V \setminus X))$. With unit capacities $\lambda(G)$ is the edge connectivity of G, i.e., the least number of edges whose removal disconnects it; the documentation of the library mentions this use first, as a test of the "reliability" of a network.

Our first fact shows that $n-1$ minimum s-t cuts in each direction are enough to find $\lambda(D)$, and it is the starting point of the algorithm of Hao and Orlin.

Theorem 15.1. *Let $r \in V$ be any node. Then*

$$\lambda(D) \;=\; \min\Big\{\, \min_{t \neq r} \lambda(r,t)\,,\; \min_{t \neq r} \lambda(t,r) \,\Big\}. \tag{15.2}$$

Proof. A cut X that contains r separates r from any node $t \notin X$, and hence $c(\delta^+(X)) \geq \lambda(r,t)$; a cut that does not contain r separates any node $t \in X$ from r. Conversely, an s-t cut is a cut. □

For an undirected graph one has $\lambda(r,t) = \lambda(t,r)$, and the second half of (15.2) can be dropped; we use this in section 15.2.2. The second fact, due to Gomory and Hu (1961), is that the $n(n-1)/2$ values $\lambda(s,t)$ of an undirected graph take at most $n-1$ distinct values, and that one tree stores them together with a minimum cut for each pair.

Theorem 15.2. *Let $G = (V, E)$ be an undirected graph with nonnegative capacities. There is a tree $T = (V, F)$, with a weight $w(f)$ on each edge $f \in F$, such that for any two distinct nodes s and t the value $\lambda(s,t)$ is the least weight $w(f)$ of an edge f on the path of T from s to t, and the two components of $T - f$ are the two sides of a minimum s-t cut of G.*

A proof is in Gomory and Hu (1961), and one that follows the construction of section 15.4 is in Gusfield (1990). Such a tree is called a *Gomory-Hu tree*, and it is rarely unique. We remark that its edges need not be edges of G, and that even when they are (as in Figure 15.2) their weights are usually not the capacities of G (which only bound them from below, cf. Theorem 15.2); also, $\lambda(G)$ is the weight of the lightest edge of T.

15.2 The Hao-Orlin algorithm

15.2.1 The method

Theorem 15.1 requires $2(n-1)$ maximum flows, each of which costs $O(n^2\sqrt{m})$ with the highest-label preflow push-relabel method of Goldberg and Tarjan (1988) (see chapter 12). Hao and Orlin (1994) observed that the minimum cuts from r to all the other nodes can be found in the time of one such computation. To this aim the sinks are taken one after the other, and a node that has been the sink becomes a source; the preflow and most distance labels of one computation then remain valid at the start of the next one. Indeed, the labels that would become wrong are those of the nodes from which the current sink can no longer be reached in the residual digraph. Hao and Orlin put these nodes aside in *dormant* sets, instead of relabelling them, and it brings the sets back (the last one first) when the current part of the digraph has been exhausted. At any moment the nodes are therefore split into the sources S, a stack of dormant sets, and the *awake* nodes W, among which lies the current sink t. Since the algorithm pushes flow only between awake nodes, the following invariant holds, and it gives the value of a cut when no awake node other than t has an excess.

Lemma 15.1. *During the algorithm, no arc of the residual digraph goes from $V \setminus W$ to W. Hence, when t is the only awake node with a positive excess, the excess of t equals $c(\delta^+(V \setminus W))$, and $V \setminus W$ is a minimum cut among those that contain S but not t.*

Proof. When a node becomes a source, the arcs out of it are saturated. A set becomes dormant only when its nodes have no residual arc into W, either because of a gap in the labels or because the single node that becomes dormant has none; also, no flow is ever pushed from an awake node to a node that is not awake, and hence no residual arc into W is created afterwards. Thus, the total excess of W is the flow that enters it, i.e., $c(\delta^+(V \setminus W))$. That no cut which separates S from t has a smaller capacity follows from the argument of the push-relabel method on the labels of the awake nodes (see Hao and Orlin 1994). □

We give below the phase that finds the cuts with r inside. For the cuts with r outside one runs the same algorithm on the reverse digraph (the pushes go backwards along the arcs), and the cut is then the awake set W, which contains the sink but not r.

Algorithm 15.1 (Hao-Orlin, the cuts that contain r).

```
 1  S <- {r}; lambda <- +inf; saturate every arc (r, v) with c(r, v) > 0
 2  label the other nodes by backward breadth-first searches: the first
    search makes the awake set W, every later one a dormant set; t <- a
    node of W of lowest label
 3  loop
 4    while a node v of W - {t} has e(v) > 0, take the one of highest label:
 5      push from v along the residual arcs (v, w), w in W, d(w) = d(v) - 1
 6      if e(v) > 0 still:
 7        if v is alone in its label: the nodes of W of label >= d(v)
          become a new dormant set (a gap)
 8        else if v has no residual arc into W: {v} becomes a dormant set
 9        else d(v) <- 1 + min{ d(w) : (v, w) residual, w in W }
10    if e(t) < lambda then lambda <- e(t); X <- V - W
11    S <- S + {t}; W <- W - {t}; saturate the residual arcs out of t
12    if W is empty: stop if no dormant set is left, else W <- the dormant
      set made last
13    t <- a node of W of lowest label
```

Step 4 is the highest-label rule of the generic method. Steps 7 to 9 replace the relabelling of the generic method (in `findMinCutOut()` of `lemon/hao_orlin.h`, the buckets are lists of nodes of equal label, grouped by dormant set). Step 10 compares the new cut with the best one by an exact `<`, and step 11 turns the old sink into a source. According to the header, the running time is $O(n^2\sqrt{m})$ for the two phases with the highest-label rule, and of $O(nm)$ with unit capacities. Furthermore, Hao and Orlin (1994) give the bound $O(nm\log(n^2/m))$ for a version with dynamic trees, which is not the one implemented.

15.2.2 The class HaoOrlin

The class is declared as

```
template< typename GR , typename CAP = typename GR::template ArcMap< int > ,
          typename TOL = Tolerance< typename CAP::Value > >
class HaoOrlin;
```

where `GR` is the digraph type, `CAP` a readable arc map of capacities of any numeric type, and `TOL` the tolerance of the comparisons; there are no named template parameters, and no traits class. Its constructor `HaoOrlin(graph, capacity, tolerance = Tolerance())` stores references to the digraph and to the capacities, and `tolerance(tol)` and `tolerance()` set and read the tolerance afterwards. An execution starts with `init()`, which takes the first node of the digraph as r, or with `init(source)`, which takes the given one. Then `calculateOut()` runs the phase of Algorithm 15.1 (the cuts with r inside) and `calculateIn()` the other one (the cuts with r outside), in either order, and one may also call only one of them. Also, `run()` and `run(s)` call `init()` (respectively `init(s)`), `calculateOut()` and `calculateIn()`.

Note that `init()` sets the stored value to the largest value of the capacity type, and each phase replaces the stored cut only if it finds one of strictly smaller value. Hence, after `calculateOut()` the stored cut is a minimum one among the sets that contain r, and after both phases it is a minimum cut of the digraph. We read the results by `minCutValue()`, which returns the value, and by `minCutMap(cutMap)`, which also returns it. The latter writes into a writable `bool` node map the characteristic vector of the set X of (15.1), i.e., of the side whose leaving arcs have the least capacity. Of course, the digraph is assumed to have at least two nodes, since a cut is a nonempty proper subset of them.

In this class the tolerance decides which arcs exist for the algorithm and which nodes are active. An arc whose capacity (or residual capacity) is not positive for the tolerance is ignored by the initial searches and by the pushes, and an excess is used up when the residual capacity is not less than it; a node keeps being processed while its excess is nonzero for the tolerance. We recall that `Tolerance< CAP::Value >` is exact by default on integer types, while on floating-point ones it compares with the absolute epsilon of the library (i.e., $a < b$ is $a + \varepsilon < b$) (see chapter 3).

abLEMON. The constructor and `tolerance()` are as in LEMON 1.3.1. In abLEMON, however, the test that a node still has an excess goes through the tolerance (`nonZero()`) instead of a comparison with an exact zero; hence, on floating-point capacities a residue of rounding no longer keeps a node active.

On an undirected graph one may run `HaoOrlin` on the digraph `Undirector<GR>` of chapter 5, with the capacities of the original arcs. By Theorem 15.1 and the symmetry of the cut function, `init()` followed by `calculateOut()` is then enough (the test `test/hao_orlin_test.cc` runs both phases on an `Undirector`). In the example below the two phases are run one at a time on the digraph of Figure 15.1. Its minimum cut with node 0 inside is $\{0, 1, 2\}$, of value 5, whereas the minimum cut of the digraph is $X = \{3, 4, 5\}$, of value 4, which only the second phase finds; each value is checked against the enumeration of the $2^6 - 2$ cuts.

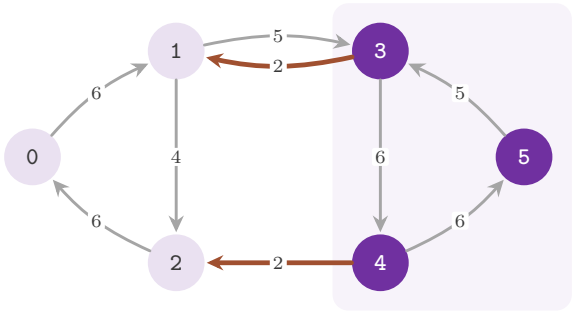


Figure 15.1. A digraph with the capacity of each arc on its plate; its minimum cut is $X = \{3, 4, 5\}$, whose two leaving arcs carry 4 units in all.

```
// The minimum cut of a digraph by HaoOrlin (section 15.2): the two phases
// run one at a time from a chosen node, then together by run(), and each
// value is checked against the cuts of all the subsets of the nodes.

#include <iostream>
```

```
#include <vector>
#include <lemon/smart_graph.h>
#include <lemon/hao_orlin.h>

using namespace lemon;

typedef SmartDigraph::ArcMap< int > CapMap;

// the capacity of the arcs that leave the set of nodes coded by mask
int outCapacity( const SmartDigraph & g , const CapMap & cap , unsigned mask )
{
 int sum = 0;
 for( SmartDigraph::ArcIt a( g ) ; a != INVALID ; ++a )
  if( ( mask >> g.id( g.source( a ) ) & 1 ) &&
      ! ( mask >> g.id( g.target( a ) ) & 1 ) )
   sum += cap[ a ];
 return( sum );
 }

int main()
{
 SmartDigraph g;
 std::vector< SmartDigraph::Node > v;
 for( int i = 0 ; i < 6 ; ++i )
  v.push_back( g.addNode() );
 CapMap cap( g );
 const int arcs[][ 3 ] = { { 0 , 1 , 6 } , { 1 , 2 , 4 } , { 2 , 0 , 6 } ,
                           { 1 , 3 , 5 } , { 4 , 2 , 2 } , { 3 , 4 , 6 } ,
                           { 4 , 5 , 6 } , { 5 , 3 , 5 } , { 3 , 1 , 2 } };
 for( const auto & a : arcs )
  cap[ g.addArc( v[ a[ 0 ] ] , v[ a[ 1 ] ] ) ] = a[ 2 ];

 // the minimum over the sets that contain node 0, and over all the sets
 const unsigned all = ( 1u << 6 ) - 1;
 int best_out = 1 << 30 , best = 1 << 30;
 for( unsigned mask = 1 ; mask < all ; ++mask ) {
  int c = outCapacity( g , cap , mask );
  if( c < best )
   best = c;
  if( ( mask & 1 ) && c < best_out )
   best_out = c;
  }

 HaoOrlin< SmartDigraph > ho( g , cap );
 SmartDigraph::NodeMap< bool > cut( g );

 ho.init( v[ 0 ] );
 ho.calculateOut();  // the sets that contain node 0
 ho.minCutMap( cut );
 int out = ho.minCutValue();
 std::cout << "with node 0 inside: " << out << std::endl;
 if( out != best_out || ! cut[ v[ 0 ] ] )
  return( 1 );

 ho.calculateIn();  // the sets that do not contain it
 int value = ho.minCutMap( cut );
 std::cout << "minimum cut: " << value << ", X =";
 unsigned mask = 0;
 for( SmartDigraph::NodeIt n( g ) ; n != INVALID ; ++n )
  if( cut[ n ] )
   mask |= 1u << g.id( n );
 for( int i = 0 ; i < 6 ; ++i )
  if( mask >> i & 1 )
   std::cout << ' ' << i;
 std::cout << std::endl;
```

```
if( value != best || outCapacity( g , cap , mask ) != value )
 return( 1 );
if( mask == 0 || mask == all )
 return( 1 );

ho.run();  // init(), calculateOut() and calculateIn() from the first node
return( ho.minCutValue() == best ? 0 : 1 );
}
```

15.3 The Nagamochi-Ibaraki algorithm

15.3.1 Maximum adjacency orderings

On an undirected graph, Nagamochi and Ibaraki (1992) find $\lambda(G)$ without any flow, by orderings of the nodes that certify a lower bound on $\lambda(u, v)$ for many pairs at once. An ordering $v_1, \ldots, v_n$ of the nodes is a *maximum adjacency ordering* if v_1 is arbitrary and each v_j, $j \geq 2$, maximizes the adjacency $c(v, \{v_1, \ldots, v_{j-1}\})$ among the nodes v not yet ordered (the adjacency being the capacity of the edges between v and the nodes already ordered). For an edge $e = v_i v_j$ with $i < j$ we write $q(e) = c(v_j, \{v_1, \ldots, v_i\})$; this is the adjacency of v_j just after v_i has been ordered, i.e., the key of v_j in the priority queue when e is scanned.

Lemma 15.2. *In a maximum adjacency ordering, every edge $e = v_i v_j$ with $i < j$ satisfies $\lambda(v_i, v_j) \geq q(e)$. In particular, the last edge scanned into v_n has $q(e) = c(\delta(v_n))$.*

A proof is in Nagamochi and Ibaraki (1992), and it is rather short. It follows that an edge with $q(e)$ not smaller than the value μ of a cut already found can be contracted: a cut that separates its ends has capacity at least $\lambda(v_i, v_j) \geq q(e) \geq \mu$, and it is not better than the one at hand. Each phase of the algorithm computes one ordering and takes as μ the least capacity of a prefix $\{v_1, \ldots, v_k\}$, $1 \leq k < n$, of the ordering (these are cuts, and their capacities are computed while the ordering is built). Then it contracts all the edges with $q(e) \geq \mu$. Since $\mu \leq c(\delta(\{v_1, \ldots, v_{n-1}\})) = c(\delta(v_n))$, by the second part of Lemma 15.2 at least one edge is contracted, and at most $n - 1$ phases are made.

Algorithm 15.2 (Nagamochi-Ibaraki).

```
1  merge the parallel edges; lambda <- min_v c(delta(v)); X <- that {v}
2  if lambda = 0: stop
3  repeat
4    build a maximum adjacency ordering v_1, ..., v_n' of the current
     nodes from the first one, with q(e) for every edge e
5    if it does not reach every node: X <- the nodes reached;
     lambda <- 0; stop
6    mu <- min{ c(delta({v_1, ..., v_k})) : 1 <= k < n' }
7    if mu < lambda: lambda <- mu; X <- the original nodes of v_1..v_k
8    contract every edge e with q(e) >= mu, merging parallel edges
9  until a single node is left
```

In the class, step 1 is `init()`. Steps 4 to 8 are one call of `processNextPhase()`. In `lemon/nagamochi_ibaraki.h` the keys of the ordering are kept in a heap that maximizes them, and the nodes merged into a node are kept in a linked list, which step 7 follows to write the original nodes. Step 5 detects a disconnected graph, whose cut of value 0 is the connected component of the first node. In abLEMON the comparisons of steps 1 and 7 go through the tolerance, while those of steps 6 and 8 are exact. A phase costs $O(m \log n)$ with the binary heap, which gives $O(nm \log n)$ in all; the header adds that a Fibonacci heap would give $O(nm + n^2 \log n)$, and a bucket heap $O(nm)$ with unit capacities.

15.3.2 The class NagamochiIbaraki

The class is declared as

```
template< typename GR , typename CM = typename GR::template EdgeMap< int > ,
          typename TR = NagamochiIbarakiDefaultTraits< GR , CM > >
class NagamochiIbaraki;
```

where `GR` is an undirected graph type, `CM` a readable edge map of capacities, and `TR` the traits class. Its default traits `NagamochiIbarakiDefaultTraits` define `Value`, `Graph`, `CapacityMap`, the cross reference `HeapCrossRef`

(i.e., `GR::NodeMap< int >`) and the heap `BinHeap< Value , HeapCrossRef , std::greater< Value > >`, a binary heap that maximizes the keys. We have two constructors: a first one, `NagamochiIbaraki(graph, capacity)`, takes the graph and the capacities; the second one, `NagamochiIbaraki(graph)`, is meant for the named template parameter `SetUnitCapacity`, whose `Create` type allocates a constant map of value 1 on the edges; e.g., `NagamochiIbaraki< G >::SetUnitCapacity::Create ni(g)` computes the edge connectivity of `g`.

Furthermore, the class declares the named parameters `SetHeap< H , CR >` and `SetStandardHeap< H , CR >` (the heap must maximize the keys), and the function `heap(hp, cr)`, which hands an external heap and its cross reference to the algorithm. However, in the present code the two named parameters do not compile: the function that allocates the structures passes the graph to the `createHeapCrossRef()` of their traits, which takes an `int`. Hence, the binary heap of the default traits is, for now, the only usable one. The test `test/nagamochi_ibaraki_test.cc` does not instantiate them.

In the execution, `init()` builds the internal copy of the graph (loops are skipped and parallel edges are merged) and makes step 1. Then `processNextPhase()` makes one phase and returns `true` when the algorithm has finished, `start()` calls it until then, and `run()` is `init()` followed by `start()`; `tolerance(tol)` and `tolerance()` set and read the `Tolerance< Value >` of the comparisons. As for the results, `minCutValue()` returns the value of the best cut found so far, which after `init()` is already the capacity of a cut (the lightest star of a node) and at the end is $\lambda(G)$. Similarly, `minCutMap(cutMap)` writes the characteristic vector of one side of that cut into a writable `bool` node map, and returns its value. Also, the header warns that the value type must hold the cut values of the graph, since their sums may otherwise overflow, and notes that the capacities are supposed to be integer. With floating-point capacities the result rests on the tolerance and on the exact comparisons of steps 6 and 8, and the tests of the library cover only integer capacities. In section 15.4.2 an example runs the algorithm phase by phase on the graph of Figure 15.2 (with three calls of `processNextPhase()` and the value 3).

abLEMON. `tolerance()` is new in abLEMON. LEMON 1.3.1 compared the cut values exactly, whereas abLEMON compares through the tolerance the stars of step 1, the value 0 that ends step 2, and the update of step 7.

15.4 Gomory-Hu trees

15.4.1 The construction

Gomory and Hu's original construction (Gomory and Hu 1961) computes $n-1$ maximum flows on graphs obtained by contracting parts of G. Gusfield (1990) showed that a tree with the properties of Theorem 15.2 can be obtained by $n-1$ maximum flows on the original graph, if the cuts are kept in a rooted tree that is rearranged after each flow. This second construction is the one of `lemon/gomory_hu.h`; there each node v other than the root r has a parent $p(v)$ and a weight $w(v)$, the weight of the edge $v\,p(v)$ of the tree.

Algorithm 15.3 (Gomory-Hu tree, the construction of Gusfield).

```
1  r <- the first node; p(v) <- r for every v != r
2  for each node v != r, in the order of the node iterator:
3    compute a minimum cut (X, V - X) between v and p(v), v in X, and its
     value f, by Preflow on G
4    w(v) <- f
5    for each u != v with u in X and p(u) = p(v): p(u) <- v
6    if p(p(v)) is defined and lies in X:
7      u <- p(v); p(v) <- p(u); p(u) <- v; w(v) <- w(u); w(u) <- f
8  number the nodes so that every node comes after its parent
```

Step 3 is a call of `Preflow::runMinCut()` with the tolerance of the class. Step 7 makes v the parent of its former parent when the cut puts its grandparent on its side, and step 8 computes the numbering by which the queries climb the two paths of the tree towards their common ancestor. In all, $n-1$ maximum flows are computed by the preflow algorithm of chapter 12, and the header states an overall bound of $O(n^3\sqrt{m})$.

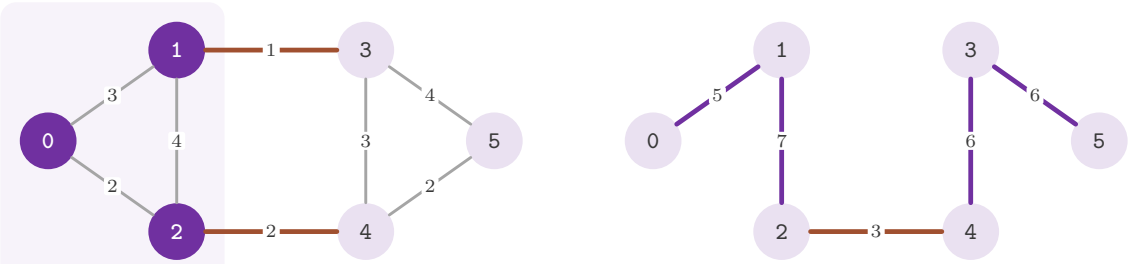
15.4.2 The class GomoryHu

The class is declared as `template< typename GR , typename CAP = typename GR::template EdgeMap< int > > class GomoryHu`, where `GR` is an undirected graph type and `CAP` a readable edge map of capacities (the constructor checks the `ReadMap` concept on it). Its public types are `Graph`, `Capacity` and `Value`, and it has no traits class. Its constructor `GomoryHu(graph, capacity)` stores the references, `tolerance(tol)` and `toleranc`

`e()` set and read the `Tolerance< Value >` given to the maximum flows, and `run()` executes Algorithm 15.3 (there is no separate `init()`). One reads the tree by `predNode(v)`, the parent of v (`INVALID` for the root), and by `predValue(v)`, the weight of the edge between v and its parent (undefined for the root, where it is in fact the largest value of `Value`). Note that the documentation of `rootDist(v)` calls it the distance of v from the root, but the function returns the number given to v by step 8: 0 for the root, and for any other node a number larger than that of its parent, distinct for distinct nodes.

A minimum cut between two nodes is read by `minCutValue(s, t)`, which climbs the two paths of the tree to their common ancestor and returns the least weight met, in $O(n)$ time. In the same way `minCutMap(s, t, cut Map)` finds the lightest edge of the path, writes `true` on the side of s and `false` on the other side into a readable and writable `bool` node map, and returns the value; for $s = t$ both functions return the largest value of `Value`. We can also list a cut by two iterators. `MinCutNodeIt(gomory, s, t, side = true)` lists the nodes of the side of s (or those of the other side if `side` is `false`), and `MinCutEdgeIt(gomory, s, t, side = true)` lists the edges of the cut, each once, as the arc that leaves the side of s (or enters it if `side` is `false`); the latter converts to both `Arc` and `Edge`. Each constructor calls `minCutMap()` on a node map that the iterator holds, at a cost of $O(n)$, and the postfix increments return a `Node` (respectively an `Arc`) instead of an iterator. Since a minimum cut need not be unique, `MinCutNodeIt(gomory, s, t, true)` and `MinCutNodeIt(gomory, t, s, false)` may list different nodes, whereas `MinCutNodeIt(gomory, s, t, true)` and `MinCutNodeIt(gomory, s, t, false)` together list each node once.

abLEMON. `tolerance()` is new in abLEMON. LEMON 1.3.1 gave its maximum flows the default tolerance of the value type, while abLEMON gives them the one set on the class.



G, $\lambda(G) = 3$ *Gomory-Hu tree T*

Figure 15.2. Left, an undirected graph G and its minimum cut $X = \{0, 1, 2\}$, of value 3, with the two cut edges; right, the Gomory-Hu tree computed by `GomoryHu`, whose lightest edge (between 2 and 4) gives the same cut.

Our example below computes the minimum cut of the graph of Figure 15.2 by `NagamochiIbaraki`, one phase at a time, and then its Gomory-Hu tree. It checks the value of the cut against the enumeration of all the cuts; for each of the 15 pairs of nodes, it also checks the value read from the tree against a maximum flow, the sides given by `minCutMap()`, and the sum of the capacities listed by `MinCutEdgeIt`. On this graph the tree is the path $0, 1, 2, 4, 3, 5$ with the weights $5, 7, 3, 6, 6$; hence, e.g., $\lambda(0, 5) = 3$ and $\lambda(1, 2) = 7$.

```
// The minimum cut of an undirected graph by NagamochiIbaraki (section 15.3)
// and the minimum cuts of all the pairs of nodes by the Gomory-Hu tree
// (section 15.4), each checked against the cuts of all the subsets of the
// nodes and against a maximum flow.

#include <iostream>
#include <vector>
#include <lemon/smart_graph.h>
#include <lemon/nagamochi_ibaraki.h>
#include <lemon/gomory_hu.h>
#include <lemon/preflow.h>

using namespace lemon;

typedef SmartGraph::EdgeMap< int > CapMap;

// the capacity of the edges between the set coded by mask and the rest
int cutCapacity( const SmartGraph & g , const CapMap & cap , unsigned mask )
{
```

```
 int sum = 0;
 for( SmartGraph::EdgeIt e( g ) ; e != INVALID ; ++e )
  if( ( mask >> g.id( g.u( e ) ) & 1 ) != ( mask >> g.id( g.v( e ) ) & 1 ) )
   sum += cap[ e ];
 return( sum );
 }

int main()
{
 const int n = 6;
 SmartGraph g;
 std::vector< SmartGraph::Node > v;
 for( int i = 0 ; i < n ; ++i )
  v.push_back( g.addNode() );
 CapMap cap( g );
 const int edges[][ 3 ] = {
  { 0 , 1 , 3 } , { 0 , 2 , 2 } , { 1 , 2 , 4 } , { 1 , 3 , 1 } ,
  { 2 , 4 , 2 } , { 3 , 4 , 3 } , { 3 , 5 , 4 } , { 4 , 5 , 2 }
  };
 for( const auto & e : edges )
  cap[ g.addEdge( v[ e[ 0 ] ] , v[ e[ 1 ] ] ) ] = e[ 2 ];

 // the global minimum cut by enumeration
 const unsigned all = ( 1u << n ) - 1;
 int best = 1 << 30;
 for( unsigned mask = 1 ; mask < all ; ++mask ) {
  int c = cutCapacity( g , cap , mask );
  if( c < best )
   best = c;
  }

 NagamochiIbaraki< SmartGraph > ni( g , cap );
 ni.init();
 int phases = 0;
 while( ! ni.processNextPhase() )  // or ni.start(), or ni.run()
  ++phases;
 SmartGraph::NodeMap< bool > cut( g );
 int value = ni.minCutMap( cut );
 unsigned mask = 0;
 for( SmartGraph::NodeIt u( g ) ; u != INVALID ; ++u )
  if( cut[ u ] )
   mask |= 1u << g.id( u );
 std::cout << "minimum cut: " << value << " after " << phases + 1 << " phases"
           << std::endl;
 if( value != best || cutCapacity( g , cap , mask ) != value )
  return( 1 );

 // the Gomory-Hu tree: its edges, then the cut of every pair of nodes
 GomoryHu< SmartGraph > gh( g , cap );
 gh.run();
 for( SmartGraph::NodeIt u( g ) ; u != INVALID ; ++u )
  if( gh.predNode( u ) != INVALID )
   std::cout << "tree edge " << g.id( u ) << " - " << g.id( gh.predNode( u ) )
             << ", weight " << gh.predValue( u ) << std::endl;
 for( int s = 0 ; s < n ; ++s )
  for( int t = s + 1 ; t < n ; ++t ) {
   Preflow< SmartGraph , CapMap > pf( g , cap , v[ s ] , v[ t ] );
   pf.runMinCut();
   int lambda = gh.minCutValue( v[ s ] , v[ t ] );
   gh.minCutMap( v[ s ] , v[ t ] , cut );
   int sum = 0;
   for( GomoryHu< SmartGraph >::MinCutEdgeIt e( gh , v[ s ] , v[ t ] ) ;
        e != INVALID ; ++e )
    sum += cap[ e ];
   if( lambda != pf.flowValue() || sum != lambda || ! cut[ v[ s ] ] ||
```

```
      cut[ v[ t ] ] )
    return( 1 );
   }
 return( 0 );
}
```

15.5 Choosing an algorithm

For a digraph `HaoOrlin` is the only class of the chapter, and it is arguably also the natural choice when one needs the best cut on a given side of a node (the stored cut after `calculateOut()`). For the minimum cut of an undirected graph, `NagamochiIbaraki` uses no flow, and its phases may be far fewer than $n-1$, since all the edges with $q(e) \geq \mu$ are contracted at once. On the other hand, `HaoOrlin` with the first phase only, on the `Undirector`, has the bound of a single maximum flow. Which of the two is faster typically depends on the instance, and neither the headers nor the works cited here give a rule that settles it in general. `GomoryHu` costs $n-1$ maximum flows; it pays off when the cuts of "many" pairs of nodes are needed, since each one is then read from the tree in $O(n)$ time, whereas for the cut of one pair (or of a "few" pairs) a maximum flow of chapter 12 is cheaper.

15.6 Exercises

Exercise 15.1. Prove Theorem 15.1, and check it on the digraph of Figure 15.1 by computing $\lambda(0,t)$ and $\lambda(t,0)$ with `Preflow` for every $t \neq 0$ and comparing the least of them with `HaoOrlin::minCutValue()` (the example `manual/examples/15-hao-orlin.cc` gives the value 4).

Exercise 15.2. Show that, on the digraph `Undirector<GR>` of an undirected graph, the value stored after `calculateOut()` is already $\lambda(G)$, and verify it on the graph of `test/hao_orlin_test.cc`, whose test runs both phases.

Exercise 15.3. Compute the edge connectivity of the graph of Figure 15.2 with `NagamochiIbaraki< SmartGraph >::SetUnitCapacity::Create`, and check the cut it gives by counting its edges (the last block of `test/nagamochi_ibaraki_test.cc` does the same on another graph).

Exercise 15.4. Print the value `minCutValue()` of `NagamochiIbaraki` on the graph of Figure 15.2 after `init()` and after each call of `processNextPhase()`. Explain why the sequence is nonincreasing, and why its first value is the lightest star of a node (5, at node 0).

Exercise 15.5. Using only `predNode()` and `predValue()` of `GomoryHu`, count the distinct values among the $\lambda(s,t)$ of a graph, and prove that there are at most $n-1$ of them; check the values against `minCutValue(s, t)` for each pair, as `test/gomory_hu_test.cc` does against `Preflow`.

Exercise 15.6. Show that the sum of the capacities listed by `MinCutEdgeIt(gomory, s, t)` equals `minCutValue(s, t)`, and that the nodes listed by `MinCutNodeIt` with `side` true and false partition V (both facts are checked by `test/gomory_hu_test.cc`, the first also by `manual/examples/15-undirected-cuts.cc`).

16. Minimum mean cycles

We are concerned in this chapter with the directed cycles of least mean cost of a digraph, i.e., with the cycles whose cost divided by their number of arcs is minimum. abLEMON has three algorithms for this problem, in the group Minimum Mean Cycle Algorithms: Karp's algorithm (`KarpMmc`, in `lemon/karp_mmc.h`), its variant with early termination by Hartmann and Orlin (`HartmannOrlinMmc`, in `lemon/hartmann_orlin_mmc.h`), and Howard's policy iteration (`HowardMmc`, in `lemon/howard_mmc.h`). All three have the same interface, stated as the concept `concepts::MinMeanCycle` of `lemon/concepts/min_mean_cycle.h`. Inside the library, the minimum mean cycle is used by the cycle-canceling algorithm of chapter 13 and by the certificates of unboundedness of chapter 14. Section 16.1 states the problem and its optimality conditions, sections 16.2 to 16.4 describe the three algorithms, and section 16.5 describes the interface they share.

16.1 The problem

Let $D = (V, A)$ be a digraph with n nodes, m arcs and a cost $c(a) \in \mathbb{R}$ on each arc a. For a directed cycle C, with $|C|$ arcs and cost $c(C) = \sum_{a \in C} c(a)$, the *mean cost* of C is $\mu(C) = c(C)/|C|$, and the minimum cycle mean of D is

$$\mu^*(D) \;=\; \min\{\, \mu(C) \;:\; C \text{ a directed cycle of } D \,\}. \tag{16.1}$$

A digraph without directed cycles has no minimum cycle mean, and the algorithms report this case. Since each directed cycle lies in one strongly connected component, $\mu^*(D)$ is the least of the minimum cycle means of the components; the three classes therefore find the components first (by `stronglyConnectedComponents()` of chapter 18), solve the problem on each component that has a cycle, and keep the best cycle. This problem is closely related to conservative costs, i.e., to costs under which no directed cycle has a negative cost (cf. Ahuja, Magnanti and Orlin 1993, chapter 5).

Theorem 16.1. *Let $\mu \in \mathbb{R}$. Every directed cycle of D has mean cost at least μ if and only if there are potentials $\pi : V \to \mathbb{R}$ such that*

$$c(u, v) - \mu + \pi(u) - \pi(v) \;\geq\; 0 \qquad \text{for every arc } (u, v) \in A. \tag{16.2}$$

Proof. If (16.2) holds, summing it along a cycle C cancels the potentials and gives $c(C) - \mu|C| \geq 0$. Conversely, if no cycle has mean less than μ, the costs $c - \mu$ are conservative, and the distances from a virtual source joined to all the nodes by arcs of cost 0 are finite and satisfy (16.2). □

Hence, $\mu^*(D)$ is the largest μ for which the costs $c - \mu$ are conservative, i.e., the negative of the smallest amount that, added uniformly to the costs, makes them conservative (as the documentation of the group puts it). Furthermore, a cycle C together with potentials that satisfy (16.2) for $\mu = \mu(C)$ certifies that C is a minimum mean cycle. Hartmann and Orlin check this certificate to stop early, Howard's algorithm maintains potentials of this kind (the distances of section 16.4), and the example of section 16.5 checks the certificate by a Bellman-Ford computation.

16.2 Karp's algorithm

Let D be strongly connected and let s be any node. For $k = 0, \ldots, n$ and $v \in V$, let $D_k(v)$ be the least cost of a directed walk from s to v with exactly k arcs ($+\infty$ if there is none), so that $D_0(s) = 0$, $D_0(v) = +\infty$ for $v \neq s$, and

$$D_k(v) \;=\; \min\{\, D_{k-1}(u) + c(u,v) \;:\; (u,v) \in A \,\} \qquad k = 1, \ldots, n. \tag{16.3}$$

We use the following characterization, due to Karp (1978).

Theorem 16.2. *If D is strongly connected, then*

$$\mu^*(D) \;=\; \min_{v \in V:\, D_n(v) < \infty} \; \max_{0 \le k < n:\, D_k(v) < \infty} \frac{D_n(v) - D_k(v)}{n - k}. \tag{16.4}$$

A short proof is in Karp (1978); it also shows that a walk of n arcs that attains $D_n(v)$ at a minimizing node v contains a cycle of mean $\mu^*(D)$. Karp's algorithm computes the table (16.3) round by round and then evaluates (16.4). In `KarpMmc` this is done on each strongly connected component, with s its first node and n its number of nodes.

Algorithm 16.1 (Karp, on a strongly connected component of n nodes).

```
1  s <- the first node; D_0(s) <- 0; D_0(v) <- +inf for v != s
2  for k = 1, ..., n:
3    for each arc (u, v) of the component with D_{k-1}(u) < +inf:
4      if D_{k-1}(u) + c(u, v) < D_k(v):
5        D_k(v) <- D_{k-1}(u) + c(u, v); pred_k(v) <- (u, v)
6  for each node v with D_n(v) < +inf:
7    M(v) <- max{ (D_n(v) - D_k(v)) / (n - k) : 0 <= k < n, D_k(v) < +inf }
8  mu <- min_v M(v), attained at v*
9  (findCycle) walk back from v* along pred_n, pred_{n-1}, ... until a
   node repeats; the arcs between its two visits form the cycle
```

In the code the rounds of steps 2 to 5 first scan only the nodes already reached, until all the nodes of the component have been reached, and then all of them (`processNextBuildRound()` and `processNextFullRound()`); the comparison of step 4 goes through the tolerance. In steps 7 and 8 the ratios are compared without division, by cross-multiplication in the type `LargeCost`. Each round costs $O(m)$, and therefore the algorithm runs in $O(nm)$ time; it stores $n+1$ values for each node, which gives $O(n^2 + m)$ space, as the header states. We remark that `findCycleMean()` stops at step 8, and that until `findCycle()` has been called, `cycleCost()` and `cycleSize()` return the numerator and the denominator of the ratio in (16.4) at v^*. Their quotient is $\mu^*(D)$, but they need not be the cost and the length of a cycle; after `findCycle()`, they are those of the cycle found in step 9.

16.3 The Hartmann-Orlin algorithm

Hartmann and Orlin (1993) observed that Karp's table often contains a minimum mean cycle rather long before round n, and that Theorem 16.1 allows one to recognize it. At some rounds k the algorithm follows backwards the walks that end at round k, takes the cycle C of least mean among those that these walks close, and computes the potentials

$$\pi(v) \;=\; \min_{0 \le j \le k} \big(|C|\, D_j(v) - j\, c(C) \big), \tag{16.5}$$

i.e., $|C|$ times the least cost under $c - \mu(C)$ of a walk from s to v with at most k arcs. If these potentials satisfy (16.2) for $\mu = \mu(C)$ (scaled by $|C|$), then C is optimal by Theorem 16.1, and the algorithm stops.

Algorithm 16.2 (Hartmann-Orlin, on a strongly connected component of n nodes).

```
1  as steps 1 to 5 of Algorithm 16.1, but after the rounds k = 4, 6, 9,
   13, ... (each the previous one times 3/2) and after the round k = n:
2    for each node v with D_k(v) < +inf, walk back from v along
     pred_k, pred_{k-1}, ...; each node met twice closes a cycle
3    C <- the cycle of least mean among those closed in step 2
4    if a cycle was found and k < n:
5      compute pi by (16.5)
6      if |C| c(u, v) - c(C) >= pi(v) - pi(u) for every arc (u, v):
```

```
7         stop: C is a minimum mean cycle of the component
8     if k = n: stop with the cycle C
```

This schedule of the checks is the one of `processRounds()` in `lemon/hartmann_orlin_mmc.h` (the first check after round 4, then at rounds 6, 9, 13, and so on, rounded down), and the test of step 6 goes through the tolerance. Also, step 6 scans the arcs of the whole digraph, although the potentials are computed only for the nodes of the component; since the arcs inside the component are tested as well, this can only make the test fail more often, and hence possibly delay the stop. In the worst case the checks add to the work of Karp's algorithm, and the bounds of the header are the same, $O(nm)$ time and $O(n^2 + m)$ space. According to the header, the early termination makes the algorithm significantly faster on some instances, but slower on others.

16.4 Howard's policy iteration

Howard's algorithm comes from the policy iteration of Markov decision processes (Howard 1960); its use for the minimum mean cycle is analysed in Cochet-Terrasson et al. (1998) and in Dasdan and Gupta (1998). A *policy* chooses one leaving arc for each node of a strongly connected component, and the arcs chosen form the *policy graph*, in which every node has out-degree 1. Each weakly connected part of the policy graph therefore contains one cycle, and the algorithm evaluates the policy by the cycle of least mean among them. Then it improves the policy by the distances under $c - \lambda$, where λ is that mean, towards the cycle.

Algorithm 16.3 (Howard, on a strongly connected component).

```
1  for each node u: policy(u) <- an arc (u, v) of least cost
2  loop
3    find the cycles of the policy graph; C <- the one of least mean
4    d(v0) <- 0 for a node v0 of C; backwards from v0 along the policy
     arcs (u, v): d(u) <- d(v) + |C| c(u, v) - c(C)
5    for the nodes not reached yet, backwards along any arc (u, v) from
     the nodes reached: policy(u) <- (u, v); d(u) <- d(v) + |C| c(u, v) - c(C)
6    improved <- false
7    for each arc (u, v): if d(v) + |C| c(u, v) - c(C) < d(u):
8      d(u) <- d(v) + |C| c(u, v) - c(C); policy(u) <- (u, v); improved <- true
9    if not improved: return C
```

Steps 1, 3 and 4 to 9 are `buildPolicyGraph()`, `findPolicyCycle()` and `computeNodeDistances()` of `lemon/howard_mmc.h`, and each loop is one iteration. Note that the distances are scaled by $|C|$, so that integer costs give integer distances, and the test of step 7 goes through the tolerance. When step 7 improves no distance, the distances satisfy (16.2) for $\mu = \mu(C)$ (scaled by $|C|$), and C is optimal by Theorem 16.1; if it improves some, the mean of the best cycle of the policy does not increase. As the header remarks, the best known bound on the number of iterations is exponential; however, the documentation of the group reports that in practice Howard's algorithm is by far the most efficient of the three, in agreement with the experiments of Dasdan (2004).

One can also limit the iterations, and then the algorithm returns a cycle that is usually good, but not necessarily optimal. Indeed, `findCycleMean(limit)` takes the largest number of iterations (by default the largest `int`, i.e., no limit), counted over all the components together. It returns a value of the enumeration `TerminationCause`: `NO_CYCLE` (0) if the digraph has no directed cycle, `OPTIMAL` (1) if the minimum mean was found, and `ITERATION_LIMIT` if the limit was reached first. In the last case the cycle kept is the best one met so far, whose mean is an upper bound on $\mu^*(D)$, and the components after the one where the limit was reached are not processed at all. Since `NO_CYCLE` is 0, the result converts to `false` only when there is no cycle, and `run()` keeps the meaning of a `bool`.

Howard's algorithm ends on each component with a policy whose cycle is a minimum mean cycle of that component. `HowardMmc` keeps these cycles, one for each component that has a cycle, and gives them by `componentCycleNum()`, the number of such components, by `componentCycleCost(i)` and `componentCycleSize(i)`, the cost and the number of arcs of the i-th cycle, and by `componentCycle(i, path)`, which appends the i-th cycle to a path that has `addBack()`. These cycles are pairwise node-disjoint, since the components are. They are of minimum mean in their components only if `findCycleMean()` was not stopped by its limit; otherwise they are those of the components processed until then. Furthermore, in the present code a run stopped by the limit just before the first iteration of a component keeps the cycle of the previous component a second time, and `componentCycleNum()` then counts it twice; hence, after an `ITERATION_LIMIT` these functions should be read with care. The min-cost flow algorithms of chapter 13 take from one run of `HowardMmc` the disjoint negative

cycles that their function `unboundedCycles(k)` returns (see chapter 14).

> **abLEMON.** `componentCycleNum()`, `componentCycleCost()`, `componentCycleSize()` and `componentCycle()` are new in abLEMON (version 1.9.0), and `unboundedCycles(k)` of the four minimum cost flow algorithms reads them. On the other hand, the iteration limit of `findCycleMean()` was already in LEMON 1.3.

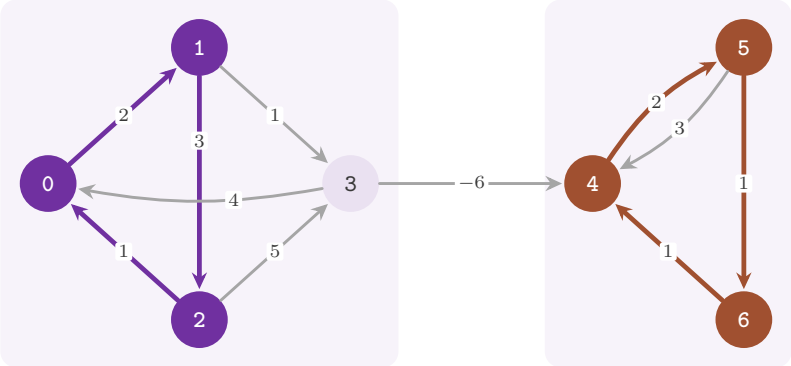


Figure 16.1. A digraph with two strongly connected components (the tinted regions) and the cost of each arc on its plate: the cycle 0, 1, 2 of mean 2 is the best of the first component, and the cycle 4, 5, 6 of mean 4/3 the minimum mean cycle of the digraph.

16.5 The common interface

Each of the three classes has the form

```
template< typename GR , typename CM = typename GR::template ArcMap< int > ,
          typename TR = KarpMmcDefaultTraits< GR , CM > >
class KarpMmc;
```

(with `HartmannOrlinMmcDefaultTraits` and `HowardMmcDefaultTraits` for the other two), where `GR` is the digraph type, `CM` a readable arc map of costs and `TR` the traits class. Their default traits define `Digraph`, `CostMap`, `Cost` (the value type of the costs), `LargeCost`, the type of the internal computations, `Tolerance`, which is `lemon::Tolerance< LargeCost >`, and `Path`, the type of the cycle found, which is `lemon::Path< Digraph >`. By default, `LargeCost` is `long long` when `Cost` is an integer type (`long` if the compiler has no `long long`) and `double` otherwise; `Cost` must be convertible to it. Also, `Path` must meet the concept of chapter 8 and have `addFront()` for `KarpMmc` and `HartmannOrlinMmc`, `addBack()` for `HowardMmc`, which are the only functions the algorithms call on it. Two named template parameters change the traits: `SetLargeCost< T >`, which sets `LargeCost` to `T` and `Tolerance` to `lemon::Tolerance< T >`, and `SetPath< T >`, which sets `Path`; e.g., `HowardMmc< G , CM >::SetPath< ListPath< G > >::Create` stores the cycle in a `ListPath`.

Their constructor takes the digraph and the costs, `cycle(path)` makes the algorithm store the cycle in an external path (otherwise it allocates one, which its destructor deletes), and `tolerance(tol)` and `tolerance()` set and read the tolerance; the first two return the object, so that calls may be chained, as in `mmc.cycle(p).run()`. To execute the algorithm one calls `run()`, which is `findCycleMean() && findCycle()` and returns `true` if the digraph has a directed cycle. It may be called again after the digraph or the costs have changed. In more detail, `findCycleMean()` computes the minimum mean (it returns a `bool` in `KarpMmc` and `HartmannOrlinMmc`, a `TerminationCause` in `HowardMmc`), and `findCycle()` builds the cycle from its data, appending the arcs to the path. Note that `findCycleMean()` clears the path, and `findCycle()` does not; hence, `findCycle()` should be called once after each `findCycleMean()`. We read the results by `cycleCost()` (a `Cost`), `cycleSize()` (an `int`), `cycleMean()`, which is `double(cycleCost()) / cycleSize()`, and `cycle()`, a constant reference to the path.

In all three, the tolerance is used where the algorithms compare costs of walks or distances, namely in step 4 of Algorithm 16.1, in steps 2 to 6 of Algorithm 16.2 and in step 7 of Algorithm 16.3. On the contrary, the comparisons of the means of two cycles are exact cross-multiplications in `LargeCost`, which must therefore hold the products of a cost of a cycle by a number of arcs; with integer costs, this is the reason why `LargeCost` is `long long` by default. Finally, the concept `concepts::MinMeanCycle< GR , Cost >` of `lemon/concepts/min_mean_cycle.h` checks this interface (with `SetPath< ListPath< GR > >` and `SetLargeCost< Cost >`), and `test/min_mean_cycle_test.cc` checks the three classes against it, with `int` and `float` costs.

abLEMON. The concept `concepts::MinMeanCycle` is in `lemon/concepts/min_mean_cycle.h` since abLEMON 1.9.0, where the classes added to the library are held to it; in LEMON 1.3.1 the same checks were written in the test.

Our example below solves the problem of Figure 16.1 by the three classes, which agree on the mean 4/3. Each cycle found is checked to be a closed walk with the cost and the length reported, and the mean is certified by Theorem 16.1: the costs $|C|\,c(a) - c(C)$ are given to `BellmanFord` of chapter 10, which, started with all the distances equal to 0, finds no negative cycle. Then `HowardMmc` gives the two cycles of the components, of means 2 and 4/3, and with a limit of one iteration a cycle whose mean is not less than the optimal one.

```
// The minimum mean cycle of a digraph by the three algorithms of chapter 16:
// they must agree on the mean, the cycle they give must be a closed walk of
// the cost and the length they report, and the mean is certified by
// Bellman-Ford, which finds no negative cycle once it is subtracted from
// every arc (Theorem 16.1). HowardMmc also gives the cycle of each strongly
// connected component, and an upper bound when its iterations are limited.

#include <iostream>
#include <vector>
#include <lemon/smart_graph.h>
#include <lemon/path.h>
#include <lemon/karp_mmc.h>
#include <lemon/hartmann_orlin_mmc.h>
#include <lemon/howard_mmc.h>
#include <lemon/bellman_ford.h>

using namespace lemon;

typedef SmartDigraph::ArcMap< int > CostMap;

// true if path is a closed walk of the given cost and number of arcs
template< typename P >
bool isCycle( const SmartDigraph & g , const CostMap & cost , const P & path ,
              int c , int size )
{
 if( path.length() != size || path.empty() )
  return( false );
 int sum = 0;
 for( int i = 0 ; i < path.length() ; ++i ) {
  SmartDigraph::Arc a = path.nth( i ) ,
                    b = path.nth( ( i + 1 ) % path.length() );
  if( g.target( a ) != g.source( b ) )
   return( false );
  sum += cost[ a ];
  }
 return( sum == c );
 }

template< typename MMC >
bool solve( const char * name , const SmartDigraph & g , const CostMap & cost ,
            int & c , int & size )
{
 MMC mmc( g , cost );
 if( ! mmc.run() )
  return( false );
 c = mmc.cycleCost();
 size = mmc.cycleSize();
 std::cout << name << ": mean " << mmc.cycleMean() << " = " << c << "/" << size
           << std::endl;
 return( isCycle( g , cost , mmc.cycle() , c , size ) );
 }

int main()
{
 SmartDigraph g;
```

```
std::vector< SmartDigraph::Node > v;
for( int i = 0 ; i < 7 ; ++i )
 v.push_back( g.addNode() );
CostMap cost( g );
// two strongly connected components, {0,1,2,3} and {4,5,6}, and an arc
// from the first to the second
const int arcs[][ 3 ] = { { 0 , 1 , 2 } , { 1 , 2 , 3 } , { 2 , 0 , 1 } ,
                          { 1 , 3 , 1 } ,
                          { 3 , 0 , 4 } , { 2 , 3 , 5 } , { 3 , 4 , -6 } ,
                          { 4 , 5 , 2 } ,
                          { 5 , 4 , 3 } , { 5 , 6 , 1 } , { 6 , 4 , 1 } };
for( const auto & a : arcs )
 cost[ g.addArc( v[ a[ 0 ] ] , v[ a[ 1 ] ] ) ] = a[ 2 ];

int c[ 3 ] , size[ 3 ];
if( ! solve< KarpMmc< SmartDigraph > >( "Karp" , g , cost , c[ 0 ] ,
                                        size[ 0 ] ) ||
    ! solve< HartmannOrlinMmc< SmartDigraph > >( "Hartmann-Orlin" , g , cost ,
                                                 c[ 1 ] , size[ 1 ] ) ||
    ! solve< HowardMmc< SmartDigraph > >( "Howard" , g , cost , c[ 2 ] ,
                                          size[ 2 ] ) )
 return( 1 );
// the same mean (compared without division), here 4/3
for( int i = 1 ; i < 3 ; ++i )
 if( c[ i ] * size[ 0 ] != c[ 0 ] * size[ i ] )
  return( 1 );
if( c[ 0 ] * 3 != 4 * size[ 0 ] )
 return( 1 );

// certificate: the costs size * cost - c have no negative cycle
CostMap reduced( g );
for( SmartDigraph::ArcIt a( g ) ; a != INVALID ; ++a )
 reduced[ a ] = size[ 0 ] * cost[ a ] - c[ 0 ];
BellmanFord< SmartDigraph , CostMap > bf( g , reduced );
bf.init( 0 );  // a virtual source joined to every node by a zero arc
if( ! bf.checkedStart() )
 return( 1 );

// one cycle of minimum mean for each component that has a cycle
HowardMmc< SmartDigraph > howard( g , cost );
if( howard.findCycleMean() != HowardMmc< SmartDigraph >::OPTIMAL )
 return( 1 );
if( howard.componentCycleNum() != 2 )
 return( 1 );
for( int i = 0 ; i < howard.componentCycleNum() ; ++i ) {
 Path< SmartDigraph > p;
 howard.componentCycle( i , p );
 std::cout << "component cycle " << i << ": "
           << howard.componentCycleCost( i ) << "/"
           << howard.componentCycleSize( i ) << std::endl;
 if( ! isCycle( g , cost , p , howard.componentCycleCost( i ) ,
                howard.componentCycleSize( i ) ) )
  return( 1 );
 }

// with one iteration only, the mean found is an upper bound
if( howard.findCycleMean( 1 ) == HowardMmc< SmartDigraph >::NO_CYCLE )
 return( 1 );
return( howard.cycleCost() * size[ 0 ] >= c[ 0 ] * howard.cycleSize() ? 0
                                                                      : 1 );
}
```

In our view, the choice among the three is usually simple: `HowardMmc` is the fastest in practice and the only one with the cycles of the components and the limit on the iterations, while `KarpMmc` and `HartmannOrlinMmc` have the polynomial bound $O(nm)$, which may matter when the running time must be bounded a priori.

16.6 Exercises

Exercise 16.1. Prove Theorem 16.1, and verify it on the digraph of Figure 16.1 by computing with `BellmanFord` the potentials of (16.2) for $\mu = 4/3$, scaled by 3 (the example `manual/examples/16-min-mean-cycle.cc` checks only that no negative cycle exists).

Exercise 16.2. Compute by hand the table $D_k(v)$ of (16.3) for the component $\{4, 5, 6\}$ of Figure 16.1, with $s = 4$, and evaluate (16.4); then compare `cycleCost()` and `cycleSize()` of `KarpMmc` after `findCycleMean()` and after `findCycle()`.

Exercise 16.3. Run the three classes on the four cost functions of `test/min_mean_cycle_test.cc`, and check that they find the cycles that the test expects (the test does it with `checkMmcAlg()`).

Exercise 16.4. For the cost function `len1` of `test/min_mean_cycle_test.cc`, find the least limit for which `HowardMmc::findCycleMean(limit)` returns `OPTIMAL` (the test checks that 2 iterations are not enough and 4 are).

Exercise 16.5. Use `componentCycleNum()` and `componentCycle()` of `HowardMmc` to list, for a digraph with several strongly connected components, the node-disjoint cycles of minimum mean of the components, and keep those of negative mean, as `unboundedCycles(k)` does (the example of section 16.5 lists the two cycles of Figure 16.1).

Exercise 16.6. Show that adding a constant α to all the costs adds α to $\mu^*(D)$ and leaves the minimum mean cycles unchanged, and check it with any of the three classes on the digraph of Figure 16.1.

17. Matchings

We are concerned in this chapter with finding, in an undirected graph $G = (V, E)$ with n nodes and m edges, a set of edges no two of which share an end, i.e., a *matching*, which is as large as possible, or of largest total weight, or of least total cost among those that cover a prescribed set of nodes. abLEMON solves these problems with three headers. The first, `bipartite_matching.h`, holds the algorithms for bipartite graphs (`HopcroftKarp` and `MinCostAssignment`); the second, `matching.h`, holds the blossom algorithms of Edmonds for general graphs (`MaxMatching`, `MaxWeightedMatching` and `MaxWeightedPerfectMatching`); the third, `fractional_matching.h`, holds the algorithms for the relaxation in which an edge may be taken by halves (`MaxFractionalMatching`, `MaxWeightedFractionalMatching` and `MaxWeightedPerfectFractionalMatching`), which `matching.h` also uses to start its weighted algorithms. Each class gives, besides the matching, a dual solution that proves it optimal, and we describe the two together, since one usually checks a result against its dual (a "proof" that the user can verify in linear time, or nearly so). The structure of the chapter is as follows. Section 17.1 states the problems and the duality results on which the algorithms rest; sections 17.2 and 17.3 treat the bipartite case, sections 17.4 and 17.5 the general one, section 17.6 the fractional relaxations, and section 17.7 gives the exercises.

17.1 Problems, duality and the choice of an algorithm

Let $\delta(u)$ denote the set of the edges incident to the node u, and $\gamma(X)$ the set of the edges with both ends in $X \subseteq V$. A matching M *covers* the ends of its edges, and it is *perfect* when it covers all the nodes; the cardinality of a maximum matching is denoted by $\nu(G)$. With a weight w_e on each edge, the maximum weight matching problem is the integer program

$$\max \Big\{ \sum_{e \in E} w_e x_e \;:\; \sum_{e \in \delta(u)} x_e \le 1 \quad \forall u \in V, \quad x \in \{0,1\}^E \Big\}, \tag{17.1}$$

of which the maximum cardinality matching problem is the case $w \equiv 1$ and the maximum weight perfect matching problem the case with equations in place of the inequalities (a minimum cost perfect matching is then found by negating the costs). An *alternating path* with respect to M is a path whose edges alternate between M and $E \setminus M$; it is *augmenting* when neither of its ends is covered by M. Exchanging the edges of M and of the path along an augmenting path gives a matching with one more edge, and the algorithms of the chapter that compute an integer matching all rest on the converse.

Theorem 17.1 (Berge). *A matching M of G is maximum if and only if no path is augmenting with respect to M.*

For a proof see Berge 1957 or Schrijver 2003, ch. 24. In a bipartite graph, whose nodes are split into the *red* nodes R and the *blue* nodes B with each edge joining a red node to a blue one (the `BpGraph` structures of chapter 4), the dual object of a matching is a *vertex cover*, i.e., a set of nodes that contains at least one end of each edge. Since the edges of a matching need distinct nodes of any cover, $|M| \le |C|$ for a matching M and a cover C; in the bipartite case the two optima coincide.

Theorem 17.2 (König). *In a bipartite graph the maximum cardinality of a matching equals the minimum cardinality of a vertex cover.*

See König 1931 and Schrijver 2003, ch. 16; `HopcroftKarp` returns such a cover with the matching (cf. section 17.2). In a general graph a minimum vertex cover may be twice as large as a maximum matching (as in a triangle), and the certificate is rather a set of nodes A, called a *barrier*, whose removal leaves many components with an odd number of nodes. In fact, if $o(G - A)$ denotes the number of the odd components of $G - A$, each of them has a node that is either matched to A or not covered at all; hence, at least $o(G - A) - |A|$ nodes are left uncovered by any matching.

Theorem 17.3 (Tutte–Berge formula and Gallai–Edmonds decomposition). *For every graph,*

$$\nu(G) \;=\; \min_{A \subseteq V} \tfrac{1}{2}\big(n + |A| - o(G - A)\big). \tag{17.2}$$

Let D be the set of the nodes left uncovered by at least one maximum matching, A the set of the nodes not in D with a neighbour in D, and $C = V \setminus (D \cup A)$. Then A attains the minimum in (17.2), and each component of the subgraph induced by D is factor-critical (i.e., it has a perfect matching after the removal of any one of its nodes). Also, the subgraph induced by C has a perfect matching, and each maximum matching matches the nodes of A to nodes of D in distinct components.

See Tutte 1947, Gallai 1964 and Schrijver 2003, ch. 24; `MaxMatching` returns the three sets as the status of each node (cf. section 17.4). For the weighted problems the inequalities of (17.1) do not describe the convex hull of the matchings, since the point with $x_e = \frac{1}{2}$ on the edges of a triangle satisfies them and is not a combination of matchings. Edmonds showed that the *odd set inequalities* are the missing ones; the weighted classes of `matchin g.h` work on his description.

Theorem 17.4 (Edmonds). *Let $\mathcal{O}$ be the family of the subsets of V of odd cardinality at least 3. The convex hull of the incidence vectors of the matchings of G is the set of the $x \geq 0$ with $\sum_{e \in \delta(u)} x_e \leq 1$ for each $u \in V$ and*

$$\sum_{e \in \gamma(S)} x_e \;\leq\; \frac{|S| - 1}{2} \qquad \forall\, S \in \mathcal{O}, \tag{17.3}$$

and that of the perfect matchings is obtained by turning the node inequalities into equations.

See Edmonds 1965b and Schrijver 2003, ch. 25 and 26. The linear programming dual of this description has a variable y_u for each node and a variable $z_S \geq 0$ for each odd set:

$$\min \Big\{ \sum_{u \in V} y_u + \sum_{S \in \mathcal{O}} \frac{|S| - 1}{2}\, z_S \;:\; y_u + y_v + \sum_{S \in \mathcal{O}:\, uv \in \gamma(S)} z_S \;\geq\; w_{uv} \quad \forall\, uv \in E \Big\}, \tag{17.4}$$

with $y \geq 0$ for (17.1) and y free for the perfect version. A matching and a dual solution of equal value prove each other optimal. This happens when (i) each edge of the matching has zero *slack*, i.e., $y_u + y_v + \sum z_S - w_{uv} = 0$, (ii) each node with $y_u > 0$ is covered (in the version with inequalities), and (iii) each set with $z_S > 0$ holds $\frac{|S|-1}{2}$ edges of the matching; these are the complementary slackness conditions of the description of Theorem 17.4 and of (17.4). Dropping (17.3) gives the *fractional matching* problem, whose vertices are half-integral: each of them is made of a matching, with value 1 on its edges, and of node-disjoint odd cycles, with value $\frac{1}{2}$ on theirs (see Schrijver 2003, ch. 30). Its dual is (17.4) without the z variables. Its optimum bounds that of (17.1) from above. A triangle with unit weights shows that the bound may be strict ($\frac{3}{2}$ against 1).

As for the choice among the classes, a bipartite graph is best handled by `bipartite_matching.h`, i.e., by `Hopc roftKarp` for the cardinality, in $O(m\sqrt{n})$ time, and by `MinCostAssignment` for a matching of least cost that covers all the red nodes, in $O(n_R\, m \log n)$ time with $n_R = |R|$. A general graph needs the blossoms (which make the algorithms somewhat more involved), i.e., `MaxMatching` for the cardinality and `MaxWeightedMatching` or `M axWeightedPerfectMatching` for the weights, the latter two in $O(nm \log n)$ time. In our view, the fractional classes are mostly of interest when the relaxation is itself the object (e.g., as a bound), and as the starting point of the weighted blossom algorithms, which call them by default. Of course, one can also write an assignment problem as a minimum cost flow problem (chapter 13), and a bipartite cardinality matching as a maximum flow problem (chapter 12). This requires building the network, which is rather wasteful when the graph is "large". The classes of this chapter work on the graph itself, and they give the dual objects described above directly.

17.2 Maximum cardinality matching in bipartite graphs

`HopcroftKarp< BPGR >` finds a maximum matching of a bipartite graph of type `BPGR`, together with the minimum vertex cover of Theorem 17.2, by the algorithm of Hopcroft and Karp 1973. The algorithm grows the matching in phases. In each phase a breadth-first search from the uncovered red nodes labels each red node with the number of matched edges on a shortest alternating path that reaches it. Then a depth-first search from each uncovered red node follows these labels, one layer at a time, until it meets an uncovered blue node, and the matching is augmented along the path found. The header states the bound $O(m\sqrt{n})$ of the original algorithm (a bound usually far from attained in practice), which rests on the fact that the length of the shortest augmenting paths grows at each phase, so that $O(\sqrt{n})$ phases suffice.

Algorithm 17.1 (Hopcroft–Karp).

```
 1  M <- greedy matching (greedyInit) or empty matching (init)
 2  loop
 3     for each red r do d(r) <- 0 if r is uncovered, d(r) <- inf otherwise
 4     breadth-first search from the uncovered red nodes, from a red x, for
        each edge xb, if b is covered by the edge yb of M and d(y) = inf,
        then d(y) <- d(x) + 1; note whether an uncovered b was seen
 5     if no uncovered blue node was seen then exit the loop
 6     for each uncovered red node s do
 7        depth-first search from s, from a red x, scan the edges xb from
           the one it stopped at; if b is uncovered, augment M along the
           path found and stop; if b is matched to y with d(y) = d(x)+1,
           go down to y
 8        a red node all of whose edges fail gets d <- inf
 9  R* <- nodes reached by alternating paths from the uncovered red nodes
10  cover <- (red nodes not in R*) + (blue nodes in R*)
```

Steps 3 to 5 are `bfs()`, steps 6 to 8 `dfs()`, and steps 9 and 10 `findCover()`, which is called once at the end of `start()`. Note that the depth-first search is written without recursion, and each red node keeps the edge it has reached in a map; hence, each edge is scanned once per phase. We remark that the breadth-first search is not stopped at the layer of the first uncovered blue node, and therefore a depth-first search may also augment along a layered path longer than the shortest one. This affects neither the correctness of the result nor its certificate, which step 10 builds from the final matching alone. Indeed, the set of step 10 is a cover because an edge from a red node of R^* to a blue node outside R^* would extend an alternating path. Furthermore, it has $|M|$ nodes. Its nodes are all covered by M (an uncovered blue node in R^* would end an augmenting path), and no edge of M has both ends in it.

This class follows the pattern of the algorithm classes of chapter 3, although it has no named template parameters, since it uses no auxiliary structure that one could replace. Its constructor `HopcroftKarp( const BpGraph & g raph )` builds the maps of the matching, of the layers and of the cover. Execution starts with `init()`, which empties the matching, or with `greedyInit()`, which also matches each red node, in the order of `RedNodeIt`, to the first uncovered blue node among its neighbours. Then `start()` augments the matching until it is maximum and computes the cover, and `run()` is simply `greedyInit()` followed by `start()`. Among the queries, `match ingSize()`, `matching( const Edge & )` (whether the edge is in the matching), `matching( const Node & )` (the matching edge at the node, or `INVALID`), `matchingMap()` (a const reference to the node map of type `Match ingMap`, i.e., `BpGraph::NodeMap< Edge >`) and `mate( const Node & )` give the primal solution. Finally, `cove r( const Node & )` returns whether the node is in the minimum vertex cover, which is meaningful only after `s tart()` or `run()`.

abLEMON. Both `HopcroftKarp` and `MinCostAssignment` are new in abLEMON (version 1.5.0, LEMON trac ticket #168). LEMON 1.3.1 has bipartite graphs but no matching algorithm written for them; its general matching classes give no vertex cover and no assignment potentials.

17.3 Minimum cost assignment

We now give a cost c_e to each edge of a bipartite graph, which may have any number of blue nodes and need not be complete. In this setting `MinCostAssignment< BPGR, CM >` solves

$$\min\Big\{\sum_{e\in E} c_e x_e \;:\; \sum_{e\in\delta(r)} x_e = 1 \;\; \forall r\in R,\quad \sum_{e\in\delta(b)} x_e \le 1 \;\; \forall b\in B,\quad x\in\{0,1\}^E\Big\}, \tag{17.5}$$

i.e., it finds a matching of least cost among those that cover all the red nodes, if one exists. Its dual solution is a *potential* π on the nodes with

$$c_e \;\ge\; \pi(b) - \pi(r) \qquad \forall\, e = rb \in E, \tag{17.6}$$

with equality on the edges of the matching, and with the blue nodes left uncovered all at the largest potential $K = \max_{b\in B}\pi(b)$ of the blue nodes. Setting $\alpha_r = K - \pi(r)$ and $\beta_b = \pi(b) - K \le 0$ gives a feasible solution of the linear programming dual of (17.5) whose value equals the cost of the matching. Thus, the optimal cost is $\sum_{r\in R}\big(\pi(b_r) - \pi(r)\big)$, where b_r is the mate of r.

The method is that of the successive shortest augmenting paths, with the potentials of Edmonds and Karp 1972 and Tomizawa 1971. Here the red nodes are matched one at a time, each along a shortest path of the residual graph, in which an edge not in the matching is traversed from its red end at cost c_e and an edge of the matching

from its blue end at cost $-c_e$. Each path is found by Dijkstra's algorithm (Dijkstra 1959) on the costs reduced by π, which (17.6) keeps nonnegative.

Algorithm 17.2 (Successive shortest paths for the assignment).

```
1  M <- empty; pi(b) <- 0 for each blue b
2  for each red r do if r has no edge then return false, else
      pi(r) <- - (least cost of an edge of r)
3  for each red node s, in the order of RedNodeIt do
4     Dijkstra from s on the reduced costs, red x -> blue b along xb not
       in M at c(xb) + pi(x) - pi(b); matched blue b -> its mate y at
       d(b) - (c(yb) + pi(y) - pi(b)); stop when an uncovered blue node t
       is taken from the heap
5     if no uncovered blue node is reached then return false
6     for each node v do pi(v) += d(v) if v was taken from the heap with
       d(v) < d(t), and pi(v) += d(t) otherwise
7     augment M along the path from s to t
8  return true
```

Step 2 makes the reduced costs nonnegative before the first search, and step 6 keeps them so, with zero reduced cost on the new path. Since the nodes not finished by the search all receive the same increase $d(t)$, the uncovered blue nodes, which are never finished, keep the largest blue potential. As for the heap of step 4, it is a `std::priority_queue` of pairs (distance, node id) with lazy deletion: a node reached again at a shorter distance is pushed again, and the stale entries are skipped when popped. A search therefore takes $O(m \log n)$ time, and the method $O(n_R\, m \log n)$, as the header states. Note that the comparisons are exact (the class has no tolerance), which is usually adequate for integer costs but may be somewhat fragile with floating-point ones, and that the cost type must be a signed arithmetic type.

The interface is reduced to what the method needs. Its constructor is `MinCostAssignment( const BpGraph & graph , const CostMap & cost )`, with `CM` defaulting to `BpGraph::EdgeMap< int >`. Its only execution function is `run()`, which returns `true` if a matching covering all the red nodes exists and `false` otherwise (in which case the matching is left as the method had grown it, and the potentials have no meaning). As in `HopcroftKarp`, `totalCost()`, `matching( const Edge & )`, `matching( const Node & )` and `matchingMap()` give the primal solution, while `potential( const Node & )` gives the dual solution (17.6).

The program below solves a "small" instance with both classes of the header. It checks the cover of Theorem 17.2, the conditions (17.6) and the value of the dual solution. Then it compares the optimal cost with an enumeration of all the assignments.

```
// The matchings of a bipartite graph (sections 17.2 and 17.3): a maximum
// matching by HopcroftKarp with the vertex cover that proves it, and a
// minimum cost assignment of the red nodes by MinCostAssignment, checked
// against its dual potentials and against all the assignments.

#include <algorithm>
#include <iostream>
#include <vector>
#include <lemon/list_graph.h>
#include <lemon/bipartite_matching.h>

using namespace lemon;

typedef ListBpGraph BpGraph;

int main()
{
 BpGraph g;
 std::vector< BpGraph::RedNode > r;
 std::vector< BpGraph::BlueNode > b;
 for( int i = 0 ; i < 4 ; ++i )
  r.push_back( g.addRedNode() );
 for( int j = 0 ; j < 5 ; ++j )
  b.push_back( g.addBlueNode() );
```

```
// cost[i][j] < 0 means that r_i and b_j are not adjacent
const int cost[ 4 ][ 5 ] = { { 4 , 1 , -1 , -1 , 7 } ,
                             { 2 , -1 , 3 , -1 , -1 } ,
                             { -1 , 2 , 5 , 4 , -1 } ,
                             { 3 , -1 , 6 , -1 , 2 } };
BpGraph::EdgeMap< int > c( g );
for( int i = 0 ; i < 4 ; ++i )
 for( int j = 0 ; j < 5 ; ++j )
  if( cost[ i ][ j ] >= 0 )
   c[ g.addEdge( r[ i ] , b[ j ] ) ] = cost[ i ][ j ];

// maximum cardinality matching and minimum vertex cover (Konig)
HopcroftKarp< BpGraph > hk( g );
hk.run();
int covered = 0;
for( BpGraph::NodeIt n( g ) ; n != INVALID ; ++n )
 if( hk.cover( n ) )
  ++covered;
for( BpGraph::EdgeIt e( g ) ; e != INVALID ; ++e )
 if( ! hk.cover( g.u( e ) ) && ! hk.cover( g.v( e ) ) )
  return( 1 );
std::cout << "maximum matching: " << hk.matchingSize()
          << ", vertex cover: " << covered << std::endl;
if( hk.matchingSize() != 4 || covered != 4 )
 return( 1 );

// minimum cost assignment of the red nodes
MinCostAssignment< BpGraph > mca( g , c );
if( ! mca.run() )
 return( 1 );
for( BpGraph::EdgeIt e( g ) ; e != INVALID ; ++e ) {
 int slack = c[ e ] - mca.potential( g.blueNode( e ) ) +
             mca.potential( g.redNode( e ) );
 if( slack < 0 || ( mca.matching( e ) && slack != 0 ) )
  return( 1 );
 }

// the best assignment by enumeration of the injections of r into b
int best = -1;
std::vector< int > p;
for( int j = 0 ; j < 5 ; ++j )
 p.push_back( j );
do {
 int sum = 0;
 bool ok = true;
 for( int i = 0 ; i < 4 && ok ; ++i ) {
  ok = cost[ i ][ p[ i ] ] >= 0;
  sum += cost[ i ][ p[ i ] ];
  }
 if( ok && ( best < 0 || sum < best ) )
  best = sum;
 } while( std::next_permutation( p.begin() , p.end() ) );

// the cost of the assignment from the potentials alone
int dual = 0;
for( int i = 0 ; i < 4 ; ++i )
 dual += mca.potential( g.blueNode( mca.matching( r[ i ] ) ) ) -
         mca.potential( r[ i ] );
if( dual != mca.totalCost() )
 return( 1 );

std::cout << "minimum cost assignment: " << mca.totalCost()
          << " (enumeration: " << best << ")" << std::endl;
for( int i = 0 ; i < 4 ; ++i )
```

```
  for( int j = 0 ; j < 5 ; ++j )
   if( mca.matching( r[ i ] ) != INVALID &&
       g.blueNode( mca.matching( r[ i ] ) ) == b[ j ] )
    std::cout << "  r" << i << " -> b" << j << std::endl;
 return( mca.totalCost() == best ? 0 : 1 );
 }
```

17.4 Maximum cardinality matching in general graphs

We now turn to general graphs, in which an alternating search tree may close an odd cycle, and the search then has to treat the cycle as a single node. This is the *blossom* of Edmonds 1965a, and `MaxMatching< GR >` implements his alternating forest algorithm on an undirected graph of type `GR`. Each node has a status, `EVEN`, `ODD`, `MATCHED` or `UNMATCHED`. A search tree is grown from an uncovered root, which is `EVEN`; a covered node reached from an `EVEN` node becomes `ODD`, and its mate `EVEN`. An edge between two `EVEN` nodes of the same tree in distinct blossoms closes a blossom, whose nodes all become `EVEN`, and an edge from an `EVEN` node to an uncovered node gives an augmenting path. Internally, the blossoms are kept by a `UnionFindEnum` and the trees by an `ExtendFindEnum` (see chapter 8).

Algorithm 17.3 (Edmonds' alternating forest, sparse variant).

```
1  M <- greedy or empty matching; covered nodes MATCHED, others UNMATCHED
2  for each node n, while more than L nodes are UNMATCHED,
       (L = 0, or L = 1 without the decomposition) do
3      if n is UNMATCHED then make n the EVEN root of a new tree; Q <- (n)
4      while Q is not empty do take u from Q; for each arc uv do
5          if v is EVEN and in another blossom then shrink the blossom closed
            by uv; its ODD nodes become EVEN and enter Q
6          if v is MATCHED then extend, v becomes ODD, its mate EVEN (into Q)
7          if v is UNMATCHED then augment along uv and the tree path of u; the
            nodes of the tree become MATCHED; go to the next root
8  (a tree that did not augment keeps its labels)
```

Step 5 is `shrinkOnEdge()`, step 6 `extendOnArc()` and step 7 `augmentOnArc()`. A tree that ends without augmenting is a Hungarian tree, i.e., no later search can enter it with an augmenting path, and its labels stay. Hence, when the loop ends with no node `UNMATCHED`, the `EVEN` nodes form the set D of Theorem 17.3, the `ODD` nodes the barrier A and the `MATCHED` nodes the set C, which proves the matching maximum by (17.2). `startDense()` runs the same search with the heuristic of postponing the shrinks. It first extends the tree from the nodes in the queue. Only when no extension or augmentation is left does it shrink the blossoms, extending again after each shrink; the header reports this to be faster on "dense" graphs (presumably a matter of constants, since the worst case is the same). The header states no bound on the running time, and we do not attempt one here. The bounds of the implementations of the blossom algorithm are surveyed in Galil 1986.

Besides the constructor `MaxMatching( const Graph & graph )`, the class has three functions that set the initial matching. These are `init()` (the empty one), `greedyInit()` (each node, in the order of `NodeIt`, matched to its first uncovered neighbour, loops excluded) and `matchingInit( const MatchingMap & matching )`, which takes a `bool` edge map and returns `false` if two edges with value `true` share a node. Two functions, `startSparse( bool decomposition = true )` and `startDense( bool decomposition = true )`, run the two variants of Algorithm 17.3, and `run( bool decomposition = true )` calls `greedyInit()` and `startSparse()` if $m < 2n$, and `init()` and `startDense()` otherwise. Of the queries, `matchingSize()` counts the covered nodes (in $O(n)$ time), and `matching( const Edge & )`, `matching( const Node & )` (the matching arc at the node, directed away from it, or `INVALID`), `matchingMap()` (a `Graph::NodeMap< Arc >`) and `mate( const Node & )` complete the primal solution. Lastly, `status( const Node & )` and `statusMap()` give the values of the enum `Status`, i.e., `EVEN` $= 1$ (also called `D`), `MATCHED` $= 0$ (also `C`), `ODD` $= -1$ (also `A`) and `UNMATCHED` $= -2$, and `barrier( const Node & )` is `true` on the `ODD` nodes.

With `decomposition` set to `false`, the loop of step 2 stops as soon as at most one node is left uncovered. At that point the matching is maximum, since an augmenting path needs two uncovered ends; however, the last uncovered node has not been searched from, and the statuses do not form the decomposition. According to the header, this may save much time, in particular on a dense graph with an odd number of nodes, where the last search, from a node that remains uncovered, usually explores the whole graph and finds no augmenting path.

The program below computes the matching of Figure 17.1. It counts the components of the subgraph induced

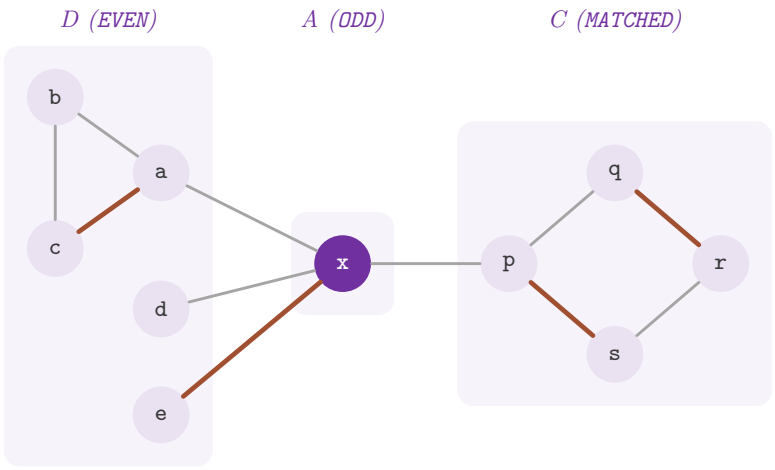


Figure 17.1. The graph of the example below, with the maximum matching found by MaxMatching (the edges in the colour of the added objects) and its Gallai–Edmonds decomposition: the EVEN nodes (D) induce the factor-critical components {a, b, c}, {d} and {e}, the barrier A is {x}, and the MATCHED nodes (C) induce a square with a perfect matching. By (17.2), at least $3 - 1 = 2$ nodes are uncovered by any matching.

by the `EVEN` nodes with a `FilterNodes` adaptor (chapter 5) and `countConnectedComponents()` (chapter 18). Then it checks the equality of (17.2) for the barrier found, and it runs the algorithm once more without the decomposition.

```
// A maximum matching of a general graph by MaxMatching (section 17.4), on
// the graph of Figure 17.1: a triangle and two single nodes hang from the
// node x, which is also joined to a square. The Gallai-Edmonds
// decomposition proves the matching maximum by the Tutte-Berge formula.

#include <iostream>
#include <lemon/list_graph.h>
#include <lemon/connectivity.h>
#include <lemon/adaptors.h>
#include <lemon/matching.h>

using namespace lemon;

int main()
{
 ListGraph g;
 ListGraph::Node a = g.addNode() , b = g.addNode() , c = g.addNode() ,
                 d = g.addNode() , e = g.addNode() , x = g.addNode() ,
                 p = g.addNode() , q = g.addNode() , r = g.addNode() ,
                 s = g.addNode();
 g.addEdge( a , b );
 g.addEdge( b , c );
 g.addEdge( c , a );
 g.addEdge( x , a );
 g.addEdge( x , d );
 g.addEdge( x , e );
 g.addEdge( x , p );
 g.addEdge( p , q );
 g.addEdge( q , r );
 g.addEdge( r , s );
 g.addEdge( s , p );

 MaxMatching< ListGraph > mm( g );
 mm.run();

 // the components of the subgraph induced by the EVEN nodes, and the
 // barrier made of the ODD nodes
```

```
ListGraph::NodeMap< bool > even( g );
int barrier = 0;
for( ListGraph::NodeIt n( g ) ; n != INVALID ; ++n ) {
 even[ n ] = mm.status( n ) == MaxMatching< ListGraph >::EVEN;
 if( mm.barrier( n ) )
  ++barrier;
 }
FilterNodes< ListGraph > d_graph( g , even );
int odd_comps = countConnectedComponents( d_graph );

// Tutte-Berge: n - 2 |M| = (odd components of G - A) - |A|
int deficiency = countNodes( g ) - 2 * mm.matchingSize();
std::cout << "matching size " << mm.matchingSize() << ", " << odd_comps
          << " components in D, " << barrier << " barrier node(s)"
          << std::endl;
for( ListGraph::NodeIt n( g ) ; n != INVALID ; ++n )
 std::cout << "  node " << g.id( n ) << ": status " << int( mm.status( n ) )
           << ", mate "
           << ( mm.mate( n ) == INVALID ? -1 : g.id( mm.mate( n ) ) )
           << std::endl;
if( mm.matchingSize() != 4 || deficiency != odd_comps - barrier )
 return( 1 );
if( ! mm.barrier( x ) )
 return( 1 );
if( mm.status( p ) != MaxMatching< ListGraph >::MATCHED )
 return( 1 );

// without the decomposition the size is the same
MaxMatching< ListGraph > fast( g );
fast.run( false );
return( fast.matchingSize() == mm.matchingSize() ? 0 : 1 );
}
```

abLEMON. The parameter `decomposition` of `startSparse()`, `startDense()` and `run()` is not in LEMON 1.3.1: it was added to the repository of LEMON in 2018, after its last release, and abLEMON is the first release to distribute it. With the default `true`, a program written for LEMON 1.3.1 behaves as before.

17.5 Maximum weight matchings in general graphs

`MaxWeightedMatching< GR, WM >` solves (17.1), and `MaxWeightedPerfectMatching< GR, WM >` its perfect version, by the primal-dual blossom algorithm of Edmonds 1965a. Throughout, the algorithm keeps a matching and a dual solution of (17.4) that satisfy the complementary slackness conditions (i) and (iii) of section 17.1, and changes the duals until condition (ii) also holds (in the perfect version, until all the nodes are covered). Its weight map `WM` defaults to `GR::EdgeMap< int >`; we remark that loops are ignored. Both classes use binary heaps (`BinHeap`) for the kinds of dual change, four in `MaxWeightedMatching` and three in the perfect version, and a `HeapUnionFind` for the nested blossoms, which gives the bound $O(nm \log n)$ stated by the header, i.e., that of Galil, Micali and Gabow 1986.

Algorithm 17.4 (Primal-dual blossom algorithm).

```
1  initial duals y(u) <- max(0, max of w(e)/2 over the edges e at u),
       z <- 0 (init); or the optimal duals and matching of the fractional
       relaxation (fractionalInit, the default of run())
2  each uncovered node is the EVEN root of a tree; each node a blossom
3  while some node is uncovered do
4      d1 <- least y(u) over the EVEN nodes (not in the perfect version)
       d2 <- least slack of an edge from an EVEN blossom to a node out of
             the forest
       d3 <- half the least slack of an edge between two EVEN blossoms
       d4 <- least change that brings to zero the z(B) of a nontrivial
             ODD blossom
5      if the least of them is infinite then return false (perfect version)
6      move the duals by the least d, y down on EVEN nodes and up on ODD
```

```
       ones, z(B) up on EVEN blossoms and down on ODD ones
 7     d1 -> the node stays uncovered (its y is now 0) and leaves the forest
       d2 -> extend the tree, or augment if the node reached is uncovered
       d3 -> same tree, shrink a new blossom; distinct trees, augment
       d4 -> expand the blossom
 8  extract the matching and the duals, the nested blossoms flattened
```

In the code, the quantities of step 4 are kept in the heaps `_delta1` to `_delta4`, and the dual changes of step 6 are applied lazily through a running sum; thus, a step costs a few heap operations instead of a pass over the graph. If the weights are of an integer type, each dual value is multiplied by the constant `dualScale`, which is 4 (and 1 for a floating-point type), so that the halvings of steps 1 and 4 stay exact. At the end, the value of the duals equals `dualScale` times the weight of the matching. By default the perfect version starts from the optimal perfect fractional matching (cf. section 17.6); when this does not exist (e.g., when two pendant edges end at the same node), `fractionalInit()` already detects the failure, which `start()` then returns. The header calls this initialization the "jumpstart" heuristic, and it may change which of several optimal matchings is found, but not the optimal value.

The two classes have the same members, except for the return type of the execution functions. Their constructor takes `( const Graph & graph , const WeightMap & weight )`. Either `init()` or `fractionalInit()` sets the initial solution, the latter by the jumpstart from the fractional relaxation (solved by `MaxWeightedFractionalMatching` or `MaxWeightedPerfectFractionalMatching`, with loops excluded); then `start()` runs the algorithm, and `run()` is `fractionalInit()` followed by `start()`. In `MaxWeightedPerfectMatching`, both `start()` and `run()` return `false` when the graph has no perfect matching. As in `MaxMatching`, `matchingWeight()`, `matchingSize()`, `matching( const Edge & )`, `matching( const Node & )`, `matchingMap()` and `mate( const Node & )` give the primal solution. For the dual one, `dualValue()` returns the value of the scaled dual solution, i.e., $\sum_u y_u + \sum_B \lfloor |B|/2 \rfloor z_B$, and `nodeValue( const Node & )` the scaled y_u. Furthermore, `blossomNum()` returns the number of the blossoms with a dual variable in the final solution, `blossomSize( int k )` and `blossomValue( int k )` (the scaled z_B) describe the k-th of them, and the nested class `BlossomIt`, built as `BlossomIt( algorithm , k )`, lists its nodes. A blossom comes before the blossoms nested in it, whose nodes form contiguous parts of its own list.

The program below computes a maximum weight matching of a triangle with two pendant edges. It verifies the conditions of section 17.1 on each edge (the slack computed from `nodeValue()` and from the blossoms that contain both ends, which `BlossomIt` lists) and the equality of the primal and dual values, and it observes that the graph has no perfect matching; it ends with the fractional classes of the next section.

```
// Weighted matchings of a general graph (sections 17.5 and 17.6): a
// triangle of weight 5 with a pendant edge at two of its nodes. The
// maximum weight matching is checked against its dual solution (node
// potentials and blossom values, scaled by dualScale); the graph has no
// perfect matching, while the triangle alone has a perfect fractional one,
// whose value exceeds that of every matching.

#include <iostream>
#include <lemon/list_graph.h>
#include <lemon/matching.h>
#include <lemon/fractional_matching.h>

using namespace lemon;

typedef ListGraph Graph;
typedef MaxWeightedMatching< Graph > MWM;

// y_u + y_v + sum of z_B over the blossoms B containing u and v - w_uv,
// all scaled by dualScale: nonnegative, and zero on the matching
int reducedWeight( const Graph & g , const Graph::EdgeMap< int > & w ,
                   const MWM & mwm , Graph::Edge e )
{
 int rw = mwm.nodeValue( g.u( e ) ) + mwm.nodeValue( g.v( e ) ) -
          MWM::dualScale * w[ e ];
 for( int k = 0 ; k < mwm.blossomNum() ; ++k ) {
  bool su = false , sv = false;
  for( MWM::BlossomIt n( mwm , k ) ; n != INVALID ; ++n ) {
```

```
   if( Graph::Node( n ) == g.u( e ) )
    su = true;
   if( Graph::Node( n ) == g.v( e ) )
    sv = true;
   }
  if( su && sv )
   rw += mwm.blossomValue( k );
  }
 return( rw );
 }

int main()
{
 Graph g;
 Graph::Node a = g.addNode() , b = g.addNode() , c = g.addNode() ,
             y = g.addNode() , z = g.addNode();
 Graph::EdgeMap< int > w( g );
 w[ g.addEdge( a , b ) ] = 5;
 w[ g.addEdge( b , c ) ] = 5;
 w[ g.addEdge( c , a ) ] = 5;
 w[ g.addEdge( a , y ) ] = 1;
 w[ g.addEdge( b , z ) ] = 2;

 MWM mwm( g , w );
 mwm.run();
 std::cout << "maximum weight matching: " << mwm.matchingWeight() << " ("
           << mwm.matchingSize() << " edges), dual value " << mwm.dualValue()
           << " / " << MWM::dualScale << ", " << mwm.blossomNum()
           << " blossom(s)" << std::endl;
 if( mwm.matchingWeight() != 7 )
  return( 1 );
 if( mwm.dualValue() != MWM::dualScale * mwm.matchingWeight() )
  return( 1 );
 for( Graph::EdgeIt e( g ) ; e != INVALID ; ++e ) {
  int rw = reducedWeight( g , w , mwm , e );
  if( rw < 0 || ( mwm.matching( e ) && rw != 0 ) )
   return( 1 );
  }

 // five nodes: no perfect matching
 MaxWeightedPerfectMatching< Graph > mwpm( g , w );
 if( mwpm.run() )
  return( 1 );

 // the maximum fractional matching covers four nodes (value 2)
 MaxFractionalMatching< Graph > mfm( g );
 mfm.run();
 std::cout << "covered nodes in a maximum fractional matching: "
           << mfm.matchingSize() << std::endl;
 if( mfm.matchingSize() != 4 )
  return( 1 );
 // the barrier S proves it: n - |covered| = |S| - |N(S)|
 Graph::NodeMap< bool > adj( g , false );
 int s_size = 0 , n_size = 0;
 for( Graph::NodeIt n( g ) ; n != INVALID ; ++n )
  if( mfm.barrier( n ) ) {
   ++s_size;
   for( Graph::IncEdgeIt e( g , n ) ; e != INVALID ; ++e )
    adj[ g.oppositeNode( n , e ) ] = true;
   }
 for( Graph::NodeIt n( g ) ; n != INVALID ; ++n )
  if( adj[ n ] )
   ++n_size;
 std::cout << "barrier of " << s_size << " nodes with " << n_size
           << " neighbours" << std::endl;
```

```
if( countNodes( g ) - mfm.matchingSize() != s_size - n_size )
 return( 1 );

// the triangle alone: value 15/2 by halves, 5 by any matching
Graph t;
Graph::Node u0 = t.addNode() , u1 = t.addNode() , u2 = t.addNode();
Graph::EdgeMap< int > tw( t );
tw[ t.addEdge( u0 , u1 ) ] = 5;
tw[ t.addEdge( u1 , u2 ) ] = 5;
tw[ t.addEdge( u2 , u0 ) ] = 5;
typedef MaxWeightedPerfectFractionalMatching< Graph > MWPFM;
MWPFM frac( t , tw );
if( ! frac.run() )
 return( 1 );
std::cout << "perfect fractional matching of the triangle: "
          << frac.matchingWeight() << " / " << MWPFM::primalScale
          << ", dual value " << frac.dualValue() << " / " << MWPFM::dualScale
          << std::endl;
for( Graph::EdgeIt e( t ) ; e != INVALID ; ++e )
 if( frac.matching( e ) != 1 )
  return( 1 );  // x_e = 1/2, scaled by 2
MaxWeightedPerfectMatching< Graph > integral( t , tw );
if( integral.run() )
 return( 1 );
return( frac.matchingWeight() == 15 &&
            frac.dualValue() * MWPFM::primalScale ==
              MWPFM::dualScale * frac.matchingWeight()
          ? 0
          : 1 );
}
```

abLEMON. In LEMON 1.3.1 the extraction of the nested blossoms at the end of `start()` (step 8 of Algorithm 17.4) was recursive, with one level per level of nesting, and it could overflow the stack on large graphs (LEMON trac ticket #638); in abLEMON it uses an explicit stack, a change made in the repository of LEMON in 2021 and never released by it. Furthermore, the node records of the two classes initialize all their fields, some of which LEMON 1.3.1 copied while still indeterminate. The results and the interface do not change.

17.6 Fractional matchings

We finally describe the three classes of `fractional_matching.h`, which solve the relaxation of section 17.1 without the odd set inequalities (17.3), whose optimal vertices are made of matching edges with value 1 and of odd cycles with value $\frac{1}{2}$. They represent a fractional matching by a node map of arcs, in which each covered node points along an arc to a neighbour; the two ends of an edge with value 1 point to each other, and the nodes of a half-valued odd cycle point to their successors on the cycle. Since the values are multiples of $\frac{1}{2}$, the primal solution is reported multiplied by the constant `primalScale`, which is 2. Thus, `matching( const Edge & )` returns an `int` in $\{0, 1, 2\}$ (the number of ends of the edge that point along it), and `matchingSize()` returns the number of the covered nodes, i.e., twice the value of the fractional matching. All three constructors take a flag `allow_loops`, `true` by default, which decides whether a loop may be taken (as a cycle of length one, with value $\frac{1}{2}$).

`MaxFractionalMatching< GR, TR >` maximizes the cardinality. The relaxation is equivalent to a bipartite matching problem on the double cover of G, in which each node has a copy on each side and each edge uv joins the first copy of u to the second copy of v and vice versa. This problem is solved by a push-relabel method in the spirit of Goldberg and Tarjan 1988, with the levels kept by an `Elevator` (chapter 8).

Algorithm 17.5 (Push-relabel for the maximum fractional matching).

```
1  each node u points to its first neighbour (loops skipped unless
      allowed); indeg(v) <- number of the nodes pointing to v;
      all nodes on level 0; the nodes with indeg 0 are active
2  while some node is active do n <- an active node on the highest level l
3     if a neighbour u of n points to a node v on a level below l then
         u points to n instead; indeg(v) -= 1, indeg(n) += 1;
```

```
          n becomes inactive, v active if indeg(v) = 0
 4     else lift n to one level above the lowest such v (or to the top);
          if level l is left empty, lift the nodes above it to the top
 5   a node pointing to a node v with indeg(v) > 1 drops its pointer until
       indeg(v) = 1
 6   if postprocess then turn the alternating paths and the even cycles of
       pointers into matching edges, leaving odd cycles of halves
```

The perfect variant `startPerfect()` stops with `false` as soon as a node would be lifted to the top or a level is left empty, either of which proves that no perfect fractional matching exists. The dual object, which we also call a *barrier*, is a set S of nodes with $|S| - |N(S)|$ as large as possible, where $N(S)$ is the set of the neighbours of the nodes of S. Each fractional matching leaves at least $|S| - |N(S)|$ nodes uncovered. Conversely, a maximum one leaves exactly that many for some S, i.e.,

$$n - \texttt{matchingSize()} \;=\; \max_{S \subseteq V} \big(|S| - |N(S)|\big), \tag{17.7}$$

which is König's theorem on the double cover. `barrier( const Node & )` returns the nodes on the top level of the elevator after `start()`, and those on or above the level found empty after a failed `startPerfect()`; the example of section 17.5 checks (17.7) on the former. The header states no bound on the running time, although push-relabel methods are often fast in practice.

We have modeled the description on the push-relabel classes of chapter 12. The traits class `MaxFractionalMatchingDefaultTraits< GR >` defines `MatchingMap` (`GR::NodeMap< GR::Arc >`) and `Elevator` (`LinkedElevator< GR, GR::Node >`) with their factory functions, and the named template parameters `SetMatchingMap< T >`, `SetElevator< T >` (an elevator that the caller passes with `elevator()`) and `SetStandardElevator< T >` (an elevator that the algorithm allocates) replace them. Its constructor is `( const Graph & graph , bool allow_loops = true )`. Two functions, `matchingMap( MatchingMap & )` and `elevator( Elevator & )`, set external structures, and `elevator()` returns the one in use. Execution is controlled by `init()`, `start( bool postprocess = true )` and `startPerfect( bool postprocess = true )` (which returns whether a perfect fractional matching exists), or by their shortcuts `run( bool postprocess = true )` and `runPerfect( bool postprocess = true )`. Its queries are `matchingSize()`, `matchingMap()`, `matching( const Edge & )`, `matching( const Node & )` and `barrier( const Node & )`. With `postprocess` set to `false`, the result may also contain even cycles and paths of half-valued edges, with the same value.

`MaxWeightedFractionalMatching< GR, WM >` and `MaxWeightedPerfectFractionalMatching< GR, WM >` solve the weighted relaxation, without and with the perfection constraint, by the primal-dual method of Algorithm 17.4, in which a closed odd cycle is not shrunk into a blossom but kept as a half-valued cycle of the solution. There are no z variables, and hence no fourth kind of dual change; the header states the bound $O(nm \log n)$ for both. Their constructor is `( const Graph & graph , const WeightMap & weight , bool allow_loops = true )`, and their execution functions are `init()`, `start()` and `run()`, which in the perfect class return `false` when no perfect fractional matching exists. Their queries are `matchingWeight()` (the weight scaled by `primalScale`), `matchingSize()`, `matching( const Edge & )`, `matching( const Node & )`, `matchingMap()`, `dualValue()` and `nodeValue( const Node & )`, with the duals scaled by `dualScale` as in section 17.5. At the optimum, `primalScale` times `dualValue()` equals `dualScale` times `matchingWeight()`. Again, the example of section 17.5 checks this on a triangle, whose perfect fractional matching has weight $\frac{15}{2}$, while the triangle has no perfect matching at all.

17.7 Exercises

Exercise 17.1. Prove that the set returned by `cover()` after `HopcroftKarp::run()` is a vertex cover with `matchingSize()` nodes, using only the fact that the final matching admits no augmenting path; then check it on random bipartite graphs against `MaxMatching` run on the same graph. (The test `test/bipartite_matching_test.cc` does the latter; the example `manual/examples/17-bipartite.cc` checks the cover on one instance.)

Exercise 17.2. Show that the potentials returned by `MinCostAssignment` give a feasible solution of the linear programming dual of (17.5) with the value of the matching, and use them to write a check that needs no other solver. Then compare `totalCost()` with the optimum of the same problem written as a minimum cost flow problem for `NetworkSimplex` (chapter 13). (The test `test/bipartite_matching_test.cc` makes this comparison and checks the conditions (17.6).)

Exercise 17.3. Modify the graph of Figure 17.1 so that its barrier has two nodes, predict the statuses of the

nodes from Theorem 17.3, and verify the prediction and the equality of (17.2) with the program `manual/examples/17-edmonds.cc`.

Exercise 17.4. On a complete graph with an odd number of nodes, measure the running time of `MaxMatching::run()` with and without the decomposition, and explain the difference from step 2 of Algorithm 17.3. (The test `test/matching_test.cc` checks that the two give matchings of the same size.)

Exercise 17.5. Find a minimum cost perfect matching of a general graph with `MaxWeightedPerfectMatching`, by negating the costs, and verify its optimality from `nodeValue()`, `blossomValue()` and `BlossomIt` as the function `reducedWeight()` of `manual/examples/17-weighted.cc` does, with the sign of y free. (The test `test/matching_test.cc` checks the dual solutions of both weighted classes in this way.)

Exercise 17.6. On a graph made of node-disjoint triangles with unit weights, compare the values of `MaxWeightedFractionalMatching` and `MaxWeightedMatching`, and find the odd set inequalities (17.3) that close the gap; then compare the running time of `MaxWeightedMatching` started by `init()` and by `fractionalInit()`. (The test `test/fractional_matching_test.cc` checks the primal and dual solutions of the fractional classes.)

18. Connectivity, planarity, isomorphism

This chapter collects the algorithms that decide a structural property of a graph, or compute the object that certifies it, instead of optimizing a weight. They are the components of a graph with the nodes, edges and arcs that separate them, the tests of acyclicity and the topological order (`lemon/connectivity.h`), the Euler tours (`lemon/euler.h`), the planarity test with its embedding, its straight-line drawing and the colorings of a planar graph (`lemon/planarity.h`), and the search for an isomorphic copy of a graph inside another (`lemon/vf2.h`, `lemon/vf2pp.h`). All of them read only the structure of the graph (and, for the isomorphism, two node labelings), and most of them run in time linear in its size (which, on the graphs met in practice, makes them rather cheap compared with the optimization algorithms of the previous chapters). We remark two exceptions: the five coloring, which is quadratic in the worst case, and the isomorphism search, which solves an NP-complete problem and may take exponential time. We write $G = (V, E)$ for an undirected graph and $D = (V, A)$ for a digraph, with $n = |V|$ and $m = |E|$ or $m = |A|$. We use the iterators, the maps and the graph structures of chapter 3 and chapter 4, while the depth-first and breadth-first searches, on which most functions are built, are those of chapter 9.

18.1 Connected components, acyclicity and simple graphs

Two nodes of an undirected graph are *connected* if a path joins them. This is an equivalence relation, whose classes are the *connected components*, and a graph is connected if it has one component (by convention, the graph without nodes is connected and has zero components, as the documentation of each function states). Three functions compute the components, i.e., `bool connected(g)`, `int countConnectedComponents(g)` and `int connectedComponents(g, compMap)`. Of these, `connected()` runs one depth-first search from the first node and checks that it has reached all the nodes, while `countConnectedComponents()` starts a breadth-first search from each node not yet reached and counts the searches. In the same way, `connectedComponents()` writes in the writable node map `compMap` the index of the component of each node, from 0 to the number of components minus one, and returns that number; the nodes of one component receive their index consecutively, in the order in which the search reaches them. Each function costs $O(n + m)$, i.e., the cost of one search over the graph.

A graph is *acyclic* (a forest) if it contains no cycle, and it is a *tree* if, furthermore, it is connected. In particular, `acyclic(g)` runs a depth-first search from each unreached node, and it returns `false` at the first arc that leads to a reached node without being the opposite of the arc by which the search entered the current node. Hence, two parallel edges form a cycle of length two for it, and a loop a cycle of length one. Similarly, `tree(g)` makes the same test from the first node only and then checks that all the nodes have been reached; thus, the empty graph is a tree.

A graph is *bipartite* if its nodes can be split into two sets with no edge inside either of them, i.e., if it has no cycle of odd length. One decides it with `bipartite(g)`, while `bool bipartitePartitions(g, partMap)` also writes the two sides into the writable `bool` node map `partMap` (`true` on the side of the first node of each component). Both run a breadth-first search that puts each discovered node on the side opposite to that of its parent, and they stop with `false` as soon as an edge joins two nodes of the same side. In that case `partMap` has been written only on the nodes reached so far, and its values are of no use. Both functions cost $O(n + m)$.

Finally, three functions test whether a graph is simple, and they apply to digraphs and to undirected graphs alike. Here `loopFree(g)` returns `true` if no arc has the same source and target, `parallelFree(g)` if no two arcs that leave a node enter the same node, and `simpleGraph(g)` if both conditions hold. The latter two mark the targets of the arcs leaving a node with the index of that node in a node map; hence, they cost $O(n + m)$ and

need no sorting. Several algorithms of this chapter (cf. §18.5 and §18.6) assume a simple graph (a graph read from a file, e.g., may well have parallel edges), and `simpleGraph()` is the check to run before them.

18.2 Biconnectivity

A connected graph may be disconnected by the removal of one node or of one edge, and the parts that survive such removals measure how "reliable" its connections are (e.g., those of a communication network). A *cut node* (or articulation node) is a node whose removal increases the number of connected components, and a *cut edge* (or bridge) is an edge with the same property. Two edges are in the same *bi-node-connected component*, or *block*, if they lie on a common cycle; this is an equivalence relation on the edges, whose classes are separated by the cut nodes. Two nodes are in the same *bi-edge-connected component* if two edge-disjoint paths join them; this is an equivalence relation on the nodes, whose classes are separated by the bridges. For instance, the graph of Figure 18.1 has four blocks (the triangles A and B, which share the cut node 2, the bridge C and the triangle D), three cut nodes and two bi-edge-connected components.

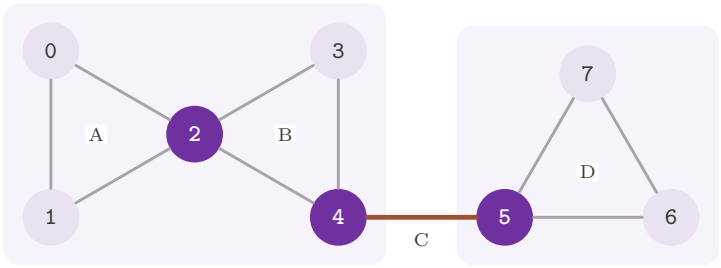


Figure 18.1. A connected graph with four blocks A, B, C and D. The cut nodes 2, 4 and 5 are in the primary colour, the bridge 4-5 in the colour of the added items, and the two bi-edge-connected components, which the bridge separates, are the shaded regions.

The header provides, for the nodes and for the edges, the same kinds of function as for the connected components. For the blocks they are `bool biNodeConnected(g)`, `int countBiNodeConnectedComponents(g)`, `int biNodeConnectedComponents(g, compMap)` and `int biNodeConnectedCutNodes(g, cutMap)`. For the bi-edge-connected components they are `bool biEdgeConnected(g)`, `int countBiEdgeConnectedComponents(g)`, `int biEdgeConnectedComponents(g, compMap)` and `int biEdgeConnectedCutEdges(g, cutMap)`. Note that the map of the blocks is an edge map, since a block is a set of edges and a cut node belongs to several blocks, whereas the map of the bi-edge-connected components is a node map; both are numbered from 0 as in §18.1. Those that find the cut nodes and the cut edges set `cutMap` to `true` on them (once each) and return their number, but they leave the other items untouched, and the caller must therefore initialize the map to `false`. Furthermore, `biNodeConnectedCutNodes()` reads the map in order not to count a node twice, although its concept check requires only a writable map; one should pass a node map of `bool`, as the example of §18.3 does.

At the boundary we follow the conventions of the header. A graph with zero or one node is bi-node-connected, two isolated nodes are not, and two nodes joined by an edge are. In general, `biNodeConnected()` returns `false` if the graph has an isolated node together with some other node, and otherwise whether it has at most one block. On the other hand, `biEdgeConnected()` returns whether there is at most one bi-edge-connected component; hence, the empty graph is bi-edge-connected. A loop forms a separate block, and `biNodeConnectedCutNodes()` reports the node that carries it as a cut node.

All these functions rely on one depth-first search, with the low points of Hopcroft and Tarjan 1973 and Tarjan 1972. We denote by $\mathrm{num}(v)$ the index of v in the order of discovery, and by $\mathrm{low}(v)$ the least index of a node that one can reach from v by going down the tree of the search and then following at most one non-tree edge.

Lemma 18.1. *In a depth-first search of a connected graph, the root is a cut node if and only if it has at least two children in the tree, and any other node u is a cut node if and only if it has a child w with* $\mathrm{low}(w) \geq \mathrm{num}(u)$. *The tree edge uw, with u the parent of w, is a bridge if and only if* $\mathrm{low}(w) > \mathrm{num}(u)$, *where the low point is computed without using the edge uw.*

Its proof rests on the fact that each non-tree edge of a depth-first search of an undirected graph joins a node to one of its ancestors; thus, the subtree of w is attached to the rest of the graph only through u and through the non-tree edges that leave it (see Tarjan 1972). In the library, the visitor of the cut nodes applies the lemma as we show next.

Algorithm 18.1 (Cut nodes by the low points).

```
1  for each node r not yet reached: root := r; rootCut := false; DFS(r)
2  DFS(v): num(v) := low(v) := next index
3     for each arc vw:
4        if w is not reached: pred(w) := v; DFS(w)
5           low(v) := min(low(v), low(w))
6           if num(v) <= low(w):
7              if v != root or rootCut: mark v as a cut node
8              else rootCut := true
9        else if w != pred(v): low(v) := min(low(v), num(w))
```

Step 7 marks the root at the second of its children that satisfy the condition of step 6, the flag `rootCut` recording the first one; since all the children of the root satisfy that condition, this is the rule of Lemma 18.1. Similarly, the count of the blocks grows by one at each tree edge that satisfies the condition of step 6, which closes a block, and the function that numbers the blocks keeps on a stack the edges discovered since the block was opened. Step 9 skips all the edges to the parent, which does not change the blocks. In contrast, the functions on the bi-edge-connected components store the arc by which each node was entered and skip only its opposite, since a second edge parallel to a tree edge does give a path back and prevents that edge from being a bridge. There, a node v with $\mathrm{low}(v) = \mathrm{num}(v)$ closes a component, and the arc that entered it is a bridge. Each function costs $O(n + m)$.

18.3 Strong components and topological order

Two nodes of a digraph are *strongly connected* if directed paths join them in both directions. This is an equivalence relation, whose classes are the *strongly connected components*, and the digraph that has one node per component, and an arc from one component to another whenever some arc of D goes so, is acyclic (we call it the *condensation* of D). The library offers `bool stronglyConnected(d)`, `int countStronglyConnectedComponents(d)`, `int stronglyConnectedComponents(d, compMap)` and `int stronglyConnectedCutArcs(d, cutMap)`, with the conventions of §18.1, i.e., the empty digraph is strongly connected and has zero components. Of these, `stronglyConnected()` runs a depth-first search from the first node on D and another on its reverse (the adaptor `ReverseDigraph` of chapter 5), and it checks that both searches reach all the nodes. Instead, the other three functions use the algorithm usually attributed to Kosaraju and published by Sharir 1981.

Algorithm 18.2 (Strongly connected components by two depth-first searches).

```
1  run a DFS of D from each node not yet reached, and list the nodes
   in the order in which the search leaves them
2  k := 0
3  for each node v in the reverse of that list:
4     if v is not reached by the second search:
5        run a DFS of the reverse of D from v; give index k to the nodes
         it reaches; k := k + 1
6  return k
```

The node that the first search leaves last lies in a component that no arc enters from another component, and in the reverse digraph its search cannot leave that component; by induction, each search of step 5 collects one component (see, e.g., Cormen et al. 2001, §22.5). Thus, the components are found in a topological order of the condensation, and the header guarantees accordingly that no arc goes from a component of higher index to one of lower index. In turn, the cut arcs of `stronglyConnectedCutArcs()` are the arcs whose ends lie in different components, i.e., the arcs of the condensation. This function sets them to `true` in `cutMap`, leaves the other arcs untouched (the map must start at `false`), and returns their number. Each of the four functions costs $O(n + m)$.

A digraph without directed cycles is a *DAG*, and a *topological order* of a DAG is a numbering of its nodes in which each arc goes from a lower to a higher number. Three functions deal with them, i.e., `bool dag(d)`, `void topologicalSort(d, order)` and `bool checkedTopologicalSort(d, order)`. The first one runs a depth-first search from each unreached node and returns `false` at the first arc whose target has been reached but not yet left (a back arc, which closes a cycle; a loop is one). Next, `topologicalSort()` writes in `order` the values from $n - 1$ down to 0, in the order in which a depth-first search leaves the nodes; hence, `order[u] < order[v]` holds for each arc uv of a DAG. It does not check that the digraph is a DAG, which is its precondition. By contrast, `checkedTopologicalSort()` does check it, and it needs a map that can also be read: it first sets

all the values to -1, and it returns `false` at the first arc that leads to a node reached and still at -1, leaving the map partly written. All three cost $O(n+m)$. One usually runs them, or the strongly connected components, before an algorithm that assumes a DAG or a strongly connected digraph; for instance, the minimum mean cycle algorithms of chapter 16 call `stronglyConnectedComponents()` and then work on one component at a time.

In the example we build the graph of Figure 18.1 and check the numbers of its components and of its cut items. We then build a digraph with three strongly connected components, whose numbering we check to be a topological order of the condensation, and we sort a "small" DAG.

```
// The components of section 18.1 to 18.3: an undirected graph of four
// blocks (two triangles sharing node 2, the bridge 4-5 and a triangle), its
// connected, bi-node-connected and bi-edge-connected components with their
// cut nodes and cut edges, and a digraph with its strongly connected
// components and the topological order of a DAG.

#include <iostream>
#include <lemon/list_graph.h>
#include <lemon/connectivity.h>

using namespace lemon;

int main()
{
 int errors = 0;

 // The graph of Figure 18.1
 ListGraph g;
 ListGraph::Node v[ 8 ];
 for( int i = 0 ; i < 8 ; ++i )
  v[ i ] = g.addNode();
 const int ends[ 10 ][ 2 ] = { { 0 , 1 } , { 1 , 2 } , { 2 , 0 } , { 2 , 3 } ,
                               { 3 , 4 } , { 4 , 2 } , { 4 , 5 } , { 5 , 6 } ,
                               { 6 , 7 } , { 7 , 5 } };
 ListGraph::Edge bridge;
 for( int i = 0 ; i < 10 ; ++i ) {
  ListGraph::Edge e = g.addEdge( v[ ends[ i ][ 0 ] ] , v[ ends[ i ][ 1 ] ] );
  if( i == 6 )
   bridge = e;
  }

 ListGraph::NodeMap< int > comp( g );
 int cc = connectedComponents( g , comp );
 ListGraph::EdgeMap< int > block( g );
 int bn = biNodeConnectedComponents( g , block );
 ListGraph::NodeMap< bool > cutNode( g , false );  // must start false
 int cn = biNodeConnectedCutNodes( g , cutNode );
 ListGraph::NodeMap< int > twoEdge( g );
 int be = biEdgeConnectedComponents( g , twoEdge );
 ListGraph::EdgeMap< bool > cutEdge( g , false );  // must start false
 int ce = biEdgeConnectedCutEdges( g , cutEdge );

 std::cout << "connected components: " << cc << ", blocks: " << bn
           << ", cut nodes: " << cn << ", 2-edge-connected components: " << be
           << ", bridges: " << ce << std::endl;
 if( ! connected( g ) || cc != 1 )
  ++errors;
 if( bn != 4 || countBiNodeConnectedComponents( g ) != 4 )
  ++errors;
 if( cn != 3 || ! cutNode[ v[ 2 ] ] || ! cutNode[ v[ 4 ] ] ||
     ! cutNode[ v[ 5 ] ] )
  ++errors;
 if( be != 2 || twoEdge[ v[ 0 ] ] != twoEdge[ v[ 3 ] ] ||
     twoEdge[ v[ 4 ] ] == twoEdge[ v[ 5 ] ] )
  ++errors;
 if( ce != 1 || ! cutEdge[ bridge ] )
```

```
 ++errors;
if( biNodeConnected( g ) || biEdgeConnected( g ) )
 ++errors;
if( acyclic( g ) || tree( g ) || bipartite( g ) || ! simpleGraph( g ) )
 ++errors;

// An even cycle is bipartite, and a path is a tree
ListGraph h;
ListGraph::Node w[ 4 ];
for( int i = 0 ; i < 4 ; ++i )
 w[ i ] = h.addNode();
for( int i = 0 ; i < 3 ; ++i )
 h.addEdge( w[ i ] , w[ i + 1 ] );
if( ! tree( h ) || ! acyclic( h ) )
 ++errors;
h.addEdge( w[ 3 ] , w[ 0 ] );
ListGraph::NodeMap< bool > side( h );
if( ! bipartitePartitions( h , side ) || side[ w[ 0 ] ] == side[ w[ 1 ] ] ||
    side[ w[ 0 ] ] != side[ w[ 2 ] ] || tree( h ) || ! biNodeConnected( h ) )
 ++errors;

// A digraph with three strongly connected components
ListDigraph d;
ListDigraph::Node u[ 6 ];
for( int i = 0 ; i < 6 ; ++i )
 u[ i ] = d.addNode();
const int arcs[ 8 ][ 2 ] = { { 0 , 1 } , { 1 , 2 } , { 2 , 0 } , { 2 , 3 } ,
                             { 3 , 4 } , { 4 , 3 } , { 4 , 5 } , { 1 , 0 } };
for( int i = 0 ; i < 8 ; ++i )
 d.addArc( u[ arcs[ i ][ 0 ] ] , u[ arcs[ i ][ 1 ] ] );
ListDigraph::NodeMap< int > scc( d );
int sn = stronglyConnectedComponents( d , scc );
ListDigraph::ArcMap< bool > cutArc( d , false );
int an = stronglyConnectedCutArcs( d , cutArc );
std::cout << "strongly connected components: " << sn
          << ", arcs between them: " << an << std::endl;
if( sn != 3 || an != 2 || stronglyConnected( d ) || dag( d ) )
 ++errors;
// the numbering of the components is a topological order of them
for( ListDigraph::ArcIt a( d ) ; a != INVALID ; ++a )
 if( scc[ d.source( a ) ] > scc[ d.target( a ) ] )
  ++errors;

// The topological order of a DAG
ListDigraph t;
ListDigraph::Node x[ 4 ];
for( int i = 0 ; i < 4 ; ++i )
 x[ i ] = t.addNode();
t.addArc( x[ 3 ] , x[ 1 ] );
t.addArc( x[ 3 ] , x[ 2 ] );
t.addArc( x[ 1 ] , x[ 0 ] );
t.addArc( x[ 2 ] , x[ 0 ] );
ListDigraph::NodeMap< int > order( t );
if( ! checkedTopologicalSort( t , order ) )
 ++errors;
for( ListDigraph::ArcIt a( t ) ; a != INVALID ; ++a )
 if( order[ t.source( a ) ] >= order[ t.target( a ) ] )
  ++errors;
t.addArc( x[ 0 ] , x[ 3 ] );  // closes a directed cycle
if( dag( t ) || checkedTopologicalSort( t , order ) )
 ++errors;

return( errors == 0 ? 0 : 1 );
}
```

18.4 Euler tours

An *Euler tour* of a graph is a closed walk that traverses each edge once (for a digraph, each arc in its direction). We recall its characterization, which is the oldest theorem of graph theory.

Theorem 18.1. *A graph whose edges lie in one connected component has an Euler tour if and only if each node has even degree. A digraph whose arcs lie in one connected component of the underlying undirected graph has an Euler tour if and only if each node has as many entering arcs as leaving ones.*

Indeed, the proof of sufficiency, due to Hierholzer and Wiener 1873, is also the algorithm. From any node, a walk that never reuses an edge can stop only at its starting node, since each other node it enters has an unused edge to leave by; if the closed walk so found misses some edge, one of its nodes has an unused edge, and a closed walk from that node can be spliced into the first one.

In abLEMON the tour is given by an iterator, `DiEulerIt< GR >` for a digraph and `EulerIt< GR >` for an undirected graph. It is constructed from the graph and, optionally, from the starting node (by default, the first node that has a leaving arc), and it is advanced with `++` until it compares equal to `INVALID`. `DiEulerIt` converts to `GR::Arc`, while `EulerIt` converts to `GR::Arc` and to `GR::Edge`, and it gives arcs in order to show in which direction each edge is traversed. As for the other iterators of the library, the postfix `++` returns the arc instead of the iterator.

Algorithm 18.3 (Euler tour, as `DiEulerIt` produces it).

```
1  for each node v: next(v) := first arc leaving v
2  L := empty list; s := start
3  while next(s) != INVALID: append next(s) to L; advance next(s);
                             s := target of the arc appended
4  (operator*) the current arc is the first of L
5  (operator++) s := target of the first arc of L; remove it from L;
6     while next(s) != INVALID: insert next(s) at the front of the rest
      of L, after the arcs inserted before; advance next(s);
      s := target of that arc
```

The constructor walks greedily from the start (step 3), and each increment splices, right after the arc just given, the closed walk that starts where that arc ends and uses the arcs not yet taken (step 6). Each arc enters the list once and the pointer `next(v)` of a node only advances; hence, the whole tour costs $O(n+m)$, and the list holds the part of the tour not yet given. `EulerIt` works in the same way with an edge map that marks the edges already used, since each edge appears as an arc leaving each of its two ends. If the graph has no Euler tour, the iterators still give a walk (possibly a rather short one), which then is not closed or does not contain all the edges. One should therefore check the property first, with `bool eulerian(g)`, which works on digraphs and on undirected graphs (the two versions are selected through the undirected tag of the graph type). On an undirected graph it returns `true` if each node has even degree (`countIncEdges()`) and the graph is connected. On a digraph it returns `true` if each node has as many entering as leaving arcs and the underlying undirected graph (the adaptor `Undirector`) is connected. We remark that `eulerian()` requires that all the nodes be connected, whereas Theorem 18.1 requires it of the edges only; thus, a graph with an Euler tour and an isolated node is not Eulerian for the function, as its documentation warns and as the example below checks.

```
// The Euler tours of section 18.4: the tour of the complete graph on five
// nodes by EulerIt and that of a digraph made of two directed cycles by
// DiEulerIt, each checked to be a closed walk through every edge or arc
// once, and the test eulerian() on graphs with and without an isolated node.

#include <iostream>
#include <lemon/list_graph.h>
#include <lemon/full_graph.h>
#include <lemon/euler.h>

using namespace lemon;

int main()
{
 int errors = 0;
```

```
// K5: every degree is 4, hence an Euler tour exists
FullGraph k5( 5 );
FullGraph::EdgeMap< int > used( k5 , 0 );
FullGraph::Node first = INVALID , last = INVALID;
int length = 0;
for( EulerIt< FullGraph > e( k5 ) ; e != INVALID ; ++e ) {
 FullGraph::Arc a = e;  // an arc gives the direction of the walk
 if( first == INVALID )
  first = k5.source( a );
 if( last != INVALID && k5.source( a ) != last )
  ++errors;  // not a walk
 last = k5.target( a );
 ++used[ FullGraph::Edge( e ) ];
 ++length;
 }
std::cout << "Euler tour of K5: " << length << " edges" << std::endl;
if( length != 10 || first != last || ! eulerian( k5 ) )
 ++errors;
for( FullGraph::EdgeIt e( k5 ) ; e != INVALID ; ++e )
 if( used[ e ] != 1 )
  ++errors;

// Two directed triangles sharing node 0, started at node 1
ListDigraph d;
ListDigraph::Node n[ 5 ];
for( int i = 0 ; i < 5 ; ++i )
 n[ i ] = d.addNode();
d.addArc( n[ 0 ] , n[ 1 ] );
d.addArc( n[ 1 ] , n[ 2 ] );
d.addArc( n[ 2 ] , n[ 0 ] );
d.addArc( n[ 0 ] , n[ 3 ] );
d.addArc( n[ 3 ] , n[ 4 ] );
d.addArc( n[ 4 ] , n[ 0 ] );
ListDigraph::ArcMap< int > seen( d , 0 );
std::cout << "Euler tour of the digraph:";
ListDigraph::Node at = n[ 1 ];
for( DiEulerIt< ListDigraph > e( d , n[ 1 ] ) ; e != INVALID ; ++e ) {
 if( d.source( e ) != at )
  ++errors;
 at = d.target( e );
 ++seen[ e ];
 std::cout << ' ' << d.id( d.source( e ) ) << "->" << d.id( d.target( e ) );
 }
std::cout << std::endl;
if( at != n[ 1 ] || ! eulerian( d ) )
 ++errors;
for( ListDigraph::ArcIt a( d ) ; a != INVALID ; ++a )
 if( seen[ a ] != 1 )
  ++errors;

// An isolated node makes the digraph not Eulerian (it is not connected),
// although the tour is still there
d.addNode();
if( eulerian( d ) )
 ++errors;
int arcs = 0;
for( DiEulerIt< ListDigraph > e( d ) ; e != INVALID ; ++e )
 ++arcs;
if( arcs != 6 )
 ++errors;

// A path of two edges has two nodes of odd degree
ListGraph p;
ListGraph::Node a = p.addNode() , b = p.addNode() , c = p.addNode();
p.addEdge( a , b );
```

```
p.addEdge( b , c );
if( eulerian( p ) )
 ++errors;

return( errors == 0 ? 0 : 1 );
}
```

18.5 Planarity, embedding and drawing

We recall that a graph is *planar* if it can be drawn in the plane with no two edges crossing. Up to deformation, such a drawing is described by its *combinatorial embedding*, i.e., by the cyclic order of the edges around each node (counterclockwise in the drawing). In turn, the *faces* of the drawing are the orbits of the map that takes an arc uv to the arc that follows vu around v, and a connected plane graph has $f = m - n + 2$ faces (Euler's formula). Since each face of a simple plane graph with $n \geq 3$ nodes is bounded by at least three edges, one has $m \leq 3n - 6$. Hence, a planar simple graph has $O(n)$ edges and a node of degree at most five, a fact used in §18.6. Furthermore, the obstacles to planarity are known.

Theorem 18.2. *A graph is planar if and only if it contains no subdivision of K_5 (the complete graph on five nodes) and no subdivision of $K_{3,3}$ (the complete bipartite graph with three nodes on each side).*

This is the theorem of Kuratowski 1930. A subdivision of either graph inside a nonplanar graph, called a *Kuratowski subdivision*, certifies that the graph is not planar, as an embedding certifies that it is.

We now describe the implementation of the algorithm of Boyer and Myrvold 2004, which computes in linear time either an embedding or a Kuratowski subdivision, and which is usually regarded as one of the simplest linear planarity tests. The function `bool checkPlanarity(g)` runs a simplified version of it that builds neither certificate, and the class `PlanarEmbedding< Graph >` runs the full one. Both expect an undirected simple graph (a precondition of the constructor), which `simpleGraph()` of §18.1 checks. Once the class is constructed from the graph, `run(bool kuratowski = true)` returns whether the graph is planar. If it is, `next(a)` gives the arc that follows the arc `a` in the cyclic order of the arcs leaving its source, and `embeddingMap()` returns the whole order as a `const` reference to an arc map of arcs (the type `EmbeddingMap`, i.e., `Graph::ArcMap< Arc >`). If it is not, and `run()` was called with `true`, `kuratowski(e)` returns whether the edge `e` belongs to the subdivision found, whose nodes of degree three or four are those of a $K_{3,3}$ or of a K_5. According to the header, the running time is $O(n)$, which is the bound for a planar simple graph, whose edges are $O(n)$; on a nonplanar graph the initial depth-first search already costs $O(n + m)$, after which the method stops at the first back edge that it cannot embed.

Algorithm 18.4 (Planarity by edge addition, outline of `PlanarEmbedding::run()`).

```
1  run a DFS; record the order of discovery, the tree edges, the low points
   and, for each node, the least ancestor joined to it by a back edge
2  for each node v in reverse order of discovery:
3     embed each tree edge from v to a child as a separate biconnected
      component, rooted at a virtual copy of v
4     for each back edge from a descendant w to v: walk up from w and mark
      the components that the edge will cross (walkUp)
5     for each marked root: walk down along the external faces, embedding
      the back edges to v and merging the components on the way, flipping
      a component when its orientation requires it (walkDown)
6     if a back edge to v is left unembedded:
         if kuratowski: isolate a Kuratowski subdivision
         return false
7  merge the remaining components and store the cyclic orders; return true
```

We only sketch the invariant of the method, which Boyer and Myrvold 2004 prove: after processing v, the tree, together with the back edges to the nodes processed so far, is embedded with each node that still has a back edge to an unprocessed ancestor on the external face of its component. Step 6 fails only when no embedding of that subgraph has this property, and the obstruction found at that point yields the Kuratowski subdivision. Also, the flips of step 5 are recorded lazily, with one flag per component, which keeps the method linear.

The class `PlanarDrawing< Graph >` computes a straight-line drawing on an integer grid by the method of Schnyder 1990. Its coordinates have the type `dim2::Point< int >` (the type `Point`, stored in the node map

type `PointMap`), all the nodes lie in the square $[0, n-2] \times [0, n-2]$, and the time is $O(n)$ according to the header. Its interface is the constructor from a simple graph and the two functions `run()` and `run(embedding)`. Of these, the former computes an embedding with `PlanarEmbedding` and returns `false` if the graph is not planar; the latter draws according to a given combinatorial embedding (e.g., `embeddingMap()` of a `PlanarEmbedding`) and has no return value. Then the coordinates are read with `operator[](node)` or, as a node map, with `coords()`. Schnyder's method draws a maximal planar graph (a triangulation, with $m = 3n - 6$) from the three trees into which its inner edges decompose. On any other graph, `run(embedding)` copies the graph into an edge set and adds edges, consistently with the embedding, to make it connected, then bi-node-connected and then maximal planar; it draws the triangulation and keeps the coordinates of the nodes. A graph with fewer than three nodes is drawn on a vertical line, with $x = 0$ and $y = 0, 1$. The grid is somewhat small (of side $n - 2$), which is convenient for a picture, although some angles between edges may be quite sharp. Figure 18.2 shows the drawing of the cube that the example below computes.

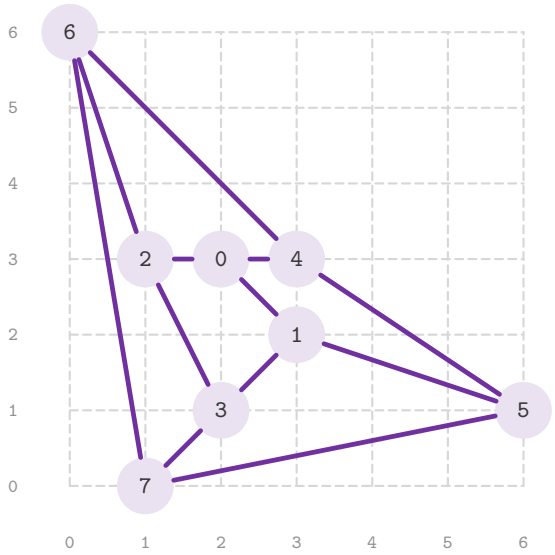


Figure 18.2. The straight-line drawing of the cube computed by `PlanarDrawing` in the example 18-planar.cc: the eight nodes on the grid $[0, 6] \times [0, 6]$, at the coordinates that the example prints, and no two edges crossing.

abLEMON. In LEMON 1.3.1, `PlanarDrawing::run()` without arguments passed the `PlanarEmbedding` object, instead of its embedding map, to `run(embedding)`, and therefore did not compile once instantiated. abLEMON has the correction made in the repository of LEMON after that release (trac #610), as well as the drawing of the graphs with fewer than three nodes (trac #611), on which the algorithm failed.

18.6 Coloring planar graphs

We call *coloring* of a graph an assignment of a color to each node, with different colors at the ends of each edge. As is well known, every planar graph can be colored with four colors, by the theorem of Appel and Haken 1977, whose proof yields a quadratic algorithm that the header judges of no practical use. Deciding whether a planar graph can be colored with three colors is NP-complete, and two colors, when they suffice, are found by `bipartitePartitions()` of §18.1. The class `PlanarColoring< Graph >` computes colorings with six and with five colors, both from the order in which the nodes are removed from the graph when one always takes a node of least degree among the remaining ones (a *smallest-last* order).

Lemma 18.2. *In the smallest-last order of a planar simple graph, each node has at most five neighbors that are removed after it.*

Indeed, when a node is removed it has the least degree in the remaining graph, which is planar and simple, and therefore it has at most five neighbors left, by the bound $m \leq 3n - 6$ of §18.5 (with fewer than three nodes the claim is trivial). Hence, we can give the colors greedily in the reverse of that order.

Algorithm 18.5 (Coloring of a planar graph with six or five colors).

```
1  repeatedly remove a node of least degree in the remaining graph
   (a bucket heap keyed by the degree), and record the order
```

```
2  for each node v in the reverse of that order:
3     give v the least color in {0,..,k-1} not used by its colored
      neighbors (k = 6 or k = 5)
4     if there is none (k = 5): take the first four colored neighbors
      u1, u2, u3, u4 of v in the cyclic order of the embedding;
5        if the Kempe chain of u1 (the component of the colors of u1, u3
         that contains u1) does not contain u3: swap its two colors,
         and give v the old color of u1
6        else swap the colors in the Kempe chain of u2 (colors of u2 and
         u4), and give v the old color of u2
```

By Lemma 18.2, step 3 cannot fail with six colors on a planar graph. With five colors it may fail, but only when the (at most five) colored neighbors use all five colors, and then the two chains of steps 5 and 6 cannot both connect their ends, since they would cross in the embedding; this is the argument of Kempe 1879, which is correct for five colors. As for its interface, `PlanarColoring` has the constructor from a simple graph and three functions. `runSixColoring()` returns `false` if step 3 fails, and then the graph is not planar (although `true` does not prove that it is); `runFiveColoring(embedding)` needs a combinatorial embedding and has no return value; `runFiveColoring()` computes the embedding with `PlanarEmbedding` and returns `false` on a nonplanar graph. One reads the results with `colorIndex(node)`, between 0 and 4 or 5, and with `color(node)`, a `Color` of a palette of six (red, green, blue, yellow, magenta and cyan, in the order of the indices). As maps, `colorIndexMap()` returns a copy of the node map of the indices (the type `IndexMap`), and `colorMap()` the map of the colors (the type `ColorMap`, a `ComposeMap` of the palette and of the indices). In all, the six coloring costs $O(n+m)$, whereas the five coloring adds a breadth-first search of a Kempe chain at each failure of step 3, and its cost is therefore $O(n^2)$ in the worst case, as the header states.

The example computes the embedding of the cube and counts its faces against Euler's formula, draws the cube on the grid and checks that no two edges cross, and colors it with six and with five colors. Finally, it finds the Kuratowski subdivision of $K_{3,3}$, which coincides with $K_{3,3}$.

```
// The planarity algorithms of sections 18.5 and 18.6: the embedding of the
// cube, whose faces are counted against Euler's formula, its straight-line
// drawing on the grid, checked to have no crossing, its colorings with six
// and five colors, and the Kuratowski subdivision found in K3,3.

#include <iostream>
#include <lemon/list_graph.h>
#include <lemon/connectivity.h>
#include <lemon/planarity.h>

using namespace lemon;
typedef ListGraph Graph;
typedef dim2::Point< int > Point;

// The sign of the turn p -> q -> r
long long turn( Point p , Point q , Point r )
{
 return( static_cast< long long >( q.x - p.x ) * ( r.y - p.y ) -
         static_cast< long long >( q.y - p.y ) * ( r.x - p.x ) );
 }

// Whether the open segments pq and rs cross (they share no end here)
bool cross( Point p , Point q , Point r , Point s )
{
 return( ( ( turn( p , q , r ) > 0 ) != ( turn( p , q , s ) > 0 ) ) &&
         ( ( turn( r , s , p ) > 0 ) != ( turn( r , s , q ) > 0 ) ) &&
         turn( p , q , r ) != 0 && turn( p , q , s ) != 0 );
 }

int main()
{
 int errors = 0;

 // The cube: nodes 0..7 are the binary strings of length 3
```

```
Graph g;
Graph::Node v[ 8 ];
for( int i = 0 ; i < 8 ; ++i )
 v[ i ] = g.addNode();
for( int i = 0 ; i < 8 ; ++i )
 for( int b = 1 ; b < 8 ; b <<= 1 )
  if( i < ( i ^ b ) )
   g.addEdge( v[ i ] , v[ i ^ b ] );

PlanarEmbedding< Graph > pe( g );
if( ! pe.run() || ! checkPlanarity( g ) )
 ++errors;

// The faces are the orbits of a -> next(opposite(a))
Graph::ArcMap< int > face( g , -1 );
int faces = 0;
for( Graph::ArcIt a( g ) ; a != INVALID ; ++a ) {
 if( face[ a ] != -1 )
  continue;
 for( Graph::Arc b = a ; face[ b ] == -1 ; b = pe.next( g.oppositeArc( b ) ) )
  face[ b ] = faces;
 ++faces;
 }
std::cout << "faces of the cube: " << faces << std::endl;
if( faces != countEdges( g ) - countNodes( g ) + 2 )
 ++errors;

// The straight-line drawing on the grid
PlanarDrawing< Graph > pd( g );
pd.run( pe.embeddingMap() );
for( int i = 0 ; i < 8 ; ++i )
 std::cout << "node " << i << " at (" << pd[ v[ i ] ].x << ", "
           << pd[ v[ i ] ].y << ")" << std::endl;
for( Graph::EdgeIt e( g ) ; e != INVALID ; ++e )
 for( Graph::EdgeIt f( g ) ; f != INVALID ; ++f ) {
  Graph::Node a = g.u( e ) , b = g.v( e ) , c = g.u( f ) , d = g.v( f );
  if( a == c || a == d || b == c || b == d )
   continue;
  if( cross( pd[ a ] , pd[ b ] , pd[ c ] , pd[ d ] ) )
   ++errors;
  }
for( int i = 0 ; i < 8 ; ++i )
 for( int j = i + 1 ; j < 8 ; ++j )
  if( pd[ v[ i ] ] == pd[ v[ j ] ] )
   ++errors;

// Six colors by the greedy rule, five with the Kempe recoloring
PlanarColoring< Graph > six( g ) , five( g );
if( ! six.runSixColoring() || ! five.runFiveColoring() )
 ++errors;
for( Graph::EdgeIt e( g ) ; e != INVALID ; ++e ) {
 if( six.colorIndex( g.u( e ) ) == six.colorIndex( g.v( e ) ) )
  ++errors;
 if( five.colorIndex( g.u( e ) ) == five.colorIndex( g.v( e ) ) )
  ++errors;
 }
for( int i = 0 ; i < 8 ; ++i )
 if( five.colorIndex( v[ i ] ) < 0 || five.colorIndex( v[ i ] ) > 4 )
  ++errors;

// K3,3 is not planar, and the subdivision found is K3,3 itself
Graph k;
Graph::Node r[ 3 ] , s[ 3 ];
for( int i = 0 ; i < 3 ; ++i )
 r[ i ] = k.addNode() , s[ i ] = k.addNode();
```

```
for( int i = 0 ; i < 3 ; ++i )
 for( int j = 0 ; j < 3 ; ++j )
  k.addEdge( r[ i ] , s[ j ] );
PlanarEmbedding< Graph > pk( k );
if( pk.run() || checkPlanarity( k ) )
 ++errors;
int marked = 0;
for( Graph::EdgeIt e( k ) ; e != INVALID ; ++e )
 if( pk.kuratowski( e ) )
  ++marked;
std::cout << "edges of the Kuratowski subdivision of K3,3: " << marked
          << std::endl;
if( marked != 9 )
 ++errors;

return( errors == 0 ? 0 : 1 );
}
```

18.7 Subgraph isomorphism: VF2 and VF2++

Given two undirected graphs $G_1 = (V_1, E_1)$ and $G_2 = (V_2, E_2)$, a *mapping* (or embedding) is an injective function $f : V_1 \to V_2$. In the *Subgraph Isomorphism Problem* one looks for a mapping with $f(u)f(v) \in E_2$ whenever $uv \in E_1$, and in the *Induced Subgraph Isomorphism Problem* one requires, furthermore, that $f(u)f(v) \notin E_2$ whenever $uv \notin E_1$. In the *Graph Isomorphism Problem* the induced mapping must also be bijective. When the nodes carry labels $l_1 : V_1 \to L$ and $l_2 : V_2 \to L$, one also requires that $l_1(u) = l_2(f(u))$ for each $u \in V_1$. An enumeration `MappingType` (in the namespace `lemon`, from `lemon/bits/vf2_internals.h`) selects the problem, with the values `SUBGRAPH` (the default), `INDUCED` and `ISOMORPH`. According to the header, `ISOMORPH` is equivalent to `INDUCED` when the two graphs have the same number of nodes, and to `SUBGRAPH` when they also have the same number of edges, but it may be faster than both, since its cutting rules are stronger.

Of course, the subgraph isomorphism problem is NP-complete, since it contains the clique problem. Hence, the two algorithms of this section are backtracking searches that may take exponential time, and their speed in practice depends mostly (and often decisively) on the order in which they consider the nodes of G_1 and on the rules with which they cut the search tree. We now give the search common to both.

Algorithm 18.6 (VF2 and VF2++ search).

```
1  order the nodes of G1 as v_0, .., v_{n1-1} (VF2: breadth first from
   each unreached node; VF2++: see the text)
2  depth := 0; all the nodes of G1 unmapped
3  while depth >= 0:
4     if depth = n1: report the mapping (find() returns true); depth -= 1
5     u := v_depth; take the next candidate w for u in G2 (a neighbor of
      the image of an already mapped neighbor of u if u has one, else any
      node of G2 compatible with the type), after the one tried last
6     if there is none: unmap u; depth -= 1; continue
7     if feasible(u, w): map u to w; depth += 1
```

The test `feasible(u, w)` checks the label (or the equivalence of the nodes), and it checks that the edges between u and the mapped nodes correspond, as the type requires, to the edges between w and their images. It then looks ahead at the neighbors of u and of w that are adjacent to the mapped part (and, for `INDUCED` and `ISOMORPH`, at those that are not): their numbers, precomputed for u along the order, must not be larger for u than for w, and must be equal for `ISOMORPH`. Each call of `find()` resumes the search where the previous one stopped, so that repeated calls give each mapping once. Note that the search is a loop with an explicit depth, and the stack of the program does not grow with the size of the graphs.

The class `Vf2< G1, G2, M, NEQ >` implements the algorithm of Cordella et al. 2004. Its template parameters are the type of the graph to be embedded (`G1`, by default `ListGraph`) and that of the graph it is embedded into (`G2`, by default `ListGraph`), the type of the read-write node map of G_1 that stores the mapping (`M`, by default `G1::NodeMap< G2::Node >`), and a binary functor on a node of G_1 and one of G_2 that decides whether the first may be mapped to the second (`NEQ`, by default a functor that always returns `true`). Its constructor `Vf2(g1, g2, m, neq = NEQ())` computes the order of step 1 and sets all the values of `m` to `INVALID`. Then `mappingType()`

returns the type and `mappingType(t)` sets it, and `find()` searches for the next mapping, writes it into `m` and returns `true`, or returns `false` when none is left.

The class `Vf2pp< G1, G2, M, M1, M2 >` implements the algorithm of Jüttner and Madarasi 2018, which differs from VF2 in two respects. First, its labels are integer node maps (`M1` and `M2`, by default `G1::NodeMap< int >` and `G2::NodeMap< int >`), whose values must be $0, 1, \ldots, K-1$ for K distinct labels, since the algorithm keeps vectors indexed by the label; there is no functor of node equivalence. Second, the order of step 1 is chosen to "fail early". Each breadth-first tree starts from a node whose label is the "rarest" among the nodes of G_2 (among those, one of largest degree), and within each level of the tree the nodes are taken by most neighbors already ordered, then by largest degree, then by rarest label; the look-ahead of `feasible()` counts the neighbors label by label. According to the experiments of Jüttner and Madarasi 2018, these two changes typically reduce the search considerably, especially on labeled graphs; we have not measured the two classes against each other, and on "easy" instances the difference may well be small. Its constructor is `Vf2pp(g1, g2, m, labels1, labels2)`, and `mappingType()` and `find()` are as for `Vf2`.

Both algorithms also have a function-type interface, `vf2(g1, g2)` and `vf2pp(g1, g2)`, which return objects of the classes `Vf2Wizard` and `Vf2ppWizard`. Their named parameters are `mapping(m)`, the map that receives the result (by default, a map is allocated and deleted inside), `nodeLabels(l1, l2)`, two node maps whose values `vf2()` compares with `==` and which `vf2pp()` takes as integer labels as above (without them, `vf2()` may map any node to any node, and `vf2pp()` gives all the nodes the label 0), `nodeEq(f)`, the functor (for `vf2()` only), and `mappingType(t)`, `induced()` and `iso()`. Such an expression ends with `run()`, which returns whether a mapping exists and leaves it in the map given by `mapping()`, with `count()`, which returns the number of mappings, or with `getPtrToVf2Object()` (respectively `getPtrToVf2ppObject()`). This last function returns a pointer to a new algorithm object, on which one calls `find()` repeatedly and which the caller deletes; the object then deletes the maps that the wizard allocated, if any. We remark that both classes work on undirected graphs only (they iterate on the incident edges), and that the count of the mappings onto a graph with symmetries includes its automorphisms (e.g., the cycle on five nodes has 10 automorphisms). Of course, on the same input the two algorithms give the same set of mappings, possibly in another order, and `Vf2` remains the one to use when a functor of node equivalence is more "natural" than integer labels.

abLEMON. `Vf2` and `Vf2pp` were added to the repository of LEMON after the release of 1.3.1 (trac #597) and were never in a release of LEMON, so that abLEMON is the first release that contains them. Furthermore, abLEMON reads the connection counter of the image of a neighbor only when the neighbor is mapped, where the code indexed a node map with `INVALID` (trac #672); `getPtrToVf2Object()` and `getPtrToVf2ppObject()` compile, the wizards being friends of the classes whose flags of deallocation they set (trac #682); and `lemon/vf2pp.h` includes `lemon/list_graph.h`, whose `ListGraph` its default arguments name, so that each of the two headers is self-contained.

The example counts the automorphisms of the cycle on five nodes and the embeddings of a path of three nodes into the cycle on four nodes and into K_4, as subgraphs and as induced subgraphs; into K_4 there are 24 of the former and none of the latter, since any two nodes of K_4 are adjacent. It then repeats the counts with labels, and it enumerates the mappings one by one with `find()`, checking each of them.

```
// The (sub)graph isomorphism of section 18.7: the automorphisms of the
// cycle on five nodes, the embeddings of a path of three nodes into the
// cycle on four nodes and into K4 (as a subgraph and as an induced
// subgraph), the same counts with node labels, by Vf2 and by Vf2pp, and the
// enumeration of the mappings one by one with find().

#include <iostream>
#include <lemon/list_graph.h>
#include <lemon/vf2.h>
#include <lemon/vf2pp.h>

using namespace lemon;
typedef ListGraph Graph;

void cycle( Graph & g , int n )
{
 std::vector< Graph::Node > v;
 for( int i = 0 ; i < n ; ++i )
  v.push_back( g.addNode() );
 for( int i = 0 ; i < n ; ++i )
```

```
  g.addEdge( v[ i ] , v[ ( i + 1 ) % n ] );
 }

int main()
{
 int errors = 0;

 Graph c5 , c4 , k4 , p3;
 cycle( c5 , 5 );
 cycle( c4 , 4 );
 for( int i = 0 ; i < 4 ; ++i )
  k4.addNode();
 for( Graph::NodeIt a( k4 ) ; a != INVALID ; ++a )
  for( Graph::NodeIt b( k4 ) ; b != INVALID ; ++b )
   if( a < b )
    k4.addEdge( a , b );
 Graph::Node x = p3.addNode() , y = p3.addNode() , z = p3.addNode();
 p3.addEdge( x , y );
 p3.addEdge( y , z );

 // The automorphisms of C5 are its 10 rotations and reflections
 int a1 = vf2( c5 , c5 ).iso().count() , a2 = vf2pp( c5 , c5 ).iso().count();
 std::cout << "automorphisms of C5: " << a1 << ' ' << a2 << std::endl;
 if( a1 != 10 || a2 != 10 )
  ++errors;

 // A path of three nodes into C4 and into K4
 int sc4 = vf2( p3 , c4 ).count() , ic4 = vf2( p3 , c4 ).induced().count();
 int sk4 = vf2pp( p3 , k4 ).count() , ik4 = vf2pp( p3 , k4 ).induced().count();
 std::cout << "P3 into C4: " << sc4 << " subgraph, " << ic4
           << " induced; into K4: " << sk4 << " subgraph, " << ik4
           << " induced" << std::endl;
 if( sc4 != 8 || ic4 != 8 || sk4 != 24 || ik4 != 0 )
  ++errors;

 // Labels: the middle node of the path must go to node 0 of K4
 Graph::NodeMap< int > lp( p3 , 0 ) , lk( k4 , 0 );
 lp[ y ] = 1;
 lk[ Graph::NodeIt( k4 ) ] = 1;
 int l1 = vf2( p3 , k4 ).nodeLabels( lp , lk ).count();
 int l2 = vf2pp( p3 , k4 ).nodeLabels( lp , lk ).count();
 std::cout << "with labels: " << l1 << ' ' << l2 << std::endl;
 if( l1 != 6 || l2 != 6 )
  ++errors;

 // All the mappings one by one, each checked to be an embedding
 Graph::NodeMap< Graph::Node > m( p3 );
 Vf2< Graph , Graph > alg( p3 , c4 , m );
 int found = 0;
 while( alg.find() ) {
  ++found;
  for( Graph::EdgeIt e( p3 ) ; e != INVALID ; ++e )
   if( findEdge( c4 , m[ p3.u( e ) ] , m[ p3.v( e ) ] ) == INVALID )
    ++errors;
  if( m[ x ] == m[ y ] || m[ y ] == m[ z ] || m[ x ] == m[ z ] )
   ++errors;
  }
 if( found != 8 )
  ++errors;

 // The same through the wizard, which allocates the object
 Vf2pp< Graph , Graph > * p =
   vf2pp( p3 , c4 ).mapping( m ).getPtrToVf2ppObject();
 found = 0;
 while( p->find() )
```

```
 ++found;
delete p;
if( found != 8 )
 ++errors;

return( errors == 0 ? 0 : 1 );
}
```

18.8 Exercises

Exercise 18.1. Add the edge 1-6 to the graph of Figure 18.1, and predict by Lemma 18.1 the blocks, the cut nodes and the bridges of the new graph. Check the prediction with the functions of §18.2, starting from the example 18-components.cc (the test `test/connectivity_test.cc` checks the same functions on other graphs).

Exercise 18.2. Build the condensation of a digraph from the map of `stronglyConnectedComponents()`, with one node per component and one arc per cut arc of `stronglyConnectedCutArcs()`. Verify with `dag()` and `checkedTopologicalSort()` that it is acyclic and that the indices of the components are a topological order of it (the example 18-components.cc checks the second property on the original digraph).

Exercise 18.3. A connected graph with two nodes of odd degree, s and t, has an Euler path from s to t. Obtain it with `EulerIt` by adding a new edge ts, starting the tour at s and dropping the new edge from the walk, and check the result as the example 18-euler.cc checks a tour (the test `test/euler_test.cc` checks the iterators on several graphs).

Exercise 18.4. Show that the faces of an embedding of a planar graph with c connected components, counted as in the example 18-planar.cc (one orbit per face of each component), satisfy $f + n - c = m + 1$, and verify it on a graph with two components; the test `test/planarity_test.cc` checks this identity in `checkEmbedding()`.

Exercise 18.5. Run `PlanarEmbedding` on K_5 and on the Petersen graph, and check that the edges marked by `kuratowski()` form a subdivision of K_5 (five nodes of degree four) or of $K_{3,3}$ (six nodes of degree three), the other nodes having degree zero or two in it; the test `test/planarity_test.cc` makes this check in `checkKuratowski()`.

Exercise 18.6. Count with `vf2(g, g).iso().count()` and with `vf2pp(g, g).iso().count()` the automorphisms of the cube and of the Petersen graph (48 and 120), and compare the number of calls of `find()` with and without labels that distinguish one node. The example 18-isomorphism.cc does the same on the cycle on five nodes, and the test `test/vf2_test.cc` checks the mappings found by both classes.

Exercise 18.7. Show that, for a pattern G_1 that is a complete graph, the counts of the `SUBGRAPH` and of the `INDUCED` mappings into any G_2 coincide, and that, for a pattern without edges, the `INDUCED` mappings correspond to the ordered independent sets of G_2 of that size. Check both claims on small graphs, starting from the example 18-isomorphism.cc.

19. Travelling salesman and maximum clique

This chapter describes two families of algorithms for NP-hard problems, for which the library mostly offers heuristics. The first family consists of the five tour construction and improvement methods for the symmetric Travelling Salesman Problem (TSP) of `nearest_neighbor_tsp.h`, `greedy_tsp.h`, `insertion_tsp.h`, `christofides_tsp.h` and `opt2_tsp.h`, which share one interface (§19.1 to §19.4). The second consists of the two algorithms for the maximum clique problem of `grosso_locatelli_pullan_mc.h` and `tomita_seki_mc.h`, an iterated local search and an exact branch and bound (§19.5 to §19.7). None of these classes takes a tolerance or uses the named template parameters of chapter 3: the TSP classes are parameterized by the type of the cost map, and the clique classes by the type of the graph.

19.1 The symmetric travelling salesman problem

We are concerned with an undirected complete graph $K_n = (V, E)$, $|V| = n$, whose edges carry a cost $c : E \to \mathbb{R}$, and with the problem of finding a Hamiltonian cycle (a *tour*) of least cost, i.e.,

$$\min \Big\{ \sum_{i=0}^{n-1} c(v_{\pi(i)}, v_{\pi(i+1 \bmod n)}) \;:\; \pi \text{ a permutation of } \{0, \dots, n-1\} \Big\}. \tag{19.1}$$

The cost is *metric* when it satisfies the triangle inequality $c(u, w) \le c(u, v) + c(v, w)$ for all u, v, w. The documentation of the group states that the TSP classes are intended for metric costs, and that on other costs they may give worse tours; this is a statement on quality, since no class checks the inequality and each of them returns a tour whatever the costs are. Of course, (19.1) is NP-hard, and without the triangle inequality it cannot be approximated within any constant factor unless P = NP. With it, the best guarantee among the classes of the library is the factor 3/2 of Christofides' algorithm (Theorem 19.1).

All five classes run on a `FullGraph` (chapter 4), the complete undirected graph in which `edge(u, v)` finds the edge between two nodes in constant time, and they have the same interface, which we describe once. The class template takes one parameter `CM`, the type of the cost map, i.e., a readable edge map of `FullGraph`; its value type is exported as `Cost`, and the map type as `CostMap`. The constructor takes the graph and the cost map and keeps references to them, which must therefore live as long as the object.

A call of `run()` computes a tour and returns its cost, which `tourCost()` gives again afterwards. Then `tourNodes()` returns a constant reference to the `std::vector< FullGraph::Node >` of the nodes in the order of the tour, `tourNodes(out)` copies this sequence through an output iterator, and `tour(path)` clears a path structure (chapter 8) and fills it with the n arcs of the tour. The last of these arcs goes from the last node of the sequence back to the first. The returned cost is recomputed at the end as the sum of the n edges of the sequence, and hence it agrees with `tourNodes()` up to the order of the floating-point sums. The degenerate graphs are handled alike by all the classes: with no node the tour is empty and costs 0, with one node it consists of that node and costs 0, and with two nodes it visits both and costs twice their edge.

19.2 Tour construction heuristics

We now describe the four classes that build a tour from scratch, ordered by the effort they spend. In the notation of (19.1), T is the current partial tour or path, $d(w, T) = \min_{t \in T} c(w, t)$ is the distance of a node w from it, and

$$\delta(u, v; w) = c(u, w) + c(w, v) - c(u, v) \tag{19.2}$$

is the increase of cost when w is inserted between the consecutive nodes u and v of T.

19.2.1 Nearest neighbor

Our first class, `NearestNeighborTsp< CM >`, implements the nearest neighbor heuristic in the variant that grows a path at both of its ends. It starts from an edge of minimum cost, found by `mapMin()` over all the edges; at each step it appends to the end whose nearest unvisited node is nearer, until the path contains all the nodes. The closing edge then turns the path into a tour.

Algorithm 19.1 (Nearest neighbor, two-ended).

```
1  let {n1, n2} be an edge of minimum cost; P := (n1, n2)
2  while |P| < n:
3      e1 := cheapest edge from n1 to an unvisited node   (kept until invalid)
4      e2 := cheapest edge from n2 to an unvisited node   (kept until invalid)
5      if c(e1) < c(e2): prepend the other end w of e1 to P; n1 := w
6      else:             append the other end w of e2 to P; n2 := w
7  close P with the edge {n2, n1}
```

Note that the candidate edge of an end is searched again only when it has been used, or when its other end has just been reached from the opposite end (lines 3 and 4). Thus, each step scans at most two stars of $n - 1$ edges, and the algorithm runs in $O(n^2)$ time, as the header states. In practice the tours are usually "reasonable", but on special instances they may be very bad. For metric costs, Rosenkrantz, Stearns and Lewis 1977 proved that a nearest neighbor tour costs at most $(\lceil \log_2 n \rceil + 1)/2$ times the optimal value, and gave instances on which the ratio exceeds $\log_2(n + 1)/3 + 4/9$. Their analysis, however, concerns the classical one-ended variant, which differs from the one of the library.

19.2.2 Greedy edge selection

Next, `GreedyTsp< CM >` builds a tour by the greedy matching heuristic. It sorts all the edges by nondecreasing cost and scans them; an edge is kept when both of its ends have degree at most one among the edges already kept and it does not close a cycle, which a union-find structure on the nodes (chapter 8) detects. After $n - 1$ edges have been kept they form a Hamiltonian path, whose two ends of degree one are joined to close the tour.

Algorithm 19.2 (Greedy).

```
1  sort the edges by nondecreasing cost
2  F := {}; each node is a singleton of the union-find structure
3  for each edge {u, v} in that order, until |F| = n - 1:
4      if deg_F(u) <= 1 and deg_F(v) <= 1 and join(u, v) merges two sets:
5          F := F + {u, v}
6  join the two nodes of degree one of F
```

According to the header, the running time is $O(n^2)$. We remark, however, that line 1 applies `std::sort` to the $n(n - 1)/2$ edges of the complete graph, which makes the worst case $O(n^2 \log n)$; the scan of lines 3 to 5 costs $O(n^2\, \alpha(n))$. As for the nearest neighbor method, the tours are usually short but may be very bad in special cases.

19.2.3 Insertion

In `InsertionTsp< CM >` one maintains a subtour T, initially of two nodes, and inserts the other nodes one at a time, each at the position that minimizes (19.2). The node that enters next is decided by one of the four selection rules of the enumeration `SelectionRule`, passed to `run(rule)`; the default is `FARTHEST`.

Rule	Initial subtour	Node selected at each step	Time
`NEAREST`	ends of an edge of minimum cost	an unvisited w of least $d(w, T)$	$O(n^2)$
`FARTHEST` (default)	ends of an edge of maximum cost	an unvisited w of largest $d(w, T)$	$O(n^2)$
`CHEAPEST`	ends of an edge of minimum cost	the unvisited w and position of least $\delta(u, v; w)$	$O(n^3)$

Rule	Initial subtour	Node selected at each step	Time
`RANDOM`	ends of an edge of minimum cost	an unvisited node drawn uniformly by the global `rnd`	$O(n^2)$

Table 19.1. The selection rules of `InsertionTsp`, as `init()` and the selection functors of the header implement them.

Algorithm 19.3 (Insertion).

```
1  T := the two ends of the initial edge of the rule (Table 19.1)
2  for each unvisited w: d(w) := d(w, T)              (NEAREST, FARTHEST)
                         or best (delta, position)    (CHEAPEST)
3  repeat n - 2 times:
4      select w by the rule; remove it from the unvisited nodes
5      insert w between the consecutive u, v of T of least delta(u, v; w)
6      update d(x) := min{ d(x), c(w, x) } for the unvisited x
                    (or, for CHEAPEST, the best insertion of each x)
```

For `NEAREST`, `FARTHEST` and `RANDOM`, the selection and the update of d cost $O(n)$ per step, and so do the search of the position and the insertion into the vector of the subtour; hence, the running time is $O(n^2)$. For `CHEAPEST`, each unvisited node keeps its best insertion cost and position. After an insertion, only the two new pairs of consecutive nodes are compared with it, unless the pair it would have been inserted into has just been broken; in this case, its best position is searched again in $O(n)$. A step therefore costs $O(n^2)$ in the worst case, and the algorithm $O(n^3)$, although the header remarks that in most cases it is almost as fast as with the other rules. In the code, the rules are private functors passed to `start()`: `ComparingSelection` (with `std::less` for `NEAREST` and `std::greater` for `FARTHEST`), `CheapestSelection` and `RandomSelection` select the node, and `DefaultInsertion` inserts it at its best position, while `CheapestInsertion` only adds the cost that `CheapestSelection` has already computed, the insertion being done by the latter.

As for `RANDOM`, it draws from the global generator `rnd` (chapter 8), which one may seed before `run()` to make the tour reproducible. The header describes `FARTHEST` as typically the most effective rule, and `RANDOM` as usually slightly worse but less sensitive to the instance. For metric costs, the nearest and the cheapest insertion give tours of cost at most twice the optimal one (Rosenkrantz, Stearns and Lewis 1977). For the farthest insertion only the logarithmic bound of any order of insertion is known, although in practice it is often (but not always) the best of the four rules.

19.2.4 Christofides' algorithm

Finally, `ChristofidesTsp< CM >` implements the heuristic of Christofides 1976. It combines a minimum cost spanning tree with a minimum cost perfect matching on the nodes of odd degree of the tree (Figure 19.1), and it is the only class of the family with a guaranteed approximation factor.

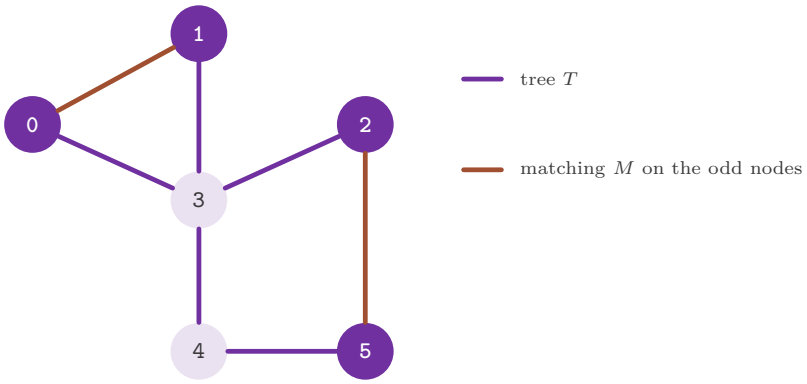


Figure 19.1. Christofides' algorithm: the minimum spanning tree T (primary), its nodes of odd degree (primary), and the minimum cost perfect matching M on them (the added colour). Each node has even degree in $T \cup M$, whose Euler tour, without the nodes already visited, is the tour.

Algorithm 19.4 (Christofides).

```
1  T := minimum cost spanning tree of K_n                  (kruskal)
2  O := nodes of odd degree in T                           (|O| is even)
3  M := minimum cost perfect matching of the complete graph on O
         (MaxWeightedPerfectMatching on the costs -c, on a SmartGraph)
4  H := the multigraph T + M on a SmartGraph               (all degrees even)
5  walk the Euler tour of H (EulerIt); append each node when first reached
```

In the code, the tree is computed by `kruskal()` (chapter 11), and the closed walk by `EulerIt` (chapter 18). The matching is computed by `MaxWeightedPerfectMatching` (chapter 17) on a copy of the subgraph induced by O whose edge weights are the negated costs; for this reason, the header warns that `CM::Value` must be a signed number type. Line 5 skips the nodes already visited, i.e., it replaces a subpath by the direct edge between its ends, which does not increase the cost when the triangle inequality holds. This gives the guarantee below, proved in Christofides 1976.

Theorem 19.1. *If c is metric, the tour returned by Algorithm 19.4 costs at most $3/2$ times the optimal value of (19.1).*

We recall that the proof rests on two facts. First, $c(T)$ is at most the optimal value, since removing an edge from an optimal tour leaves a spanning tree. Second, $c(M)$ is at most half of it, since an optimal tour shortcut to the nodes of O splits into two perfect matchings of O. The running time is dominated by the matching, and the header gives $O(n^3 \log n)$ for the whole method. In practice, the tours are usually much better than the bound: in the example of §19.4 the tour costs less than 1.3 times the spanning tree.

19.3 Improvement by 2-opt

We now turn to `Opt2Tsp< CM >`, which implements the 2-opt local search of Croes 1958. Given a tour, it looks for two of its edges $\{u, u^+\}$ and $\{v, v^+\}$, with u^+ and v^+ the successors of u and v, such that

$$c(u, u^+) + c(v, v^+) > c(u, v) + c(u^+, v^+), \tag{19.3}$$

and replaces them by $\{u, v\}$ and $\{u^+, v^+\}$; this reverses the path from u^+ to v and decreases the cost (Figure 19.2). The search stops when no pair satisfies (19.3), i.e., at a *2-optimal* tour.

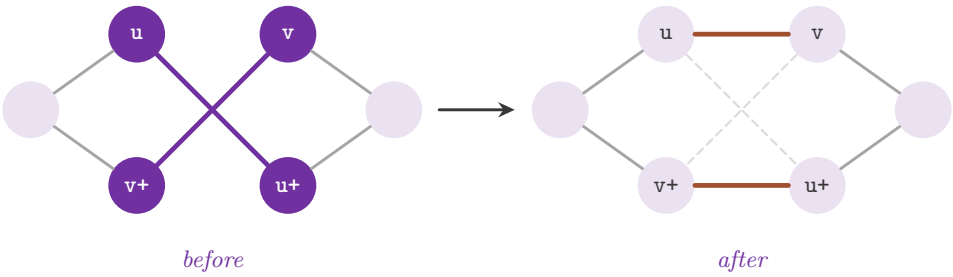


Figure 19.2. A 2-opt move: the two edges of the tour that cross (primary) are replaced by the two that do not (the added colour), and the path between u^+ and v is traversed in the opposite direction.

There are three ways to give the starting tour. A call of `run()` starts from the order of the node ids; `run(path)` starts from a path structure, which must be a valid path of n arcs (cf. `checkPath()` in chapter 8); and `run(nodes)` starts from a `std::vector< Node >` that holds all the nodes in the order of the tour. The typical use is the improvement of the tour of a construction heuristic, e.g., `opt2.run(ins.tourNodes())`. The tour is kept as a list of two neighbors per node, without an orientation; hence, a move rewires four entries and costs $O(1)$, with no explicit reversal of a path.

Algorithm 19.5 (2-opt, first improvement).

```
1  restart:
2  for each node u of the tour, from node 0 in the order of the tour:
3      for each v after u+ in the tour, before the predecessor of u:
4          if (19.3) holds for (u, u+) and (v, v+):
5              replace {u,u+}, {v,v+} by {u,v}, {u+,v+}   (O(1) rewiring)
6              goto restart
```

```
7  the tour is 2-optimal; list it from node 0 and recompute its cost
```

Each pass takes the first improving move, and it restarts from the first node after each move (the `goto` of `start()`). A pass without moves thus costs $O(n^2)$ evaluations of (19.3), and each move is paid by the scan that precedes it. The number of moves is not polynomially bounded in general, and the header describes the method as rather slow but effective. Note that (19.3) is a strict comparison without tolerance; for floating-point costs, a move whose gain is below the rounding of the four costs may or may not be taken.

On points of the plane with Euclidean costs, two edges that cross always satisfy (19.3). A 2-optimal tour has therefore no crossing edges, and on points in convex position it is the optimal tour, i.e., the boundary of their convex hull (Exercise 19.2); the example of §19.4 checks this.

19.4 Choosing a heuristic, and an example

We summarize the trade-off as the documentation of the group does. The fastest methods are `NearestNeighborTsp`, `GreedyTsp` and `InsertionTsp`, and the insertion is usually quite effective; `ChristofidesTsp` is somewhat slower, but it has the best guarantee. Finally, `Opt2Tsp` is the slowest, it usually gives the best tours in practice, and it can improve the tour of any other method.

On random Euclidean instances, the farthest insertion followed by 2-opt is a sensible default. The run of the example below on 40 points of the unit square illustrates this (on one instance, and therefore only as an illustration). The nearest neighbor and the greedy tours cost 5.58 and 5.33, the four insertion rules give 5.37 (nearest), 4.76 (farthest), 5.20 (cheapest) and 4.93 (random), and Christofides' tour costs 4.90, against 3.81 for the spanning tree. Also, 2-opt brings the farthest insertion tour to 4.76, and the other three to between 4.83 and 5.21.

Our program first takes twelve points of a circle, numbered out of their angular order. On them, each tour must cost at least the perimeter of the regular polygon, Christofides' tour at most 3/2 of it, and the 2-opt tour (whose starting order, by node id, is far from the polygon) cost the perimeter. Then the program runs each method on the random points. It checks that each tour visits each node once and costs the returned value, that it costs at least the minimum spanning tree, and that Christofides' tour costs at most twice the tree, a consequence of the proof of Theorem 19.1.

```
// The five TSP heuristics of chapter 19 on twelve points of a circle,
// numbered out of their angular order: the optimal tour is the regular
// polygon, every tour must cost at least its perimeter, Christofides'
// tour at most 3/2 of it, and the 2-opt tour, which on points in convex
// position has no crossing edge, exactly the perimeter. Then on 40 random
// points of the unit square, where the heuristics differ, every tour must
// cost at least the minimum spanning tree, Christofides' tour at most twice
// it, and 2-opt must not worsen the tour it starts from.

#include <cmath>
#include <iostream>
#include <vector>
#include <lemon/full_graph.h>
#include <lemon/path.h>
#include <lemon/nearest_neighbor_tsp.h>
#include <lemon/greedy_tsp.h>
#include <lemon/insertion_tsp.h>
#include <lemon/christofides_tsp.h>
#include <lemon/opt2_tsp.h>
#include <lemon/kruskal.h>
#include <lemon/random.h>

using namespace lemon;

typedef FullGraph::EdgeMap< double > CostMap;

// whether the sequence visits every node once and its cost is c
bool checkTour( const FullGraph & g , const CostMap & cost ,
                const std::vector< FullGraph::Node > & tour , double c )
{
```

```
 if( int( tour.size() ) != g.nodeNum() )
  return( false );
 FullGraph::NodeMap< bool > seen( g , false );
 double sum = 0;
 for( std::size_t i = 0 ; i < tour.size() ; ++i ) {
  if( seen[ tour[ i ] ] )
   return( false );
  seen[ tour[ i ] ] = true;
  sum += cost[ g.edge( tour[ i ] , tour[ ( i + 1 ) % tour.size() ] ) ];
  }
 return( std::fabs( sum - c ) < 1e-9 );
 }

int main()
{
 const int n = 12;
 const double pi = 3.14159265358979323846;
 FullGraph g( n );
 std::vector< double > x( n ) , y( n );
 for( int i = 0 ; i < n ; ++i ) {  // node i sits at the position
  int pos = ( 5 * i ) % n;       // 5 i mod 12 of the polygon
  x[ i ] = std::cos( 2 * pi * pos / n );
  y[ i ] = std::sin( 2 * pi * pos / n );
  }
 CostMap cost( g );
 for( FullGraph::EdgeIt e( g ) ; e != INVALID ; ++e ) {
  int u = g.id( g.u( e ) ) , v = g.id( g.v( e ) );
  cost[ e ] = std::hypot( x[ u ] - x[ v ] , y[ u ] - y[ v ] );
  }
 const double opt = n * 2 * std::sin( pi / n );  // the perimeter
 bool ok = true;

 NearestNeighborTsp< CostMap > nn( g , cost );
 GreedyTsp< CostMap > gr( g , cost );
 InsertionTsp< CostMap > ins( g , cost );
 ChristofidesTsp< CostMap > chr( g , cost );
 double c_nn = nn.run() , c_gr = gr.run() , c_ins = ins.run() ,
        c_chr = chr.run();  // FARTHEST by default
 ok = ok && checkTour( g , cost , nn.tourNodes() , c_nn ) &&
      checkTour( g , cost , gr.tourNodes() , c_gr ) &&
      checkTour( g , cost , ins.tourNodes() , c_ins ) &&
      checkTour( g , cost , chr.tourNodes() , c_chr );
 ok = ok && c_nn >= opt - 1e-9 && c_gr >= opt - 1e-9 && c_ins >= opt - 1e-9 &&
      c_chr <= 1.5 * opt + 1e-9;

 // 2-opt from the order of the ids, and from the insertion tour
 Opt2Tsp< CostMap > opt2( g , cost );
 double c_id = opt2.run();
 ok = ok && checkTour( g , cost , opt2.tourNodes() , c_id ) &&
      std::fabs( c_id - opt ) < 1e-9;
 Path< FullGraph > p;
 ins.tour( p );  // the tour as a path of n arcs
 double c_impr = opt2.run( p );
 ok = ok && c_impr <= c_ins + 1e-9 && std::fabs( c_impr - opt ) < 1e-9;

 std::cout << "optimum " << opt << "\nnearest neighbor " << c_nn << "\ngreedy "
           << c_gr << "\ninsertion " << c_ins << "\nChristofides " << c_chr
           << "\n2-opt " << c_id << "\n2-opt after insertion " << c_impr
           << std::endl;

 const int m = 40;
 FullGraph h( m );
 Random r( 19 );
 std::vector< double > px( m ) , py( m );
 for( int i = 0 ; i < m ; ++i ) {
```

```
 px[ i ] = r();
 py[ i ] = r();
 }
CostMap d( h );
for( FullGraph::EdgeIt e( h ) ; e != INVALID ; ++e ) {
 int u = h.id( h.u( e ) ) , v = h.id( h.v( e ) );
 d[ e ] = std::hypot( px[ u ] - px[ v ] , py[ u ] - py[ v ] );
 }
FullGraph::EdgeMap< bool > tree( h );
double mst = kruskal( h , d , tree );
NearestNeighborTsp< CostMap > nn2( h , d );
GreedyTsp< CostMap > gr2( h , d );
ChristofidesTsp< CostMap > chr2( h , d );
Opt2Tsp< CostMap > o2( h , d );
std::cout << "\nrandom points: spanning tree " << mst;
double c[ 7 ] = { nn2.run() , gr2.run() , 0 , 0 , 0 , 0 , chr2.run() };
const char * rule[ 4 ] = { "nearest" , "farthest" , "cheapest" , "random" };
for( int k = 0 ; k < 4 ; ++k ) {
 InsertionTsp< CostMap > ins2( h , d );
 c[ 2 + k ] = ins2.run( InsertionTsp< CostMap >::SelectionRule( k ) );
 ok = ok && checkTour( h , d , ins2.tourNodes() , c[ 2 + k ] );
 double impr = o2.run( ins2.tourNodes() );
 ok = ok && impr <= c[ 2 + k ] + 1e-9;
 std::cout << "\ninsertion (" << rule[ k ] << ") " << c[ 2 + k ]
           << ", after 2-opt " << impr;
 }
for( int k = 0 ; k < 7 ; ++k )
 ok = ok && c[ k ] >= mst - 1e-9;
ok = ok && c[ 6 ] <= 2 * mst + 1e-9 &&
     checkTour( h , d , chr2.tourNodes() , c[ 6 ] );
std::cout << "\nnearest neighbor " << c[ 0 ] << ", greedy " << c[ 1 ]
          << ", Christofides " << c[ 6 ] << std::endl;
return( ok ? 0 : 1 );
}
```

19.5 The maximum clique problem

Given an undirected graph $G = (V, E)$, a *clique* is a set $C \subseteq V$ of pairwise adjacent nodes. In the maximum clique problem one looks for a clique of largest cardinality $\omega(G)$, i.e.,

$$\omega(G) = \max\{|C| \ : \ C \subseteq V, \ \{u, v\} \in E \text{ for all distinct } u, v \in C\}. \tag{19.4}$$

Since the problem is NP-hard and hard to approximate, the library offers two algorithms with different aims. `GrossoLocatelliPullanMc` is a heuristic that quickly finds a "large" clique, without a proof of its maximality. `TomitaSekiMc` is an exact branch and bound that proves the clique it finds maximum, in a time that may grow exponentially with the size of the graph, unless a limit on its search tree stops it first. Both copy the graph into an $n \times n$ adjacency matrix of `char`, which costs $O(n^2)$ memory and time before the search starts. Both also ignore loops (a loop does not make a node adjacent to itself) and parallel edges.

abLEMON. `TomitaSekiMc` is new in abLEMON (version 1.5.0). In LEMON 1.3.1, `GrossoLocatelliPullanMc` ran forever on a graph with a loop, which made a node adjacent to itself and kept it among the candidates of the clique it already belonged to; abLEMON ignores loops when it builds the adjacency matrix.

19.6 The iterated local search of Grosso, Locatelli and Pullan

Our heuristic class, `GrossoLocatelliPullanMc< GR >`, implements the iterated local search of Grosso, Locatelli and Pullan 2008 on an undirected graph of type `GR`. It keeps a current clique C and, for each node w, the number $\delta(w)$ of nodes of C not adjacent to w, where a node of C counts itself. Thus, $\delta(w) = 0$ holds for the nodes that can be added to C, and $\delta(w) = 1$, with $w \notin C$, for the nodes that can replace one node of C. An iteration perturbs C and improves it by a local search with a tabu set; the largest clique found over all the iterations is the result.

Algorithm 19.6 (One iteration of the local search).

```
1  R := { w : delta(w) >= 4 }; pick r uniformly in R, or in V if R is empty
2  remove from C the nodes not adjacent to r; add r to C          (restart)
3  tabu := {}; max_swap := |C|
4  while the step limit is not reached:
5      if a non-tabu w has delta(w) = 0:            add w to C
           (and max_swap := |C| while tabu is still empty)
6      elif a non-tabu w not in C has delta(w) = 1:
           add w, remove the one node v of C not adjacent to w, tabu += v
           max_swap := max_swap - 1; stop the iteration if max_swap <= 0
7      elif some w has delta(w) = 0 (tabu allowed): add w to C
8      else stop the iteration
9  if |C| > |best|: best := C; return SIZE_LIMIT if |best| >= the size limit
```

Each selection of lines 5, 6 and 7 is a *step*, and it costs $O(n)$, since the update of δ after an addition or a removal is one pass over a row of the matrix. The node selected among the eligible ones depends on the rule passed to `run(rule)`, of the enumeration `SelectionRule`. With `RANDOM` it is the first eligible node in cyclic order from a random starting index; with `DEGREE_BASED` it is an eligible node of maximum degree, ties going to the first one in the same cyclic order; with `PENALTY_BASED`, the default, it is an eligible node of minimum penalty. The header describes the penalties as updated adaptively after each stage, as in Grosso, Locatelli and Pullan 2008. In the code, however, the penalties start at zero and the function `update()` that would change them is empty; in fact, `PENALTY_BASED` selects the same node as `RANDOM`. The three rules are the private classes `RandomSelectionRule`, `DegreeBasedSelectionRule` and `PenaltyBasedSelectionRule`, of which `start()` is a template.

Three limits control the search; each is set by a function that returns a reference to the object, and read by the function of the same name without argument. The function `iterationLimit(int)` sets the number of iterations (restarts), 1000 by default; `stepLimit(int)` sets the number of steps over all the iterations, by default -1, i.e., no limit; and `sizeLimit(int)` sets a clique size at which the search stops at once, by default -1, which stands for the number of nodes. The value -1 removes the iteration limit too, and the header warns that one should then set a step limit, since otherwise the search does not end. The function `run(rule)` returns the cause of the termination, of the enumeration `TerminationCause`: `ITERATION_LIMIT`, `STEP_LIMIT` or `SIZE_LIMIT`. The last one is also the answer on a graph with no node (clique of size 0) and on a graph with one node (clique of size 1).

The random choices use a generator of type `Random` (chapter 8) owned by the object, which the three constructors initialize differently. `GrossoLocatelliPullanMc(graph)` copies the state of the global generator `rnd` at the time of the construction; the global generator is not advanced, and two objects built one after the other draw the same sequence. `GrossoLocatelliPullanMc(graph, seed)` seeds the generator with `seed`, and `GrossoLocatelliPullanMc(graph, random)` copies the given one. The results are read by `cliqueSize()` and by `cliqueMap(map)`, which writes `true` on the nodes of the clique and `false` elsewhere, in a writable node map with `bool` (or convertible) values. Also, the iterator `CliqueNodeIt`, constructed from the algorithm object, lists the nodes of the clique (note that its postfix increment returns a `Node`, not an iterator).

19.7 The exact algorithm of Tomita and Seki

For the exact solution, `TomitaSekiMc< GR >` implements the branch and bound MCQ of Tomita and Seki 2003. A node of its search tree is a clique C together with the set P of the candidates, i.e., the nodes adjacent to all the nodes of C. The bound that prunes the tree is the number of colors of a proper coloring of the subgraph induced by P, which no clique of P can exceed.

Lemma 19.1. *If the nodes of P are partitioned into k independent sets, a clique contained in P has at most k nodes; hence, no clique that extends C by nodes of P is larger than $|C| + k$.*

This lemma is immediate, since a clique has at most one node in each independent set. In `colorSort()`, the coloring is computed greedily: each candidate, in the order of P, goes into the first color class that contains none of its neighbors. The candidates are then listed by color, and the k-th of them carries the number col_k of colors used by the first k; Lemma 19.1 thus applies to each prefix of the list.

Algorithm 19.7 (Tomita and Seki, `expand(P)`).

```
1  order P by greedy coloring; col[k] := number of colors of P[0..k]
2  for k from |P| - 1 down to 0:
```

```
3       if |C| + col[k] <= |best|: return                    (Lemma 19.1)
4       if the tree limit is reached: stopped := true; return
5       count a node of the search tree; v := P[k]; C := C + v
6       P' := { P[j] : j < k, P[j] adjacent to v }
7       if P' is empty: best := C if |C| > |best|
8       else expand(P')
9       C := C - v; return if stopped
```

At the root, the candidates are all the nodes, sorted by nonincreasing degree as Tomita and Seki order them; the sort is stable, and ties keep the order of the node iterator. The function `treeLimit(long long)` sets the largest number of nodes of the search tree, i.e., of the cliques that line 5 extends, after which `run()` stops; a negative value, the default (-1), means no limit. The function `run()` returns `true` when the clique found is proved maximum, and `false` when the limit stopped the search first; in this case the clique is the largest one found so far. The results are read by `cliqueSize()`, by `treeSize()` (the number of nodes of the search tree of the last run) and by `cliqueMap(map)`, with the same contract as in §19.6.

In the worst case the time is exponential, as for any exact algorithm for (19.4) known so far, and the size of the tree depends strongly on the density of the graph. On sparse graphs the coloring bound prunes almost all the branches: in the example below, a planted clique of 9 nodes in a random graph of 60 nodes and density 0.2 is proved maximum with a search tree of 9 nodes. On dense random graphs of a few hundred nodes, instead, the tree may already be very large.

Our example compares the exact algorithm with the enumeration of all the subsets on five random graphs of 14 nodes. On the larger graph, it checks that the heuristic, seeded and stopped at the maximum size, finds a clique no larger than the maximum one, and that the rules `RANDOM` and `PENALTY_BASED` with the same seed give the same clique; finally, it stops an exact search after 5 nodes of its tree.

```
// The two maximum clique algorithms of chapter 19: on small random graphs
// TomitaSekiMc proves its clique maximum (checked against the enumeration
// of all the subsets), and on a larger graph with a planted clique the
// heuristic GrossoLocatelliPullanMc finds a clique no larger than the
// maximum one, while a limit on the search tree stops the exact search.

#include <iostream>
#include <vector>
#include <lemon/list_graph.h>
#include <lemon/random.h>
#include <lemon/tomita_seki_mc.h>
#include <lemon/grosso_locatelli_pullan_mc.h>

using namespace lemon;

typedef ListGraph Graph;

void randomGraph( Graph & g , std::vector< Graph::Node > & v , int n ,
                  double p ,
                  Random & r )
{
 for( int i = 0 ; i < n ; ++i )
  v.push_back( g.addNode() );
 for( int i = 0 ; i < n ; ++i )
  for( int j = i + 1 ; j < n ; ++j )
   if( r() < p )
    g.addEdge( v[ i ] , v[ j ] );
 }

bool adjacent( const Graph & g , Graph::Node a , Graph::Node b )
{
 return( findEdge( g , a , b ) != INVALID );
 }

// whether the nodes marked in the map are pairwise adjacent and k
template< typename Map >
bool isClique( const Graph & g , const Map & m , int k )
```

```
{
 std::vector< Graph::Node > c;
 for( Graph::NodeIt u( g ) ; u != INVALID ; ++u )
  if( m[ u ] )
   c.push_back( u );
 for( std::size_t i = 0 ; i < c.size() ; ++i )
  for( std::size_t j = i + 1 ; j < c.size() ; ++j )
   if( ! adjacent( g , c[ i ] , c[ j ] ) )
    return( false );
 return( int( c.size() ) == k );
 }

// the largest clique by the enumeration of the 2^n subsets
int bruteForce( const Graph & g , const std::vector< Graph::Node > & v )
{
 int n = int( v.size() ) , best = 0;
 for( long s = 1 ; s < ( 1L << n ) ; ++s ) {
  int k = 0;
  bool clique = true;
  for( int i = 0 ; i < n && clique ; ++i ) {
   if( ! ( s >> i & 1 ) )
    continue;
   ++k;
   for( int j = i + 1 ; j < n && clique ; ++j )
    if( ( s >> j & 1 ) && ! adjacent( g , v[ i ] , v[ j ] ) )
     clique = false;
   }
  if( clique && k > best )
   best = k;
  }
 return( best );
 }

int main()
{
 Random r( 2026 );
 bool ok = true;

 for( int t = 0 ; t < 5 && ok ; ++t ) {  // small graphs
  Graph g;
  std::vector< Graph::Node > v;
  randomGraph( g , v , 14 , 0.5 , r );
  TomitaSekiMc< Graph > mc( g );
  Graph::NodeMap< bool > in( g );
  ok = mc.run() && mc.cliqueSize() == bruteForce( g , v );
  mc.cliqueMap( in );
  ok = ok && isClique( g , in , mc.cliqueSize() );
  }

 Graph g;  // a planted clique
 std::vector< Graph::Node > v;
 randomGraph( g , v , 60 , 0.2 , r );
 for( int i = 0 ; i < 9 ; ++i )
  for( int j = i + 1 ; j < 9 ; ++j )
   if( ! adjacent( g , v[ 3 * i ] , v[ 3 * j ] ) )
    g.addEdge( v[ 3 * i ] , v[ 3 * j ] );

 TomitaSekiMc< Graph > exact( g );
 bool proved = exact.run();
 Graph::NodeMap< bool > in( g );
 exact.cliqueMap( in );
 ok = ok && proved && exact.cliqueSize() >= 9 &&
      isClique( g , in , exact.cliqueSize() );

 GrossoLocatelliPullanMc< Graph > heur( g , 7 );  // seeded generator
```

```
GrossoLocatelliPullanMc< Graph >::TerminationCause cause =
  heur.iterationLimit( 200 ).sizeLimit( exact.cliqueSize() ).run();
Graph::NodeMap< bool > hin( g );
heur.cliqueMap( hin );
ok = ok && heur.cliqueSize() <= exact.cliqueSize() &&
     isClique( g , hin , heur.cliqueSize() ) &&
     ( cause != heur.SIZE_LIMIT || heur.cliqueSize() == exact.cliqueSize() );

// PENALTY_BASED never updates its penalties: with the same seed it
// picks the same nodes as RANDOM, and finds the same clique
GrossoLocatelliPullanMc< Graph > rnd_rule( g , 11 ) , pen_rule( g , 11 );
rnd_rule.iterationLimit( 50 ).run( rnd_rule.RANDOM );
pen_rule.iterationLimit( 50 ).run( pen_rule.PENALTY_BASED );
Graph::NodeMap< bool > rin( g ) , pin( g );
rnd_rule.cliqueMap( rin );
pen_rule.cliqueMap( pin );
for( Graph::NodeIt u( g ) ; u != INVALID ; ++u )
 ok = ok && rin[ u ] == pin[ u ];

TomitaSekiMc< Graph > stopped( g );  // a stopped search
ok = ok && ! stopped.treeLimit( 5 ).run() && stopped.treeSize() == 5;

std::cout << "maximum clique " << exact.cliqueSize() << " (search tree "
          << exact.treeSize() << " nodes), heuristic clique "
          << heur.cliqueSize() << ", cause " << int( cause ) << std::endl;
return( ok ? 0 : 1 );
}
```

19.8 Exercises

Exercise 19.1. Show that on a metric instance each tour returned by the five TSP classes costs at least the minimum spanning tree, and that the tour of `ChristofidesTsp` costs at most twice it. Check both on random Euclidean instances (`manual/examples/19-tsp.cc` does it for 40 points).

Exercise 19.2. Prove that on points of the plane with Euclidean costs a 2-optimal tour has no two crossing edges. Deduce that `Opt2Tsp` returns the optimal tour on points in convex position, whatever its starting tour (`manual/examples/19-tsp.cc` checks it on twelve points of a circle).

Exercise 19.3. Build a metric instance on which `NearestNeighborTsp` returns a tour much more expensive than `InsertionTsp` with the `FARTHEST` rule, e.g., points on a line with exponentially growing gaps, and measure the ratio as n grows.

Exercise 19.4. Give a non-metric instance on which the shortcuts of Algorithm 19.4 increase the cost of the Euler tour, and compare the resulting tour with the bound of Theorem 19.1 (`test/tsp_test.cc` checks only metric instances).

Exercise 19.5. Count the evaluations of (19.3) made by `Opt2Tsp` from the order of the node ids and from the tour of `InsertionTsp`, on random Euclidean instances of growing size, and relate them to the $O(n^2)$ cost of a pass of Algorithm 19.5.

Exercise 19.6. Verify on random graphs that `TomitaSekiMc` finds a clique of the size given by the enumeration of all the subsets, and that a tree limit of 1 stops the search after one node (`test/tomita_seki_mc_test.cc` and `manual/examples/19-clique.cc`).

Exercise 19.7. Run `GrossoLocatelliPullanMc` with the rules `RANDOM` and `PENALTY_BASED` and the same seed on the same graph, observe that they return the same clique, and explain why from §19.6 (`manual/examples/19-clique.cc` checks it; `test/max_clique_test.cc` checks the termination causes of the three rules).

Exercise 19.8. Plot the size of the search tree of `TomitaSekiMc` (`treeSize()`) on random graphs of 100 nodes as the density grows from 0.1 to 0.9, and find where the coloring bound stops being effective.

20. The LP and MIP interface

This chapter describes the interface through which a program builds a Linear Program (LP) or a Mixed Integer Program (MIP) and has it solved by an external solver. The interface is declared in `lp_base.h`, the default solvers are chosen in `lp.h`, the skeletons of `lp_skeleton.h` do nothing but compile, and `glpk.h`, `cplex.h`, `clp.h`, `cbc.h` and `soplex.h` connect the interface to GLPK, CPLEX, Clp, Cbc and SoPlex. Unlike the rest of the library, these classes are not templates: they are compiled into the library, each backend only when its solver is found at configuration time (chapter 2). A program written against the base classes compiles with any backend; it also compiles with none, although then only the skeletons are available.

20.1 The structure of the interface

We are concerned with problems of the form

$$\min \text{ (or } \max) \;\; c^\top x + c_0 \quad \text{s.t.} \quad l^r \le Ax \le u^r, \quad l^c \le x \le u^c, \quad x_j \in \mathbb{Z} \;\; (j \in J), \tag{20.1}$$

where each row i of A has the bounds $l_i^r \in \mathbb{R} \cup \{-\infty\}$ and $u_i^r \in \mathbb{R} \cup \{+\infty\}$, each column j has the bounds l_j^c and u_j^c alike, c_0 is a constant, and J is the set of the integer columns, which is empty for an LP. A row with $l_i^r = u_i^r$ is an equation, and one with both bounds finite is a *range* constraint. The interface describes (20.1) with three classes (Figure 20.1): `LpBase` holds the model, i.e., the columns, the rows, the coefficients, the bounds and the objective; `LpSolver` adds the solution of an LP, with its dual values and its basis; `MipSolver` adds the integrality of the columns and the solution of a MIP. Both `LpSolver` and `MipSolver` derive virtually from `LpBase`, and a backend derives from one of them. Hence, a function that takes an `LpSolver &` works with any LP backend, and one that takes an `LpBase &` builds a model for both kinds of solvers.

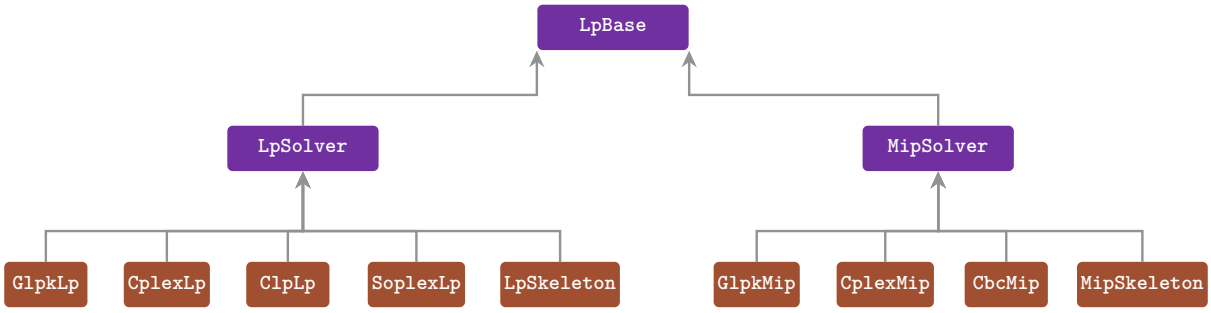


Figure 20.1. The classes of the interface: the model is held by `LpBase` (primary), the solving and the solution by `LpSolver` and `MipSolver` (primary), and each backend (the added colour) derives from one of the two. The common bases `GlpkBase`, `CplexBase` and `SkeletonSolverBase` are not drawn.

All the numbers of the interface are of the type `LpBase::Value`, which is `double`. The constants `LpBase::INF` and `LpBase::NaN` are the infinity and the quiet NaN of `double`; an infinite bound is written as `INF` or `-INF`, and NaN marks an unset bound of a constraint (§20.2). The enumerations of `LpBase` are `Sense` (`MIN`, `MAX`), `MessageLevel` (`MESSAGE_NOTHING`, `MESSAGE_ERROR`, `MESSAGE_WARNING`, `MESSAGE_NORMAL`, `MESSAGE_VERBOSE`) and `SolveExitStatus`. The last one is the value returned by `solve()`: it is `SOLVED` (0) when the solver has found an optimal solution or proved the problem infeasible or unbounded, and `UNSOLVED` (1) in any other case, e.g., when a limit of the solver stopped it. Whether a solved problem is optimal, infeasible or unbounded is then read from the *problem type* (§20.4 and §20.5).

Then `lp.h` defines the default solvers. When at least one backend is found at configuration time, the macro `LEMON_HAVE_LP` (and, for GLPK, CPLEX and Cbc, `LEMON_HAVE_MIP`) is defined in `lemon/config.h`, together with `LEMON_HAVE_GLPK`, `LEMON_HAVE_CPLEX`, `LEMON_HAVE_CLP`, `LEMON_HAVE_CBC` and `LEMON_HAVE_SOPLEX` for the backends found. In this case `Lp` is the default LP solver, and `Mip` the default MIP solver. Usually the CMake options `LEMON_DEFAULT_LP` and `LEMON_DEFAULT_MIP` choose them; when they are not given, or they name a solver that was not found, the first solver found is taken in the order CPLEX, Clp (with Cbc for the MIP), GLPK, SoPlex. SoPlex is an LP solver only, and therefore `Mip` is not defined when it is the only backend; neither `Lp` nor `Mip` is defined when no backend is found. A program that must compile in all cases guards their use with `#ifdef LEMON_HAVE_LP` and `#ifdef LEMON_HAVE_MIP`, as `test/lp_test.cc` and `test/mip_test.cc` do, and as the example of §20.8 does.

20.2 Columns, rows and expressions

A column (a variable) is referred to by an object of the class `LpBase::Col`, and a row (a constraint) by an object of `LpBase::Row`. These handles behave as the items of a graph (chapter 3): they are copied freely, they compare with `==`, `!=` and `<` (an artificial order, for the associative containers), a default constructed one is undefined, and one constructed from `INVALID` is invalid. A handle stays valid when other columns or rows are added or erased, since the interface keeps its own index of the columns and of the rows and translates it into the indices of the solver. The static functions `id(col)`, `colFromId(id)`, `id(row)` and `rowFromId(id)` convert between a handle and its integer id. The iterators `ColIt` and `RowIt`, constructed from the solver, list the columns and the rows, and `cols()` and `rows()` return ranges for the loops of C++11.

A linear expression of the columns with a constant term is an object of the class `LpBase::Expr`. It stores the nonzero coefficients in a `std::map` from the column id to the value, and the constant apart. A number and a column convert implicitly to an `Expr`, and the operators `+`, `-`, `*` and `/` by a number, with their compound forms, combine expressions; for instance, `2 * x + 3 * (y - x) + 4` is the expression $-x + 3y + 4$. The coefficient of a column is read with `e[c]` (zero when absent) and changed through the reference returned by the non-constant `e[c]`, and the constant term is accessed by the unary `*e`. Also, `clear()` sets everything to zero, `simplify(eps)` removes the coefficients whose absolute value is at most `eps` (by default 0) and sets the constant to zero under the same condition, and the iterators `CoeffIt` and `ConstCoeffIt` list the terms, converting to the column of the term and dereferencing to its coefficient. Note that `set(c, v)` removes the term when `v` is zero, but otherwise inserts it only when the column has no term yet, since it calls `std::map::insert`; one changes an existing coefficient through `e[c] = v`.

A constraint is an object of the class `LpBase::Constr`, made of an expression and of a lower and an upper bound, read and written by `expr()`, `lowerBound()` and `upperBound()`; `lowerBounded()` and `upperBounded()` tell whether a bound is set and finite. A bound equal to NaN is unset, which is the state of both bounds of a default constructed constraint. The comparison operators between expressions and numbers build constraints, as Table 20.1 shows.

Written	Lower bound	Expression	Upper bound
`e <= s`	unset	`e`	`s`
`s <= e`	`s`	`e`	unset
`e <= f`	0	`f - e`	unset
`e == s`	`s`	`e`	`s`
`e == f`	0	`f - e`	0
`(s <= e) <= t`	`s`	`e`	`t`

Table 20.1. The constraints built by the comparison operators, `e` and `f` expressions (or columns) and `s` and `t` numbers; `>=` gives the mirrored cases.

A second bound can be added to a constraint by comparing it with a number, as in the last row of Table 20.1, but only on the side that is still unset. This is checked at run time by `LEMON_ASSERT`, and hence a wrong double comparison such as `x[1] <= x[2] <= 5` compiles but fails the assertion (when assertions are enabled, chapter 8). Since C++ evaluates `s <= e <= t` as `(s <= e) <= t`, the form `-3 <= v <= 4` builds the range constraint intended. When a constraint is added to the model, its constant term is moved to the bounds, i.e., the row $l \le a^\top x + c \le u$ is stored as $l - c \le a^\top x \le u - c$, and an unset or infinite bound becomes $\mp\infty$.

Similarly, `LpBase::DualExpr` is the analogous expression of the rows, i.e., a column of the matrix, or a linear combination of the dual variables; it has the same members as `Expr` except the constant term. Rows convert to

it implicitly, and it is used to set or read a whole column (§20.3) and to evaluate a combination of the dual values (§20.4).

20.3 Building and modifying a problem

We now list the members of `LpBase` that build and change the model; all of them work through the virtual functions of the backend, and therefore their cost is the one of the solver. A column is added by `addCol()`, which gives an empty column with the bounds $-\infty$ and $+\infty$ in all the backends of the library, and by `addCol(e , o)`, which also sets its coefficients from the `DualExpr e` and its objective coefficient to `o` (0 by default). The function `addColSet(t)` fills a container with new columns and returns their number; `t` can be a container of `Col` (e.g., a `std::vector`), an associative container whose mapped type is `Col` (e.g., a `std::map< K , Col >`), or a writable map of the library with values of type `Col`, e.g., a `ListDigraph::ArcMap< LpBase::Col >`, which gives one column per arc. The rows are added in the same ways: `addRow()` gives an empty row, `addRowSet(t)` fills a container, and `addRow(l, e, u)` and `addRow(c)` add a row from an expression with two bounds or from a constraint, moving the constant term as described in §20.2.

Once created, a column is changed by `col(c, e)`, which sets its coefficients, and read back as a `DualExpr` by `col(c)`; the analogous `row(r, l, e, u)` and `row(r, c)` set a row, and `row(r)` returns its coefficients as an `Expr`. A single coefficient is set and read by `coeff(r, c, v)` and `coeff(r, c)`, and `erase(c)` and `erase(r)` remove a column or a row. The bounds are set and read by `colLowerBound(c, v)`, `colUpperBound(c, v)`, `colBounds(c, l, u)`, `rowLowerBound(r, v)` and `rowUpperBound(r, v)`, with the getters of the same names; the column setters also accept a container or a map of columns, as `addColSet()` does, to give the same bounds to all of them. Names are given by `colName(c, s)` and `rowName(r, s)`, read by `colName(c)` and `rowName(r)`, and searched by `colByName(s)` and `rowByName(s)`, which return `INVALID` when no item has the name.

The objective is set by `obj(e)`, whose constant term is kept by `LpBase` itself and added to the objective value of the solution, and read by `obj()`; `objCoeff(c, v)` and `objCoeff(c)` set and read one coefficient. The direction is set by `sense(MIN)` or `sense(MAX)`, or by the shorthands `min()` and `max()`, and read by `sense()`. Finally, `clear()` empties the model, `messageLevel(level)` sets how much the solver prints (all the backends start at `MESSAGE_NOTHING`), and `solverName()` returns the name of the backend. Also, `write(file, format)` writes the model to a file, by default in the format `"MPS"`; a format that the backend does not support throws `LpBase::UnsupportedFormatError` (Table 20.2).

20.4 Solving an LP and reading the solution

For an LP, i.e., (20.1) with $J = \emptyset$, the class `LpSolver` adds the solution. We recall the optimality conditions that its dual values satisfy, for a minimization with finite bounds. With y the dual values of the rows and $d = c - A^\top y$ the reduced costs of the columns, a feasible x is optimal if and only if there is a y such that

$$y_i > 0 \Rightarrow (Ax)_i = l^r_i, \quad y_i < 0 \Rightarrow (Ax)_i = u^r_i, \quad d_j > 0 \Rightarrow x_j = l^c_j, \quad d_j < 0 \Rightarrow x_j = u^c_j. \tag{20.2}$$

Thus, the dual value of a row is the derivative of the optimal value with respect to the bound that is active, and its sign indicates which bound it is (the interface passes on the dual values of the solver unchanged, and hence their sign convention, in particular for a maximization, is the one of the solver). The theory is in any textbook on linear programming, e.g., Dantzig 1963.

A call of `solve()` solves the problem and returns the `SolveExitStatus` of §20.1. The *problem type* of the primal and of the dual problem is then read by `primalType()` and `dualType()`, of the enumeration `LpSolver::ProblemType`: `UNDEFINED` (no feasible solution found, although one may exist), `INFEASIBLE`, `FEASIBLE` (a feasible solution found, its optimality not proved), `OPTIMAL` and `UNBOUNDED`. The values of a solution are read by `primal(c)` for a column, `primal(e)` for an expression (the scalar product with the primal solution plus the constant term), `dual(r)` for a row and `dual(e)` for a `DualExpr`. As for the objective value, `primal()` includes the constant term of the objective; according to the documentation, it is `INF` or `-INF` for an infeasible or unbounded problem, depending on the direction, and NaN when no primal solution is found. Also, the basis is read by `colStatus(c)` and `rowStatus(r)`, of the enumeration `VarStatus`: `BASIC`, `FREE` (nonbasic and free), `LOWER` and `UPPER` (nonbasic at a bound) and `FIXED` (nonbasic and fixed).

When the problem has no optimal solution, the rays certify it, in the backends that provide them. The *primal ray* `primalRay(c)` is a solution of the primal problem with each finite bound replaced by 0 and a negative (for

a minimization) objective value; it proves that the dual problem is infeasible, and, when the primal problem is feasible, that the latter is unbounded. The *dual ray* `dualRay(r)` is, symmetrically, a solution of the dual problem with each finite objective coefficient of it, i.e., each finite primal bound, replaced by 0, with a positive objective value; it proves the primal problem infeasible. The header warns that some solvers may not compute the rays. Finally, `newSolver()` returns a new empty solver of the same type, and `cloneSolver()` a copy of the problem; both return a pointer that the caller deletes.

20.5 Mixed integer programs

In `MipSolver` one describes (20.1) with integer columns. The type of a column is set by `colType(c, type)` and read by `colType(c)`, of the enumeration `MipSolver::ColTypes`: `REAL` (the default) or `INTEGER`; a binary column is an integer one with the bounds 0 and 1. As for an LP, `solve()` returns `SOLVED` or `UNSOLVED`, and `type()` returns the problem type, with the same five values as `LpSolver::ProblemType`; `UNBOUNDED` means that the MIP, or at least its continuous relaxation, is unbounded. The solution is read by `sol(c)`, `sol(e)` and `solValue()`, the last one including the constant term of the objective. A MIP solver gives no dual values and no basis.

The integrality of a column whose bounds are not integer deserves a remark (cf. Exercise 20.5). The integer values of such a column are those of $[\lceil l_j^c \rceil, \lfloor u_j^c \rfloor]$, and the problem is infeasible when this interval is empty. GLPK refuses such a column, and Cbc does not round its bounds; hence, `GlpkMip` and `CbcMip` round the bounds of each integer column inwards before the solve, up to an integer tolerance of 10^{-6}, as in Algorithm 20.1, which is the one of `GlpkMip::_solve()`.

Algorithm 20.1 (The solve of `GlpkMip`).

```
1  solve the continuous relaxation by the dual simplex method
   (on a bad basis, build an advanced basis and solve once more)
2  if the relaxation is not optimal: return SOLVED (type: of the relaxation)
3  for each integer column j with a finite bound:
4      l' := ceil(l_j - 1e-6); u' := floor(u_j + 1e-6)
5      if l' > u': the problem is INFEASIBLE; skip the search
6      else give the column the bounds [l', u'] (fixed if l' = u')
7  run the branch and cut of GLPK (glp_intopt)
8  give back the bounds given by the user; return SOLVED or UNSOLVED
```

`CbcMip` rounds the bounds in the same way in the copy of the problem that it gives to Cbc, and it also leaves the bounds of the model unchanged. The test `test/mip_test.cc` checks both that the bounds stay as they were given and that a column without integer values makes the problem infeasible.

abLEMON. The rounding of the bounds of the integer columns in `GlpkMip` and `CbcMip` is new in abLEMON (version 1.4.1): GLPK 5.0 refused such a column, which left the problem unsolved, and Cbc gave a solution beyond the bound. Also, `CbcMip` no longer reports a proven infeasible problem as feasible (trac #626, version 1.4.0).

20.6 The backends

Each backend wraps one solver and adds a few functions of its own. Table 20.2 summarizes them, and the paragraphs below give the details.

Class	Solver	`solve()`	Other solves	Formats of `write()`
`GlpkLp`	GLPK	primal simplex	`solvePrimal()`, `solveDual()`	`"MPS"`, `"LP"`
`GlpkMip`	GLPK	Algorithm 20.1		`"MPS"`, `"LP"`
`CplexLp`	CPLEX	`CPXlpopt`	`solvePrimal()`, `solveDual()`, `solveBarrier()`	`"MPS"`, `"LP"`, `"SOL"`
`CplexMip`	CPLEX	branch and cut of CPLEX		`"MPS"`, `"LP"`, `"SOL"`
`ClpLp`	Clp	primal simplex	`solvePrimal()`, `solveDual()`, `solveBarrier()`	`"LP"`
`CbcMip`	Cbc	branch and cut of Cbc		none
`SoplexLp`	SoPlex	simplex of SoPlex		none

Table 20.2. The backends, the method of `solve()`, the other solve functions, and the formats accepted by `write()`; any other format throws `UnsupportedFormatError`.

GLPK. `GlpkLp` and `GlpkMip` share the base `GlpkBase`, whose `lpx()` returns the pointer to the problem object of GLPK (as a `void *` that converts to the pointer type of GLPK), and whose `lpxRow(r)` and `lpxCol(c)` give the indices that GLPK uses. Each solve function of `GlpkLp` runs the simplex method of GLPK; when it reports a bad, singular or ill-conditioned basis, it builds an advanced basis by `glp_adv_basis()` and runs once more, and it returns `UNSOLVED` if this fails too. The function `presolver(bool)` turns the presolver of GLPK on or off, and it is off by default. Also, a static member of `GlpkBase` of the private type `FreeEnvHelper` frees the environment of GLPK (`glp_free_env()`) when the program ends.

CPLEX. `CplexLp` and `CplexMip` share the base `CplexBase`, whose `cplexEnv()` and `cplexLp()` return the environment and the problem objects of CPLEX, and whose `env()` returns the `CplexEnv` in use. A `CplexEnv` wraps the environment of CPLEX with a reference count protected by a lock, so that several problems can share one environment (and one license): the constructors of `CplexLp` and `CplexMip` that take a `CplexEnv` use the given one, and the others open a new one. When the license check fails, the constructor of `CplexEnv` throws `CplexEnv::LicenseError`, whose `what()` gives the message of CPLEX. In `CplexLp`, `solve()` calls `CPXlpopt`, which lets CPLEX choose the method, and the three other functions force the primal simplex, the dual simplex and the barrier method.

Clp and Cbc. `ClpLp` wraps a `ClpSimplex` object of Clp; `solve()` runs the primal simplex method, and `clpRow(r)` and `clpCol(c)` give the indices of Clp. Note that `ClpLp` writes only the format `"LP"`, and therefore `write(file)` with the default `"MPS"` throws. `CbcMip` keeps the model in a `CoinModel` of Cbc, and at each solve it copies the model into a new `OsiClpSolverInterface` (with the rounding of §20.5) and solves the continuous relaxation. When this is optimal, it runs the branch and cut of Cbc with a fixed set of cut generators (probing, Gomory, knapsack cover, odd hole, clique, mixed integer rounding and flow cover) and of primal heuristics (rounding, local search, greedy cover, feasibility pump and RINS).

SoPlex. `SoplexLp` wraps the class `SoPlex` of SoPlex. Its message level `MESSAGE_NOTHING` maps to the verbosity of errors of SoPlex, which is (apparently) the lowest one SoPlex offers. Its primal ray is the one of SoPlex, and its dual ray is the Farkas proof of the infeasibility of the primal problem.

> **abLEMON.** `SoplexLp` works with SoPlex 5 and later, through the API of its class `SoPlex`; in LEMON 1.3.1 it did not compile with these versions, and it gave the Farkas proof of the dual as its primal ray (version 1.5.0). SoPlex is found through its CMake configuration.

20.7 The skeletons

The header `lp_skeleton.h` declares `LpSkeleton` and `MipSkeleton`, which derive from `LpSolver` and `MipSolver` and from the common base `SkeletonSolverBase`. They implement the virtual functions of the interface by doing (almost) no work: the coefficients, the bounds and the names are not stored, the getters return 0 (or `MIN` for the sense, and `-1`, i.e., `INVALID`, for a search by name), `solve()` returns `SOLVED`, the problem types are `UNDEFINED`, the basis statuses are `BASIC`, `write()` writes nothing, and `newSolver()` and `cloneSolver()` return a null pointer. Their names are `"LpSkeleton"` and `"MipSkeleton"`. On a skeleton, however, all that `LpBase` keeps works as with a real solver: the index of the columns and of the rows, and hence `ColIt`, `RowIt`, `cols()`, `rows()` and the handles, and the classes `Expr`, `DualExpr` and `Constr` with their operators, which do not involve the solver at all.

We see two uses of the skeletons. First, a program can build and check a model, e.g., count its columns and rows, without any solver; `test/lp_test.cc` does so in all configurations, and so does the example of §20.8. Second, they are the starting point for a new backend, which must implement the pure virtual functions declared in `lp_base.h`: `_addCol()`, `_addRow()`, `_eraseCol()`, `_eraseRow()`, the functions of the names, of the coefficients (`_setRowCoeffs()`, `_getRowCoeffs()`, `_setColCoeffs()`, `_getColCoeffs()`, `_setCoeff()`, `_getCoeff()`), of the bounds, of the objective and of the sense, `_clear()`, `_solverName()` and `_messageLevel()`. An LP backend also implements `_solve()`, the getters of the solution, of the rays, of the basis and of the problem types, `newSolver()` and `cloneSolver()`; a MIP backend implements `_solve()`, `_setColType()`, `_getColType()`, `_getType()`, `_getSol()` and `_getSolValue()`. The coefficients reach the backend through two protected iterators of `LpBase`: `ExprIterator` lists the terms of an expression as pairs of a solver index and a value, and `InsertIterator` stores the pairs returned by the solver into an expression, translating the indices. The function `_addRow(l, b, e, u)`, which adds a row with its coefficients and bounds in one call, and `_write()` have default implementations, the latter throwing `UnsupportedFormatError`.

20.8 An example

Our program writes the maximum flow problem on a digraph $G = (N, A)$ with capacities u, a source s and a sink t as the LP

$$\max \sum_{a \in \delta^+(s)} x_a - \sum_{a \in \delta^-(s)} x_a \quad \text{s.t.} \quad \sum_{a \in \delta^-(v)} x_a = \sum_{a \in \delta^+(v)} x_a \;\; (v \in N \setminus \{s,t\}), \quad 0 \le x_a \le u_a \;\; (a \in A), \qquad (20.3)$$

in a function that takes any `LpSolver`, with one column per arc obtained by `addColSet()` on an arc map. First, the model is built on an `LpSkeleton`, on which the program checks the number of columns and rows and the algebra of the expressions and constraints of Table 20.1. When the library has an LP backend, the same function builds the model on the default solver `Lp`, which solves it, and the program compares the optimal value with the one of `Preflow` (chapter 12); without a backend this part is left out by `#ifdef LEMON_HAVE_LP`, and the program still compiles and checks the rest.

```
// The LP interface of chapter 20: the maximum flow problem written as an
// LP over the arcs of a digraph, one column per arc and one row per inner
// node, by a function that takes any LpSolver. It is built on LpSkeleton,
// which compiles with no solver and checks the model and the algebra of
// the expressions; when the library has an LP solver (LEMON_HAVE_LP), the
// same model is solved by the default one and its value compared with
// that of Preflow.

#include <cmath>
#include <iostream>
#include <lemon/config.h>
#include <lemon/list_graph.h>
#include <lemon/preflow.h>
#include <lemon/lp_skeleton.h>
#ifdef LEMON_HAVE_LP
#include <lemon/lp.h>
#endif

using namespace lemon;

typedef ListDigraph Digraph;
typedef Digraph::ArcMap< int > CapMap;

// the maximum flow LP: max sum of the flow out of s minus into s,
// conservation at every node but s and t, 0 <= x_a <= cap_a
void buildMaxFlow( LpSolver & lp , const Digraph & g , const CapMap & cap ,
                   Digraph::Node s , Digraph::Node t ,
                   Digraph::ArcMap< LpSolver::Col > & x )
{
 lp.addColSet( x );  // one column per arc
 for( Digraph::ArcIt a( g ) ; a != INVALID ; ++a )
  lp.colBounds( x[ a ] , 0 , cap[ a ] );
 for( Digraph::NodeIt v( g ) ; v != INVALID ; ++v ) {
  if( v == s || v == t )
   continue;
  LpSolver::Expr in , out;
  for( Digraph::InArcIt a( g , v ) ; a != INVALID ; ++a )
   in += x[ a ];
  for( Digraph::OutArcIt a( g , v ) ; a != INVALID ; ++a )
   out += x[ a ];
  lp.addRow( in == out );
  }
 LpSolver::Expr value;
 for( Digraph::OutArcIt a( g , s ) ; a != INVALID ; ++a )
  value += x[ a ];
 for( Digraph::InArcIt a( g , s ) ; a != INVALID ; ++a )
  value -= x[ a ];
 lp.obj( value );
```

```
 lp.max();
 }

int countCols( const LpBase & lp )
{
 int k = 0;
 for( LpBase::ColIt c( lp ) ; c != INVALID ; ++c )
  ++k;
 return( k );
 }

int countRows( const LpBase & lp )
{
 int k = 0;
 for( LpBase::RowIt r( lp ) ; r != INVALID ; ++r )
  ++k;
 return( k );
 }

int main()
{
 Digraph g;
 Digraph::Node s = g.addNode() , a = g.addNode() , b = g.addNode() ,
               t = g.addNode();
 CapMap cap( g );
 cap[ g.addArc( s , a ) ] = 4;
 cap[ g.addArc( s , b ) ] = 2;
 cap[ g.addArc( a , b ) ] = 3;
 cap[ g.addArc( a , t ) ] = 1;
 cap[ g.addArc( b , t ) ] = 5;
 bool ok = true;

 // the model on the skeleton: two rows (a and b), five columns
 LpSkeleton skel;
 Digraph::ArcMap< LpSolver::Col > xs( g );
 buildMaxFlow( skel , g , cap , s , t , xs );
 ok = ok && countCols( skel ) == 5 && countRows( skel ) == 2;

 // the algebra of expressions and constraints, kept by LpBase itself
 LpBase::Col p = skel.addCol() , q = skel.addCol();
 LpBase::Expr e = 2 * p + 3 * ( q - p ) + 4;  // -p + 3 q + 4
 ok = ok && e[ p ] == -1 && e[ q ] == 3 && *e == 4;
 LpBase::Constr c = ( 1 <= e ) <= 10;  // a two-sided row
 ok = ok && c.lowerBound() == 1 && c.upperBound() == 10;
 LpBase::Constr d = e <= q;  // 0 <= q - e
 ok = ok && d.lowerBounded() && ! d.upperBounded() && d.expr()[ p ] == 1;
 e += 0.001 * q - 3 * q;
 e.simplify( 0.01 );  // drops the term of q
 ok = ok && e[ q ] == 0 && e[ p ] == -1;

 Preflow< Digraph , CapMap > pre( g , cap , s , t );
 pre.run();
 std::cout << "maximum flow by Preflow: " << pre.flowValue() << std::endl;

#ifdef LEMON_HAVE_LP
 Lp lp;  // the default LP solver
 Digraph::ArcMap< LpSolver::Col > x( g );
 buildMaxFlow( lp , g , cap , s , t , x );
 lp.solve();
 ok = ok && lp.primalType() == LpSolver::OPTIMAL &&
      std::fabs( lp.primal() - pre.flowValue() ) < 1e-6;
 std::cout << "maximum flow by " << lp.solverName() << ": " << lp.primal()
           << std::endl;
#else
 std::cout << "no LP solver: the model was only built" << std::endl;
```

```
#endif
 return( ok ? 0 : 1 );
 }
```

20.9 Exercises

Exercise 20.1. Write the shortest path problem from s to t as an LP with one column per arc, solve it with the default solver, and compare its value with the distance computed by `Dijkstra` (chapter 10); read the dual values of the rows and check that they are distances from s (up to a constant and to the sign convention of the solver).

Exercise 20.2. Build an infeasible LP, e.g., by adding to the model of §20.8 the row $\sum_{a\in\delta^+(s)} x_a \geq 1 + \sum_a u_a$, and verify that `primalType()` is `INFEASIBLE` and that the dual ray, when the backend provides it, satisfies the conditions of §20.4 (`test/lp_test.cc` checks the problem types of a small LP).

Exercise 20.3. Show with a short program that `Expr::set(c, v)` does not change an existing coefficient of `c`, while `e[c] = v` does, and that `simplify(eps)` also zeroes the constant term when it is at most `eps` in absolute value.

Exercise 20.4. Write the maximum weight matching problem of a small bipartite graph as a MIP with binary columns, solve it with `Mip`, and compare the result with `MinCostAssignment` of chapter 17 (on the negated weights); then relax the integrality and observe that the LP gives the same value, as the total unimodularity of the bipartite incidence matrix implies.

Exercise 20.5. Give an integer column the bounds 0.5 and 0.7, and check that `GlpkMip` and `CbcMip` report the problem as `INFEASIBLE` and leave the bounds of the model unchanged (`test/mip_test.cc`).

Exercise 20.6. Write a function that builds the model of §20.8 on an `LpBase &`, and use it with both an `LpSkeleton` and a `MipSkeleton`; count the columns and the rows of both through `ColIt` and `RowIt` (`manual/examples/20-lp.cc` does it for the LP skeleton).

21. Input and output

This chapter describes how graphs and the data attached to them are read from and written to files. The format of the library is the LEMON Graph Format (LGF), read by the classes of `lgf_reader.h` and written by those of `lgf_writer.h` (§21.1 to §21.4). Besides it, `dimacs.h` reads the formats of the first DIMACS implementation challenge and writes one of them (§21.5), `nauty_reader.h` reads the graph6 format of the nauty package (§21.6), and `graph_to_eps.h` draws a graph in Encapsulated PostScript (§21.7). A malformed input makes the readers and the writers throw `FormatError`, and a file that cannot be opened makes them throw `IoError` (chapter 8); both derive from `Exception`.

21.1 The LEMON Graph Format

An LGF file is a text file made of *sections*, each opened by a header line whose first character is `@`, followed by the type of the section and by an optional name, the *caption*, which distinguishes sections of the same type. A line whose first non-blank character is `#` is a comment, and the empty lines are ignored; whatever precedes the first header is skipped. The standard types are `@nodes`, `@arcs` (with its synonym `@edges`) and `@attributes` for digraphs and graphs, and `@red_nodes` and `@blue_nodes` for the two node sets of a bipartite graph. A file may contain other sections, whose content is free (§21.4).

Standard sections are made of *tokens* separated by white space. A token is either plain, i.e., a sequence of characters other than white space, or quoted, i.e., enclosed in double quotes, in which case it may contain white space. In both kinds a backslash introduces an escape sequence of C: `\\`, `\"`, `\'`, `\?`, `\a`, `\b`, `\f`, `\n`, `\r`, `\t`, `\v`, a character in octal (one to three digits) or in hexadecimal (`\x` with one or two digits). Moreover, the node and arc sections are *column oriented*: their first line gives the names of the maps, and each following line, up to the next header, gives the values of these maps for one item. A short file shows the structure:

```
@nodes
label  name
0      "the source"
1      a
2      t
@arcs
           label  capacity
0    1     0      16
0    2     1      12
1    2     2      4
@attributes
source  0
target  2
```

In a node section, the map named `label` plays a special role, since its values identify the nodes in the rest of the file; it is needed whenever the arcs or the attributes refer to the nodes. In an arc section, each line starts with the labels of the source and of the target node, followed by the values of the maps; the map `label` is optional here, and it identifies the arcs for the attributes. A section without maps must contain a single `-` in its first line. For an undirected graph, the section `@edges` (or `@arcs`) describes the edges, and two columns whose names are the name of a map prefixed with `+` and `-` hold the values of an arc map on the two arcs of each edge. Each line of the section `@attributes` consists of two tokens, a name and a value; the value may be the label of a node or of an edge, and the label of an edge prefixed with `+` or `-` refers to the forward or the backward arc of that edge. For a bipartite graph, the sections `@red_nodes` and `@blue_nodes` replace `@nodes`, each with its own maps; a map present in both sections can be read as a node map.

21.2 Reading

We now describe `DigraphReader< DGR >`, which reads an LGF file into a digraph of type `DGR` by *batch processing*. One creates a reader, adds to it the rules that specify which maps and attributes are wanted and where they go, and executes them by `run()`, which reads the file once. The functions that add the rules return a reference to the reader, and are therefore chained:

```
DigraphReader< ListDigraph >( g , std::cin )
  .arcMap( "capacity" , cap ).node( "source" , s ).run();
```

Its three constructors take the digraph and an input stream (by default `std::cin`), a file name as a `std::strin g` or a file name as a `const char *`; the last two open the file and throw `IoError` when they cannot. With the same three kinds of source, `digraphReader()` returns a reader, which lets one write the chain without naming the type (the reader returned is moved, not copied).

> **abLEMON.** The readers and the writers take over the stream of the object returned by `digraphReader()` and the like through a constructor that takes it by rvalue reference, which C++23 requires; in LEMON 1.3.1 it took a non-constant lvalue reference, and the LGF classes did not compile under C++23 (version 1.4.0).

We now list the reading rules. A rule `nodeMap(caption, map)` reads the column `caption` of the node section into `map`, a writable node map, and `arcMap(caption, map)` does the same for the arc section; one column can be read into several maps. The value of each token is converted to the value type of the map by `operator>>` of a `std::istringstream`, and the token must be read whole: a token that cannot be converted, or that leaves characters behind, throws `FormatError`. A token read into a `std::string` is taken as it is. The overloads `node Map(caption, map, converter)` and `arcMap(caption, map, converter)` take instead a functor from `std:: string` to the value type. The rules `attribute(caption, value)` and `attribute(caption, value, conver ter)` read an attribute into a variable, and `node(caption, n)` and `arc(caption, a)` read an attribute whose value is a label into a node or an arc handle.

By default the reader uses the first section of each type. The functions `nodes(caption)`, `arcs(caption)` and `a ttributes(caption)` select instead the section of that type with the given caption. With `useNodes(map)` the nodes are taken as already existing, and they are not created: `map` gives the label of each existing node (its values are turned into strings by `operator<<`, or by a converter in the overload `useNodes(map, converter)`), and the node section only identifies the nodes by their labels, reading the other maps into them. Similarly, `useA rcs(map)` and `useArcs(map, converter)` reuse the existing arcs. These functions allow a *multipass* reading, e.g., to read a node set and one arc section into a digraph in a first pass, and a second arc section into an arc set of chapter 4 in a second pass. The functions `skipNodes()` and `skipArcs()` leave a section out: the nodes are then not created, and the arcs can only be read if `useNodes()` gives the labels.

Algorithm 21.1 (The reading of an LGF file by `run()`).

```
1  skip the lines before the first header
2  for each header "@type caption":
3      if type is nodes and none was read, and the caption matches:
4          read the map names; each line: tokens -> new node (or the node
           of its label, with useNodes); store the label; apply the rules
5      elif type is arcs or edges, likewise, the first two tokens being
           the labels of the source and of the target
6      elif type is attributes, likewise: each line is a name and a value,
           given to all the rules of that name
7      else skip the section
8  throw FormatError if the node or the arc section was not found (unless
   skipped), or if attribute rules exist and no attribute section was read
```

Note that the checks are rather strict. A map named by a rule and absent from the header, a column missing in a line, an extra token at the end of a line, a label not found, a map name repeated in a header and an attribute repeated in its section all throw `FormatError`; so does an attribute required by a rule and absent from the section. In all cases, the error carries the line number and, when the reader was opened on a file, the file name, and its `what()` gives both. Since the labels are kept in a `std::map` from the token to the item, reading n nodes and m arcs costs $O((n+m)\log n)$ string comparisons, besides the conversions.

For an undirected graph one uses `GraphReader< GR >`, which reads it in the same way, with the rules `nodeMap( )`, `edgeMap()`, `attribute()`, `node()` and `edge()`, the selection functions `nodes()`, `edges()` and `attributes()`, and `useNodes()`, `useEdges()`, `skipNodes()` and `skipEdges()`. Its rule `arcMap(caption, map)` reads the two

columns `+caption` and `-caption` into the values of an arc map on the forward and the backward arc of each edge, and `arc(caption, a)` reads an attribute whose value is an edge label with a sign. Similarly, `BpGraphReader< BGR >` reads a bipartite graph from the sections `@red_nodes`, `@blue_nodes` and `@edges`, all three of which must be present; it adds the rules `redNodeMap()`, `blueNodeMap()`, `redNode()` and `blueNode()`, and its `nodeMap()` reads a map that both node sections contain. The functions `graphReader()` and `bpGraphReader()` return readers as `digraphReader()` does. In the code, each rule is an object of the namespace `_reader_bits`: a `MapStorage` (or a `GraphArcMapStorage`, for the two columns of an arc map) for a map, and a `ValueStorage` for an attribute. The label of an attribute is resolved by a `MapLookUpConverter`, a `DoubleMapLookUpConverter` (for a node of a bipartite graph, searched among the red and the blue labels) or a `GraphArcLookUpConverter` (for a signed edge label).

21.3 Writing

Symmetrically, `DigraphWriter< DGR >` writes a digraph in the same batch style: the rules `nodeMap(caption, map)`, `arcMap(caption, map)`, `attribute(caption, value)`, `node(caption, n)` and `arc(caption, a)`, executed by `run()`. Its constructors take the digraph and an output stream (by default `std::cout`) or a file name, and throw `IoError` when the file cannot be opened; `digraphWriter()` returns a writer in the same three ways. Each value of a map or of an attribute is converted to a token by `operator<<` of a `std::ostringstream`, or by a functor from the value type to `std::string` given as the third argument of the rule. A token that is empty, starts with `@`, or contains white space or one of the characters `\`, `"`, `'`, `\a` and `\b`, is written quoted and with escapes; the others are written as they are. We remark that the default conversion writes a `double` with the default precision of the streams, i.e., 6 significant digits, so that a file written with it does not give back the same values when it is read; a converter that writes 17 significant digits (as in the example of §21.8) avoids this.

Each item gets a label from the writer. If a map is written under the name `label`, its values are the labels, and the items are written in the order of these values (an order that `operator<` must define, otherwise `FormatError` is thrown). Otherwise the writer adds a column `label` with the ids of the items, and writes them in the order of the ids. Hence, the arc section always has the label column, and therefore it never needs the `-` of §21.1. The functions `nodes(caption)`, `arcs(caption)` and `attributes(caption)` give a caption to the sections, `skipNodes()` and `skipArcs()` leave a section out (its labels are still computed, for the sections that follow), and `ostream()` returns the stream of the writer. These allow writing two arc sections in two passes on the same stream: the second writer skips the nodes and appends its arc section to the output of the first.

Note that the writer keeps the maps and the values of the rules by reference, and reads them only in `run()`. Thus, they must exist until then. A temporary passed to a rule lives until the end of the full expression that contains it; this is enough in a chain that ends with `run()`, but not when the rules and `run()` are separated.

> **abLEMON.** The documentation of the writers states that the values of their rules are kept by reference; in LEMON 1.3.1 the test of the format passed a temporary arc to a writer, which read it after its end (trac #683, version 1.4.0).

For an undirected graph, `GraphWriter< GR >` is used, with `edgeMap()` and `edge()` for the edges and an `arcMap()` that writes the columns `+caption` and `-caption`; the rule `arc(caption, a)` writes the label of the edge of `a` with the sign of its direction. Similarly, `BpGraphWriter< BGR >` writes the sections `@red_nodes`, `@blue_nodes` and `@edges`, with the rules `redNodeMap()`, `blueNodeMap()`, `redNode()` and `blueNode()`; its `nodeMap()` writes the map in both node sections, and its `nodes(caption)` gives the caption to both. The functions `graphWriter()` and `bpGraphWriter()` return writers. Their rules are objects of the namespace `_writer_bits` with the same names as those of the readers (`MapStorage`, `GraphArcMapStorage`, `ValueStorage`, `MapLookUpConverter`, `DoubleMapLookUpConverter` and `GraphArcLookUpConverter`), and the items are sorted by their labels through the comparators `MapLess` and `GraphArcMapLess`.

21.4 Extra sections and the contents of a file

Sections of other types are read by `SectionReader` and written by `SectionWriter`, which are constructed, like the other readers and writers, from a stream or a file name, or returned by `sectionReader()` and `sectionWriter()`. In a `SectionReader`, `sectionLines(type, functor)` gives each line of the section `@type` to a functor that takes a `std::string`, after the empty lines and the comments have been removed and the leading white space trimmed; `sectionStream(type, functor)` instead calls once a functor that takes the `std::istream &` and an `int &` line number, which it reads from and updates. A section is read at most once (a second section of the same type throws `FormatError`), and `run()` throws `FormatError` when a section with a rule is missing. In a

`SectionWriter`, `sectionLines(type, generator)` writes the strings returned by a functor without arguments until it returns an empty string, and `sectionStream(type, functor)` lets a functor write to the stream, its last line ending with a newline. The two kinds of rules are stored as objects of the classes `LineSection` and `StreamSection` of the namespaces `_reader_bits` and `_writer_bits`. These classes act on the extra sections only, and they can be run on the same file as the graph readers, which skip the sections they do not know.

Finally, `LgfContents` reads a file without knowing its contents and records its structure, for general tools that must guess what a file holds. After `run()`, `nodeSectionNum()`, `nodeSection(i)` and `nodeMapNames(i)` give the number of node sections, the caption of the i-th one and the names of its maps; `arcSectionNum()`, `arcSection(i)` and `arcMapNames(i)` do the same for the arc and edge sections, with the synonyms `edgeSectionNum()`, `edgeSection(i)` and `edgeMapNames(i)`. Also, `attributeSectionNum()`, `attributeSectionNames(i)` and `attributes(i)` describe the attribute sections, and `extraSectionNum()` and `extraSection(i)` give the types of the other sections.

21.5 The DIMACS formats

We now turn to the formats of the first DIMACS implementation challenge (Johnson and McGeoch 1993). They are line oriented: a line starting with `c` is a comment, the line `p type n m` gives the type of the problem and the numbers of nodes and arcs, a line starting with `n` describes a node, and a line starting with `a` an arc, the nodes being numbered from 1. A call of `dimacsType(is)` reads the comments and the problem line, and returns a `DimacsDescriptor`, whose fields are the `type` (`NONE`, `MIN`, `MAX`, `SP` or `MAT`), the `nodeNum` and the `edgeNum`; an unknown type, a missing problem line or another line before it throws `FormatError`. Each reader takes the descriptor as its last, optional, argument; without it, the reader calls `dimacsType()` itself, and with it, the reader continues on the stream where `dimacsType()` stopped. A reader clears the graph, creates the n nodes in order, and throws `FormatError` when the type of the file is not the one it reads.

Function	Type	Lines read	Data written
`readDimacsMin(is, g, lower, capacity, cost, supply, infty, desc)`	`min`	`n id supply`, `a u v low cap cost`	lower bounds, capacities, costs, supplies (0 when absent)
`readDimacsMax(is, g, capacity, s, t, infty, desc)`	`max`	`n id s`, `n id t`, `a u v cap`	capacities, source, target
`readDimacsSp(is, g, length, s, desc)`	`sp`	`n id`, `a u v len`	lengths, source
`readDimacsCap(is, g, capacity, infty, desc)`	`max` or `sp`	`a u v cap`	capacities (or lengths)
`readDimacsMat(is, g, desc)`	`mat`	`a u v`	the arcs (or edges)

Table 21.1. The DIMACS readers of `dimacs.h`; the problem line and the comments are read as described in the text.

An infinite capacity is written in the DIMACS files in two ways, and the readers translate both into the value `infty`. In a `min` file, an arc whose capacity is smaller than its lower bound gets the capacity `infty`, and in a `max` file (also through `readDimacsCap()`) an arc with a negative capacity gets it. When `infty` is 0, its default, the readers take `std::numeric_limits< Capacity >::infinity()` if the type has an infinity, and its maximum value otherwise. As for `readDimacsMat()`, it adds an edge for each `a` line to an undirected graph, and an arc to a digraph; note that it reads only the `a` lines, and the `e` lines that some matching and clique files use are skipped. Conversely, `writeDimacsMat(os, g, comment)` writes a digraph in the format `mat`, i.e., an optional comment line, the problem line, and one `a` line per arc; the nodes are numbered from 1 in the order of the node iterator.

abLEMON. `dimacsType()` is inline, and hence two translation units that include `dimacs.h` link together, which they did not in LEMON 1.3.1 (version 1.4.0).

21.6 The graph6 format of nauty

Our next format comes from the program `geng` of the nauty package (McKay and Piperno 2014) generates all the non-isomorphic undirected graphs of a class with a given number of nodes, writing one graph per line in the *graph6* format. A call of `readNautyGraph(graph, is)` reads one line of the stream (by default `std::cin`) and builds the graph it describes, after clearing `graph`; it returns the stream, and it can thus be used as the

condition of a loop that reads a whole file. In graph6, each character carries six bits, its code minus 63, and the upper triangle of the adjacency matrix is listed column by column.

Algorithm 21.2 (The decoding of a graph6 line).

```
1  skip the header ">>graph6<<" if the line starts with '>'
2  c := next character - 63
3  if c < 63: n := c
4  else: n := the 18 bits of the next three characters (minus 63 each)
5  create n nodes
6  for j := 1 to n - 1, for i := 0 to j - 1:
7      take the next bit, from the most significant of each character;
       if it is 1, add the edge {i, j}
```

For instance, the line `D~{` describes the complete graph on 5 nodes: `D` gives $n = 68 - 63 = 5$, and `~{` give the bits 111111 and 1111 of the 10 pairs, followed by two bits of padding. Note that the function does not check the length of the line, and therefore a truncated line is read past its end. The header suggests to pipe the output of `geng` into the program, since the files are large.

21.7 Drawing a graph in EPS

A call of `graphToEps(g, os)` draws a graph or a digraph as an Encapsulated PostScript picture on a stream (by default `std::cout`); the overloads `graphToEps(g, file_name)` write to a file, which they open and close themselves, and throw `IoError` when they cannot open it. It returns an object of the class `GraphToEps`, whose member functions set the parameters of the picture by the technique of the named parameters (chapter 3), and whose `run()` writes it. Nothing is written without `run()`, which therefore must close the chain.

```
graphToEps( g , "flow.eps" ).coords( coords ).arcWidths( cap )
  .autoArcWidthScale().drawArrows().title( "a flow problem" ).run();
```

The maps of the picture are given by functions that return a new object with the type of the map, and they are kept by reference. First, `coords(map)` gives the position of each node, a node map with values of type `dim2::Point` (chapter 8), without which all the nodes are drawn at the point $(1, 1)$. Then `nodeSizes(map)`, `nodeShapes(map)`, `nodeColors(map)`, `nodeTexts(map)`, `nodeTextColors(map)` and `nodePsTexts(map)` set the size, the shape (`CIRCLE`, `SQUARE`, `DIAMOND`, `MALE`, `FEMALE`, i.e., 0 to 4), the color, a text written on the node (of any type that `operator<<` writes), its color, and a block of PostScript code drawn at the node; and `arcWidths(map)` and `arcColors(map)` set the width and the color of the arcs. Colors are objects of the class `Color` of `color.h`, and a `Palette` composed with an integer map (`composeMap()`, chapter 6) gives distinct colors.

Other parameters are set by functions that return the same object. Usually one scales the sizes by `nodeScale(d)` (0.01 by default) and `arcWidthScale(d)` (0.003), and both are relative to the diagonal of the bounding box of the nodes, unless `absoluteNodeSizes()` and `absoluteArcWidths()` make them absolute; `autoNodeScale()` and `autoArcWidthScale()` rescale them so that the largest size equals the scale. Similarly, `scale(d)` scales the picture; by default, `preScale()` multiplies or divides the scale by powers of 10 until the bounding box is between 100 and 1000 points, and `preScale(false)` turns this off. Also, `scaleToA4()` fits the picture to an A4 page, `border(b)` and `border(x, y)` set the margin (10 points by default), and `negateY()` flips the vertical axis. As for the arcs, `drawArrows()` draws the arrowheads, whose size `arrowLength(d)` and `arrowWidth(d)` set, and `enableParallel()` draws parallel arcs as distinct curves at the distance `parArcDist(d)`. Finally, `hideNodes()` and `hideArcs()` leave the nodes or the arcs out, `nodeTextSize(d)` sets the size of the texts, and `distantColorNodeTexts()` and `distantBWNodeTexts()` color each text so that it stands out on its node. The functions `undirected()` and `directed()` choose whether the edges of an undirected graph are drawn once (the default for undirected graphs) or as two arcs, and `title(t)` and `copyright(t)` write the DSC fields `%%Title` and `%%Copyright` of the header, which also gets the creation date. Also, `edgeWidths()`, `edgeColors()`, `edgeWidthScale()`, `autoEdgeWidthScale()`, `absoluteEdgeWidths()`, `parEdgeDist()` and `hideEdges()` are aliases of the functions on the arcs. The demo `graph_to_eps_demo` (chapter 22) uses most of these parameters.

21.8 Examples

Our first program writes a network with integer capacities, floating-point costs, a name for each node (one of which contains a blank and is therefore quoted) and the source, the target and a caption as attributes, and reads it back into another digraph. It writes the costs twice, once with the default conversion and once with a

converter that prints 17 significant digits, and checks that only the second gives back the same values. Then it checks that a rule for a map absent from the file throws `FormatError`.

```
// The LGF reader and writer of chapter 21: a network with integer
// capacities, floating-point costs, a name for each node and the source
// and the target as attributes is written to a string and read back into
// another digraph. The costs are written once with the default converter,
// which prints 6 significant digits, and once with a converter that
// prints 17, which gives back the same doubles.

#include <cstdio>
#include <iostream>
#include <sstream>
#include <string>
#include <lemon/smart_graph.h>
#include <lemon/lgf_reader.h>
#include <lemon/lgf_writer.h>

using namespace lemon;

typedef SmartDigraph Digraph;

// a converter from a double to a token with all its digits
struct Exact
{
 std::string operator()( double v ) const {
  char buf[ 32 ];
  std::snprintf( buf , sizeof( buf ) , "%.17g" , v );
  return( buf );
  }
 };

int main()
{
 Digraph g;
 Digraph::NodeMap< std::string > name( g );
 Digraph::ArcMap< int > cap( g );
 Digraph::ArcMap< double > cost( g );
 Digraph::Node v[ 4 ];
 const char * names[ 4 ] = { "s" , "a" , "b" , "the sink" };
 for( int i = 0 ; i < 4 ; ++i ) {
  v[ i ] = g.addNode();
  name[ v[ i ] ] = names[ i ];
  }
 int arcs[ 5 ][ 2 ] = { { 0 , 1 } , { 0 , 2 } , { 1 , 2 } , { 1 , 3 } ,
                        { 2 , 3 } };
 for( int k = 0 ; k < 5 ; ++k ) {
  Digraph::Arc e = g.addArc( v[ arcs[ k ][ 0 ] ] , v[ arcs[ k ][ 1 ] ] );
  cap[ e ] = 10 + k;
  cost[ e ] = 1.0 / ( k + 3 );  // 1/3, 1/4, ... not exact in 6 digits
  }
 std::string caption = "a small network";

 std::ostringstream plain , exact;
 digraphWriter( g , plain )
   .nodeMap( "name" , name )
   .arcMap( "capacity" , cap )
   .arcMap( "cost" , cost )
   .node( "source" , v[ 0 ] )
   .node( "target" , v[ 3 ] )
   .attribute( "caption" , caption )
   .run();
 digraphWriter( g , exact )
   .nodeMap( "name" , name )
   .arcMap( "capacity" , cap )
```

```
   .arcMap( "cost" , cost , Exact() )
   .node( "source" , v[ 0 ] )
   .node( "target" , v[ 3 ] )
   .attribute( "caption" , caption )
   .run();
 std::cout << exact.str();

 bool ok = true;
 for( int pass = 0 ; pass < 2 ; ++pass ) {
  Digraph h;
  Digraph::NodeMap< std::string > hname( h );
  Digraph::ArcMap< int > hcap( h );
  Digraph::ArcMap< double > hcost( h );
  Digraph::Node src , trg;
  std::string hcaption;
  std::istringstream is( pass == 0 ? plain.str() : exact.str() );
  digraphReader( h , is )
    .nodeMap( "name" , hname )
    .arcMap( "capacity" , hcap )
    .arcMap( "cost" , hcost )
    .node( "source" , src )
    .node( "target" , trg )
    .attribute( "caption" , hcaption )
    .run();
  ok = ok && countNodes( h ) == 4 && countArcs( h ) == 5 &&
       hname[ src ] == "s" && hname[ trg ] == "the sink" &&
       hcaption == caption;
  // the labels are the ids, so the arcs come back in the same order
  bool same = true;
  for( Digraph::ArcIt e( g ) ; e != INVALID ; ++e ) {
   Digraph::Arc f = h.arcFromId( g.id( e ) );
   ok = ok && hcap[ f ] == cap[ e ] &&
        hname[ h.source( f ) ] == name[ g.source( e ) ];
   same = same && hcost[ f ] == cost[ e ];
   }
  ok = ok && same == ( pass == 1 );  // 6 digits lose, 17 do not
  }

 // a missing map is reported as a FormatError with its line
 try {
  Digraph h;
  Digraph::ArcMap< int > w( h );
  std::istringstream is( plain.str() );
  digraphReader( h , is ).arcMap( "weight" , w ).run();
  ok = false;
  }
 catch( const FormatError & err ) {
  std::cout << "expected error: " << err.what() << std::endl;
  }
 return( ok ? 0 : 1 );
 }
```

Our second program reads a maximum flow instance in DIMACS format from a string, with an arc of infinite capacity, and checks the maximum flow found by `Preflow`. Then it writes the digraph in the format `mat` and reads it back as an undirected graph, decodes two graph6 lines, and draws the digraph in EPS on a string, checking the header of the picture.

```
// The other formats of chapter 21: a maximum flow instance and a plain
// graph in DIMACS format, a graph in the graph6 format of nauty, and the
// EPS drawing of a digraph, all through strings.

#include <iostream>
#include <sstream>
#include <string>
```

```
#include <lemon/smart_graph.h>
#include <lemon/dimacs.h>
#include <lemon/nauty_reader.h>
#include <lemon/graph_to_eps.h>
#include <lemon/preflow.h>

using namespace lemon;

int main()
{
 bool ok = true;

 // a "p max" file: arc 2 -> 4 has a negative capacity, i.e., infinite
 std::istringstream max_file( "c a small maximum flow problem\n"
                              "p max 4 5\n"
                              "n 1 s\n"
                              "n 4 t\n"
                              "a 1 2 4\n"
                              "a 1 3 2\n"
                              "a 2 3 3\n"
                              "a 2 4 -1\n"
                              "a 3 4 5\n" );
 DimacsDescriptor desc = dimacsType( max_file );
 ok = ok && desc.type == DimacsDescriptor::MAX && desc.nodeNum == 4 &&
      desc.edgeNum == 5;
 SmartDigraph g;
 SmartDigraph::ArcMap< int > cap( g );
 SmartDigraph::Node s , t;
 readDimacsMax( max_file , g , cap , s , t , 1000 , desc );
 Preflow< SmartDigraph , SmartDigraph::ArcMap< int > > pre( g , cap , s , t );
 pre.run();
 ok = ok && countArcs( g ) == 5 && pre.flowValue() == 6 &&
      cap[ g.arcFromId( 3 ) ] == 1000;

 // a plain digraph written as "p mat" and read back as a graph
 std::ostringstream mat;
 writeDimacsMat( mat , g , "the arcs of the flow problem" );
 std::istringstream mat_in( mat.str() );
 SmartGraph u;
 readDimacsMat( mat_in , u );
 ok = ok && countNodes( u ) == 4 && countEdges( u ) == 5;

 // graph6: "D" is 5 nodes, "~{" the 10 bits of the upper triangle
 std::istringstream g6( "D~{\nC]\n" );
 SmartGraph k5 , c4;
 readNautyGraph( k5 , g6 );
 readNautyGraph( c4 , g6 );  // 4 nodes, bits 011110: a 4-cycle
 ok = ok && countNodes( k5 ) == 5 && countEdges( k5 ) == 10 &&
      countNodes( c4 ) == 4 && countEdges( c4 ) == 4;

 // the EPS drawing of the flow problem, the arc widths by capacity
 SmartDigraph::NodeMap< dim2::Point< double > > coords( g );
 double xy[ 4 ][ 2 ] = { { 0 , 0 } , { 2 , 1 } , { 2 , -1 } , { 4 , 0 } };
 for( int i = 0 ; i < 4 ; ++i )
  coords[ g.nodeFromId( i ) ] =
    dim2::Point< double >( xy[ i ][ 0 ] , xy[ i ][ 1 ] );
 std::ostringstream eps;
 graphToEps( g , eps )
   .coords( coords )
   .arcWidths( cap )
   .autoArcWidthScale()
   .arcWidthScale( 0.01 )
   .drawArrows()
   .title( "a small maximum flow problem" )
   .run();
```

```
ok = ok && eps.str().compare( 0 , 23 , "%!PS-Adobe-2.0 EPSF-2.0" ) == 0 &&
     eps.str().find( "%%BoundingBox:" ) != std::string::npos &&
     eps.str().find( "showpage" ) != std::string::npos;

std::cout << mat.str() << "max flow " << pre.flowValue() << ", EPS of "
          << eps.str().size() << " bytes" << std::endl;
return( ok ? 0 : 1 );
}
```

21.9 Exercises

Exercise 21.1. Write a digraph with two arc maps into an LGF file in two passes, each with its own arc section and caption, and read it back into a digraph and an arc set of chapter 4 with `useNodes()` and `arcs(caption)` (the multipass reading is exercised by `test/lgf_reader_writer_test.cc`).

Exercise 21.2. Write a `double` map with the default conversion and read it back; measure the largest relative error, and explain it by the precision of the streams (`manual/examples/21-lgf.cc`).

Exercise 21.3. Write an undirected graph with an arc map by `GraphWriter`, inspect the columns `+caption` and `-caption`, and read the map back by `GraphReader::arcMap()`; then write an arc as an attribute and read it back by `arc()` (`test/lgf_test.cc`).

Exercise 21.4. Use `SectionWriter` and `SectionReader` to store a list of paths, one per line as a sequence of arc labels, in an extra section of a file that also contains the digraph, and read both back.

Exercise 21.5. Convert a `min` file of DIMACS into LGF as the tool `dimacs-to-lgf` of chapter 22 does, and check that `NetworkSimplex` (chapter 13) finds the same optimal cost on both.

Exercise 21.6. Count the non-isomorphic connected planar graphs with 7 nodes from the output of `geng -c 7` , read by `readNautyGraph()` and tested by `PlanarityChecking` (chapter 18), and compare the count with the known value 646.

Exercise 21.7. Draw a spanning tree of a random geometric graph by `graphToEps()`, with the tree arcs wider and in another color than the others, and compare the bounding box with and without `preScale()` (`manual/examples/21-formats.cc` draws a small digraph).

22. The tools

This chapter describes the programs distributed with the library: the three command-line tools of `tools/`, i.e., `dimacs-solver`, `dimacs-to-lgf` and `lgf-gen`, the script `lemon-0.x-to-1.x.sh`, and the three demos of `demo/`. These tools are small applications of the classes of the previous chapters, and their options are read from their sources; we also give, for each of them, the classes it uses, so that one can reproduce its work in a program (as the example of §22.6 does for two of them).

22.1 Building and calling the tools

By default CMake builds the tools, when the option `LEMON_BUILD_TOOLS` is on, and installs them into the directory of the executables (`bin` under the prefix, by `GNUInstallDirs`); the script `lemon-0.x-to-1.x.sh` is installed with them, except on Windows. Similarly, the demos are built when `LEMON_BUILD_DEMOS` is on, also by default, but they are not installed. Both can also be built with GNU make, through the makefiles of `tools/` and `demo/`, without a configuration step (chapter 2).

All three tools parse their command line with `ArgParser` (chapter 8), and hence they share its conventions. An option is a word preceded by a single dash, e.g., `-q` or `-infcap 100`, the options of a group marked as exclusive may not be given together, and `-h`, `-help` and `--help` print the list of the options with their descriptions and exit. The words that are not options are the file arguments; in `dimacs-solver` and `dimacs-to-lgf` they are an optional input file and an optional output file, which default to the standard input and output, and more than two of them is an error. Since the input file is read once and the problem line comes first, both tools also read from a pipe.

22.2 dimacs-solver

Our first tool, `dimacs-solver`, reads a problem in one of the DIMACS formats of §21.5 and solves it with the algorithm of the library for that type, reporting the times of the reading, of the construction of the algorithm object, and of the run. It is meant (apparently) as a benchmark of the algorithms on the instances of the DIMACS challenges: apart from the optimal value, it writes no solution, and the output file, which it opens when it is given, stays empty.

Option	Meaning	Default
`-q`	print no report	off
`-int`	`int` for capacities, costs and supplies	on
`-long`	`long long` (only where the compiler has it)	off
`-double`	`double`	off
`-ldouble`	`long double`	off
`-infcap v`	the value of an infinite capacity (§21.5)	`0`, i.e., the infinity or the maximum of the type

Table 22.1. The options of `dimacs-solver`; the four options of the type form an exclusive group.

At the start, the tool reads the problem line by `dimacsType()`, and, unless `-q` is given, writes the type of the problem and the numbers of nodes and arcs on the standard output; a file of unknown type makes `dimacsType()` throw `FormatError`, which the tool does not catch. Also, the rest of the report goes to the standard error. Then it builds a `SmartDigraph` (or a `SmartGraph` for the type `mat`) and proceeds by type, as follows. For a `sp` file, it reads the digraph and the lengths by `readDimacsSp()` and computes the shortest paths from the source by `Dijkstra` (chapter 10), without printing the distances. For a `max` file, it reads the capacities by `readDimacsM`

`ax()`, computes a maximum flow by `Preflow` (chapter 12), and prints its value. For a `mat` file, it reads the graph by `readDimacsMat()` and prints the cardinality of a maximum matching computed by `MaxMatching` (chapter 17). For a `min` file, it reads the problem by `readDimacsMin()` and solves it by `NetworkSimplex` (chapter 13), as in Algorithm 22.1, printing whether a feasible flow was found and, unless the problem is infeasible, its cost, which is computed in a wider type (`long long` when the values are `int`, the type of the values otherwise).

Algorithm 22.1 (The minimum cost flow problem in `dimacs-solver`).

```
1  read lower, capacity, cost, supply (readDimacsMin, infinite = -infcap)
2  sigma := sum of the supplies
3  eps := the data tolerance of NetworkSimplex for the largest magnitude
          of the supplies, of the lower bounds and of the finite capacities
4  if |sigma| <= eps: sigma := 0
5  if sigma > 0: supply type LEQ, else GEQ (the default of NetworkSimplex)
6  run NetworkSimplex; print "found" or "not found", and the total cost
```

Note that the choice of line 5 follows from the meaning of the two supply types of chapter 13. With a total supply larger than the total demand the problem can only be feasible if some supply is left unused, which the type `LEQ` allows; with a smaller one, it needs the type `GEQ`, under which some demand stays unsatisfied, and with a zero sum the two types are equivalent to the equations. Lines 3 and 4 treat as zero a sum that only the rounding of fractional supplies keeps away from zero, with the tolerance that `NetworkSimplex` itself uses to balance the supplies (chapter 13).

> **abLEMON.** In the version after 1.9.0, `dimacs-solver` takes as zero a sum of the supplies within the tolerance of `NetworkSimplex`; before, a sum of 10^{-12} left by fractional supplies of a `double` instance made it choose and announce the type `LEQ`.

22.3 dimacs-to-lgf

Next, `dimacs-to-lgf` converts a DIMACS file into an LGF file (§21.1), and it has no option besides the input and the output file. It reads the problem by `dimacsType()` and by the reader of its type, all values being of type `double`, into a `SmartDigraph`, and it writes the digraph by `DigraphWriter` with the maps and attributes of Table 22.2; the nodes are labeled by their ids, i.e., by the numbers of the DIMACS file minus one.

Type	Node maps	Arc maps	Attributes
`min`	`supply`	`lower`, `capacity`, `cost`	`problem` = `min`
`max`		`capacity`	`source`, `target`, `problem` = `max`
`sp`		`capacity` (the lengths)	`source`, `problem` = `sp`
`mat`			`problem` = `mat`

Table 22.2. The contents of the LGF file written by `dimacs-to-lgf`, by type of the DIMACS file.

Two remarks are in order. First, the infinite capacities of a `min` or `max` file are written as the infinity of `double`, i.e., as the token `inf`; whether a program can read this token back depends on the `operator>>` of its standard library, and it may need a converter (§21.2). Second, the values are written with the default conversion, i.e., with 6 significant digits (§21.3), which is exact for the integer data of the usual DIMACS instances below 10^6 but not for larger or fractional values. As in `dimacs-solver`, a file of unknown type makes `dimacsType()` throw `FormatError`, which ends the tool.

22.4 lgf-gen

Finally, `lgf-gen` generates random plane graphs, i.e., graphs whose nodes are points of the plane, and writes them as LGF files with the coordinates of the nodes, optionally with an EPS drawing. It is a convenient source of test instances for the geometric and graph algorithms of the library.

Option	Meaning	Default
`-n N`	number of nodes	100

Option	Meaning	Default
`-disc`, `-square`, `-gauss`	the nodes uniformly in the unit disc, uniformly in the square $[-1,1]^2$, or by a two-dimensional Gaussian distribution	`-disc`
`-cities k`	the nodes in k clusters, with random centres and sizes	1
`-area a`	the scale of the clusters (their full relative area)	1
`-2con`, `-tree`, `-tsp`, `-tsp2`, `-dela`	the edges: a 2-connected planar graph, a minimum spanning tree, a TSP tour, a tree-based TSP tour, the Delaunay triangulation	no edges
`-g d`	the girth parameter of `-2con`	10
`-dir`	write a digraph, each edge as two opposite arcs	off
`-eps`	also write `prefix.eps`	off
`-nonodes`	draw only the edges in the EPS file	off
`-rand`, `-seed s`	seed the generator with the time, or with `s`	the default seed

Table 22.3. The options of `lgf-gen`; the distributions, the algorithms and the two ways of seeding form three exclusive groups. The only file argument is the prefix of the output files, `lgf-gen-out` by default.

Thus, the nodes are placed first. With k clusters, each cluster s receives a random weight w_s, uniform in $[0,1)$, and a share of the N nodes proportional to it; its centre is a random point of the unit disc (the origin when $k=1$), and its points are drawn by the chosen distribution and scaled by $a\sqrt{w_s/\sum_r w_r}$. Then the edges are built by the chosen algorithm, the lengths being the Euclidean distances, as follows.

Algorithm 22.2 (The edges of `lgf-gen`).

```
-dela : the Delaunay triangulation of the points (delaunay() of the tool)
-tree : the Delaunay triangulation; keep the spanning tree that
        kruskal() finds on the squared lengths
-tsp  : the cycle of the nodes in the order of creation, improved by
        2-opt moves on the lengths until none applies
-tsp2 : the tree of -tree; join its nodes of odd degree in pairs, by
        increasing distance; shortcut the Euler tour of the result to
        a tour; improve it by 2-opt
-2con : for each pair of nodes by increasing length: add the edge if it
        crosses no edge already added and the two nodes do not have two
        edge-disjoint paths of total length at most d (Suurballe, unit
        lengths); then remove, in reverse order, each edge whose ends
        keep two such paths without it
```

The algorithm `-2con` thus gives a plane graph, since no two edges cross, and d is the value of `-g`; `Suurballe` is the algorithm of chapter 10. At the end, the tool prints the number of nodes and edges and the total length of the edges, and, for `-tsp` and `-tsp2`, the number of 2-opt moves. It writes `prefix.lgf`, by `GraphWriter`, or by `DigraphWriter` with `-dir`; the only node maps are `coordinates_x` and `coordinates_y`, the coordinates multiplied by 600. With `-eps`, it also writes `prefix.eps` by `graphToEps()` (§21.7), scaled to an A4 page.

22.5 lemon-0.x-to-1.x.sh and the demos

We now turn to `lemon-0.x-to-1.x.sh`, which helps to port a program written for the old series 0.x of LEMON: it rewrites in place each file given as an argument by a long sequence of `sed` substitutions, and with no argument, or with `-h`, `-help` or `--help`, it prints its usage. The substitutions rename the graph concepts of the old series: a graph becomes a digraph, an undirected graph a graph, an edge an arc and an undirected edge an edge, and the `A`- and `B`-nodes of the bipartite graphs become red and blue nodes. They also rename the headers (e.g., `digraph_adaptor.h` becomes `adaptors.h` and `topology.h` becomes `connectivity.h`), the named template parameters `Def*` into `Set*`, the adaptors into their current names (e.g., `SubDigraphAdaptor` becomes `SubDigraph` and `ResDigraphAdaptor` becomes `ResidualDigraph`), several maps (e.g., `InvertableMap` becomes `CrossRefMap`), and the LP classes (`LpGlpk` becomes `GlpkLp`, and so on). Since the substitutions are textual, the result must be read and compiled; in particular, the words *graph* and *edge* in the comments are renamed too.

Each of the three demos of `demo/` is a short program that shows the use of one tool of the library. First, `arg_parser_demo` declares options of all the kinds of `ArgParser` (a mandatory integer `-n`, doubles with and without a reference, a synonym, a string, switches, an exclusive and mandatory group of three options and the file arguments), parses the command line with `throwOnProblems()`, and prints what it found. Then `graph_to_eps_demo` builds a small digraph and writes seven EPS files, `graph_to_eps_demo_out_1_pure.eps` to `graph_to_eps_demo_out_7_colors.eps`, which show the plain drawing, the sizes, shapes, colors and widths given by maps, the arrowheads, the parallel arcs, the fit to an A4 page and, on a second graph, the colors of a `Palette` with the texts of the nodes in distant colors. Finally, `lgf_demo` reads the digraph, the capacities and the source and target of the file `digraph.lgf` of the directory of the demos, which it must be started from, and writes them back on the standard output.

22.6 An example

Our program does, on a small minimum cost flow problem, what two of the tools do. It declares the options of `dimacs-solver` and parses a given command line with them, reads a DIMACS `min` file from a string with an arc of infinite capacity, chooses the supply type by the sign of the sum of the supplies as in Algorithm 22.1, and solves the problem by `NetworkSimplex`. Then it writes the problem as `dimacs-to-lgf` writes it, reads the LGF text back, and checks the costs and the attribute `problem`.

```
// What the tools of chapter 22 do on a minimum cost flow instance, in one
// program: the options of dimacs-solver parsed by ArgParser from a given
// command line, the DIMACS file read as dimacs-solver reads it, the type of
// the supply constraints chosen by the sign of the sum of the supplies,
// and the conversion to LGF that dimacs-to-lgf writes, read back.

#include <iostream>
#include <sstream>
#include <string>
#include <lemon/arg_parser.h>
#include <lemon/smart_graph.h>
#include <lemon/dimacs.h>
#include <lemon/lgf_reader.h>
#include <lemon/lgf_writer.h>
#include <lemon/network_simplex.h>

using namespace lemon;

typedef SmartDigraph Digraph;

int main()
{
 // the command line "dimacs-solver -q -double -infcap 100 net.min"
 const char * argv[] = { "dimacs-solver" , "-q" ,  "-double" ,
                         "-infcap" ,       "100" , "net.min" };
 ArgParser ap( 6 , argv );
 ap.throwOnProblems();  // an exception rather than exit(1)
 ap.other( "[INFILE [OUTFILE]]" )
   .boolOption( "q" , "Do not print any report" )
   .boolOption( "int" , "Use 'int' for capacities, costs etc. (default)" )
   .optionGroup( "datatype" , "int" )
   .boolOption( "double" , "Use 'double' for capacities, costs etc." )
   .optionGroup( "datatype" , "double" )
   .onlyOneGroup( "datatype" )
   .stringOption( "infcap" , "Value used for 'very high' capacities" , "0" )
   .run();
 bool ok = ap.given( "q" ) && ap.given( "double" ) && ! ap.given( "int" ) &&
           ap.files().size() == 1 && ap.files()[ 0 ] == "net.min";
 double infty;
 std::istringstream( static_cast< std::string >( ap[ "infcap" ] ) ) >> infty;
 ok = ok && infty == 100;

 // a "p min" file: node 1 supplies 4, node 4 demands 3, so that the sum
```

```
// is positive and the supplies are constraints of the type LEQ; the arc
// 2 -> 4 has an upper bound below its lower bound, i.e., infinite
std::istringstream min_file( "p min 4 5\n"
                             "n 1 4\n"
                             "n 4 -3\n"
                             "a 1 2 0 4 1\n"
                             "a 1 3 0 4 2\n"
                             "a 2 3 0 2 1\n"
                             "a 2 4 0 -1 3\n"
                             "a 3 4 1 5 1\n" );
Digraph g;
Digraph::ArcMap< double > lower( g ) , cap( g ) , cost( g );
Digraph::NodeMap< double > sup( g );
readDimacsMin( min_file , g , lower , cap , cost , sup , infty );
double sum = 0;
for( Digraph::NodeIt n( g ) ; n != INVALID ; ++n )
 sum += sup[ n ];
NetworkSimplex< Digraph , double > ns( g );
ns.lowerMap( lower ).upperMap( cap ).costMap( cost ).supplyMap( sup );
if( sum > 0 )
 ns.supplyType( ns.LEQ );
ok =
  ok && sum == 1 && cap[ g.arcFromId( 3 ) ] == 100 && ns.run() == ns.OPTIMAL;
// the 3 units demanded go along 1 -> 3 -> 4 at cost 3 each (the path
// 1 -> 2 -> 4 costs 4), and node 1 keeps the fourth unit
ok = ok && ns.totalCost() == 9;

// dimacs-to-lgf: the same maps under the same names, read back
std::ostringstream lgf;
digraphWriter( g , lgf )
  .nodeMap( "supply" , sup )
  .arcMap( "lower" , lower )
  .arcMap( "capacity" , cap )
  .arcMap( "cost" , cost )
  .attribute( "problem" , "min" )
  .run();
Digraph h;
Digraph::ArcMap< double > hcost( h );
std::string problem;
std::istringstream lgf_in( lgf.str() );
digraphReader( h , lgf_in )
  .arcMap( "cost" , hcost )
  .attribute( "problem" , problem )
  .run();
ok = ok && countArcs( h ) == 5 && problem == "min" &&
     hcost[ h.arcFromId( 3 ) ] == 3;

std::cout << lgf.str() << "minimum cost " << ns.totalCost() << std::endl;
return( ok ? 0 : 1 );
}
```

22.7 Exercises

Exercise 22.1. Generate a Delaunay triangulation with `lgf-gen -dela -n 1000`, read it by `GraphReader` with the coordinates, and check that its number of edges is at most $3N - 6$, as for any simple plane graph (and that `PlanarityChecking` of chapter 18 accepts it).

Exercise 22.2. Generate a TSP tour with `lgf-gen -tsp` and with `lgf-gen -tsp2` on the same seed, and compare their lengths with the tours of the classes of chapter 19 on the same points (read by `GraphReader`, with the costs given by the coordinates).

Exercise 22.3. Write a `min` file whose supplies sum to a positive value, and check that `dimacs-solver` announces the type `LEQ` and finds the same cost as a program that solves the problem by `NetworkSimplex` with this supply type (`manual/examples/22-tools.cc`).

Exercise 22.4. Convert a `max` file with an arc of negative capacity by `dimacs-to-lgf`, and read the result back by `DigraphReader` with a converter that maps the token `inf` to a large finite capacity; check that `Preflow` gives the same value as `dimacs-solver` on the original file.

Exercise 22.5. Run `arg_parser_demo` without the mandatory option `-n`, with two options of the group `-gra`, `-grb`, `-grc`, and with none of them, and explain each message from the declarations of the demo.

A. Writing a graph, a map or a heap

We are concerned in this appendix with the classes that a user may add to the library. These are typically a graph structure that computes its items from some private data (an implicit graph, or a view of another data structure), a map that computes its values on demand, an observer that follows the changes of a graph, and a heap. Such a class is accepted by the algorithms as soon as it conforms to the concept of its kind; also, most of the work is done by the classes of `lemon/bits/`, which turn a small base class into a complete graph. Section A.1 states the concepts as contracts, and section A.2 describes the base class of a graph and the extenders that complete it. Section A.3 deals with the alteration notifier through which a graph informs its maps, sections A.4 and A.5 with the maps and the heaps, and section A.6 with the checking of a new class. In the program of this appendix (`manual/examples/A-cycle-digraph.cc`) we write a digraph, a map and an observer, and we run `Dijkstra` on the first two.

A.1 The concepts as contracts

A concept class of `lemon/concepts/` (cf. chapter 3) lists what a generic algorithm may use, and hence what a new class must provide. For a digraph this is `concepts::Digraph`, i.e., the base, idable, iterable and mappable components of chapter 4, while an undirected graph conforms to `concepts::Graph` and a bipartite one to `concepts::BpGraph`. A structure that can grow, shrink or be cleared also conforms to the extendable, erasable and clearable components. Each of them requires the alterable one, since a structure that changes must notify its maps. However, the concepts state only the syntax; some semantic requirements are written only in their documentation, and the algorithms rely on them as well.

- The identifiers of the items of one kind are distinct and immutable, and they lie between 0 and `maxNodeId()` (or the other maxima). The maximum should not be much larger than the number of items, since the standard maps allocate one slot for each identifier up to it.
- Each iterator visits each item exactly once and then becomes `INVALID`. In an undirected graph `ArcIt` visits each of the two arcs of an edge once, and `InArcIt` and `OutArcIt` visit the arcs of the incident edges.
- The ends `u(e)` and `v(e)` do not change while the edge exists, and `direct(e, true)` goes from `u(e)` to `v(e)`.

For a map the contracts are `concepts::ReadMap< K , V >`, `concepts::WriteMap< K , V >`, `concepts::ReadWriteMap< K , V >` and `concepts::ReferenceMap< K , V , R , CR >` of `lemon/concepts/maps.h` (chapter 6), and for a heap `concepts::Heap< PR , IM , CMP >` of `lemon/concepts/heap.h` (section A.5). Each concept class has a nested template `Constraints`, whose function `constraints()` is the executable form of the contract that `checkConcept` instantiates (section A.6).

A.2 The base class and the extenders

Most of the interface of a graph does not depend on how the graph is stored, or depends on it only somewhat. Indeed, the iterators can be written once from functions such as `first(Node &)` and `next(Node &)`, the standard maps need only the identifiers and a notifier, and the ranges, `oppositeNode()`, `baseNode()` and `runningNode()` follow from the iterators. Hence, the header `lemon/bits/graph_extender.h` defines three class templates, `DigraphExtender< Base >`, `GraphExtender< Base >` and `BpGraphExtender< Base >`, which derive from a base class written by the user and add this common part. All the structures of chapter 4 are made in this way (e.g., `ListDigraph` derives from `DigraphExtender< ListDigraphBase >`). So are the arc and edge sets, by `ArcSetExtender` and `EdgeSetExtender` of `lemon/bits/edge_set_extender.h`, and the adaptors of chapter 5, by the extenders of `lemon/bits/graph_adaptor_extender.h`.

The base of a digraph. A base class for `DigraphExtender` defines the following members, which are those of `BaseDigraphComponent`, `IDableDigraphComponent` and the low-level part of `IterableDigraphComponent`.

- The handle types `Node` and `Arc`, with default and `Invalid` constructors, `==`, `!=` and `<`.
- The functions `source(a)` and `target(a)`.
- The functions `first(Node &)`, `next(Node &)`, `first(Arc &)` and `next(Arc &)`, which enumerate the items and set the handle to the invalid value after the last one. Also, `firstOut(Arc &, Node)`, `nextOut(Arc &)`, `firstIn(Arc &, Node)` and `nextIn(Arc &)` enumerate the arcs at a node.
- The functions `id(Node)`, `id(Arc)`, `nodeFromId(int)`, `arcFromId(int)`, `maxNodeId()` and `maxArcId()`.

If the structure can change, the base also has `addNode()`, `addArc(s, t)`, `erase(Node)`, `erase(Arc)` and `clear()`, which modify the storage only, since the extender adds the notifications. A static structure has instead `build(g, nodeRef, arcRef)` and the tag `BuildTag`. Optionally, the tags `NodeNumTag` and `ArcNumTag`, with `nodeNum()` and `arcNum()`, make the counting of chapter 4 take $O(1)$ time, and `FindArcTag`, with `findArc(s, t, prev)`, replaces the scan of `findArc()`.

The extender. On top of the functions above, `DigraphExtender< Base >` defines `NodeIt`, `ArcIt`, `OutArcIt` and `InArcIt` (Algorithm A.1), with the ranges `nodes()`, `arcs()`, `outArcs(v)` and `inArcs(v)`. It also defines `baseNode()`, `runningNode()`, `oppositeNode()`, `maxId()` and `fromId()`, and one `AlterationNotifier` for the nodes and one for the arcs, returned by `notifier(Node())` and `notifier(Arc())`. Its map templates `NodeMap< V >` and `ArcMap< V >` are `DefaultMaps` with the iterators of `MapExtender` (cf. chapter 6). Finally, its functions `addNode()`, `addArc()`, `erase()`, `clear()` and `build()` call those of the base and notify the observers in the order of section A.3; in particular, `erase(Node)` erases the arcs at the node first, and the base need not do it.

For an undirected graph, `GraphExtender< Base >` requires from the base the type `Edge` as well, with `u(e)`, `v(e)`, `direction(a)`, `direct(e, bool)`, `first(Edge &)`, `next(Edge &)`, `firstInc(Edge &, bool &, Node)`, `nextInc(Edge &, bool &)`, `id(Edge)`, `edgeFromId()` and `maxEdgeId()`. In return it defines the tag `UndirectedTag`, `EdgeIt`, `IncEdgeIt`, `edges()`, `incEdges(v)`, `oppositeArc()`, `direct(e, node)`, a third notifier for the edges and `EdgeMap< V >`, and its `addEdge()` notifies the new edge and its two arcs. In the same way, `BpGraphExtender< Base >` adds the red and the blue nodes, with their iterators, notifiers and maps.

Algorithm A.1 (The arc iterators that DigraphExtender derives from the base).

```
ArcIt(g):          g.first(a)
++a:               g.next(a)
OutArcIt(g, v):    g.firstOut(a, v)
++a:               g.nextOut(a)
a != INVALID:      Arc::operator!= of the base, against Arc(INVALID)
```

The program of this appendix writes the directed cycle on n nodes in this way. Its base class, `CycleDigraphBase`, stores only n, gives node i and arc i the identifier i, and computes the arc i as going from node i to node $(i+1) \bmod n$. Each node has one arc out and one arc in; hence, `nextOut()` and `nextIn()` set the handle to the invalid value. Then `CycleDigraph` derives from `DigraphExtender< CycleDigraphBase >`, and its `resize(n)` notifies the maps that all the items were erased and that the new ones were built, as `FullDigraph::resize()` does (Figure A.1). The program checks the digraph against `concepts::Digraph` and runs `Dijkstra` on it, with no change to the algorithm.

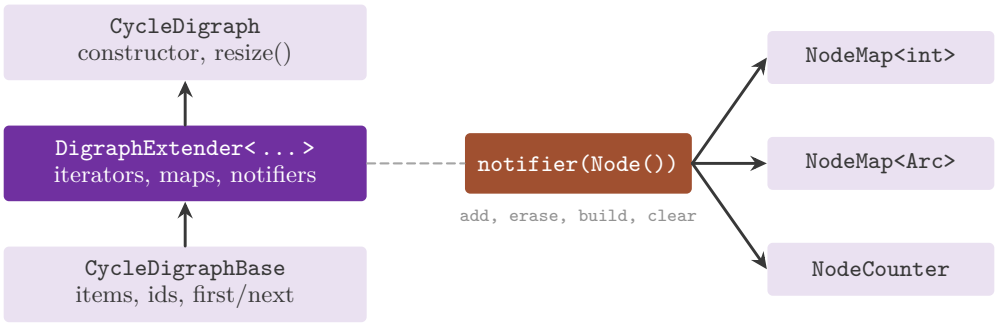


Figure A.1. The layers of the digraph of the program: the base class computes the items, DigraphExtender adds the iterators, the maps and the notifiers, and the notifier of the nodes informs the observers attached to it (the node maps among them).

```
// A new graph, a new map and an observer (appendix A): the directed cycle
// on n nodes written as a base class that computes everything from the
// indices, completed by DigraphExtender and checked against the Digraph
// concept; a read map computed on the fly, checked against ReadMap; an
// observer of the nodes of a ListDigraph; and Dijkstra run on the new
// digraph with the new map.

#include <iostream>
#include <vector>
#include <lemon/bits/graph_extender.h>
#include <lemon/concept_check.h>
#include <lemon/concepts/digraph.h>
#include <lemon/concepts/maps.h>
#include <lemon/dijkstra.h>
#include <lemon/list_graph.h>

using namespace lemon;

// The base: node i and arc i, the arc i going from node i to node i + 1
// (mod n); the extender derives the iterators, the maps and the notifiers
class CycleDigraphBase
{
 public:
 class Node
 {
  friend class CycleDigraphBase;

  protected:
  int _id;
  explicit Node( int id ) : _id( id ) { }

  public:
  Node() : _id( -1 ) { }
  Node( Invalid ) : _id( -1 ) { }
  bool operator==( const Node & n ) const { return( _id == n._id ); }
  bool operator!=( const Node & n ) const { return( _id != n._id ); }
  bool operator<( const Node & n ) const { return( _id < n._id ); }
  };

 class Arc
 {
  friend class CycleDigraphBase;

  protected:
  int _id;
  explicit Arc( int id ) : _id( id ) { }

  public:
  Arc() : _id( -1 ) { }
  Arc( Invalid ) : _id( -1 ) { }
  bool operator==( const Arc & a ) const { return( _id == a._id ); }
  bool operator!=( const Arc & a ) const { return( _id != a._id ); }
  bool operator<( const Arc & a ) const { return( _id < a._id ); }
  };

 typedef True NodeNumTag;  // countNodes() calls nodeNum()
 typedef True ArcNumTag;
 int nodeNum() const { return( _n ); }
 int arcNum() const { return( _n ); }

 Node source( Arc a ) const { return( Node( a._id ) ); }
 Node target( Arc a ) const { return( Node( ( a._id + 1 ) % _n ) ); }

 // the items from the last index down to 0
 void first( Node & v ) const { v._id = _n - 1; }
```

```
 static void next( Node & v ) { --v._id; }
 void first( Arc & a ) const { a._id = _n - 1; }
 static void next( Arc & a ) { --a._id; }
 // one arc out of each node and one into it
 void firstOut( Arc & a , const Node & v ) const { a._id = v._id; }
 static void nextOut( Arc & a ) { a._id = -1; }
 void firstIn( Arc & a , const Node & v ) const {
  a._id = ( v._id + _n - 1 ) % _n;
  }
 static void nextIn( Arc & a ) { a._id = -1; }

 static int id( Node v ) { return( v._id ); }
 static int id( Arc a ) { return( a._id ); }
 static Node nodeFromId( int id ) { return( Node( id ) ); }
 static Arc arcFromId( int id ) { return( Arc( id ) ); }
 int maxNodeId() const { return( _n - 1 ); }
 int maxArcId() const { return( _n - 1 ); }

 protected:
 CycleDigraphBase() : _n( 0 ) { }
 int _n;
 };

typedef DigraphExtender< CycleDigraphBase > ExtendedCycleDigraphBase;

class CycleDigraph : public ExtendedCycleDigraphBase
{
 public:
 explicit CycleDigraph( int n ) { _n = n; }
 // the maps of the digraph are told that every item has gone, and then
 // that the new ones are there, as FullDigraph::resize() does
 void resize( int n ) {
  notifier( Arc() ).clear();
  notifier( Node() ).clear();
  _n = n;
  notifier( Node() ).build();
  notifier( Arc() ).build();
  }
 Node operator()( int i ) const { return( nodeFromId( i ) ); }
 };

// A read map computed on the fly: the length of arc i is 1 + i % 3
class LengthByIndex
{
 public:
 typedef CycleDigraph::Arc Key;
 typedef int Value;
 explicit LengthByIndex( const CycleDigraph & g ) : _g( g ) { }
 Value operator[]( const Key & a ) const { return( 1 + _g.id( a ) % 3 ); }

 private:
 const CycleDigraph & _g;
 };

// An observer of the nodes of a ListDigraph that counts them
class NodeCounter : public ListDigraph::NodeNotifier::ObserverBase
{
 public:
 explicit NodeCounter( const ListDigraph & g ) : _count( 0 ) {
  attach( g.notifier( ListDigraph::Node() ) );
  build();
  }
 int count() const { return( _count ); }

 protected:
```

```
 virtual void add( const ListDigraph::Node & ) { ++_count; }
 virtual void add( const std::vector< ListDigraph::Node > & v ) {
  _count += static_cast< int >( v.size() );
  }
 virtual void erase( const ListDigraph::Node & ) { --_count; }
 virtual void erase( const std::vector< ListDigraph::Node > & v ) {
  _count -= static_cast< int >( v.size() );
  }
 virtual void build() {
  _count = 0;
  ListDigraph::Node v;
  for( notifier()->first( v ) ; v != INVALID ; notifier()->next( v ) )
   ++_count;
  }
 virtual void clear() { _count = 0; }

 private:
 int _count;
 };

int main()
{
 int errors = 0;

 checkConcept< concepts::Digraph , CycleDigraph >();
 checkConcept< concepts::ReadMap< CycleDigraph::Arc , int > ,
               LengthByIndex >();

 CycleDigraph g( 7 );
 if( countNodes( g ) != 7 || countArcs( g ) != 7 )
  ++errors;
 int out = 0;
 for( CycleDigraph::OutArcIt a( g , g( 3 ) ) ; a != INVALID ; ++a )
  ++out;
 if( out != 1 ||
     g.runningNode( CycleDigraph::OutArcIt( g , g( 6 ) ) ) != g( 0 ) )
  ++errors;

 // Dijkstra on the new digraph with the new map: the distance of node k
 // from node 0 is the sum of 1 + i % 3 for i < k
 LengthByIndex len( g );
 Dijkstra< CycleDigraph , LengthByIndex > dij( g , len );
 dij.run( g( 0 ) );
 int expected = 0;
 for( int k = 0 ; k < 7 ; ++k ) {
  if( dij.dist( g( k ) ) != expected )
   ++errors;
  expected += 1 + k % 3;
  }

 // a node map follows a resize() through the notifier
 CycleDigraph::NodeMap< int > label( g , 5 );
 g.resize( 10 );
 if( label[ g( 9 ) ] != 0 )
  ++errors;  // rebuilt, with Value() everywhere
 label[ g( 9 ) ] = 1;

 // the observer counts the nodes as they come and go
 ListDigraph lg;
 lg.addNode();
 NodeCounter counter( lg );
 ListDigraph::Node v = lg.addNode();
 lg.addNode();
 lg.erase( v );
 if( counter.count() != 2 || counter.count() != countNodes( lg ) )
```

```
  ++errors;

std::cout << "a new digraph, map and observer: " << errors << " errors"
          << std::endl;
return( errors == 0 ? 0 : 1 );
}
```

A.3 The alteration notifier

A graph informs the objects that depend on its items of each change through one notifier for each kind of item. This is an object of the class `AlterationNotifier< Container , Item >` of `lemon/bits/alteration_notifier.h`, which keeps a list of observers, i.e., of objects of classes derived from its nested class `ObserverBase`. An observer is attached to a notifier by its constructor `ObserverBase( notifier )` or by `attach( notifier )`, and it is detached by `detach()` or by its destructor; `notifier()` and `attached()` give its notifier and its state. An observer overrides six pure virtual functions, which the notifier calls.

- `add(item)` and `add(items)`, after one item or a `std::vector` of items has been added to the container.
- `erase(item)` and `erase(items)`, before one item or several items are erased from it.
- `build()`, after the container has been built from empty, and `clear()`, before all its items are erased.

A graph calls the functions `add()`, `erase()`, `build()` and `clear()` of its notifier in a fixed order with respect to its storage. An item is added to the storage first and notified afterwards, so that the observer can read it (e.g., its identifier), while an item to be erased is notified first and removed afterwards. The notifier also lets an observer enumerate the current items by `first(item)` and `next(item)`, and read their identifiers by `id(item)` and the bound on them by `maxId()`. We remark that an observer must not trigger another alteration of the same notifier, nor detach another observer, from within these functions.

Exceptions. The calls of `add()` and `build()` go from the observer attached first to the one attached last. If an observer throws, those already notified receive `erase()` (or `clear()`) for the same items, and the exception is propagated (Algorithm A.2); hence, a graph whose `addArc()` fails in one of its maps (e.g., for lack of memory) leaves all its maps consistent. The calls of `erase()` and `clear()` go from the last attached to the first, and they must not throw, with one exception. An observer may throw `ImmediateDetach`, after which the notifier detaches it and goes on with the others. The `Snapshot` of a list graph uses it when an item older than the snapshot is erased, and so do the `Smart` arc and edge sets when an erased node has arcs in them (chapter 4).

Algorithm A.2 (Notifying an addition, with rollback, and an erasure).

```
add(x):
 1  for each observer o, from the first attached to the last:
 2    try o.add(x)
 3    on an exception:  for each observer o' notified before o:
 4                        o'.erase(x)
 5                      rethrow the exception
erase(x):
 6  for each observer o, from the last attached to the first:
 7    try o.erase(x)
 8    on ImmediateDetach:  detach o and continue
```

Thread safety and limits. Attaching and detaching go through a lock. It is a mutex of POSIX threads or of Windows when the library is configured with `LEMON_USE_PTHREAD` or `LEMON_USE_WIN32_THREADS`, and an empty class otherwise; thus, maps may be created and destroyed on one graph from several threads, while the notifications are not locked. The header also remarks that the notifications are not always complete. A reversal of an arc by `reverseArc()` is not notified (the observers see neither an erasure nor an addition), and therefore the degree maps `InDegMap` and `OutDegMap` of chapter 6, which observe the arcs, may become wrong after it. Also, a filter adaptor of chapter 5 cannot notify the change of its filter map at all.

The program of section A.2 includes `NodeCounter`, an observer of the nodes of a `ListDigraph` that counts them. Its constructor attaches it to `g.notifier(ListDigraph::Node())` and calls `build()`, which counts the nodes present by `first()` and `next()`. Afterwards, `add()` and `erase()` keep the count, and the program checks it against `countNodes()` after an addition and an erasure.

A.4 Writing a map

A map that an algorithm only reads is a class with the types `Key` and `Value` and a constant `operator[]( const Key & )`, which returns a `Value` (or a reference to one). A map that the algorithm writes also has `set( const Key & , const Value & )`. No other member is needed; in fact, the map need not store its values and need not know the graph, unless it computes from it. It is passed by reference to the algorithm, which keeps a pointer to it (and therefore requires it to outlive the algorithm). For instance, the class `LengthByIndex` of the program of section A.2 gives each arc of `CycleDigraph` the length $1 + (i \bmod 3)$ from the identifier i of the arc, and `Dijkstra< CycleDigraph , LengthByIndex >` reads it in place of an arc map. A map may derive from `MapBase< K , V >` of `lemon/maps.h`, which only defines the two types. Also, a reference map defines the types `Reference` and `ConstReference` and the tag `typedef True ReferenceMapTag`, by which `MapTraits` (cf. chapter 3) recognizes it.

A map that stores a value for each item of a graph, and must therefore follow its changes, is an observer of the notifier of that kind of item: its `add()` and `erase()` make room for the new items and release the erased ones. The standard maps of the extenders, `VectorMap` and `ArrayMap` (described in chapter 6), are written in this way, and so are the degree maps of `lemon/maps.h`, while `CrossRefMap` and `RangeIdMap` derive from a standard map and extend its observer functions. A new graph structure obtains the standard maps from its extender; it usually needs a specific map only for a different storage (e.g., a map of the arcs of an implicit graph that stores values only for some of them).

A.5 Writing a heap

The algorithms that keep a priority queue of nodes (`Dijkstra`, `BidirDijkstra`, `AStar`, `MinCostArborescence` and several others of Part III) take the type of their heap from their traits. A user may replace it by the named template parameter `SetHeap< H , CR >` (cf. chapter 3), with a heap that conforms to `concepts::Heap< PR , IM , CMP >`. The parameters of the concept are the type `PR` of the priorities, the type `IM` of a read-write map from the items to `int`, and the comparison functor `CMP`, by default `std::less< PR >`. Through the map `IM`, called the cross reference map, the heap finds the place of an item in $O(1)$ time, and its key type `IM::Key` is the type of the items. The concept requires the following members, which one can compare with those of the heaps of the library.

- The types `Item`, `Prio`, `ItemIntMap` and `State`, the last one an enumeration with `IN_HEAP` $= 0$, `PRE_HEAP` $= -1$ and `POST_HEAP` $= -2$.
- The constructors `Heap( IM & map )` and `Heap( IM & map , const CMP & comp )`, and a copy constructor (the `Constraints` of the concept copy a heap).
- The functions `size()`, `empty()` and `clear()`, the last of which empties the heap without touching the cross reference map.
- The functions `push(i, p)` for an item not in the heap, `top()` and `prio()` for the item of least priority and its priority, `pop()`, which removes that item, and `erase(i)` for an item in the heap.
- The query `heap[i]`, the priority of an item in the heap, and the functions `set(i, p)`, which inserts the item or changes its priority, and `decrease(i, p)` and `increase(i, p)`, whose precondition is that the new priority is not larger (resp. smaller) than the old one.
- The functions `state(i)`, which returns `PRE_HEAP` for an item that has never been in the heap, `IN_HEAP` for one in it and `POST_HEAP` for one removed, and `state(i, st)`, which sets the state of an item that is not in the heap.

The cross reference map is owned by the caller and shared with the heap. It must give `PRE_HEAP` for each item that may be pushed, and the heap typically writes into it the position of each item in its storage (a nonnegative value) and `POST_HEAP` after its removal. For instance, `Dijkstra::init()` calls `clear()` on its heap and then sets the cross reference of each node to `PRE_HEAP`, in $O(n)$ time. The implementations of the library, their costs, and the test of all of them against the concept (`test/heap_test.cc`) are given in chapter 7.

A.6 Checking a new class

A new class is checked at compile time by `checkConcept< Concept , T >()` of `lemon/concept_check.h`, as the program of section A.2 does for `CycleDigraph` (against `concepts::Digraph`) and for `LengthByIndex` (against `concepts::ReadMap< CycleDigraph::Arc , int >`). The call instantiates the function `constraints()` of the concept on the class, and a missing type or function (or one with an incompatible signature) gives a compilation error at that point. It usually costs nothing at run time, and it is empty when `NDEBUG` is defined. A structure

that offers more than the concept is also checked against the components it adds, e.g., `checkConcept< concepts::ExtendableDigraphComponent<> , G >()` and the erasable, clearable and alterable components, as `test/digraph_test.cc` does for the structures of the library.

Since the concepts check the syntax only, the semantic requirements of section A.1 are checked by running the class. In the tests, the header `test/graph_test.h` has function templates that do it for any graph type, and a test of a new structure may call them. Thus, `checkGraphNodeList(g, n)` and `checkGraphArcList(g, m)` count the items visited by the iterators, `checkGraphOutArcList(g, v, d)` and `checkGraphInArcList(g, v, d)` the arcs at a node, and `checkGraphIncEdgeArcLists(g, v, d)` the incidences of an undirected graph. Also, `checkNodeIds(g)` and `checkArcIds(g)` verify that the identifiers are distinct and that `nodeFromId()` inverts them, `checkArcDirections(g)` that the arcs of each edge are directed consistently, and `checkGraphNodeMap(g)` and `checkGraphArcMap(g)` that a map has a value for each item. Finally, an algorithm run on the new structure should give the results known in closed form or computed on a copy of it into a `ListDigraph` (by `digraphCopy()` of chapter 4); the program of section A.2, e.g., checks the distances of `Dijkstra` against their closed form.

Y. Glossary

The terms defined in the chapters, each with the section that defines it.

2-optimal tour: a tour that no 2-opt move, i.e., the replacement of two of its edges by the two other edges that reconnect it, makes cheaper (§19.3).

active item: in an elevator, an item marked as active, e.g., a node with a positive excess in a push-relabel algorithm; in the Bellman-Ford algorithm, a node whose distance changed in the last round (§8.3, §10.4).

admissible arc: in a push-relabel or cost scaling algorithm, a residual arc along which flow may be pushed: one whose tail is one level above its head (maximum flow), or one of negative reduced cost (cost scaling) (§12.2, §13.6).

admissible heuristic: a heuristic of A* that never exceeds the distance of a node to the target (§10.3).

alteration notifier: the object of a graph, one for each kind of item, that informs the attached observers (the maps among them) of each item added or erased, and of the graph being built or cleared (§A.3).

alternating path: a path whose edges are alternately in a matching and out of it; it is augmenting when neither of its ends is covered by the matching (§17.1).

arborescence: a set of arcs in which each node reachable from the root, the root excepted, has one entering arc and can be reached from the root (§11.2).

arc set: a digraph (or, as an edge set, a graph) that has its own arcs on the nodes of another graph, such as ListArcSet and SmartEdgeSet (§4.7).

archetype: a concept class whose members are declared with empty bodies, so that a template that compiles with it uses only what the concept guarantees (§3.5).

assignment: a matching of a bipartite graph that covers every red node; MinCostAssignment finds one of least cost (§17.3).

augmenting path: a directed path from the source to the sink in the residual digraph of a flow; sending along it its smallest residual capacity increases the value of the flow (§5.4).

awake set: in the Hao-Orlin algorithm, the nodes among which the preflow is pushed towards the current sink, as opposed to the sources and to the dormant sets (§15.2).

barrier (circulation): a set of nodes whose supply exceeds what the upper bounds of the arcs leaving it, less the lower bounds of the arcs entering it, can carry, which proves a circulation problem infeasible (§12.4).

barrier (matching): a set of nodes whose removal leaves more odd components than it has nodes, which bounds from below the number of nodes uncovered by any matching; for fractional matchings, a set S with $|S| - |N(S)|$ as large as possible (§17.1).

base node: the node on which an incidence iterator was constructed; the other end of its current arc or edge is the running node (§3.3).

basis: in the network simplex method, a spanning tree of the arcs together with the bound at which each other arc lies; a basis of a vertex of the polyhedron of the flows is a spanning tree containing its free arcs (§13.4, §14.4).

bi-edge-connected component: a class of the nodes of an undirected graph joined by two edge-disjoint paths; the components are separated by the bridges (§18.2).

bind arc: the arc of a SplitNodes adaptor from the in-node to the out-node of a node of the digraph, which carries the data of the node (§5.5).

block: a bi-node-connected component of an undirected graph, i.e., a class of the edges that lie on a common cycle; the blocks are separated by the cut nodes (§18.2).

blossom: an odd cycle closed by an alternating search tree, which Edmonds' algorithms shrink into a single node and expand later (§17.4).

breadth-first search: the visit of the nodes reachable from the sources in the order of their distance, i.e., of the least number of arcs of a path from a source (§9.2).

bridge: an edge whose removal increases the number of connected components of a graph, also called a cut edge (§18.2).

bucket heap: a heap of non-negative integer priorities that keeps a list of the items of each priority in a vector indexed by the priority (§7.4).

caption: the optional name that follows the type in the header line of an LGF section, by which a reader selects one of several sections of the same type (§21.1).

cascading cut: in a Fibonacci heap, the cut of a node that has lost a second child since it became a child, repeated upwards (§7.3).

clique: a set of pairwise adjacent nodes of an undirected graph (§19.5).

combinatorial embedding: the cyclic order of the edges around each node of a planar graph, which describes a plane drawing up to deformation (§18.5).

concept: the list of the types and functions that a class of a kind (graph, map, heap) must provide, written as a class of lemon/concepts/ and checked by checkConcept (§3.5).

condensation: the acyclic digraph with one node per strongly connected component of a digraph and an arc between two components whenever an arc of the digraph joins them (§18.3).

consistent heuristic: a heuristic h of A* with $h(u) \leq \ell(uv) + h(v)$ on each arc, with which each node is scanned at most once (§10.3).

core number: the largest k such that a node belongs to a subgraph of minimum degree k; the largest core number is the degeneracy of the graph (§6.6).

cross reference map: the map from the items to integers that a heap shares with its user, through which it finds the place of an item and records its state (PRE_HEAP, IN_HEAP, POST_HEAP) (§A.5).

cross-reference map: the map from the items to int through which a heap finds an item in its storage, holding its position while the item is in the heap and its state otherwise (§7.1).

cut: a set of nodes containing the source and not the target; its capacity is the sum of the capacities of the arcs that leave it (§12.1).

cut node: a node whose removal increases the number of connected components of a graph, also called an articulation node (§18.2).

d-ary heap: a heap stored as a complete tree in which each node has at most d children, laid out level by level in a vector (§7.2).

degenerate vertex: a vertex of the polyhedron of the flows whose free arcs do not span the nodes, and which therefore has more than one basis (§14.4).

depth-first search: the visit that advances from the last node reached along an arc not yet scanned and backtracks when there is none, a node being processed (finished) when the search backtracks from it (§9.3).

dormant set: in the Hao-Orlin algorithm, a set of nodes put aside, without relabelling them, because the current sink can no longer be reached from them in the residual digraph; the sets are woken up in the reverse order of their creation (§15.2).

dual ray: a solution of the dual LP with its finite objective coefficients (the primal bounds) set to 0 and an improving objective value, which proves the primal problem infeasible (§20.4).

dual scale: the constant (4 for integer weights, 1 otherwise) by which the weighted matching classes multiply their dual values, so that the halvings of the algorithm stay exact (§17.5).

edge connectivity: the least number of edges whose removal disconnects an undirected graph, i.e., its minimum cut with unit capacities (§15.1).

elevator: the data structure of the labels of the push-relabel algorithms, which keeps the items by level with the active ones first and gives the highest active item in constant time (§8.3).

eligible arc: in the network simplex method, a nonbasic arc whose reduced cost is negative at its lower bound or positive at its upper bound, which may enter the basis (§13.4).

epsilon-optimality: the property of a flow and of potentials under which no residual arc has a reduced cost below the opposite of a given epsilon (§13.6).

Euler tour: a closed walk that traverses each edge of a graph (each arc of a digraph, in its direction) once (§18.4).

extender: a class template of lemon/bits/ (DigraphExtender, GraphExtender, BpGraphExtender) that completes a base class with the iterators, the maps, the notifiers and the notifying modifications (§A.2).

extreme flow: a vertex of the polyhedron of the flows of a minimum cost flow problem, i.e., a feasible flow whose arcs strictly within their bounds form a forest (§14.4).

factor-critical graph: a graph that has a perfect matching after the removal of any one of its nodes (§17.1).

filter: a bool map that decides which nodes, arcs or edges a filtering adaptor shows (§5.2).

forward arc: the arc of an edge that goes from u(e) to v(e), for which direction() is true; the other arc of the edge is the backward one (§3.2).

fractional matching: a solution of the linear relaxation of the matching problem without the odd set inequalities, whose vertices take the values 0, 1/2 and 1 (§17.1).

function-type interface: a function (e.g., dijkstra()) that returns a wizard object, on which the optional outputs of an algorithm are given by name before a single run (§3.6).

Gallai–Edmonds decomposition: the partition of the nodes into those left uncovered by some maximum matching (D), their other neighbours (A) and the rest (C), which MaxMatching returns as the status of each node (§17.1).

gap heuristic: the lift to the top level of all the items above a level that has become empty, which can no longer reach the sink (§8.3).

GEQ and LEQ forms: the minimum cost flow problem with supply constraints “greater or equal” (all the supplies leave their nodes, some demand may be unmet) or “less or equal” (all the demands are met, some supply may stay) (§13.1).

Gomory-Hu tree: a tree on the nodes of an undirected graph, with a weight on each edge, such that the minimum cut between two nodes is the lightest edge of the path between them in the tree, and the components left by removing that edge are the two sides of the cut (§15.1).

graph adaptor: a class that presents a digraph or a graph as another one (filtered, reversed, without orientation, residual, split) by translating each query, without copying it (§5.1).

graph6: the format of the nauty package that encodes an undirected graph in one line of printable characters, six bits of the upper triangle of the adjacency matrix per character (§21.6).

handle: a small value object (Node, Arc, Edge, RedNode, BlueNode) that identifies an item of a graph, meaningful only together with its graph (§3.2).

heap order: the invariant of a heap-ordered tree that no item has a smaller priority than its parent (§7.2).

identifier: the integer that a graph gives to an item, unique among the items of its kind and immutable while the item exists, used as the index of the maps (§3.2).

implicit graph: a graph structure that stores only its sizes and computes its items from their indices, such as FullGraph, GridGraph and HypercubeGraph (§4.5).

inherent orientation: the orientation of an edge from u(e) to v(e), given by the order of the end nodes at its creation (from red to blue in a bipartite graph) (§3.2).

INVALID: the only object of the type Invalid, to which each handle and iterator converts, and which marks the end of an iteration or the absence of an item (§3.2).

iterable map: a graph map that also lists, in time proportional to their number, the items that have a given value (§6.6).

Kempe chain: a connected component of the subgraph induced by the nodes of two given colors, whose colors can be swapped without spoiling a coloring (§18.6).

Kuratowski subdivision: a subdivision of K_5 or of $K_{3,3}$ contained in a graph, which certifies that the graph is not planar (§18.5).

label: in an LGF file, the value of the map named `label`, which identifies a node or an arc in the rest of the file (§21.1).

Lawler's partition: the splitting of a set of solutions, once its best one is known, into parts in which the elements of that solution are fixed in turn, used to find the k best solutions in order (§14.1, §14.3).

LGF: the LEMON Graph Format, a column-oriented text format made of sections (`@nodes`, `@arcs`, `@edges`, `@attributes`, `@red_nodes`, `@blue_nodes` and free extra sections) (§21.1).

low point: in a depth-first search, the least index of discovery of a node reachable from a given node by going down the tree and then along at most one non-tree edge (§18.2).

map adaptor: a map whose values are computed at each read from one or two other maps, a constant or a functor, which it keeps by reference (§6.4).

matching: a set of edges no two of which share an end; it is perfect when it covers every node (§17.1).

maximum adjacency ordering: an ordering of the nodes of an undirected graph in which each node maximizes the capacity of its edges to the nodes before it; the orderings of the Nagamochi-Ibaraki algorithm (§15.3).

maximum cardinality search: the order of the nodes in which each node, when chosen, has the largest total capacity of arcs to the nodes chosen before it (§9.6).

mean cost (of a cycle): the cost of a directed cycle divided by its number of arcs (§16.1).

metric cost: a cost on the edges of a complete graph that satisfies the triangle inequality $c(u, w) \leq c(u, v)+c(v, w)$ (§19.1).

minimum cut (global): a nonempty proper subset of the nodes whose leaving arcs (or boundary edges, in an undirected graph) have the least total capacity, with no source and no sink fixed (§15.1).

named template parameter: a nested template of an algorithm class (e.g., SetDistMap< M >) whose member Create is the class with one type of its traits replaced (§3.6).

negative cycle: a directed cycle of negative total length, whose presence makes the distances of the nodes reachable from it $-\infty$ (§10.4).

observer: an object of a class derived from AlterationNotifier::ObserverBase, attached to a notifier, whose functions add, erase, build and clear are called at each change of the graph (§A.3).

odd set inequalities: the constraints that a set S of odd cardinality holds at most $(|S| - 1)/2$ edges of a matching, which with the degree constraints describe the matching polytope (§17.1).

operation traits: the class of the operations (`zero()`, `plus()`, `less()` and, for Bellman-Ford, `infinity()`) by which `Dijkstra` and `BellmanFord` compute and compare the distances (§10.2).

path compression: the rule of a find in a union-find structure that makes each item met on the way to the root point to the root (§8.2).

path dumper: an object that lists the arcs of a path, possibly from the last to the first (when its `RevPathTag` is `True`), returned by the `path()` queries of the search algorithms and meant to be assigned to a path class (§8.1).

perfect elimination order: an order of the nodes of a graph in which the neighbours of each node that follow it form a clique; a graph has one if and only if it is chordal (§9.6).

policy graph: in Howard's algorithm, the subgraph made of one chosen leaving arc for each node, in which each weakly connected part contains one cycle (§16.4).

potentials: the dual variables of a minimum cost flow problem, one per node; the reduced cost of an arc is its cost plus the potential of its tail minus that of its head (§13.1).

preflow: a map within the bounds of the arcs whose excess, the flow into a node less the flow out of it, is nonnegative at every node other than the source (§12.2).

primal ray: a solution of the primal LP with its finite bounds set to 0 and an improving objective value, which proves the dual problem infeasible (§20.4).

primal scale: the constant 2 by which the fractional matching classes multiply their primal values, so that the half-valued edges are reported as integers (§17.6).

priority queue: the abstract data type of a set of items with priorities, with the item of least priority, its removal and the change of a priority as operations; the heaps implement it (§7.1).

problem type: the outcome of a solved LP or MIP (`UNDEFINED`, `INFEASIBLE`, `FEASIBLE`, `OPTIMAL` or `UNBOUNDED`), read after `solve()` by `primalType()`, `dualType()` or `type()` (§20.4).

radix heap: a heap of integer priorities, monotone in the minimum, whose items are kept in boxes of widths 1, 1, 2, 4, ... from the current minimum (§7.4).

range constraint: a row of a linear program with both a finite lower and a finite upper bound, written in the interface as `(s <= e) <= t` (§20.1).

reduced length: the length $\ell(uv) + \pi(u) - \pi(v)$ of an arc under a potential π, which changes the length of all the paths between two nodes by the same amount (§10.1).

reference map: a read-write map whose operator[] returns a reference to the value, recognized by its tag ReferenceMapTag (§6.1).

relaxation: the update of the tentative distance of the target of an arc to the distance of its source plus the length of the arc, when this is smaller (§10.2).

residual digraph: for a flow on a digraph, the digraph of the forward arcs below their capacity and of the backward arcs of positive flow, each with its residual capacity (§5.4).

residual network: for a flow, the network of the arcs along which flow may still be sent, forward up to the upper bound and backward down to the lower bound, the backward arcs with the opposite cost (§12.1, §13.1).

smallest-last order: the order in which the nodes are removed from a graph by always taking a node of least degree among the remaining ones (§18.6).

smallest-last ordering: the order of the nodes obtained by removing, one at a time, a node of least degree in the graph that remains, read backwards (§6.6).

snapshot: a saved state of a list or smart graph, to which restore() brings the graph back by erasing the items added after it (§4.2).

spanning tree: a set of $n - 1$ edges of a connected graph with no cycle, which therefore connects all the nodes (§11.1).

splay tree: the self-adjusting binary search tree of Sleator and Tarjan, used by DynArcLookUp, whose operations take logarithmic amortized time (§4.8).

static graph: a graph structure built in one call and not modifiable afterwards, which stores the arcs out of each node contiguously (StaticDigraph, StaticGraph, CompactDigraph) (§4.4).

strongly connected component: a class of the nodes of a digraph joined by directed paths in both directions (§18.3).

subgraph isomorphism: an injective map of the nodes of a graph into those of another that takes each edge to an edge; it is induced if it also takes each non-edge to a non-edge (§18.7).

successive shortest paths: the algorithm of minimum cost flow that sends flow along shortest paths of the residual digraph, keeping the reduced lengths nonnegative by potentials, used by `Suurballe` (§10.5).

tolerance: the object of class Tolerance< T > that compares numbers allowing for an absolute error up to its epsilon, exactly for the exact types (§3.7).

topological order: an order of the nodes of an acyclic digraph in which the source of each arc precedes its target, e.g., the reverse of the finishing order of a depth-first search (§9.3).

tour: a Hamiltonian cycle of a complete graph, i.e., a cycle that visits each node once; the solution of the travelling salesman problem (§19.1).

traits class: the class, a template parameter of an algorithm, that defines the types the algorithm uses (heap, maps, operations) and creates their objects (§3.6).

unbounded cycle: a directed cycle of negative cost whose arcs have infinite upper bounds, which proves a feasible minimum cost flow problem unbounded (§14.2).

union-find structure: a partition of a set of items into classes under the union of two classes and the find of the representative of the class of an item, stored as a forest with union by size and path compression (§8.2).

vertex cover: a set of nodes containing at least one end of every edge; in a bipartite graph its least cardinality equals that of a maximum matching (§17.1).

violation: of a set of nodes, its supply less the upper bounds of the arcs leaving it plus the lower bounds of the arcs entering it; a positive violation proves a minimum cost flow problem infeasible (§14.1).

visitor: an object whose members a search with a visitor interface (`BfsVisit`, `DfsVisit`) calls at each event of the search (§9.5).

wizard: the object returned by a function-type interface (e.g., `bfs()`, `dijkstra()`), on which the maps and the outputs are given by named function parameters before `run()` (§9.4).

Z. Bibliography

The works that the headers of the library cite, those that the README and NEWS name, and those that the chapters cite, in the alphabetical order of their first authors. The anchor of each entry is the key under which the chapters cite it.

W. Schnyder. Embedding planar graphs on the grid. Proceedings of the First Annual ACM-SIAM Symposium on Discrete Algorithms (SODA '90), 138–148, 1990.

Seth Pettie. Towards a final analysis of pairing heaps. In 46th Annual IEEE Symposium on Foundations of Computer Science (FOCS 2005), pages 174–183, 2005. doi:10.1109/SFCS.2005.75

N. Christofides. Worst-case analysis of a new heuristic for the travelling salesman problem. Report 388, Graduate School of Industrial Administration, Carnegie Mellon University, 1976. Reprinted in Operations Research Forum, 3(1):20, 2022. doi:10.1007/s43069-021-00101-z

J. W. J. Williams. Algorithm 232: Heapsort. Communications of the ACM, 7(6):347–348, 1964. doi:10.1145/512274.3734138

Jean Vuillemin. A data structure for manipulating priority queues. Communications of the ACM, 21(4):309–315, 1978. doi:10.1145/359460.359478

Robert B. Dial. Algorithm 360: shortest-path forest with topological ordering. Communications of the ACM, 12(11):632–633, 1969. doi:10.1145/363269.363610

Robert E. Tarjan. Efficiency of a good but not linear set union algorithm. Journal of the ACM, 22(2):215–225, 1975. doi:10.1145/321879.321884

Ravindra K. Ahuja, Kurt Mehlhorn, James B. Orlin and Robert E. Tarjan. Faster algorithms for the shortest path problem. Journal of the ACM, 37(2):213–223, 1990. doi:10.1145/77600.77615

Ravindra K. Ahuja, Thomas L. Magnanti and James B. Orlin. Network Flows: Theory, Algorithms, and Applications. Prentice-Hall, 1993.

M. Sharir. A strong-connectivity algorithm and its applications in data flow analysis. Computers & Mathematics with Applications, 7(1):67–72, 1981. doi:10.1016/0898-1221(81)90008-0

Andrew V. Goldberg. An efficient implementation of a scaling minimum-cost flow algorithm. Journal of Algorithms, 22(1):1–29, 1997. doi:10.1006/jagm.1995.0805

Claude Berge. Two theorems in graph theory. Proceedings of the National Academy of Sciences of the United States of America, 43(9):842–844, 1957. doi:10.1073/pnas.43.9.842

Alan J. Hoffman. Some recent applications of the theory of linear inequalities to extremal combinatorial analysis. In Combinatorial Analysis, Proceedings of Symposia in Applied Mathematics, 10:113–127, 1960. doi:10.1090/psapm/010/0114759

K. Appel and W. Haken. Every planar map is four colorable. Part I: Discharging. Illinois Journal of Mathematics, 21(3):429–490, 1977. doi:10.1215/ijm/1256049011

Morton Klein. A primal method for minimal cost flows with applications to the assignment and transportation problems. Management Science, 14(3):205–220, 1967. doi:10.1287/mnsc.14.3.205

George E. P. Box and Mervin E. Muller. A note on the generation of random normal deviates. The Annals of Mathematical Statistics, 29(2):610–611, 1958. doi:10.1214/aoms/1177706645

J. M. Boyer and W. J. Myrvold. On the cutting edge: simplified O(n) planarity by edge addition. Journal of Graph Algorithms and Applications, 8(3):241–273, 2004. doi:10.7155/jgaa.00091

Ursula Bünnagel, Bernhard Korte and Jens Vygen. Efficient implementation of the Goldberg-Tarjan minimum-cost flow algorithm. Optimization Methods and Software, 10(2):157–174, 1998. doi:10.1080/10556789808805709

Cbc, the COIN-OR Branch and Cut solver. https://github.com/coin-or/Cbc

Boris V. Cherkassky and Andrew V. Goldberg. On implementing the push-relabel method for the maximum flow problem. Algorithmica, 19(4):390–410, 1997. doi:10.1007/PL00009180

Yoeng-Jin Chu and Tseng-Hong Liu. On the shortest arborescence of a directed graph. Scientia Sinica, 14:1396–1400, 1965.

Clp, the COIN-OR Linear Programming solver. https://github.com/coin-or/Clp

Jean Cochet-Terrasson, Guy Cohen, Stéphane Gaubert, Michael McGettrick and Jean-Pierre Quadrat. Numerical computation of spectral elements in max-plus algebra. IFAC Proceedings Volumes, 31(18):667–674, 1998. doi:10.1016/S1474-6670(17)42067-2

Dan Gusfield. Very simple methods for all pairs network flow analysis. SIAM Journal on Computing, 19(1):143–155, 1990. doi:10.1137/0219009

L. P. Cordella, P. Foggia, C. Sansone and M. Vento. A (sub)graph isomorphism algorithm for matching large graphs. IEEE Transactions on Pattern Analysis and Machine Intelligence, 26(10):1367–1372, 2004. doi:10.1109/TPAMI.2004.75

Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest and Clifford Stein. Introduction to Algorithms, 2nd edition. The MIT Press, 2001.

Ali Dasdan and Rajesh K. Gupta. Faster maximum and minimum mean cycle algorithms for system-performance analysis. IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems, 17(10):889–899, 1998. doi:10.1109/43.728912

B. Dezső, A. Jüttner and P. Kovács. LEMON – an open source C++ graph template library. Electronic Notes in Theoretical Computer Science, 264(5):23–45, 2011. doi:10.1016/j.entcs.2011.06.003

Jack Edmonds and Richard M. Karp. Theoretical improvements in algorithmic efficiency for network flow problems. Journal of the ACM, 19(2):248–264, 1972. doi:10.1145/321694.321699

Lester R. Ford and Delbert R. Fulkerson. Maximal flow through a network. Canadian Journal of Mathematics, 8:399–404, 1956. doi:10.4153/CJM-1956-045-5

Lester R. Ford, Jr. Network flow theory. Paper P-923, The RAND Corporation, Santa Monica, 1956.

Michael L. Fredman and Robert E. Tarjan. Fibonacci heaps and their uses in improved network optimization algorithms. Journal of the ACM, 34(3):596–615, 1987. doi:10.1145/28869.28874

Michael L. Fredman, Robert Sedgewick, Daniel D. Sleator and Robert E. Tarjan. The pairing heap: a new form of self-adjusting heap. Algorithmica, 1(1):111–129, 1986. doi:10.1007/BF01840439

Harold N. Gabow, Zvi Galil, Thomas Spencer and Robert E. Tarjan. Efficient algorithms for finding minimum spanning trees in undirected and directed graphs. Combinatorica, 6(2):109–122, 1986. doi:10.1007/BF02579168

Zvi Galil, Silvio Micali and Harold Gabow. An O(EV log V) algorithm for finding a maximal weighted matching in general graphs. SIAM Journal on Computing, 15(1):120–130, 1986. doi:10.1137/0215009

GLPK, the GNU Linear Programming Kit. https://www.gnu.org/software/glpk/

Andrew V. Goldberg and Robert E. Tarjan. A new approach to the maximum-flow problem. Journal of the ACM, 35(4):921–940, 1988. doi:10.1145/48014.61051

Andrew V. Goldberg and Robert E. Tarjan. Finding minimum-cost circulations by canceling negative cycles. Journal of the ACM, 36(4):873–886, 1989. doi:10.1145/76359.76368

Andrew V. Goldberg and Robert E. Tarjan. Finding minimum-cost circulations by successive approximation. Mathematics of Operations Research, 15(3):430–466, 1990. doi:10.1287/moor.15.3.430

Ralph E. Gomory and T. C. Hu. Multi-terminal network flows. Journal of the Society for Industrial and Applied Mathematics, 9(4):551–570, 1961. doi:10.1137/0109047

Joseph B. Kruskal. On the shortest spanning subtree of a graph and the traveling salesman problem. Proceedings of the American Mathematical Society, 7(1):48–50, 1956. doi:10.1090/S0002-9939-1956-0078686-7

A. Grosso, M. Locatelli, and W. Pullan. Simple ingredients leading to very efficient heuristics for the maximum clique problem. Journal of Heuristics, 14(6):587–612, 2008. doi:10.1007/s10732-007-9055-x

Horst W. Hamacher, Jean-Claude Picard and Maurice Queyranne. On finding the K best cuts in a network. Operations Research Letters, 2(6):303–305, 1984. doi:10.1016/0167-6377(84)90083-X

J. Hao and James B. Orlin. A faster algorithm for finding the minimum cut in a directed graph. Journal of Algorithms, 17(3):424–446, 1994. doi:10.1006/jagm.1994.1043

Peter E. Hart, Nils J. Nilsson and Bertram Raphael. A formal basis for the heuristic determination of minimum cost paths. IEEE Transactions on Systems Science and Cybernetics, 4(2):100–107, 1968. doi:10.1109/TSSC.1968.300136

Mark Hartmann and James B. Orlin. Finding minimum cost to time ratio cycles with small integral transit times. Networks, 23(6):567–574, 1993. doi:10.1002/net.3230230607

Michael L. Fredman. On the efficiency of pairing heaps and related data structures. Journal of the ACM, 46(4):473–501, 1999. doi:10.1145/320211.320214

C. Hierholzer and C. Wiener. Ueber die Möglichkeit, einen Linienzug ohne Wiederholung und ohne Unterbrechung zu umfahren. Mathematische Annalen, 6(1):30–32, 1873. doi:10.1007/BF01442866

J. Hopcroft and R. Tarjan. Algorithm 447: efficient algorithms for graph manipulation. Communications of the ACM, 16(6):372–378, 1973. doi:10.1145/362248.362272

John E. Hopcroft and Richard M. Karp. An n^{5/2} algorithm for maximum matchings in bipartite graphs. SIAM Journal on Computing, 2(4):225–231, 1973. doi:10.1137/0202019

IBM ILOG CPLEX Optimization Studio. https://www.ibm.com/products/ilog-cplex-optimization-studio

Ira Pohl. Bi-directional search. Machine Intelligence, 6:127–140, 1971.

D. S. Johnson and C. C. McGeoch (editors). Network Flows and Matching: First DIMACS Implementation Challenge. DIMACS Series in Discrete Mathematics and Theoretical Computer Science, volume 12. American Mathematical Society, 1993. doi:10.1090/dimacs/012

A. Jüttner and P. Madarasi. VF2++: an improved subgraph isomorphism algorithm. Discrete Applied Mathematics, 242:69–81, 2018. doi:10.1016/j.dam.2018.02.018

Damian J. Kelly and Garrett M. O'Neill. The Minimum Cost Flow Problem and The Network Simplex Method. Master's thesis, University College, Dublin, 1991.

Zoltán Király and Péter Kovács. Efficient implementations of minimum-cost flow algorithms. Acta Universitatis Sapientiae, Informatica, 4:67–118, 2012.

Tibor Gallai. Maximale Systeme unabhängiger Kanten. A Magyar Tudományos Akadémia Matematikai Kutató Intézetének Közleményei, 9:401–413, 1964.

Dénes König. Gráfok és mátrixok. Matematikai és Fizikai Lapok, 38:116–119, 1931.

George Marsaglia and Thomas A. Bray. A convenient method for generating normal variables. SIAM Review, 6(3):260–264, 1964. doi:10.1137/1006063

Jack Edmonds. Maximum matching and a polyhedron with 0,1-vertices. Journal of Research of the National Bureau of Standards, Section B, 69B(1–2):125–130, 1965. doi:10.6028/jres.069B.013

C. Kuratowski. Sur le problème des courbes gauches en topologie. Fundamenta Mathematicae, 15:271–283, 1930. doi:10.4064/fm-15-1-271-283

Karl Menger. Zur allgemeinen Kurventheorie. Fundamenta Mathematicae, 10:96–115, 1927. doi:10.4064/fm-10-1-96-115

A. B. Kempe. On the geographical problem of the four colours. American Journal of Mathematics, 2(3):193–200, 1879. doi:10.2307/2369235

David Gale. A theorem on flows in networks. Pacific Journal of Mathematics, 7(2):1073–1082, 1957. doi:10.2140/pjm.1957.7.1073

Richard Bellman. On a routing problem. Quarterly of Applied Mathematics, 16(1):87–90, 1958. doi:10.1090/qam/102435

Richard M. Karp. A characterization of the minimum cycle mean in a digraph. Discrete Mathematics, 23(3):309–311, 1978. doi:10.1016/0012-365X(78)90011-0

Edsger W. Dijkstra. A note on two problems in connexion with graphs. Numerische Mathematik, 1(1):269–271, 1959. doi:10.1007/BF01386390

Makoto Matsumoto and Takuji Nishimura. Mersenne twister: a 623-dimensionally equidistributed uniform pseudo-random number generator. ACM Transactions on Modeling and Computer Simulation, 8(1):3–30, 1998. doi:10.1145/272991.272995

David W. Matula and Leland L. Beck. Smallest-last ordering and clustering and graph coloring algorithms. Journal of the ACM, 30(3):417–427, 1983. doi:10.1145/2402.322385

B. D. McKay and A. Piperno. Practical graph isomorphism, II. Journal of Symbolic Computation, 60:94–112, 2014. doi:10.1016/j.jsc.2013.09.003

Hiroshi Nagamochi and Toshihide Ibaraki. Computing edge-connectivity in multigraphs and capacitated graphs. SIAM Journal on Discrete Mathematics, 5(1):54–66, 1992. doi:10.1137/0405004

John W. Suurballe. Disjoint paths in a network. Networks, 4(2):125–145, 1974. doi:10.1002/net.3230040204

N. Tomizawa. On some techniques useful for solution of transportation network problems. Networks, 1(2):173–194, 1971. doi:10.1002/net.3230010206

Jack Edmonds. Paths, trees, and flowers. Canadian Journal of Mathematics, 17:449–467, 1965. doi:10.4153/CJM-1965-045-4

Jean-Claude Picard and Maurice Queyranne. On the structure of all minimum cuts in a network and applications. Mathematical Programming Study, 13:8–16, 1980. doi:10.1007/BFb0120902

Alexander Schrijver. Combinatorial Optimization: Polyhedra and Efficiency. Springer-Verlag, 2003.

Eugene L. Lawler. A procedure for computing the K best solutions to discrete optimization problems and its application to the shortest path problem. Management Science, 18(7):401–405, 1972. doi:10.1287/mnsc.18.7.401

Donald E. Knuth. The Art of Computer Programming, Volume 2: Seminumerical Algorithms. Addison-Wesley, 1969.

George B. Dantzig. Linear Programming and Extensions. Princeton University Press, 1963.

Ronald A. Howard. Dynamic Programming and Markov Processes. MIT Press, 1960.

Ali Dasdan. Experimental analysis of the fastest optimum cycle ratio and mean algorithms. ACM Transactions on Design Automation of Electronic Systems, 9(4):385–418, 2004. doi:10.1145/1027084.1027085

G. A. Croes. A method for solving traveling-salesman problems. Operations Research, 6(6):791–812, 1958. doi:10.1287/opre.6.6.791

Horst W. Hamacher. A note on K best network flows. Annals of Operations Research, 57(1):65–72, 1995. doi:10.1007/BF02099691

Katta G. Murty. Solving the fixed charge problem by ranking the extreme points. Operations Research, 16(2):268–279, 1968. doi:10.1287/opre.16.2.268

D. J. Rosenkrantz, R. E. Stearns, and P. M. Lewis II. An analysis of several heuristics for the traveling salesman problem. SIAM Journal on Computing, 6(3):563–581, 1977. doi:10.1137/0206041

Robert E. Tarjan. Depth-first search and linear graph algorithms. SIAM Journal on Computing, 1(2):146–160, 1972. doi:10.1137/0201010

D. D. Sleator and R. E. Tarjan. Self-adjusting binary search trees. Journal of the ACM, 32(3):652–686, 1985. doi:10.1145/3828.3835

William T. Tutte. The factorization of linear graphs. Journal of the London Mathematical Society, s1–22(2):107–111, 1947. doi:10.1112/jlms/s1-22.2.107

SoPlex, the Sequential object-oriented simPlex. https://soplex.zib.de/

Jack Edmonds. Optimum branchings. Journal of Research of the National Bureau of Standards, 71B(4):233–240, 1967. doi:10.6028/jres.071B.032

Robert E. Tarjan. Data Structures and Network Algorithms. SIAM, 1983. doi:10.1137/1.9781611970265

Zvi Galil. Efficient algorithms for finding maximum matching in graphs. ACM Computing Surveys, 18(1):23–38, 1986. doi:10.1145/6462.6502

John W. Suurballe and Robert E. Tarjan. A quick method for finding shortest pairs of disjoint paths. Networks, 14(2):325–336, 1984. doi:10.1002/net.3230140209

Robert E. Tarjan and Mihalis Yannakakis. Simple linear-time algorithms to test chordality of graphs, test acyclicity of hypergraphs, and selectively reduce acyclic hypergraphs. SIAM Journal on Computing, 13(3):566–579, 1984. doi:10.1137/0213035

E. Tomita and T. Seki. An efficient branch-and-bound algorithm for finding a maximum clique. In Discrete Mathematics and Theoretical Computer Science (DMTCS 2003), Lecture Notes in Computer Science, volume 2731, pages 278–289. Springer, 2003. doi:10.1007/3-540-45066-1_22

Donald B. Johnson. Priority queues with update and finding minimum spanning trees. Information Processing Letters, 4(3):53–57, 1975. doi:10.1016/0020-0190(75)90001-0